

The Road to React



2025 Edition
with React 19 and React Hooks
Required Knowledge: JavaScript

Robin Wieruch

El camino para reaccionar

El libro React.js en JavaScript (edición 2025)

Robin Wieruch

Este libro está disponible en <https://leanpub.com/the-road-to-learn-react>

Esta versión fue publicada el 21/05/2025



Este es un [Leanpub](#) Libro. Leanpub empodera a autores y editores con el proceso de Lean Publishing. [Lean Publishing](#) es el acto de publicar un libro electrónico en progreso utilizando herramientas livianas y muchas iteraciones para obtener comentarios de los lectores, modificarlo hasta tener el libro correcto y generar tracción una vez que lo tenga.

© 2016 - 2025 Robin Wieruch

¡Twittea este libro!

¡Por favor ayuda a Robin Wieruch difundiendo la información sobre este libro en [Twitter!](#)

El tweet sugerido para este libro es:

Voy a aprender [#ReactJs](#) con The Road to React de [@rwieruch](#). Únete a mí en mi viaje <https://roadtoreact.com>

El hashtag sugerido para este libro es [#ReactJs](#).

Descubra lo que otras personas dicen sobre el libro haciendo clic en este enlace para buscar este hashtag en Twitter:

[#ReactJs](#)

Contenido

Prólogo	1
Acerca del autor	2
Preguntas frecuentes	3
¿Para quién es este libro?	4
¿Cómo leer el libro?	5
 Fundamentos de React	 6
Hola React	7
Requisitos	9
Configuración de un proyecto React.	11
Estructura del	13
proyecto . Scripts npm :	15
Conozca el componente React	17
Reaccionar JSX.	21
Listas en React	26
Conozca otro componente de React	32
Instanciación de componentes de React	36
Reaccionar DOM	39
Declaración de componentes de React.	41
Función de controlador en JSX.	45
Accesorios de React.	48
Estado de reacción	52
Manejadores de devolución de llamada en JSX.	57
Estado de elevación en React.	60
Componentes controlados por React.	66
Manejo de accesorios (avanzado)	69
Efectos secundarios de React	80
Ganchos personalizados de React (avanzados)	84
Fragmentos de React.	89
Componente React reutilizable	92
Composición de componentes de React	95
Imperativo Reaccionar.	98
Manejador en línea en JSX.	104
Datos asíncronos de React	111

CONTENIDO

Representación condicional de React.	114
Estado avanzado de React.	119
Reaccionar Estados Imposibles.	124
Obtención de datos con React.	129
Recuperación de datos en React	132
Funciones memorizadas en React (avanzado)	136
Obtención explícita de datos con React	139
Bibliotecas de terceros en React.	143
Async/Await en React.	146
Formularios en React.	149
Formularios con acciones	153
Una hoja de ruta para React	155
Estilo en React	159
CSS en React	162
Módulos CSS en React.	168
Componentes con estilo en React	174
SVG en React.	180
Mantenimiento de React	183
Rendimiento en React (Avanzado)	184
TypeScript en React	195
Pruebas en React.	208
Estructura del proyecto React.	238
Reacción en el mundo real (avanzado)	244
Ordenando	245
Ordenamiento inverso	251
Recordar últimas búsquedas	254
Búsqueda paginada.	264
Implementación de una aplicación React	274
Proceso de construcción.	275
Implementar en Firebase	276
Describir	279

Prefacio

He sido desarrollador de React desde sus inicios. Cuando lo conocí, me causó cierta curiosidad, ya que se distinguía de la competencia por su énfasis en el uso exclusivo de componentes. Más de una década después, no me imagino trabajando con ningún otro framework en un futuro próximo. React continúa reinventándose, impulsando también la evolución de otros frameworks. Como desarrollador web freelance que colabora estrechamente con empresas, React es mi compañero indispensable en el día a día, mejorando mi productividad en cada proyecto.

"El Camino a React" se publicó en 2016 y, desde entonces, lo he reescrito prácticamente cada año. Este libro enseña los principios básicos de React y te guía en la creación de una aplicación práctica en React puro sin herramientas complejas. Abarca todo, desde la configuración del proyecto hasta su implementación en un servidor. Cada capítulo incluye lecturas recomendadas y ejercicios adicionales. Al finalizar, tendrás las habilidades necesarias para desarrollar tus propias aplicaciones en React.

En The Road to React, establezco una base sólida antes de explorar el ecosistema React más amplio.

El libro aclara conceptos generales, patrones y mejores prácticas para aplicaciones React del mundo real.

En definitiva, aprenderás a crear una aplicación React desde cero, incorporando funciones como paginación, búsqueda del lado del cliente y del servidor, e interacciones avanzadas de interfaz de usuario como la ordenación. Espero que este libro transmita mi pasión por React y JavaScript, ayudándote a emprender tu camino con confianza.

Espero que disfrutes de la lectura de este libro y que te ayude a empezar. Si tienes algún comentario, no dudes en contactarme en Twitter: [@rwieruch](https://twitter.com/rwieruch)¹ o [LinkedIn](https://www.linkedin.com/in/rwieruch/)². ¡Me encantaría saber de usted!

Como escribir este libro me ha costado mucho esfuerzo, te agradecería que lo compartieras con tus amigos y compañeros de trabajo. Significaría mucho para mí y me ayudaría a llegar a más personas que desean aprender desarrollo web. Dedico este libro a todos los aspirantes a desarrolladores web con ganas de aprender y crecer.

¡Feliz codificación!

¹<https://x.com/rwieruch>

²<https://tinyurl.com/c2y283mk>

Acerca del autor

Soy un desarrollador full-stack alemán apasionado por aprender y enseñar JavaScript. Tras finalizar mi máster en informática, me sumergí en el mundo de las startups, utilizando JavaScript extensamente tanto a nivel profesional como en mi tiempo libre. Colaborando con un equipo excepcional de ingenieros en Berlín, Alemania, desarrollamos aplicaciones JavaScript a gran escala, lo que despertó mi interés en compartir estos conocimientos.

Durante este tiempo, escribí regularmente artículos sobre desarrollo web para mi sitio web. Los comentarios positivos de los lectores interesados en aprender de mis artículos me motivaron a perfeccionar mi estilo de escritura y enseñanza. Con cada artículo, mi capacidad para educar eficazmente a otros seguía creciendo. Ver cómo los estudiantes prosperan al brindarles objetivos claros y retroalimentación rápida es particularmente gratificante.

Actualmente, trabajo como desarrollador freelance y colaboro estrechamente con empresas en sus productos, tanto como freelance como consultor. Puedes encontrar más información sobre cómo colaborar conmigo en mi [sitio web](https://www.robinwieruch.de/)³.

³<https://www.robinwieruch.de/>

Preguntas frecuentes

¿Cómo obtener actualizaciones?

Manténgase informado sobre las últimas actualizaciones a través de mi [Newsletter](#) . Tenga la seguridad de que priorizo compartir solo contenido de alta calidad.

¿El material de aprendizaje está actualizado?

A diferencia de los libros de programación tradicionales, que se desactualizan rápidamente, este libro autoeditado permite actualizaciones inmediatas cada vez que se lanzan nuevas versiones de herramientas o tecnologías relevantes. Siempre tendrás acceso a la información más reciente.

¿Puedo obtener una copia digital del libro si lo he comprado en Amazon?

¿Sabías que también está disponible en mi sitio web? Para acceder al contenido, simplemente envíame un correo electrónico con el comprobante de tu compra en Amazon. Luego, desbloquearé el contenido para ti en mi sitio web, donde siempre tendrás acceso a la última versión del libro. Agradezco mucho tu apoyo, así que una reseña en Amazon sería fantástica.

¿Por qué la versión impresa es tan grande?

Si ha adquirido la versión impresa del libro, considere tomar notas directamente en sus páginas. La decisión de mantener el libro impreso extra grande se tomó deliberadamente para ofrecer amplio espacio para fragmentos de código extensos y para que usted tenga suficiente espacio para sus anotaciones y notas personales. Esta decisión de tamaño se ideó con la intención de mejorar su experiencia general de lectura y aprendizaje.

¿Por qué el libro está escrito como un tutorial de lectura larga?

La forma poco convencional en que este libro está escrito y estructurado podría sorprender a quienes estén más acostumbrados al formato convencional de los textos de programación. Cuando empecé a programar, había escasez de recursos prácticos y directos. Como estudiante, encontré muy valiosos los materiales que proporcionaban instrucciones paso a paso, guiándome no solo por el qué y el cómo, sino también por el porqué de cada concepto. Con el objetivo de replicar esta experiencia de aprendizaje inmersiva, me he propuesto autopublicarlo, con la esperanza de ofrecer esta valiosa oportunidad de compartir conocimientos a otros desarrolladores de nuestra comunidad.

¿Dónde puedo obtener ayuda?

Si encuentras alguna sección difícil en el código, te animo a que te unas a la comunidad en [Discord](#) , donde puedes compartir tus pensamientos y comentarios con otros estudiantes.

<https://rwieruch.substack.com/>

<https://discord.gg/ssE5VMSPkV>

¿Para quién es este libro?

Principiantes de JavaScript

Principiantes de JavaScript con conocimientos básicos de JS, CSS y HTML: Si acabas de empezar con el desarrollo web y tienes conocimientos básicos de JS, CSS y HTML, este libro te proporcionará todo lo necesario para aprender React. Sin embargo, si sientes que te faltan conocimientos de JavaScript, no dudes en revisarlo antes de continuar. Encontrarás numerosas referencias a conceptos fundamentales de JavaScript.

Veteranos de JavaScript

Veteranos de JavaScript que vienen de jQuery: Si has usado JavaScript extensamente con jQuery, MooTools o Dojo, el nuevo panorama de JavaScript puede resultar abrumador al retomar el ritmo. Sin embargo, la mayoría de los conceptos fundamentales no han cambiado: sigue siendo JavaScript y HTML. Este libro te proporcionará un sólido punto de partida para React.

Entusiastas de JavaScript

Entusiastas de JavaScript con conocimientos de otros frameworks SPA modernos: Si vienes de Angular o Vue, habrá diferencias en la forma de escribir aplicaciones con React. Sin embargo, todos estos frameworks comparten los mismos fundamentos de JavaScript y HTML. Una vez que te adaptes al enfoque de React, no deberías tener problemas para adoptarlo.

Desarrolladores que no utilizan JavaScript

Si vienes de otro lenguaje de programación, probablemente estés más familiarizado con los diversos aspectos de la programación. Después de aprender los fundamentos de JavaScript y HTML, te resultará fácil aprender React con este libro.

Diseñadores y entusiastas de UI/UX

Si tu profesión principal es el diseño, la interacción con el usuario o la experiencia de usuario, no dudes en adquirir este libro. Es posible que ya estés familiarizado con HTML y CSS, lo cual es una ventaja. Tras cubrir algunos fundamentos de JavaScript, estarás bien preparado para trabajar con el resto del libro.

Hoy en día, la UI/UX está cada vez más entrelazada con la implementación, y React suele encargarse de estos detalles. Saber cómo funcionan las cosas en el código será una valiosa ventaja.

Líderes de equipo, propietarios de productos o gerentes de productos

Si eres líder de equipo, propietario de producto o gerente de producto en tu departamento de desarrollo, este libro te proporcionará un desglose claro de los componentes esenciales de una aplicación React. Cada sección explica un concepto, patrón o técnica de React que añade una funcionalidad o mejora la arquitectura general. Es una guía de referencia completa para React.

¿Cómo leer el libro?

La mayoría de los libros de programación son de alto nivel y profundizan en detalles técnicos, pero a menudo carecen de la capacidad de incitar a los lectores a la programación. Este libro puede ser diferente a los que estás acostumbrado a leer en este ámbito, ya que intenta enseñar programación práctica a los aspirantes a desarrolladores. Intento encontrar un equilibrio entre ser pragmático, proporcionándote las herramientas necesarias para realizar el trabajo, y ser detallista, brindándote la información necesaria para comprender estas herramientas y cómo se utilizan en la práctica.

Tomar apuntes

Si tienes la versión impresa del libro, no dudes en subrayar párrafos, tomar notas o anotar fragmentos de código. Por eso es tan grande. Si no tienes la versión impresa, guarda un cuaderno aparte para tus aprendizajes. Tomar notas refuerza lo aprendido y siempre podrás retomarlas. Con cada nuevo aprendizaje, comprenderás mejor el panorama general y cómo encajan las piezas pequeñas, así que escribir lo aprendido en una hoja de papel es un buen ejercicio.

Código Código Código

Cada sección te presenta un nuevo tema de forma pragmática. Por eso, leer la sección completa no es suficiente para convertirte en desarrollador, ya que abarca mucho en una sola sección. Así que no deberías apresurarte de una sección a otra. En su lugar, recomiendo tener una computadora a mano para programar mientras tanto.

No te limites a copiar y pegar código; escríbelo tú mismo. No te conformes con usar el código del libro; experimenta con él. Observa qué falla y cómo solucionarlo. Observa cómo ciertos cambios afectan el resultado. Y ve cómo puedes ampliar o incluso mejorar el código añadiéndole algunas líneas. Al fin y al cabo, de eso se trata la programación. De nada sirve leer el libro a toda prisa si no has escrito ni una sola línea. ¡Así que ponte manos a la obra y programa más que leyendo!

Anticipar

En este libro se presentarán muchos problemas de codificación. En ocasiones, te daré la opción de resolverlos tú mismo antes de leer la solución en el siguiente párrafo o fragmento de código.

Sin embargo, esto interrumpe el flujo de repetición, por lo que mantengo estos estímulos al mínimo.

En cambio, espero que tengas ganas de adelantarte. Intenta resolver los problemas antes de que tenga la oportunidad de presentarte la solución. Solo probando, fallando y resolviendo un problema te convertirás en un mejor desarrollador.

Tomar descansos

Como cada sección presenta un tema nuevo, es fácil olvidar lo aprendido en la sección anterior. Además de programar en cada sección, recomiendo tomar descansos entre ellas para asimilar lo aprendido. Lee la sección, programa mientras lo haces, programa un poco más si quieres, y luego descansa. Piensa en lo que has aprendido mientras das un paseo o hablas con alguien sobre lo que has aprendido, incluso si a esa persona no le gusta programar. **Después de todo, tomar descansos siempre es esencial si quieres aprender algo nuevo.**

Fundamentos de React

En la fase inicial de este proceso de aprendizaje, exploraremos los fundamentos esenciales de React y te guiaremos en la creación de tu primer proyecto. A medida que avancemos, profundizaremos en las capacidades de React, implementando funciones prácticas como la búsqueda del lado del cliente y del servidor, la obtención remota de datos y la gestión avanzada de estados. Este enfoque práctico refleja el desarrollo de una aplicación web real. Al finalizar, tendrás una aplicación React completamente funcional que interactúa fluidamente con datos reales.

Hola React

Solicitudes de una sola página (SPA) Han ganado popularidad con frameworks de SPA de primera generación como Angular (de Google), Ember, Knockout y Backbone. Estos frameworks facilitaron la creación de aplicaciones web que iban más allá de JavaScript y jQuery. React, presentado por Facebook en 2013, surgió como otra solución para las SPA, ofreciendo una potente forma de crear aplicaciones web modernas en JavaScript.

Retrocedamos en el tiempo antes de que existieran las SPA. En el pasado, los sitios web y las aplicaciones web se renderizaban en el servidor. Cuando un usuario accedía a una URL en un navegador, se enviaba una solicitud a un servidor web, que devolvía un archivo HTML junto con sus archivos CSS y JavaScript asociados. Tras un retraso en la red, el usuario veía el HTML renderizado y podía empezar a interactuar con él. Cada transición de página posterior repetía este proceso. En este modelo, el servidor gestionaba la mayoría de las tareas esenciales, mientras que el cliente desempeñaba un papel mínimo, principalmente renderizando páginas. HTML y CSS básicos estructuraban y aplicaban estilo a la aplicación, mientras que JavaScript, a menudo en forma de jQuery, permitía interacciones (p. ej., activar y desactivar un menú desplegable) o estilos avanzados (p. ej., posicionar una información sobre herramientas).

En cambio, los frameworks SPA desplazaron el enfoque del servidor al cliente. En las SPA, el servidor entrega principalmente JavaScript a través de la red, junto con un archivo HTML mínimo. El navegador ejecuta los archivos JavaScript vinculados, renderizando toda la aplicación dinámicamente mediante HTML y CSS, apoyándose en JavaScript para las interacciones. En su forma más extrema, un usuario que visita una URL recibe un archivo HTML pequeño y un archivo JavaScript más grande. Tras un breve retraso en la red y el renderizado, JavaScript renderiza el contenido en el navegador. Las transiciones de página posteriores ya no requieren solicitudes adicionales al servidor para nuevos archivos; en su lugar, el JavaScript cargado inicialmente renderiza dinámicamente las nuevas páginas.

React, junto con otras soluciones de SPA, desempeñó un papel fundamental en esta transformación. En esencia, una SPA es un conjunto estructurado de JavaScript, organizado en carpetas y archivos, que forma una aplicación completa. Un framework de SPA como React proporciona las herramientas para diseñar esta aplicación basada en JavaScript. Cuando un usuario visita la URL de una aplicación web, esta se entrega una sola vez a través de la red. A partir de ese momento, React, o cualquier otro framework de SPA, toma el control, renderizando todo en el navegador como HTML y gestionando las interacciones del usuario con JavaScript.

Con el auge de React, el concepto de componentes se volvió fundamental. Cada componente encapsula sus aspectos visuales y funcionales mediante HTML, CSS y JavaScript. Una vez definidos, los componentes pueden combinarse en una jerarquía para crear una aplicación completa. Si bien React se centra principalmente en los componentes como biblioteca, su ecosistema flexible le permite funcionar como un framework. Con una API optimizada, un ecosistema estable pero en constante evolución y una comunidad que te apoya, ¡React te da la bienvenida con los brazos abiertos! :-)

Ceremonias

- Lea más sobre [sitios web y aplicaciones web](#) .

<https://bit.ly/3BZOL1o>

<https://www.robinwieruch.de/aplicaciones-web/>

- Ver [React.js: El Documental](#) . • Lea más sobre [los fundamentos de JavaScript necesarios para React](#) . •
- Opcionalmente, si necesitas un impulso motivacional:
 - Lea más sobre [cómo aprender un marco](#)¹ .
 - Lea más sobre [cómo aprender React](#)¹¹.

<https://bit.ly/3xrvxkl>

<https://www.robinwieruch.de/fundamentos-de-javascript-requisitos-de-react/>

¹ <https://www.robinwieruch.de/how-to-learn-framework/>

¹¹<https://www.robinwieruch.de/aprender-react-js/>

Requisitos

Para navegar este libro eficazmente, es necesario tener conocimientos básicos de desarrollo web, incluyendo HTML, CSS y JavaScript.

Familiaridad con [las API](#)¹² Es útil, ya que se explicarán más adelante. Además, necesitarás las siguientes herramientas de codificación para seguir el proceso.

Editor y Terminal

Para esta experiencia de aprendizaje, recomiendo usar un [IDE](#)¹³, como Visual Studio Code (VSCode), especialmente para principiantes. Ofrece un editor avanzado con terminal integrada y es la opción más popular entre los desarrolladores web. He creado una [guía de configuración](#)¹ Para ayudarte a iniciarte en el desarrollo web general. Incluye todos los detalles y se mantiene separado de este libro, ya que ofrece opciones tanto para usuarios de Windows como de macOS. Opcionalmente, también puedes consultar mi [guía completa de configuración de macOS](#)¹ .

Si prefiere no configurar un editor y una terminal en su máquina local, [CodeSandbox](#)¹ Es una alternativa viable en línea. Si bien CodeSandbox es ideal para compartir código, una configuración local ofrece un mejor entorno de aprendizaje para crear aplicaciones web. Si planea desarrollar aplicaciones profesionalmente, eventualmente necesitará una configuración local.

A lo largo de este libro, utilizaré «línea de comandos» como término general para referirse a herramientas de línea de comandos, terminales y terminales integradas. De igual forma, editor, editor de texto e IDE se usarán indistintamente, según la configuración.

Además, recomiendo usar GitHub para gestionar proyectos a medida que avanzamos en los ejercicios. He proporcionado una [breve guía](#)¹ Sobre el uso de Git y GitHub. El control de versiones es invaluable: permite rastrear cambios, corregir errores y realizar un seguimiento más eficaz. También es una excelente manera de compartir tu código con otros.

Nodo y NPM

Antes de comenzar, necesitará instalar [Node y NPM](#)¹ . Estas herramientas ayudan a administrar las bibliotecas (paquetes de Node) necesarias en este libro. Estos paquetes pueden abarcar desde pequeñas utilidades hasta frameworks completos. Usaremos npm (Administrador de Paquetes de Node) para instalarlos.

Para comprobar si Node y npm están instalados, ejecute los siguientes comandos en su terminal:

¹²<https://www.robinwieruch.de/que-es-una-api-javascript/>

¹³<http://bit.ly/3OWCnan>

¹ <https://www.robinwieruch.de/developer-setup/>

¹ <https://www.robinwieruch.de/configuración-mac-desarrollo-web/> ¹ <https://codesandbox.io> ¹ <https://www.robinwieruch.de/git-essential-commands/> ¹ <https://nodejs.org/es/>

Línea de comandos

```
nodo --versión  
*vXX.YY.ZZ  
  
npm --versión  
*vXX.YY.ZZ
```

Si no aparecen los números de versión, deberá instalar Node y npm. Si ya los tiene instalados, asegúrese de usar la última versión. Si es nuevo en npm o necesita un repaso, mi [curso intensivo de npm](#)¹ Te ayudará a ponerte al día.

Ceremonias:

- Opcional: Leer más sobre [yarn](#)² y [pnpm](#)²¹. Ambos pueden usarse como reemplazo de npm. Sin embargo, no recomiendo usarlas como principiante. Este ejercicio solo sirve para que conozcas las alternativas.

¹ <https://www.robinwieruch.de/npm-crash-course/>

² <https://yarnpkg.com/>

²¹<https://pnpm.io/>

Configurar un proyecto React

En El camino hacia React, usaremos [Vite](#)²² Para configurar nuestra aplicación React. Vite (palabra francesa que significa rápido) es una herramienta de desarrollo moderna para frameworks web contemporáneos (p. ej., React). Incluye valores predeterminados prácticos (léase: configuración integrada) y, al mismo tiempo, es altamente extensible para casos de uso específicos (p. ej., SVG, linting, TypeScript, renderizado del lado del servidor).

El núcleo de Vite consta de:

- **Un servidor de desarrollo**, que le permite ejecutar su aplicación React localmente (léase: desarrollo). entorno laboral).
- **Un empaquetador** que genera archivos altamente optimizados para una implementación lista para producción (léase: entorno de producción).

Para quienes se inician en React, la principal ventaja de Vite es que les permite centrarse exclusivamente en aprender React sin distraerse con herramientas complejas. Esto convierte a Vite en el compañero perfecto para iniciarse en React.

Hay dos formas de crear un proyecto React con Vite:

- **Utilizando una plantilla en línea**²³ Puedes elegir entre React (recomendado para este libro) o React con TypeScript (para usuarios avanzados que requieren la implementación manual de tipos). Esta opción te permite trabajar en línea sin configurar un entorno local.
- **Configurar Vite localmente** (recomendado): este método implica crear un proyecto React con Vite en su máquina local y trabajar en su IDE preferido (por ejemplo, VSCode).

Dado que la plantilla en línea funciona de inmediato, en esta sección nos centraremos en configurar Vite en tu equipo local. En una sección anterior, instalaste Node y npm. Este último te permite instalar dependencias de terceros (es decir, bibliotecas, frameworks, etc.) desde la línea de comandos.

Para empezar, abre tu herramienta de línea de comandos y navega hasta la carpeta donde quieres crear tu proyecto de React. Aquí tienes un curso intensivo sobre navegación en la línea de comandos:

- use `pwd` (en Windows: `cd`) para mostrar la carpeta actual
- use `ls` (en Windows: `dir`) para mostrar todas las carpetas y archivos en la carpeta actual
- use `mkdir <nombre_de_carpeta>` para crear una carpeta
- use `cd <nombre_de_carpeta>` para moverse dentro de una carpeta
- use `cd ..` para moverse fuera de una carpeta

Después de navegar a la carpeta donde quieres crear tu proyecto de React, introduce el siguiente comando. Nos referiremos a este proyecto como `hacker-stories`, pero puedes elegir el nombre que prefieras.

²²<https://bit.ly/3BsG1TH>

²³<https://bit.ly/3RPAZWz>

Línea de comandos

```
npm crea vite@últimas historias de hackers -- --plantilla react
```

Opcionalmente, puedes elegir un proyecto React + TypeScript si te sientes seguro. Consulta el sitio web de instalación de Vite para obtener instrucciones sobre cómo configurar un proyecto React + TypeScript. Este libro incluye una sección sobre TypeScript más adelante; sin embargo, no proporciona una guía paso a paso para convertir JavaScript a TypeScript. En su lugar, al final de cada sección, encontrarás una implementación alternativa de TypeScript.

A continuación, siga las instrucciones de la línea de comandos para navegar a la carpeta del proyecto, instalar todas las dependencias de terceros y ejecutar el proyecto localmente en su máquina:

Línea de comandos

Historias de hackers de CD

```
npm instalar
```

```
npm ejecutar dev
```

La línea de comandos mostrará la URL donde se ejecuta tu proyecto en el navegador. Abre el navegador, accede a la URL proporcionada y verifica que el proyecto de React se muestre correctamente.

Además, comprueba en tu archivo package.json si tienes la última versión de React. Al momento de escribir esto, Vite viene con React 18, pero React 19 ya está disponible. Si quieres usar React 19, puedes actualizar manualmente React en tu proyecto.

Línea de comandos

```
npm instalar react@latest react-dom@latest
```

Si está utilizando React con TypeScript, también necesita actualizar los tipos de React:

Línea de comandos

```
npm install --save-dev @types/react@latest @types/react-dom@latest
```

Continuaremos desarrollando este proyecto en las siguientes secciones; sin embargo, durante el resto de esta sección, exploraremos la estructura del proyecto y los scripts (por ejemplo, npm run dev).

Ceremonias:

- Lea más sobre [cómo iniciar un proyecto React²](https://www.robinwieruch.de/react-starter/) .

² <https://www.robinwieruch.de/react-starter/>

Estructura del proyecto

Primero, abramos la aplicación en un editor/IDE. Si usa VSCode, simplemente escriba ``code.`` en la línea de comandos. Debería aparecer la siguiente estructura de carpetas (o una variación, según la versión de Vite):

Estructura del proyecto

```
historias de hackers/
--módulos_de_nodo/
--público/ ----
vite.svg --src/

----activos/
-----react.svg ----
Aplicación.css
----Aplicación.jsx
----index.css
----main.jsx
--.gitignore --
eslint.config.js
--índice.html

--package-lock.json --
package.json
--LÉAME.md
--vite.config.js
```

A continuación se muestra un desglose de las carpetas y archivos más importantes:

- **package.json**: Este archivo muestra una lista de todas las dependencias de terceros (léase: paquetes de nodos que se encuentran en la carpeta `node_modules/`) y otras configuraciones esenciales del proyecto relacionadas con Node/npm. • **package-lock.json**: Este archivo indica a npm cómo desglosar (léase: resolver) todos los paquetes de nodos. Versiones y sus dependencias internas de terceros. No tocaremos este archivo. • **node_modules/**: Esta carpeta contiene todos los paquetes de Node instalados. Dado que usamos Vite para crear nuestra aplicación React, ya tenemos varios módulos de Node (por ejemplo, React) instalados. No tocaremos esta carpeta.
- **.gitignore**: Este archivo indica todas las carpetas y archivos que no deben añadirse a tu repositorio Git al usar Git, ya que deben estar ubicados únicamente en tu equipo local. La carpeta `node_modules/` es un ejemplo. Basta con compartir los archivos `package.json` y `package-lock.json` con otros desarrolladores del equipo para que puedan instalar las dependencias con `npm install` sin tener que compartir la carpeta `node_modules/` completa con todos.

- **vite.config.js**: Un archivo para configurar Vite. Al abrirlo, debería aparecer el plugin de React de Vite. Si ejecuta Vite con otro framework web, aparecerá el plugin de Vite de dicho framework. Finalmente, hay muchas más opciones que se pueden configurar aquí.
- **public/**: Esta carpeta contiene activos estáticos para el proyecto como un **favicon**². Se utiliza para la miniatura de la pestaña del navegador al iniciar el servidor de desarrollo o compilar el proyecto para producción.
- **index.html**: El HTML que se muestra en el navegador al iniciar el proyecto. Si lo abres, no deberías ver mucho contenido. Sin embargo, deberías ver una etiqueta de script que enlaza a la carpeta de origen, donde se encuentra todo el código de React para generar HTML/CSS en el navegador.

Al principio, todo lo necesario se encuentra en la carpeta `src/`. El enfoque principal está en el archivo `src/App.jsx`, donde se implementan los componentes de React. Este archivo servirá como base para tu aplicación, pero más adelante, podrías querer dividir tus componentes de React en varios archivos, cada uno de los cuales gestionará uno o más componentes. Más adelante llegaremos a ese punto.

Además, encontrarás el archivo `src/main.jsx`, que sirve como punto de entrada al mundo de React. Te familiarizarás con este archivo en secciones posteriores. También existen los archivos `src/index.css` y `src/App.css` para aplicar estilos a tu aplicación y componentes. Ambos incluyen estilos predeterminados al abrirlas. Los modificarás más adelante.

² <https://bit.ly/3QvRupG>

Scripts npm

Después de conocer la estructura de carpetas y archivos de tu proyecto React, veamos los comandos disponibles. Todos los comandos específicos de tu proyecto se encuentran en el archivo `package.json`, en la propiedad `scripts`. Dependiendo de tu versión de Vite, pueden ser similares a estos:

`package.json`

```
"dev": "vite", "build":  
"construcción de vite",  
"lint": "eslint.", "preview":  
"vista previa de vite"
```

Estos scripts se ejecutan con el comando `npm run <script>` en una terminal integrada en el IDE o en su herramienta de línea de comandos independiente. Los comandos son los siguientes:

Línea de comandos

```
# Ejecuta la aplicación localmente para el navegador npm run  
dev
```

```
# Revise la aplicación localmente para detectar errores de estilo de  
código npm run lint
```

```
# Construye la aplicación para producción
```

```
npm ejecutar compilación
```

Otro comando de los scripts npm anteriores, llamado `preview`, se puede usar para ejecutar la compilación lista para producción en tu equipo local con fines de prueba. Para que funcione, debes ejecutar `npm run build` antes de ejecutar `npm run preview`. En esencia, `npm run dev` y `npm run preview` (después de `npm run build`) deberían mostrar la misma salida en el navegador. Sin embargo, el primero no está optimizado para producción y debe usarse exclusivamente para el desarrollo local de la aplicación.

Ceremonias:

- Lea más sobre [Vite²](#).

Ejercicios de scripts npm:

- Inicie su aplicación React con `npm run dev` en la línea de comandos y compruébelo en el navegador.

*

Salga del comando en la línea de comando presionando Control + C.

² <https://bit.ly/3BsG1TH>

– Ejecute el script `npm run build` y verifique que se haya agregado una carpeta `dist/` a su proyecto. Tenga en cuenta que la carpeta de compilación puede usarse más adelante para implementar su aplicación. Después, ejecute `npm run preview` para ver la aplicación lista para producción en el navegador. • Cada vez que modifiquemos algo en nuestro código fuente en las siguientes secciones, asegúrese de revisar la salida en su navegador para obtener información visual. Use `npm run dev` para mantener su aplicación en ejecución. • Opcional: Si usa Git y GitHub, agregue y confirme los cambios después de cada sección del... libro.

Conozca el componente React

Toda aplicación React se basa en componentes React. En esta sección, se le presentará su primer componente React, ubicado en el archivo `src/App.jsx`. Debería ser similar al ejemplo a continuación. Dependiendo de su versión de Vite, el contenido del archivo puede variar ligeramente:

`src/App.jsx`

```
importar { useState } de 'react'; importar
reactLogo de './assets/react.svg'; importar viteLogo de './
vite.svg'; importar './App.css';

función App() { const
  [count, setCount] = useState(0);

  devolver (
    <>
      <div>
        <a href="https://vite.dev" target="_blank"> <img src={viteLogo}
          className="logo" alt="Logotipo de Vite" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img
            src={reactLogo}
            className="logotipo de React"
            alt="Logotipo de React" /
          >
        </a>
      </div>
      Vite + Reaccionar
      <div className="tarjeta">
        <button onClick={() => setCount((count) => count + 1)}> el conteo es {count} </
          button>

        <p>
          Edite <code>src/App.jsx</code> y guárdelo para probar HMR </p> </div>

      <p className="leer-los-documentos">
        Haga clic en los logotipos de Vite y React para obtener más
        información .
      </p>
    </>
  )
}
```

```
    </>
  );
}

exportar aplicación predeterminada;
```

Este archivo será el enfoque principal de este libro, a menos que se especifique lo contrario. Aunque aumentará de tamaño, al no dividirlo desde el principio en varios archivos, será más fácil de entender para principiantes, ya que todo está en un solo lugar. Una vez que te familiarices con React, te mostraré cómo dividir tu proyecto de React con tus componentes en varios archivos.

Comencemos por reducir este componente React a una versión más liviana para que pueda comenzar sin demasiado código repetitivo que distraiga² :

```
src/App.jsx
función App() { return (

  <div>
    Hola React
  </div> );

}
```

exportar aplicación predeterminada;

Opcionalmente, recomiendo dejar los archivos `src/index.css` y `src/App.css` en blanco para empezar desde cero en cuanto a estilo. A continuación, inicia tu aplicación con `npm run dev` en la línea de comandos y comprueba lo que se muestra en el navegador. Deberías ver el encabezado "Hola React". Antes de profundizar en cada tema, aquí tienes un breve resumen de tu código y de lo que abordaremos con más detalle en las siguientes secciones:

- **En primer lugar**, este componente React, llamado específicamente componente App, es solo una función de JavaScript. A diferencia de las funciones tradicionales de JavaScript, se define en **PascalCase**² . Un componente debe empezar con mayúscula; de lo contrario, no se considera un componente en React. El componente de aplicación se conoce comúnmente como componente de función. Los componentes de función son la forma moderna de usar componentes en React; sin embargo, tenga en cuenta que también existen otras variantes de componentes de React (consulte los tipos de componentes en una sección posterior).
- **En segundo lugar**, el componente App aún no tiene parámetros en la firma de su función. En las siguientes secciones, aprenderá a pasar información (consulte las propiedades en una sección posterior) de un componente a otro. Estas propiedades serán accesibles a través de la firma de la función como parámetros.

² <https://bit.ly/3IZckS>

² <https://www.robinwieruch.de/convencciones-de-nomenclatura-de-javascript/>

- En tercer lugar, el componente App devuelve código similar a HTML. Verá cómo esta nueva sintaxis (véase JSX en una sección posterior) permite combinar JavaScript y HTML para mostrar contenido altamente dinámico e interactivo en un navegador.

Como cualquier otra función de JavaScript, un componente de función puede tener detalles de implementación entre la firma de la función y la declaración de retorno. Verás esto en la práctica durante tu experiencia con React:

src/App.jsx

```
función App() {  
  // puedes hacer algo intermedio  
  
  devolver (  
    <div>  
      Hola React  
    </div>  
  );  
}
```

exportar aplicación predeterminada;

Las variables definidas en el cuerpo de la función se redefinirán cada vez que se ejecute esta función, lo que no debería ser algo nuevo si está familiarizado con JavaScript y sus funciones:

src/App.jsx

```
función App() {  
  constante título = 'Reaccionar';  
  
  devolver (  
    <div>  
      Hola React  
    </div>  
  );  
}
```

exportar aplicación predeterminada;

La función de un componente se ejecuta cada vez que se muestra en el navegador. Esto ocurre durante la visualización inicial (léase: renderizado) del componente, pero también cuando este se actualiza porque debe mostrar algo diferente debido a cambios (re-renderizado). Más adelante aprenderemos más sobre esto.

Como no queremos redefinir una variable dentro de una función cada vez que se ejecuta, también podríamos definirla fuera del componente. En este caso, el título no depende de ninguna información contenida en el componente de la función (por ejemplo, los parámetros de la firma de la función), por lo que es posible moverlo fuera. Por lo tanto, se definirá solo una vez y no cada vez que se invoque la función:

```
src/App.jsx
```

```
constante título = 'Reaccionar';
```

```
función App() { return  
  (  
    <div>  
      Hola React  
    </div>  
  );  
}
```

```
exportar aplicación predeterminada;
```

En tu trayectoria como desarrollador de React, y en esta experiencia de aprendizaje, te encontrarás con ambos escenarios: variables (y funciones) definidas fuera y dentro de un componente. Como regla general: si una variable no necesita nada del cuerpo del componente de función (por ejemplo, parámetros), defínela fuera del componente para evitar tener que redefinirla en cada llamada a la función.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#)² .
 - Recapitular todos los [cambios del código fuente](#)³ de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)³¹. Piensa en maneras de mostrar la variable de título en el HTML devuelto por el componente de tu aplicación. En la siguiente sección, usaremos esta variable.

² <https://tinyurl.com/43uekprv>

³ <https://tinyurl.com/n3wujpmu>

³¹<https://bit.ly/3OvfqLO>

Reaccionar JSX

Todo lo que devuelva un componente React se mostrará en el navegador. Hasta ahora, solo devolvíamos HTML del componente App. Sin embargo, recuerden que mencioné que la salida devuelta por el componente App no solo se asemeja a HTML, sino que también puede combinarse con JavaScript. De hecho, esta salida se llama JSX (JavaScript XML), que combina eficazmente HTML y JavaScript. Veamos cómo funciona esto para mostrar la variable de la sección anterior:

```
src/App.jsx
```

```
constante título = 'Reaccionar';
```

```
función App() { return  
  (  
    <div>  
      Hola {título}  
    </div>  
  );  
}
```

```
exportar aplicación predeterminada;
```

Puedes reiniciar tu aplicación con `npm run dev` (o comprobar si sigue funcionando) y buscar el título mostrado (léase: renderizado) en el navegador. La salida debería ser "Hola React". Si modificas la variable en el código fuente, el navegador debería reflejar ese cambio.

Cambiar la variable en el código fuente y ver el cambio reflejado en el navegador no solo está relacionado con React, sino también con el servidor de desarrollo subyacente al iniciar nuestra aplicación desde la línea de comandos. Cada vez que uno de nuestros archivos cambia, el servidor de desarrollo lo detecta y recarga todos los archivos afectados en el navegador. El puente entre React y el servidor de desarrollo que posibilita este comportamiento se denomina React Fast Refresh (anteriormente React Hot Loader) en el lado de React y Hot Module Replacement en el lado del servidor de desarrollo.

A continuación, intenta definir un campo de entrada HTML (léase: etiqueta `<input>`) y una etiqueta HTML (léase: etiqueta `<label>`) en tu JSX. También debería ser posible enfocar el campo de entrada al hacer clic en la etiqueta, ya sea anidándolo en la etiqueta o usando atributos HTML específicos para ambos. El siguiente fragmento de código te mostrará la implementación de esta tarea en el libro, y quizá te sorprendas al descubrir que el HTML difiere ligeramente al usarlo en JSX:

```
src/App.jsx
```

```
constante título = 'Reaccionar';
```

```
función App() { return (
```

```
  <div>
```

```
    Hola {título}
```

```
    <label htmlFor="search">Buscar: </label> <input
```

```
      id="search" type="text" />
```

```
  </div> );
```

```
}
```

```
exportar aplicación predeterminada;
```

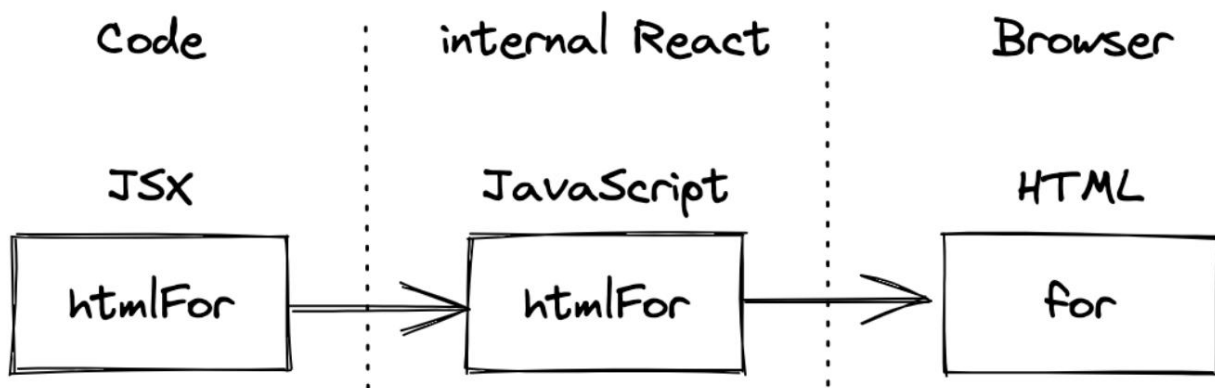
Para nuestra combinación de campo de entrada y etiqueta, especificamos tres atributos HTML: `htmlFor`, `id` y `type`. El atributo `type` es prácticamente obligatorio y no tiene nada que ver con enfocar el campo de entrada al hacer clic en la etiqueta. Sin embargo, aunque `id` y `type` deberían resultarte familiares del HTML nativo, `htmlFor` podría resultarte nuevo.

El atributo `htmlFor` refleja el atributo `for` en HTML estándar. Quizás te preguntes por qué este atributo difiere del HTML nativo. JSX reemplaza algunos atributos HTML internos debido a detalles de implementación internos de React. Sin embargo, puedes encontrar todos los [atributos HTML compatibles](#)³² En la documentación de React. Dado que JSX se parece más a JavaScript que a HTML, React usa [camelCase](#)³³ Convención de nomenclatura. A medida que aprendas más sobre React, encontrarás más atributos específicos de JSX, como `className` y `onClick`, en lugar de `class` y `onclick`.

Al usar HTML en JSX, React traduce internamente todos los atributos HTML a JavaScript, donde ciertas palabras, como `"class"` o `"for"`, se reservan durante el proceso de renderizado. Por lo tanto, React creó alternativas como `"className"` y `"htmlFor"`. Sin embargo, una vez que el HTML se renderiza en el navegador, los atributos se traducen de nuevo a su versión nativa.

³²<https://bit.ly/2Z42zcK>

³³<https://bit.ly/3jjQFn>



Más adelante, revisaremos el campo de entrada HTML y su etiqueta para obtener más detalles de implementación con JavaScript. Por ahora, para comparar cómo se usan HTML y JavaScript en JSX, usaremos tipos de datos JavaScript más complejos. En lugar de definir una cadena primitiva de JavaScript como "title", definamos un objeto JavaScript llamado "welcome" que tenga como propiedades un título (p. ej. , "React") y un saludo (p. ej. , "Hey") . Después, intentemos representar ambas propiedades del objeto en JSX, una junto a la otra, en la etiqueta `<h1>` .

El siguiente fragmento de código le mostrará la solución. Anteriormente, definimos una cadena primitiva de JavaScript para mostrarla en el componente App. Ahora, podemos hacer lo mismo con un objeto JavaScript accediendo a sus propiedades dentro de JSX:

src/App.jsx

```

constante bienvenida = {
  saludo: 'Hola', título:
  'Reaccionar', };

función App() { return (

  <div>
    <h1>
      {bienvenida.saludo} {bienvenida.título} </h1>

    <label htmlFor="search">Buscar: </label>
    <input id="búsqueda" tipo="texto" />
  </div>
);
}

exportar aplicación predeterminada;
  
```

Aunque HTML se puede usar casi (excepto los atributos) de forma nativa en JSX, todo lo que está entre llaves se puede usar para interpolar JavaScript. Por ejemplo, se podría definir una función que devuelva el título y ejecutarlo dentro de las llaves:

src/App.jsx

```
función getTitle(título) {
  título de devolución;
}

función App() { return (

  <div>
    Hola {getTitle('React')}</h1>

    <label htmlFor="search">Buscar: </label>
    <input id="búsqueda" tipo="texto" />
  </div>
);
}
```

exportar aplicación predeterminada;

JSX es una extensión de sintaxis de JavaScript. Anteriormente, los archivos JavaScript que usaban JSX debían usar³ La extensión .jsx en lugar de .js. Sin embargo, actualmente, varias herramientas de compilación (léase: compiladores/agrupadores) pueden configurarse para reconocer JSX en un archivo .js³ . Si se configuran de esta manera, transpilarán JSX a JavaScript. Sin embargo, herramientas como Vite aceptan la extensión .jsx porque la hace más explícita para los desarrolladores.

Patio de juegos de código

```
constante título = 'Reaccionar';

// JSX ... const
myElement = <h1>Hola {título}</h1>;

// ... se transpila a JavaScript const myElement =
React.createElement('h1', null, `Hello ${title}`);

// ... se representa como HTML por React <h1>Hola
React</h1>
```

³ <https://bit.ly/3tT0DDD>

³ <https://www.robinwieruch.de/minimal-react-webpack-babel-setup/>

JSX permite a los desarrolladores expresar lo que debe representarse mezclando HTML con JavaScript.

Mientras que antes se pensaba desacoplar el marcado (léase: HTML) de la lógica (léase: JavaScript), React lo integra todo en un componente. Como se puede ver en el último fragmento de código, React no requiere el uso de JSX; en su lugar, es posible usar métodos como `createElement()`. Sin embargo, a la mayoría de las personas les resulta más intuitivo usar JSX por su naturaleza declarativa en lugar de usar métodos JavaScript (en este caso, métodos ofrecidos por React), que solo permiten expresar la interfaz de usuario de forma imperativa.

JSX, inicialmente inventado para React, también ganó popularidad en otras bibliotecas y marcos modernos.

Hoy en día, no está estrictamente vinculado a React, pero la gente suele asociarlo con él. En fin, [JSX es una de mis cosas favoritas cuando me preguntan sobre React³](#). Sin necesidad de sintaxis de plantilla adicional (excepto las llaves), podemos usar JavaScript en HTML. [Todas las estructuras de datos de JavaScript, desde las más primitivas hasta las más complejas, pueden usarse en HTML con la ayuda de JSX.](#)

Ceremonias:

- Compare su código fuente con el [código fuente del autor³](#).
 - Recapitular todos los [cambios del código fuente³](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí³](#). • Principiante: Lea más sobre [las variables de JavaScript](#).
 - Principiante: Defina tipos de datos de JavaScript más primitivos y complejos y representelos en JSX.

Avanzado : Intenta renderizar un array de JavaScript en JSX usando el método `map()` integrado del array para devolver JSX para cada elemento de la lista. Si es demasiado complicado, no te preocupes, aprenderás más sobre esto en la siguiente sección.

Preguntas de la entrevista:

- Pregunta: ¿Qué es JSX en React?
 - Respuesta: JSX es una extensión de sintaxis para JavaScript recomendada por React para describir cómo debería verse la interfaz de usuario.
- Pregunta: ¿Pueden los navegadores renderizar JSX directamente?
 - Respuesta : No, los navegadores no pueden entender JSX. Es necesario compilarlo a JavaScript normal con herramientas como Babel.
- Pregunta: ¿Es obligatorio usar JSX en React?
 - Respuesta: No, JSX no es obligatorio, pero es una forma ampliamente utilizada y conveniente de escribir componentes de React.
- Pregunta: ¿Cómo se representa una variable en JSX?
 - Respuesta: Utilice llaves `{}` para incrustar variables en JSX, como `{myVariable}`.

³ <https://bit.ly/3aZbdM0>

³ <https://tinyurl.com/apb9nnbj>

³ <https://tinyurl.com/v8zczahje>

³ <https://bit.ly/3SukC3A>
<https://www.robinwieruch.de/variable-javascript/>

Listas en React

Al trabajar con datos en JavaScript, estos suelen presentarse como un array de objetos. Por lo tanto, a continuación aprenderemos a renderizar una lista de elementos en React. Para prepararte para renderizar listas en React, repasemos uno de los métodos de manipulación de datos más comunes: el método `map()` integrado en el array ¹. Se utiliza para iterar sobre cada elemento de una lista con el fin de devolver una nueva versión de cada elemento:

Patio de juegos de código

```
números constantes = [1, 2, 3, 4];
```

```
const exponentialNumbers = números.map(función (número) {  
  devolver número * número; });
```

```
console.log(NumerosExponenciales); // [1,  
4, 9, 16]
```

En React, el método `map()` integrado del array se usa para transformar una lista de elementos en JSX, devolviendo JSX para cada elemento. A continuación, queremos mostrar una lista de elementos (en este caso, objetos JavaScript) en React. Primero, definiremos el array fuera del componente. Después, intente renderizar cada objeto con su propiedad `title` en React, insertando el método `map()` del array en JSX:

src/App.jsx

```
lista constante = [ {
```

```
  título: 'React', URL:
```

```
  'https://react.dev/', autor: 'Jordan
```

```
Walke', número de comentarios:
```

```
  3,
```

```
  puntos: 4,
```

```
  objectID: 0, }, {
```

```
  título: 'Redux', URL:
```

```
  'https://redux.js.org/', autor: 'Dan
```

```
Abramov, Andrew Clark', num_comentarios: 2,
```

```
  puntos: 5,
```

```
  ID de objeto:
```

```
  1, }, ];
```

```
  'https://mzl.la/3B3a7tf
```

```
función App() { ... }
```

```
// tenga en cuenta el ... como marcador de  
posición // que se utiliza para el código fuente que no cambió // y no es  
relevante para este fragmento de código
```

```
exportar aplicación predeterminada;
```

Cada elemento de la lista tiene un título, una URL, un autor, un identificador (objectId), puntos (que indican la popularidad del elemento) y un recuento de comentarios (num_comments). Los nombres de las propiedades se eligen así porque se asemejan a los datos reales que usaremos más adelante. No se ajustan a las [convenciones de nomenclatura](#) deseadas ² Sin embargo, para JavaScript, uno de ellos utiliza un guión bajo, por ejemplo.

A continuación, renderizaremos la lista en línea en JSX con el método `map()` integrado del array . Por lo tanto, no mapearemos de un tipo de datos de JavaScript a otro, sino que devolveremos JSX que renderiza cada elemento de la lista:

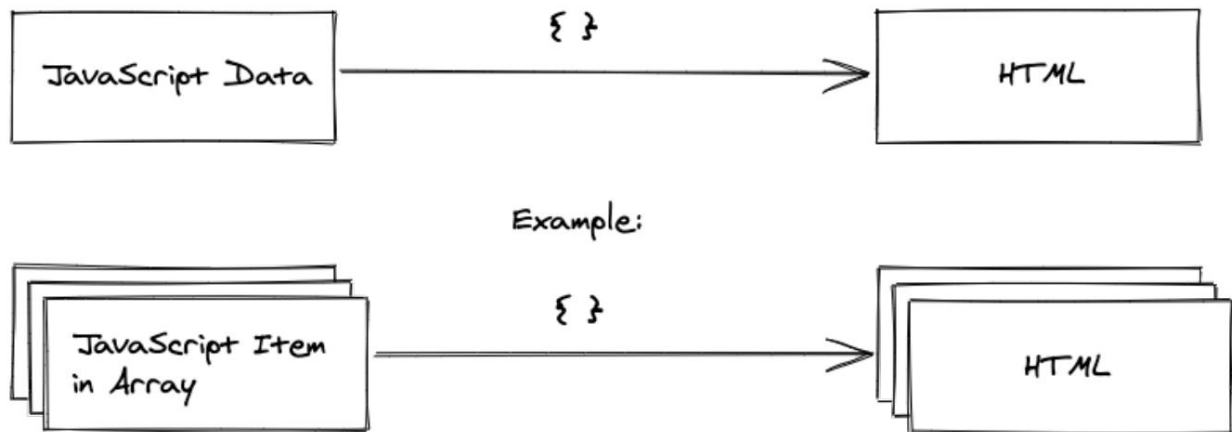
```
src/App.jsx
```

```
función App() { return (  
  
  <div>  
    Mis historias de hackers  
  
    <label htmlFor="search">Buscar: </label> <input  
    id="search" type="text" />  
  
    <hr />  
  
    <ul>  
      {lista.map(función (elemento) {  
        devolver <li>{item.title}</li>; }} </ul> </div>  
  
  );  
}
```

De hecho, renderizar una lista de elementos en React fue uno de mis momentos de revelación personal con JSX. Sin necesidad de una sintaxis de plantillas predefinida, es posible usar JavaScript para mapear desde un array de JavaScript.

²<https://www.robinwieruch.de/convenciones-de-nomenclatura-de-javascript/>

objetos a una lista de elementos HTML. Eso es, en definitiva, lo que JSX ofrece al desarrollador: simplemente JS mezclado con HTML.



Por fin, React muestra cada elemento. Pero falta un elemento importante. Si revisas las herramientas de desarrollo de tu navegador, deberías ver una advertencia en la pestaña "Consola" que indica que **cada elemento de React de una lista debe tener una clave asignada. La clave es un atributo HTML y debe ser un identificador estable.** Afortunadamente, nuestros elementos cuentan con un identificador estable, ya que tienen un id (en este caso: objectId).

src/App.jsx

```
función App() { return (

  <div>
    ...

    <ul>
      {lista.map(función (elemento) { return
        <li key={elemento.objectID}>{elemento.título}</li>; })} </ul> </div>

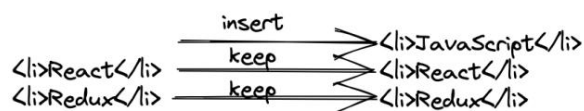
    );
  }
```

El atributo clave se utiliza por una razón específica: cada vez que React necesita volver a renderizar una lista, comprueba si un elemento ha cambiado. Al usar claves, React puede intercambiar eficientemente los elementos modificados. Al no usar claves, React puede actualizar la lista de forma ineficiente. Tomemos el siguiente ejemplo, donde se añade un nuevo elemento al principio de la lista.

without keys



with keys



La clave no es difícil de encontrar, ya que, normalmente, al tener datos en formato array, podemos usar el identificador estable de cada elemento (por ejemplo, la propiedad `id`). Sin embargo, a veces no se tiene un `id`, por lo que es necesario crear otro identificador (por ejemplo, el título, si no cambia y es único en el array). Como último recurso, también se puede usar el índice del elemento en la lista:

Patio de juegos de código


```

{list.map(function (item, index) { return ( <li
  key={index}
  > { /* usa un índice solo
    como último recurso */ /* y por cierto: así es como se
    hacen los comentarios en JSX */

    {item.title} </
  li> ); }}}
</
ul>
  
```

Generalmente, se debe evitar el uso de un índice, ya que conlleva los mismos problemas de rendimiento de renderizado mencionados anteriormente. Además, puede causar errores en la interfaz de usuario ³ al cambiar el orden de los elementos (por ejemplo, al reordenarlos, añadirlos o eliminarlos). Sin embargo, como último recurso, si la lista no cambia su orden, usar el índice es suficiente.

Hasta ahora, solo mostramos el título de cada elemento. Continúe y represente también la URL, el autor, el número de comentarios y los puntos del elemento. En el caso especial de la URL, utilice un elemento de anclaje HTML (etiqueta `<a>`) que rodee el título. ¡Pruébelo usted mismo! Como guía, la siguiente solución le mostrará cómo el libro implementa esto para estar preparado para las siguientes secciones:

³<https://www.robinwieruch.de/react-list-key/>

src/App.jsx

```

función App() { return (

  <div>
    ...

    <ul>
      {lista.map(función (elemento) {
        devolver
        ( <li clave={item.objectID}>
          <span>
            <a href={item.url}>{item.title}</a> <span>{item.author}

            <span>{item.num_comments}
            <span>{item.points} </li>

          ); }}}
        </ul>
      </div>
    );
  }
}

```

El método `map()` del array se integra concisamente en JSX para renderizar una lista. Dentro del método `map()`, tenemos acceso a cada objeto y sus propiedades. La propiedad `url` de cada elemento se utiliza como atributo `href` para el elemento HTML de anclaje. JavaScript en JSX no solo permite mostrar elementos, sino también asignar atributos HTML dinámicamente. Esta sección solo muestra superficialmente la eficacia de combinar JavaScript y HTML; sin embargo, el uso del método `map()` de un array y la asignación de atributos HTML deberían darte una buena primera impresión.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) .
- Repasar los [métodos de matriz estándar integrados](#), especialmente `map`, `filter` y `reduce`, disponibles en JavaScript.
- Ampliar la lista con más elementos para que el ejemplo sea más realista.
- Practicar el uso de diferentes expresiones de JavaScript en JSX.

<https://tinyurl.com/2rt98u3u>
<https://tinyurl.com/xj9sakd6>
<https://bit.ly/484qh6p>
<https://mzl.la/3b9V9rf>

Preguntas de la entrevista:

- Pregunta: ¿Cómo representar una lista de elementos en JSX?

– Respuesta: Use `map()` para iterar sobre la matriz y devolver elementos JSX para cada elemento. • Pregunta:

¿Qué sucede si devuelve `null` en lugar de JSX?

Respuesta : Se permite devolver un valor nulo en JSX. Siempre se usa si no se desea renderizar nada.

- Pregunta: ¿A qué se refiere el término “expresiones JSX”?

– Respuesta: Las expresiones JSX son expresiones de JavaScript incrustadas entre llaves en JSX, lo que permite contenido dinámico. • Pregunta:

¿Se puede incrustar HTML directamente en JSX?

– Respuesta: Sí, puedes incrustar HTML directamente en JSX, pero generalmente no se recomienda.

Debido a riesgos de seguridad, use `dangerouslySetInnerHTML` con precaución. •

Pregunta: ¿Cómo se comenta en JSX?

– Respuesta: Utilice llaves para los comentarios de JavaScript, como `{/* Su comentario aquí */}`.

Conozca otro componente de React

Los componentes son la base de toda aplicación React. A medida que un proyecto React crece, tendrás que gestionar cada vez más componentes. Cada componente encapsula funcionalidades (por ejemplo, renderizar una lista de elementos). Hasta ahora solo hemos usado el componente App. Esto no será un buen resultado, ya que los componentes deben escalar con el tamaño de la aplicación. Por lo tanto, en lugar de hacer que un componente sea más grande y complejo con el tiempo, lo dividiremos en varios componentes. Por lo tanto, comenzaremos con un nuevo componente List que extrae funcionalidades del componente App:

```
src/App.jsx
```

```
lista constante = [ ... ];
```

```
función App() { ... }
```

```
función Lista() { return (
```

```
  <ul>
```

```
    {lista.map(función (elemento) { return
```

```
      ( <li
```

```
        key={elemento.objectID}> <span>
```

```
          <a href={item.url}>{item.title}</a> <span>{item.author}
```

```
          <span>{item.num_comments}
```

```
          <span>{item.points} </li>
```

```
        ); }}})
```

```
  </ul> );
```

```
}
```

Luego, el nuevo componente List se puede usar en el componente Aplicación donde hemos estado usando los elementos HTML en línea anteriormente:

src/App.jsx

```
función App() { return
  (
    <div>
      Mis historias de hackers

      <label htmlFor="search">Buscar: </label>
      <input id="búsqueda" tipo="texto" />

      <hr />

      <Lista />
    </div>
  );
}
```

Acabas de crear tu primer componente de React. Con este ejemplo, podemos ver cómo los componentes encapsulan tareas importantes y contribuyen al bienestar general de un proyecto de React más grande. Extraer un componente es una tarea que realizarás con frecuencia como desarrollador de React, ya que un componente siempre crece en tamaño y complejidad. Continúa y extrae los elementos de etiqueta e entrada del componente de aplicación en su propio componente de búsqueda.

El siguiente fragmento de código muestra cómo el libro resolvería esta tarea:

src/App.jsx

```
función App() { return
  (
    <div>
      Mis historias de hackers

      <Buscar />

      <hr />

      <Lista />
    </div>
  );
}

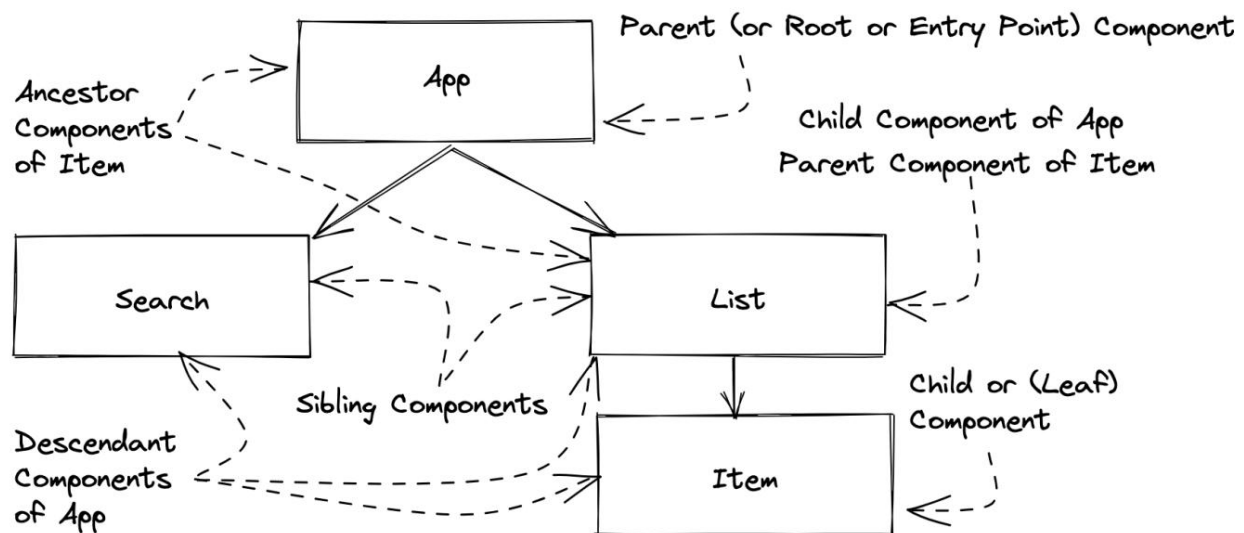
función Buscar() { return (
  <div>
    <label htmlFor="search">Buscar: </label>
```

```

    <input id="búsqueda" tipo="texto" />
  </div>
);
}

```

Finalmente, nuestra aplicación tiene tres componentes: App, List y Search. En general, una aplicación React consta de muchos componentes jerárquicos, que podemos clasificar en las siguientes categorías. La siguiente ilustración asume que también hemos separado el componente Item del componente List, lo que nos ayuda a aclarar la taxonomía.



Las aplicaciones React tienen jerarquías de componentes (también llamadas árboles de componentes). Generalmente, hay un componente de punto de entrada superior (p. ej., App) que abarca un árbol de componentes inferior. App es el componente padre de List y Search, por lo que List y Search son componentes secundarios de App y componentes hermanos entre sí. La ilustración lo lleva un paso más allá, donde el componente Item es un componente secundario de List. En un árbol de componentes, siempre hay un componente raíz (p. ej., App), y los componentes que no representan otros componentes se denominan componentes hoja (p. ej., el componente List/Search en nuestro código fuente actual o el componente Item/Search de la ilustración). Todos los componentes pueden tener cero, uno o varios componentes secundarios.

Puedes ver cómo una aplicación React crece al crear más componentes conectados en una jerarquía. Normalmente, comenzarás con el componente App, desde donde expandirás tu árbol de componentes. Puedes saber de antemano qué componentes quieres crear o empezar con un componente y extraer componentes de él con el tiempo. Para un principiante, puede ser difícil saber cuándo crear un nuevo componente o cuándo extraer un componente de otro.

Suele ocurrir de forma natural cuando un componente aumenta demasiado de tamaño o complejidad, o cuando se detectan límites naturales en los dominios o la funcionalidad (por ejemplo, el componente Lista muestra una lista de elementos, el componente Búsqueda muestra un formulario de búsqueda). En definitiva, cada componente representa una unidad en la aplicación, lo que la hace mantenible y predecible.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) . • Todavía no podemos extraer un componente Elemento del componente Lista (como en la ilustración) porque no sabemos cómo pasar elementos individuales de la lista a cada componente Elemento.
Piensa en una manera de hacerlo.

Preguntas de la entrevista:

- Pregunta: ¿Por qué es beneficioso extraer componentes en React?
 - Respuesta: La extracción de componentes promueve la reutilización, la capacidad de mantenimiento y una estructura de componentes más limpia.
 - Pregunta: ¿Cómo decide cuándo extraer un componente?
 - Respuesta: Extraiga un componente cuando encuentre patrones de IU o funcionalidades repetidas dentro de tu código.
 - Pregunta: ¿Cómo se llama el proceso de extracción de un componente en React?
 - Respuesta: Se llama refactorización, específicamente extraer un componente para mejorar el código. organización. •
- Pregunta: ¿Es posible extraer componentes en diferentes archivos?
- Respuesta: Sí, extraer componentes en archivos separados promueve una mejor organización de los archivos. y modularidad.

<https://tinyurl.com/ytecbbxx>
<https://tinyurl.com/59ctb524>
<https://bit.ly/3SuMogq>

Instanciación de componentes de React

Has aprendido a declarar un componente (p. ej., la función `List() { ... }`) y a instanciarlo (p. ej., `<List />`). En esta sección, profundizaremos en este aprendizaje mediante una analogía y la terminología. Comenzaremos con la analogía usando la clase de JavaScript. **Técnicamente, las clases de JavaScript y los componentes de React no están relacionados, lo cual es importante tener en cuenta, pero aun así es una analogía adecuada para que comprendas el concepto de componente usando algo que quizás hayas usado en el pasado.**

Una clase se usa con mayor frecuencia en lenguajes de programación orientados a objetos. JavaScript, como lenguaje de programación multiparadigma, permite la coexistencia de la programación funcional y la programación orientada a objetos. Para resumir las clases de JavaScript para la programación orientada a objetos, considere la siguiente clase `Persona`:

Patio de juegos de código

```
class Persona
{
  constructor(nombre, apellido) { this.nombre =
    nombre; this.apellido = apellido;

  }

  obtenerNombre() {
    devuelve este.nombre + ' ' + este.apellido;
  }
}
```

Cada clase tiene un constructor que toma argumentos y los asigna a la instancia de la clase al instanciarla. Una clase también puede definir funciones asociadas a la instancia (p. ej., `getName()`), que se denominan métodos o métodos de clase. Declarar la clase `Person` una vez es solo una parte; instanciarla es la otra. La declaración de la clase es el modelo de sus capacidades y su uso se produce cuando se crea una instancia con la declaración `new`. Si existe una declaración de clase en JavaScript, se pueden crear varias instancias de ella:

Patio de juegos de código

```
// declaración de clase class
Persona { ... }

// instanciación de clase const
robin = new Persona('Robin', 'Wieruch');

console.log(robin.getName()); // "Robin
Wieruch"
```

```
// otra instancia de clase
const dennis = nueva Persona('Dennis', 'Wieruch');

console.log(dennis.getName()); // "Dennis
Wieruch"
```

El concepto de una clase JavaScript con declaración e instancia es similar al de un componente React, que también tiene solo una declaración de componente, pero puede tener múltiples instancias de componente:

src/App.jsx

```
// declaración de la función del componente
App App() { return (

  <div>
    ...

    {/* creando una instancia del componente Lista */}
    <List /> {/*
      creando otra instancia del componente List */}
    <Lista />
  </div> );

}
```

```
// declaración de la función del componente
Lista List() { ... }
```

Una vez definido un componente, podemos usarlo como elemento en cualquier parte de nuestro JSX. El elemento genera una instancia de tu componente; en otras palabras, se instancia. Puedes crear tantas instancias de un componente como quieras, siempre que tengas una declaración de componente. No es muy diferente de la declaración e instancia de una clase de JavaScript; sin embargo, como se mencionó anteriormente, técnicamente una clase de JavaScript y un componente de React no son lo mismo. Su simple uso facilita la demostración de sus similitudes.

Ceremonias:

- Lea más sobre [componentes, elementos e instancias en React](https://www.robinwieruch.de/react-element-component/) ¹.
 - Familiarícese con los términos declaración de componente, instancia de componente y elemento.

¹<https://www.robinwieruch.de/react-element-component/>

- Lea más sobre [los tipos de componentes React](https://www.robinwieruch.de/react-component-types/) ².

Experimente creando múltiples instancias de un componente Lista. Si seguimos tratando la variable Lista como una variable global, cada componente Lista usaría la misma lista. Piense en cómo sería posible asignar a cada componente Lista su propia variable Lista .

²<https://www.robinwieruch.de/react-component-types/>

React DOM

Hemos aprendido sobre la declaración/instanciación de componentes y ya la hemos visto en acción para los componentes Lista y Búsqueda. Sin embargo, al principio empezamos con la declaración del componente Aplicación, pero nunca vimos su instanciación. Debe estar presente; de lo contrario, el componente Aplicación y todos sus componentes descendientes en la jerarquía de componentes no se renderizarían.

Abra el archivo `src/main.jsx` para ver la instanciación de componentes de la aplicación con el elemento `<App />`. El archivo puede ser ligeramente diferente al suyo; sin embargo, el siguiente fragmento muestra todos sus aspectos esenciales:

`src/main.jsx`

```
import { StrictMode } from 'react'; import
{ createRoot } from 'react-dom/client'; import './index.css';
import App from './
App.jsx';

createRoot(document.getElementById('root')).render(
  <Modo estricto>
    <Aplicación />
  </Modo estricto>
);
```

Hay dos bibliotecas importadas al principio del archivo: `react` y `react-dom`. Si bien `React` se usa para las tareas diarias de un desarrollador de `React`, `React DOM` suele usarse una vez en una aplicación `React` para integrar `React` en el entorno HTML nativo. Abra el archivo `index.html` en el lateral y localice el elemento HTML donde el atributo `id` es `"root"`. Ese es exactamente el elemento donde `React` se inserta en el HTML para arrancar toda la aplicación `React`, comenzando por el componente `App`.

En el archivo JavaScript, el método `createRoot()` espera el elemento HTML utilizado para instanciar `React`. En este caso, usamos el método `getElementById()` integrado en JavaScript para devolver el elemento HTML que vimos en el archivo `index.html`. Una vez que tenemos el objeto raíz, podemos ejecutar `render()` en el objeto raíz devuelto con JSX como parámetro, que generalmente representa el componente de punto de entrada (también llamado componente raíz). Normalmente, el componente de punto de entrada es la instancia del componente `App`, pero también puede ser cualquier otro JSX:

Patio de juegos de código

```
importar { createRoot } desde 'react-dom/client';

constante título = 'Reaccionar';

createRoot(document.getElementById('root')).render( <h1>Hola {título}
  </h1> );
```

En esencia, React DOM es todo lo necesario para integrar React en cualquier sitio web que use HTML. Si inicias una aplicación React desde cero, normalmente solo hay una llamada a ReactDOM.createRoot() en ella. Sin embargo, si trabajas en una aplicación heredada que usaba algo distinto a React, es posible que veas varias llamadas a ReactDOM.createRoot() , ya que solo ciertas áreas de la aplicación funcionan con React.

¿Recuerdas la introducción sobre el auge de las aplicaciones de una sola página que funcionan con un pequeño archivo HTML y un gran archivo JavaScript? Ahora puedes ver cómo todo encaja. Mientras que un pequeño archivo HTML (aquí: index.html) y un gran archivo JavaScript (aquí: archivos src/main.jsx y src/App.jsx compilados y empaquetados) se transfieren del servidor web al navegador, los archivos JavaScript son los principales responsables de renderizar todo el HTML en el navegador. El archivo HTML solo sirve para solicitar el archivo JavaScript y renderizar el elemento HTML donde React se inserta.

Desde allí, React llama a todos los componentes de función necesarios para representarse como una jerarquía de componentes.

Ceremonias:

- Lea más sobre `createRoot` ³ de React.
- Lea más sobre `StrictMode` de React.

³<https://bit.ly/3vx3uT2>
<https://bit.ly/48TUA0k>

Declaración de componentes de React

Hasta ahora hemos declarado varios componentes de React. Dado que estos componentes se denominan componentes de función, podemos aprovechar las diferentes maneras de declarar funciones en JavaScript. Hasta ahora, hemos utilizado la declaración de función estándar, aunque las funciones de flecha pueden usarse de forma más concisa y, por lo tanto, pueden establecer un nuevo estándar para la declaración de componentes de función:

Patio de juegos de código

```
// declaración de función function
```

```
App() { ... }
```

```
// expresión de función de flecha const
```

```
App = () => { ... }
```

Con este conocimiento, revise su proyecto de React y refactorice todas las declaraciones de función a expresiones de función de flecha. Si bien esta refactorización se puede aplicar a componentes de función, también se puede usar para cualquier otra función del proyecto. Continuaremos refactorizando todas las declaraciones de función del componente de función a expresiones de función de flecha:

src/App.jsx

```
constante App = () =>
```

```
  { return ( ... ); };
```

```
constante Buscar = () => { return
```

```
  ( ... ); };
```

```
constante Lista = () =>
```

```
  { return ( ... ); };
```

Como se dijo, no solo se pueden refactorizar los componentes de función, sino también otras funciones como la [función de devolución de llamada](#) que hemos utilizado para el método `map()` de la matriz :

src/App.jsx

```
constante Lista = () =>
  { return (
    <ul>
      {lista.map((elemento) =>
        { return ( <li
          key={elemento.objectID}>
            ...
          </li>

        ); }}} </ul>

  ); };
```

Además, si el único propósito de una función de flecha es devolver un valor y no tiene lógica de negocio intermedia, se puede eliminar el cuerpo del bloque (llaves) de la función. En un cuerpo conciso, se adjunta una declaración de retorno implícita, por lo que se puede eliminar. Vea la siguiente demostración:

Patio de juegos de código

// con cuerpo de bloque

```
const addOne = (count) => {
  // realizar cualquier tarea intermedia

  devolver cuenta + 1; };
```

// con cuerpo conciso como multi línea const

```
addOne = (count) =>
  contar + 1;
```

// con cuerpo conciso como una línea const

```
addOne = (count) => count + 1;
```

Veamos esto en acción para el componente Búsqueda:

src/App.jsx

```
const Buscar = () => (  
  <div>  
    <label htmlFor="search">Buscar: </label>  
    <input id="búsqueda" tipo="texto" />  
  </div>  
);
```

Esto también se puede hacer para los componentes App y List, ya que solo devuelven JSX y no realizan ninguna tarea intermedia. Además, también se aplica a la función de flecha que se usa en el método map() :

src/App.jsx

```
constante Aplicación = () => (  
  <div>  
    ...  
  </div>  
);  
  
constante Lista = () => (  
  <ul>  
    {lista.mapa((elemento) =>  
      ( <li clave={elemento.IDdeobjeto}>  
        ...  
      </li> )}}  
  </ul> );
```

Todo JSX ahora es más conciso, ya que omite la declaración de función, las llaves y la declaración de retorno. Sin embargo, es importante recordar que este paso es opcional y que se aceptan declaraciones de función sobre expresiones de función de flecha, así como cuerpos de bloque con llaves sobre cuerpos concisos con retornos implícitos para funciones de flecha.

A menudo, los cuerpos de bloque serán necesarios para introducir más lógica de negocio entre la firma de la función y la declaración de retorno. Asegúrese de comprender este concepto de refactorización, ya que pasaremos rápidamente de componentes de función de flecha con y sin cuerpos de bloque a medida que avancemos. El que usemos dependerá de los requisitos del componente:

Patio de juegos de código

```
const App = () => { //
  realizar una tarea en el medio

  devolver (
    <div>
      ...
    </div>

  );
};
```

Como regla general, utilice declaraciones de función o expresiones de función de flecha para las declaraciones de componentes en toda la aplicación. Ambas versiones son válidas, pero asegúrese de que usted y su equipo compartan el mismo estilo de implementación. Además, si bien una declaración de retorno implícita al usar expresiones de función de flecha hace que la declaración de componente sea más concisa, podría introducir refactorizaciones tediosas de `concise` a `block body` al realizar tareas entre la firma de la función y la declaración de retorno. Por lo tanto, es recomendable mantener la expresión de función de flecha con un `block body` (como en el último fragmento de código) en todo momento.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) . •

Familiarícese con las funciones de flecha con cuerpo de bloque y retorno explícito y cuerpo conciso sin retorno (retorno implícito).

- Opcional: Lea más sobre [las funciones de flecha de JavaScript](#) .

Preguntas de la entrevista:

- Pregunta: ¿Cómo se declara un componente de función utilizando una declaración de función?
 - Respuesta: Utilice la palabra clave `function`, como `function MyComponent() {...}`. •

Pregunta: ¿Cómo se declara un componente de función utilizando una expresión de función de flecha?

- Respuesta: Utilice la sintaxis de la función de flecha, como `const MyComponent = () => {...}`.

<https://tinyurl.com/3pwajd4u>
<https://tinyurl.com/yve6n2df>
<https://bit.ly/3SJbE42>
<https://mzl.la/3BYCOcp>

Función de controlador en JSX

Hemos aprendido mucho sobre los componentes de React, pero aún no hay interacciones. Si desarrollas una aplicación con React, llegará el momento en que tendrás que implementar una interacción con el usuario. El mejor punto de partida en nuestro proyecto es el componente de búsqueda, que ya incluye un elemento de campo de entrada.

En HTML nativo, podemos agregar controladores de eventos en elementos mediante el método `addEventListener()` programáticamente en un nodo DOM. En React, descubriremos cómo añadir controladores en JSX de forma declarativa. Primero, refactoricemos la función del componente Search de un cuerpo conciso a un cuerpo de bloque, para poder añadir detalles de implementación antes de la declaración de retorno:

src/App.jsx

```
const Search = () => { // realizar
  una tarea en el medio

  devolver (
    <div>
      <label htmlFor="search">Buscar: </label> <input
        id="search" type="text" />
      </div>

    );
  };
```

A continuación, defina una función, que puede ser una declaración de función o una expresión de función de flecha, para el evento de cambio del campo de entrada. En React, esta función se denomina controlador de eventos. Posteriormente, la función se puede pasar al atributo `onChange` (atributo con nombre JSX) del campo de entrada HTML:

src/App.jsx

```
const Buscar = () => { const
  manejarCambio = (evento) => {
    // evento sintético
    console.log(event); // valor
    del objetivo (aquí: elemento HTML de entrada)
    console.log(event.target.value); };

  devolver (
    <div>
      <label htmlFor="search">Buscar: </label> <input
        id="search" type="text" onChange={handleChange} />
    );
  };
```

<https://mzl.la/2ZbTcYZ>

```

    </div>

  );
};

```

Después de abrir la aplicación en un navegador web, abra la pestaña "Consola" de las herramientas de desarrollo para ver el registro después de escribir en el campo de entrada. Lo que ve se denomina evento sintético, que consiste en un objeto JavaScript y el valor interno del campo de entrada.

El evento sintético de React es esencialmente un envoltorio alrededor del evento nativo del navegador ¹. Dado que React comenzó como una biblioteca para aplicaciones de una sola página, existía la necesidad de funcionalidades mejoradas en el evento para evitar el comportamiento nativo del navegador ². Por ejemplo, en HTML nativo, enviar un formulario activa la actualización de la página. Sin embargo, en React, esta actualización debería evitarse, ya que el desarrollador debe encargarse de lo que sucede a continuación. De todas formas, si necesitas acceder al evento HTML nativo, puedes hacerlo usando `event.nativeEvent`; sin embargo, tras varios años desarrollando en React, nunca me he encontrado con este caso.

Al fin y al cabo, así es como pasamos elementos HTML en las funciones de los controladores JSX para añadir escuchas para las interacciones del usuario. Siempre pase funciones a estos controladores, no el valor de retorno de la función, excepto cuando este sea una función. Conocer esto es crucial, ya que es una fuente conocida de errores en las aplicaciones de React para principiantes.

Patio de juegos de código

```

// si handleChange es una función // que no
// devuelve una función
// no hagas esto <input
// onChange={handleChange()} />

// haz esto en su lugar
<input onChange={handleChange} />

```

Como puede ver, HTML y JavaScript funcionan bien juntos en JSX. JavaScript en HTML puede mostrar variables de JavaScript (por ejemplo, la cadena de título en `<span>{title}</span>`), pasar primitivas de JavaScript a atributos HTML (por ejemplo, la cadena de URL al elemento HTML `<a href={url}>`) y pasar funciones a los atributos de un elemento HTML para gestionar las interacciones del usuario (por ejemplo, la función `handleChange` a `<input onChange={handleChange} />`). Al desarrollar aplicaciones React, combinar HTML y JavaScript en JSX se convertirá en su principal recurso.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) ³.

¹<https://mzl.la/30Dk8kt>

²<https://www.robinwieruch.de/react-preventdefault/>

³<https://tinyurl.com/2uamttzd>

- Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#).
 - Lea más sobre [el controlador de eventos de React](#).
 - Obtenga más información sobre [la captura y propagación de eventos en React](#).
- Además del atributo `onChange`, agregue un atributo `onBlur` con un controlador de eventos a su campo de entrada y verificar su registro en las herramientas para desarrolladores del navegador.

Preguntas de la entrevista:

- Pregunta: ¿Cómo se define un controlador de eventos en React?
 - Respuesta: Cree una función que maneje el evento, como la función `handleClick() {...}`.
- Pregunta: ¿Cómo se adjunta un controlador de eventos en JSX?
 - Respuesta: use el atributo apropiado, como `onClick={handleClick}`.
- Pregunta: ¿Cuál es el patrón común para nombrar funciones de controlador de eventos?
 - Respuesta: Anteponga el nombre de la función con "handle" seguido del nombre del evento, como `handleClick` para un evento de clic.
- Pregunta: ¿Puedes usar funciones de flecha directamente en JSX para controladores de eventos?
 - Respuesta: Sí, usar funciones de flecha directamente en JSX es un patrón común para eventos concisos.
- Pregunta: ¿Cómo se pasan argumentos a un controlador de eventos en JSX?
 - Respuesta: Use una función de flecha para llamar al controlador con argumentos, como `onClick={() => handleClick(arg)}`.
- Pregunta: ¿Puede reutilizar controladores de eventos en varios elementos?
 - Respuesta: Sí, los controladores de eventos se pueden reutilizar para múltiples elementos con el mismo tipo de evento.
- Pregunta: ¿Cuál es el propósito de la propiedad `e.target` en un controlador de eventos?
 - Respuesta: Se refiere al elemento DOM que activó el evento, lo que le permite acceder a sus propiedades o manipularlo.
- Pregunta: ¿Cómo se accede al objeto de evento en un controlador de eventos?
 - Respuesta: Incluya (evento) como parámetro en la función del controlador, como la función `handleClick(evento) {...}`.
- Pregunta: ¿Qué hace `event.preventDefault()` en un controlador de eventos?
 - Respuesta: Evita el comportamiento predeterminado del evento, como enviar un formulario o siguiendo un enlace.
- Pregunta: ¿Cuál es el propósito del método `e.stopPropagation()` en un controlador de eventos?
 - Respuesta: Detiene la propagación del evento hacia arriba o hacia abajo en el árbol DOM, impidiendo que el padre... o elementos secundarios que manejan el mismo evento.

Accesorios de React

Actualmente, usamos la lista como variable global en nuestro proyecto. Al principio, la usábamos directamente desde el ámbito global en el componente App y posteriormente en el componente List. Esto podría funcionar si solo se tuviera una variable global y un solo archivo con todos los componentes, pero no es sostenible con múltiples variables en varios componentes (dentro de varias carpetas/archivos). **Al usar las llamadas propiedades en React, podemos pasar variables como información de un componente a otro, incluso si estos componentes no están ubicados en el mismo archivo en algún momento.** Exploremos cómo funciona esto.

Antes de usar las propiedades por primera vez, trasladaremos la lista del ámbito global al componente App y le asignaremos un nombre más autodescriptivo. No olvides refactorizar la función del componente App de concisa a cuerpo de bloque para declarar la lista antes de la declaración de retorno:

src/App.jsx

```
const App = () => { const
  historias = [
    {
      título: 'React', URL:
      'https://react.dev/', autor: 'Jordan
      Walke', número de comentarios:
      3, puntos: 4, objectID:
      0, }, {

      título: 'Redux', URL:
      'https://redux.js.org/', autor: 'Dan
      Abramov, Andrew Clark', número de comentarios:
      2,
      puntos: 5,
      objectID: 1, }, ];

  devolver ( ... ); };
```

A continuación, usaremos las propiedades de React para pasar la lista de elementos al componente Lista. La variable se llama "stories" en el componente Aplicación y la pasamos con este nombre al componente Lista. Sin embargo, al instanciar el componente Lista, se asigna a un nuevo atributo HTML de lista :

src/App.jsx

```
const App = () => { const
  historias = [ ... ];

  devolver (
    <div>
      ...

      <Lista lista={historias} />
    </div>

  ); };
```

Ahora intente recuperar la lista de la firma de función del componente Lista introduciendo un parámetro. Si encuentra la solución, ¡felicitaciones por pasar la primera información de un componente a otro! Si no, el siguiente fragmento de código muestra cómo funciona:

src/App.jsx

```
constante Lista = (props) => (
  <ul>
    {props.list.map((item) => ( <li
      key={item.objectID}>
        ...
      </li> ))}
    </
  ul>
);
```

Todo lo que pasamos de un componente padre a un componente hijo mediante el atributo HTML del elemento del componente se puede acceder en el componente hijo. El componente hijo recibe un parámetro (props) como objeto en la firma de su función, que incluye todos los atributos pasados como propiedades (abreviado: props).

Un punto a considerar con respecto a ESLint: podría aparecer un error que indique "Falta el error 'lista' en la validación de propiedades". En un escenario ideal donde se usa React sin TypeScript, una solución sería incorporar **tipos de propiedades** para proporcionar a su componente una mejor comprensión de los apoyos que recibe. Sin embargo, cabe mencionar que, si bien los tipos de propiedad cumplen una función similar a la de TypeScript, se consideran menos robustos. Si lograr este objetivo es un requisito previo para su proyecto, se recomienda adoptar TypeScript en su desarrollo de React. En los casos en que JavaScript sea la opción exclusiva, mi sugerencia es deshabilitar esta regla específica de ESLint en su archivo de configuración para resolver el problema.

```
eslint.config.js
```

```
normas: {  
  ...  
  'react/prop-types': 'desactivado',  
},
```

Otro caso de uso para las props de React es el componente List y su posible componente secundario. Anteriormente, no podíamos extraer un componente Item del componente List porque no sabíamos cómo pasar cada elemento al componente Item extraído. Con este nuevo conocimiento sobre las props de React, podemos realizar la extracción del componente y pasar cada elemento al nuevo componente secundario del componente List.

Antes de probar la siguiente solución, pruébela usted mismo: extraiga un componente Item del componente List y pase cada elemento de la función de devolución de llamada del método map() a este nuevo componente. Si después de un tiempo no encuentra una solución, consulte cómo se implementa en el libro:

```
src/App.jsx
```

```
constante Lista = (props) => (  
  <ul>  
    {props.list.map((elemento) => (  
      <Clave del artículo={item.objectID} artículo={item} /> ))} </  
    <ul> );
```

```
constante Item = (props) => (  
  <li>  
    <span>  
      <a href={props.item.url}>{props.item.title}</a>  
  
    <span>{props.item.author}  
    <span>{props.item.num_comments}  
    <span>{props.item.points}  
  </li>  
)
```

No olvides el atributo clave que presentamos en una sección anterior. Al trabajar con listas en JSX dentro de una aplicación React, es fundamental recordar su importancia.

Como se explicó anteriormente en una sección dedicada, el atributo clave desempeña un papel crucial en la representación y actualización eficiente de los elementos de una lista. Al asignar una clave única a cada elemento de la lista, React puede rastrear y gestionar los elementos con precisión.

Al final, deberías ver la lista renderizada de nuevo. El punto más importante sobre las propiedades: no se pueden modificar, ya que deben tratarse como una estructura de datos inmutable. Solo se utilizan para transmitir información a través de la jerarquía de componentes. Siguiendo con esta idea, la información (propiedades) solo se puede transmitir de un componente principal a uno secundario, y no viceversa. Más adelante aprenderemos a superar esta limitación. Por ahora, hemos encontrado la forma de compartir información de arriba a abajo en un árbol de componentes de React.

Ceremonias:

- Compara tu código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) ¹. • Lea más sobre [cómo usar accesorios en React](#) ².

Preguntas de la entrevista:

- Pregunta: ¿Qué son las propiedades en React?
 - Respuesta: Las props (abreviatura de propiedades) son un mecanismo para pasar datos de un padre componente a un componente secundario. •
- Pregunta: ¿Cómo se pasan propiedades a un componente en JSX?
 - Respuesta: Inclúyalos como atributos, como `<MyComponent prop1={value1} prop2={value2} />`.
- Pregunta: ¿Cómo se accede a las propiedades en un componente de función?
 - Respuesta: Utilice los parámetros de la función para acceder a las propiedades, como la función `MyComponent(props) {...}`.
- Pregunta: ¿Puedes modificar el valor de las propiedades dentro de un componente?
 - Respuesta : No, las propiedades son inmutables. Deben tratarse como de solo lectura.

<https://tinyurl.com/56wad6rt>

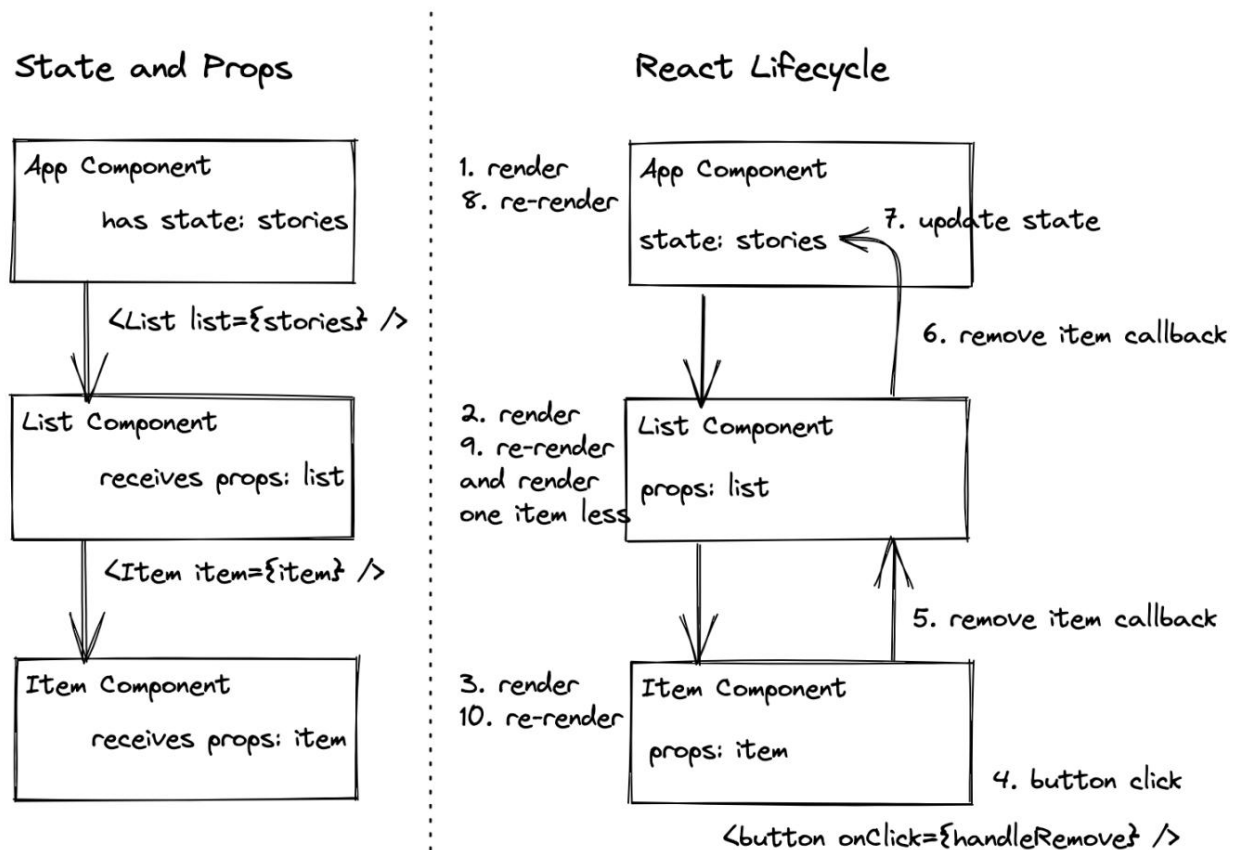
<https://tinyurl.com/yvvhx4nzz>

¹<https://bit.ly/3SzqclA>

²<https://www.robinwieruch.de/react-pass-props-to-component/>

Estado de reacción

Si bien no se permite mutar las propiedades de React como desarrollador, ya que solo sirven para transmitir información de los componentes principales a los secundarios, el estado de React introduce una estructura de datos mutable (es decir, valores con estado). Estos valores con estado se instancian en un componente de React como estado, se pueden transmitir con propiedades como vehículo a los componentes secundarios, pero también se pueden mutar mediante una función para modificar el estado. Cuando se muta un estado, el componente con el estado y todos los componentes secundarios se vuelven a renderizar.



Ambos conceptos, propiedades y estado, tienen propósitos claramente definidos: mientras que las propiedades se utilizan para transmitir información a través de la jerarquía de componentes, el estado se utiliza para modificarla con el tiempo.

Comencemos con el estado en React con el siguiente caso práctico: cada vez que un usuario escribe texto en nuestro elemento de campo de entrada HTML en el componente de búsqueda, desea ver esta información (estado) junto a él. Un enfoque intuitivo (pero no funcional) sería el siguiente:

src/App.jsx

```
const Buscar = () => {  
  deje que searchTerm = "";  
  
  const handleChange = (evento) => {  
    searchTerm = evento.objetivo.valor; };  
  
  devolver (  
    <div>  
      <label htmlFor="search">Buscar: </label> <input id="search"  
        type="text" onChange={handleChange} />  
  
      <p>Buscando <strong>{searchTerm}</strong>. </p>  
  
    </div>  
  
  );  
};
```

Al probar esto en el navegador, verá que el resultado no aparece debajo del campo de entrada HTML después de escribir. Sin embargo, este enfoque no se aleja demasiado de la solución real.

Lo que falta es indicarle a React que searchTerm es un valor con estado. Afortunadamente, React ofrece un método llamado `useState` para ello:

src/App.jsx

```
importar * como React desde 'react';  
  
...  
  
const Buscar = () => { const  
  [término de búsqueda, establecertérmino de búsqueda] = React.useState("");  
  
  const handleChange = (evento) => {  
    setSearchTerm(evento.objetivo.valor); };  
  
  ...  
  
};
```

Al usar `useState`, le indicamos a React que queremos un valor con estado que cambie con el tiempo. Y cuando este valor cambie, los componentes afectados (en este caso, el componente de búsqueda) se volverán a renderizar para usarlo (en este caso, para mostrar el valor reciente).

El método `useState` de React toma un estado inicial como argumento; en nuestro caso, es una cadena vacía.

Además, al llamar a este método se devolverá una matriz con dos entradas: la primera (`searchTerm`) representa el estado actual y la segunda (`setSearchTerm`) es una función para actualizar este estado.

El libro se referirá a esta función como función de actualización de estado. Ambas entradas contienen todo lo necesario para mostrar el estado actual (lectura) y actualizarlo (escritura).

state

state updater function

```
const [value, setValue] = React.useState("");
```

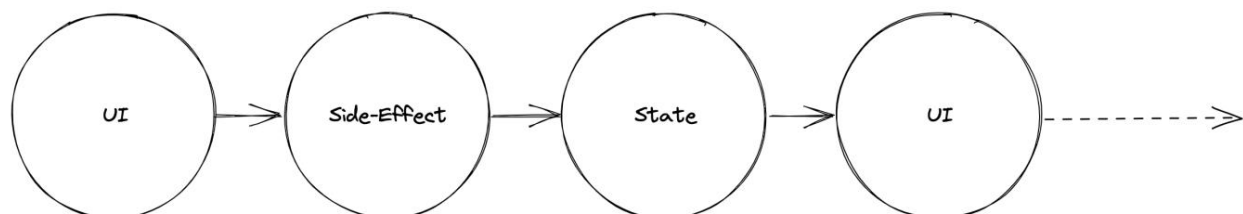
initial state

Cuando el usuario escribe en el campo de entrada, el evento de cambio del campo utiliza el controlador de eventos. La lógica del controlador utiliza el valor del evento del objetivo y la función de actualización de estado para establecer el nuevo estado. Posteriormente, el componente se vuelve a renderizar (léase: se ejecuta la función del componente). El estado actualizado se convierte en el estado actual (en este caso: `searchTerm`) y se muestra en el JSX del componente.

Como ejercicio, añade un comando `console.log()` a cada uno de tus componentes. Por ejemplo, el componente `App` recibe un comando `console.log('App renders')`, el componente `List` recibe un comando `console.log('List renders')`, y así sucesivamente. Ahora revisa tu navegador: al primer renderizado, deberían aparecer todos los registros; sin embargo, al escribir en el campo de entrada HTML, solo debería aparecer el registro del componente `Search`. React solo vuelve a renderizar este componente (y todos sus posibles descendientes) después de que su estado haya cambiado.

Ya conoces los términos **renderizado** y **re-renderizado** en un contexto técnico. En esencia, cada componente de una aplicación React tiene un renderizado inicial seguido de posibles re-renderizados.

Normalmente, el renderizado inicial ocurre cuando un componente de React se muestra en el navegador. Luego, cuando ocurre un efecto secundario, como una interacción del usuario (por ejemplo, escribir en un campo de entrada), el cambio se captura en el estado de React, lo que obliga a volver a renderizar todos los componentes afectados por este cambio; es decir, el componente que gestiona el estado y todos sus componentes descendientes.



Es importante tener en cuenta que la función `useState` se denomina **gancho** de React. Es solo uno de los varios ganchos que ofrece React, y esta sección solo ha abordado superficialmente los ganchos en React. Aprenderá más sobre ellos en las siguientes secciones. Por ahora, debe saber que puede tener tantos como desee.

useState engancha como desee en uno o varios componentes, mientras que el estado puede ser cualquier cosa, desde una cadena de JavaScript (como en este caso) hasta una estructura de datos más compleja, como una matriz o un objeto.

También es importante tener en cuenta que es posible que veas registros duplicados en las herramientas de desarrollo de tu navegador, como resultado del modo estricto de React (consulta el archivo src/main.jsx). El modo estricto, que funciona exclusivamente en el entorno de desarrollo, realiza comprobaciones y advertencias adicionales para identificar posibles problemas.

Al aplicarlo al componente raíz, resalta eficazmente cualquier problema que pueda surgir. Más adelante, en producción, solo habrá una renderización.

Al renderizar la interfaz de usuario por primera vez, el gancho useState de cada componente renderizado se inicializa con un estado inicial que se devuelve como el estado actual. Al volver a renderizar la interfaz de usuario debido a un cambio de estado, el gancho useState utiliza el estado más reciente de su cierre interno³. Esto puede parecer extraño, ya que se asumiría que useState se declara desde cero cada vez que se ejecuta la función de un componente. Sin embargo, junto a cada componente, React asigna un objeto donde se almacena información como el estado en memoria. Finalmente, la memoria se limpia cuando un componente deja de renderizarse mediante la recolección de elementos no utilizados de JavaScript.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) .
- Lea más sobre [el gancho useState de React](#) .
- Opcional: Lea más sobre [la desestructuración de matrices de JavaScript](#) .

Preguntas de la entrevista:

- Pregunta: ¿Qué es useState en React?
 - Respuesta: useState es un gancho de React que permite que los componentes de función administren y actualicen estado.
- Pregunta: ¿Cómo se utiliza useState para declarar el estado en un componente de función?
 - Respuesta: `const [state, setState] = useState(initialState);`
- Pregunta: ¿Qué desencadena una nueva renderización en React?
 - Respuesta: Los cambios de estado o las actualizaciones de propiedades pueden desencadenar una nueva renderización en React.
- Pregunta: ¿Cuál es el propósito del estado inicial en useState?
 - Respuesta: Establece el valor inicial de la variable de estado y solo se aplica durante la primera renderización.
- Pregunta: ¿Cómo se actualiza el estado usando useState?
 - Respuesta: Utilice la segunda entrada devuelta por useState para actualizar el estado.

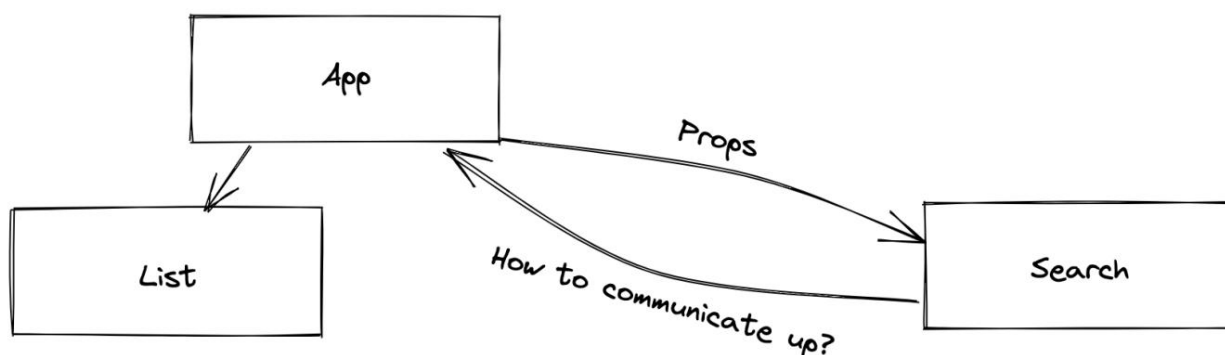
³<https://www.robinwieruch.de/javascript-closure/>
<https://tinyurl.com/4dy7s74t>
<https://tinyurl.com/mryfs8pm>
<https://bit.ly/4bn9DSm>
<https://www.robinwieruch.de/react-usestate-hook/>
<https://mzl.la/3ncC7Wl>

- Pregunta: ¿Llamar a `setState` activa una nueva renderización inmediata?
 - Respuesta: No, React procesa las actualizaciones de estado por lotes y realiza nuevas renderizaciones de forma asíncrona por razones de rendimiento.
- Pregunta: ¿Cuál es la diferencia entre usar varias llamadas `useState` y una sola `useState`?
 - ¿llamar con un objeto?
 - Respuesta: El uso de múltiples llamadas crea variables de estado independientes, mientras que una sola llamada con un objeto le permite administrar múltiples valores de estado dentro de una variable.
- Pregunta: ¿Puede mutar directamente la variable de estado obtenida de `useState`?
 - Respuesta: No, siempre debes usar la función `setState` para actualizar el estado de forma inmutable.
 - forma.
- Pregunta: ¿La actualización del estado siempre activa una nueva renderización?
 - Respuesta: Sí, actualizar el estado con `setState` activa una nueva representación del componente.

Manejadores de devolución de llamadas en JSX

Mientras que las propiedades se transmiten como información de los componentes principales a los secundarios, el estado puede usarse para cambiar la información con el tiempo. Sin embargo, aún no tenemos todas las piezas necesarias para que nuestros componentes se comuniquen entre sí. **Al usar propiedades como vehículo para transportar información, solo podemos comunicarnos con los componentes descendientes. Al usar el estado, podemos hacer que la información tenga estado, pero esta información también solo puede transmitirse mediante propiedades.**

Por ejemplo, actualmente, el componente de búsqueda no comparte su estado con otros componentes, por lo que solo lo usa (aquí: se muestra) y actualiza. Esto es suficiente para mostrar el estado más reciente en el componente de búsqueda; sin embargo, al final, queremos usar este estado en otro lugar. En esta sección, por ejemplo, queremos usar el estado en el componente de aplicación para filtrar las historias por término de búsqueda antes de que se pasen al componente de lista. Por lo tanto, sabemos que podríamos comunicarnos con un componente secundario mediante propiedades, pero no sabemos cómo comunicar el estado a un componente principal (aquí: del componente de búsqueda al componente de aplicación).



No es posible pasar información hacia arriba en el árbol de componentes, ya que las propiedades, naturalmente, solo se pasan hacia abajo. Sin embargo, podemos introducir un controlador de devolución de llamada : un controlador de devolución de llamada se introduce como controlador de eventos (A), se pasa como función en propiedades a otro componente (B), se ejecuta allí como controlador (C) y vuelve a llamar al lugar donde se introdujo (D).

src/App.jsx

```
const App = () => { const
  historias = [ ... ];

  // Una
  constante handleSearch = (evento) => {
    // D
    console.log(evento.objetivo.valor); };

  devolver (
```

```
<div>

  Mis historias de hackers

  { /* // B */ }
  <Buscar en búsqueda={manejarBúsqueda} />

  <hr />

  <Lista lista={historias} />
</div>

); };

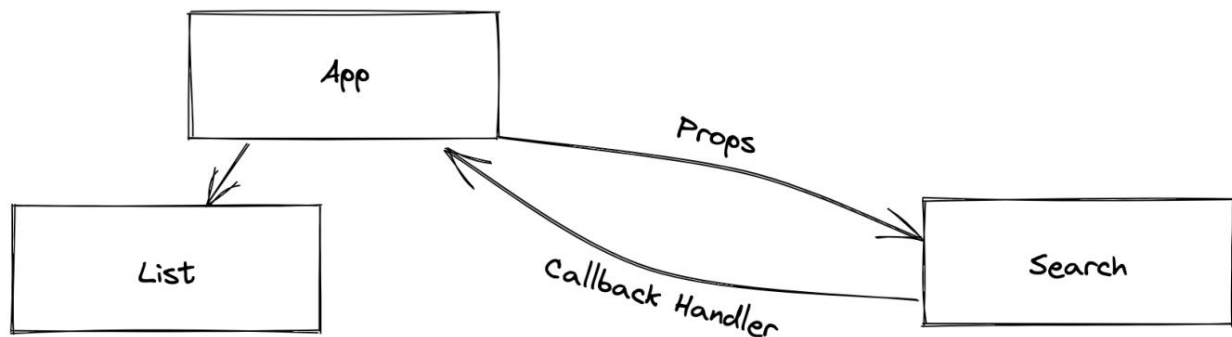
constante Buscar = (props) => {
  const [término de búsqueda, establecertérmino de búsqueda] = React.useState("");

  const handleChange = (evento) => {
    setSearchTerm(evento.objetivo.valor);

    // C
    props.onSearch(evento); };

  devolver ( ... ); };
```

Ahora, cada vez que un usuario escribe en el campo de entrada, se ejecuta la función transferida del componente App al componente Search. De esta forma, podemos notificar al componente App cuando un usuario escribe en el campo de entrada del componente Search. En esencia, un controlador de devolución de llamada, que es simplemente un tipo más específico de controlador de eventos, se convierte en nuestro vehículo implícito para comunicarnos con el árbol de componentes.



El concepto del manejador de devolución de llamada en resumen: Pasamos una función de un componente principal (App) a un componente secundario (Search) mediante propiedades; llamamos a esta función en el componente secundario (Search), pero tenemos la implementación real de la función llamada en el componente principal (App). En otras palabras, cuando un `manejador` (de eventos) se pasa como propiedad de un componente principal a su componente secundario, se convierte en un `manejador de devolución de llamada`. Las propiedades de React siempre se pasan a lo largo del árbol de componentes y, por lo tanto, las funciones que se pasan como `manejadores de devolución de llamada` en las propiedades pueden usarse para comunicarse a lo largo del árbol de componentes.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) ¹. • Revisar los conceptos de [controlador \(de eventos\)](#) y [controlador de devolución de llamada](#) ² tantas veces como necesites.

Preguntas de la entrevista:

- Pregunta: ¿Qué es un controlador de devolución de llamada en React?
 - Respuesta: Un controlador de devolución de llamada es una función que se pasa como apoyo a un componente secundario, lo que permite que el secundario se comuniqué con el primario. •
- Pregunta: ¿Cómo se pasa un controlador de devolución de llamada a un componente secundario?
 - Respuesta: Inclúyalo como un accesorio, como `<ChildComponent callback={handleCallback} />`.
- Pregunta: ¿Cómo se define un controlador de devolución de llamada en un componente principal?
 - Respuesta: Cree una función en el componente principal, por ejemplo, la función `handleCallback(data)`

```
{...}
```
- Pregunta: ¿Puede un controlador de devolución de llamada recibir parámetros?
 - Respuesta: Sí, los controladores de devolución de llamada pueden recibir parámetros pasados por el componente secundario. •
- Pregunta: ¿Pueden los controladores de devolución de llamada ser asíncronos?
 - Respuesta: Sí, los controladores de devolución de llamadas pueden ser asíncronos, lo que permite el manejo asíncrono de operaciones.
- Pregunta: ¿Se puede pasar un controlador de devolución de llamada a través de múltiples capas de componentes?
 - Respuesta: Sí, puedes pasar controladores de devolución de llamadas a través de múltiples capas de componentes. •
- Pregunta: ¿Puede un componente secundario tener múltiples controladores de devolución de llamadas del mismo componente principal?
 - Respuesta: Sí, un componente secundario puede recibir y usar múltiples controladores de devolución de llamada pasados del mismo componente principal. •
- Pregunta: ¿Es común usar controladores de devolución de llamada para envíos de formularios en React?
 - Respuesta: Sí, los controladores de devolución de llamada se utilizan comúnmente para gestionar envíos de formularios y actualizando el estado en los componentes principales.

<https://tinyurl.com/bdat6vsa>

<https://tinyurl.com/4uchxre7>

¹<https://bit.ly/3UnOJMQ>

²<https://www.robinwieruch.de/react-event-handler/>

Estado de elevación en React

En esta sección, nos enfrentamos a la siguiente tarea: usar el término de búsqueda con estado del componente Búsqueda para filtrar las historias por su propiedad de título en el componente Aplicación antes de que se pasen como propiedades al componente Lista. Hasta ahora, hemos aprendido a pasar información explícitamente con propiedades y a pasar información implícitamente con controladores de devolución de llamada. Sin embargo, si observamos el controlador de devolución de llamada en el componente Aplicación, no resulta natural aplicar el término de búsqueda del controlador `handleSearch()` del componente Aplicación como filtro para las historias.

Una solución podría ser establecer otro estado en el componente App que capture el término de búsqueda (`searchTerm`) que llega al componente App y lo use para filtrar las historias antes de que se pasen al componente List como propiedades. Sin embargo, **esto genera duplicación, lo que constituye una mala práctica, ya que el término de búsqueda tendría un estado en los componentes Search y App**. Por lo tanto, piénselo de otra manera: si el componente App necesita el estado `searchTerm` para filtrar las historias, ¿por qué no instanciarlo en el componente App en lugar de en el componente Search?

Trasladaremos el gancho `useState` del componente de búsqueda al componente de aplicación, usaremos la función de actualización de estado en el controlador de devolución de llamada proporcionado y usaremos dicho controlador en el componente de búsqueda para pasar el evento al componente principal. Así, cada vez que un usuario escriba en el campo de entrada, se actualizará el estado en el componente de aplicación. Después, usaremos el nuevo estado en el componente de aplicación para filtrar las historias antes de pasarlas al componente de lista. La siguiente implementación muestra la primera parte:

src/App.jsx

```
const App = () => { const
  historias = [ ... ];

  const [término de búsqueda, establecertérmino de búsqueda] = React.useState("");

  const handleSearch = (evento) => {
    setSearchTerm(evento.objetivo.valor); };

  devolver (
    <div>
      Mis historias de hackers

      <Buscar en búsqueda={manejarBúsqueda} />

      <hr />

      <Lista lista={historias} />
    </div>
```

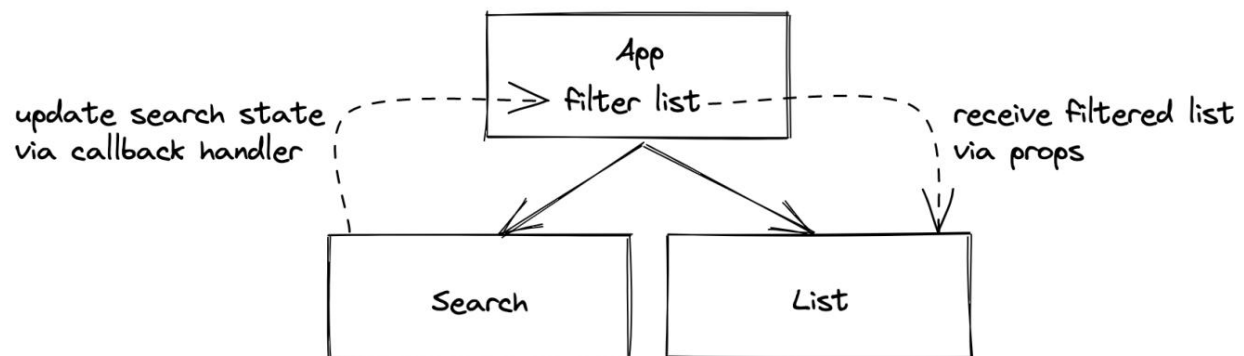
```

); };

const Buscar = (props) => (
  <div>
    <label htmlFor="search">Buscar: </label> <input
      id="search" type="text" onChange={props.onSearch} />
    </div>
  );

```

Ya aprendimos sobre el **controlador de devolución de llamada**, ya que **nos ayuda a mantener un canal de comunicación abierto entre el componente secundario** (en este caso, el componente de búsqueda) **y el componente principal** (en este caso, el componente de aplicación). Ahora, el componente de búsqueda ya no gestiona el estado, sino que solo pasa el evento al componente de aplicación mediante un controlador de devolución de llamada después de introducir el texto en el campo de entrada HTML. A partir de ahí, el componente App actualiza su estado. **El proceso de mover el estado de un componente a otro, como hicimos en el último fragmento de código, se denomina "lifting state"**. A continuación, podría volver a mostrar el término de búsqueda en el componente App (desde el estado, al usar "searchTerm") o en el componente Search (desde las propiedades, al pasar el estado de "searchTerm" como propiedades).



Regla general: Gestionar siempre el estado a nivel de componente, donde cada componente interesado lo gestione (usando información directamente del estado, p. ej., el componente App) o un componente inferior (usando información de propiedades, p. ej., los componentes List o Search). Si un componente inferior necesita actualizar el estado (p. ej., Search), se le pasa un controlador de devolución de llamada que le permite actualizar el estado superior en el componente principal. Si un componente inferior necesita usar el estado (p. ej., mostrarlo), se lo pasa como propiedad.

Finalmente, al gestionar el estado de búsqueda en el componente App, podemos filtrar las historias con el término de búsqueda con estado antes de pasarlas como propiedad de lista al componente List. Pruébalo usted mismo usando el método `filter()` integrado del array junto con las historias y el término de búsqueda antes de consultar la siguiente implementación:

src/App.jsx

```
const App = () => { const
  historias = [ ... ];

  const [término de búsqueda, establecertérmino de búsqueda] = React.useState("");

  const handleSearch = (evento) => {
    setSearchTerm(evento.objetivo.valor); };

  const searchedStories = historias.filter(función (historia) { return
    historia.título.includes(término de búsqueda); });

  devolver (
    <div>
      Mis historias de hackers

      <Buscar en búsqueda={manejarBúsqueda} />

      <hr />

      <Lista lista={Historias buscadas} />
    </div>

  ); };
```

Aquí, el [método de filtro integrado de la matriz de JavaScript](#) ³ Se utiliza para crear una nueva matriz filtrada. El método filter() toma una función como argumento, que accede a cada elemento de la matriz y devuelve verdadero o falso. Si la función devuelve verdadero (lo que significa que se cumple la condición), el elemento permanece en la nueva matriz; si devuelve falso, se elimina.

³<https://mzl.la/3BYFAOR>

Patio de juegos de código

```
const palabras =  
  [ 'spray',  
    'límite',  
    'élite',  
    'exuberante',  
    'destrucción',  
    'presente'  
  ];  
  
const filteredWords = palabras.filter(función (palabra) { return  
  palabra.length > 6; });  
  
console.log(filteredWords); //  
["exuberante", "destrucción", "presente"]
```

El método filter() podría hacerse más conciso utilizando una función de flecha con una función inmediata.
devolver:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  const searchedStories = historias.filter((historia) =>  
    historia.título.incluye(término de búsqueda) );  
  
  ...  
};
```

Eso es todo en cuanto a los pasos de refactorización de la función en línea para el método filter() . Tiene muchas variantes, y no siempre es fácil mantener un buen equilibrio entre legibilidad y concisión. Sin embargo, creo que mantenerla concisa siempre que sea posible también la hace legible la mayor parte del tiempo.

Lo que aún no funciona bien: El método filter() comprueba si el término de búsqueda (searchTerm) está presente como cadena en la propiedad title de cada historia, pero distingue entre mayúsculas y minúsculas. Si buscamos "react", no hay ninguna historia de "React" filtrada en la lista renderizada. Intenta solucionar este problema tú mismo haciendo que la condición del método filter() no distinga entre mayúsculas y minúsculas con JavaScript. El siguiente fragmento de código te muestra cómo lograrlo escribiendo en minúsculas el término de búsqueda (searchTerm) y el título de la historia:

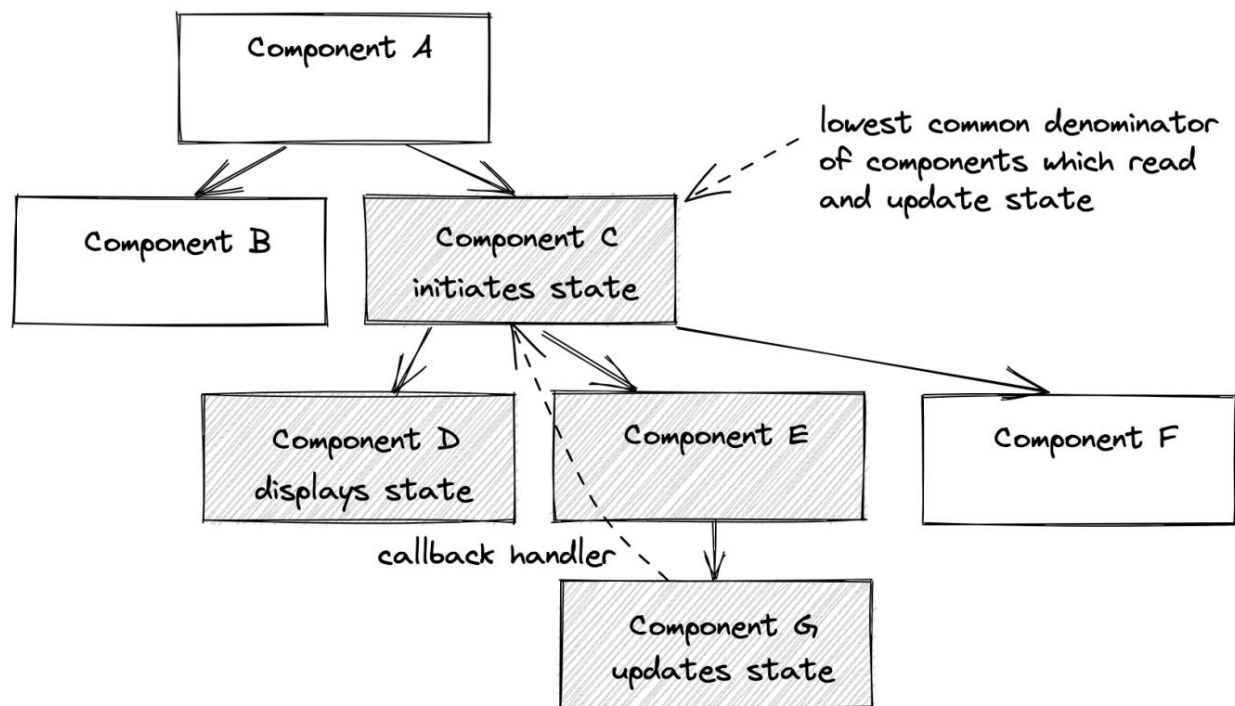
src/App.jsx

```
const App = () => {
  ...

  const searchedStories = historias.filter((historia) =>
    historia.titulo.toLowerCase().includes(searchTerm.toLowerCase()));

  ...
};
```

Ahora deberías poder buscar "eact", "React" o "react" y ver una de las dos historias mostradas. ¡Felicitaciones! Acabas de añadir tu primera función interactiva real a tu aplicación aprovechando el estado (para obtener una lista filtrada de historias) y las propiedades (pasando un controlador de devolución de llamada al componente de búsqueda).



Después de todo, saber dónde instanciar el estado en React resulta ser una habilidad importante en la carrera de todo desarrollador de React. El estado siempre debe estar ahí, donde todos los componentes que dependen de él puedan leerlo (mediante propiedades) y actualizarlo (mediante un controlador de devolución de llamada). Todos estos son componentes descendientes del componente que instancia el estado.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios del código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) .
- Lea más sobre [cómo levantar el estado en React](#) .

Preguntas de la entrevista:

- Pregunta: ¿Qué es el estado de elevación en React?
 - Respuesta: El levantamiento de estado se refiere a la práctica de mover el estado de un componente secundario a su componente principal.
- Pregunta: ¿Por qué se levanta el estado en React?
 - Respuesta: Para compartir y gestionar el estado a un nivel superior, haciéndolo accesible a varios niños.
- componentes.
- Pregunta: ¿Cómo se levanta el estado en React?
 - Respuesta: Mover el estado y las funciones relacionadas a un ancestro común (normalmente un padre)
- componente.
- Pregunta: ¿Pueden varios componentes secundarios compartir el mismo estado levantado?
 - Respuesta: Sí, el estado de elevación permite que varios componentes secundarios compartan el mismo estado.
- Pregunta: ¿Cuál es la ventaja de levantar el estado sobre el uso del estado local en un componente?
 - Respuesta: Elevar el estado promueve compartir el estado entre componentes.
- Pregunta: ¿Cuál es la función de las devoluciones de llamadas en la elevación del estado?
 - Respuesta: Las funciones de devolución de llamada se utilizan para pasar datos de los componentes secundarios a los principales cuando estado de elevación.
- Pregunta: ¿Puede un componente secundario modificar el estado de un componente principal directamente a través de un ¿manejador de devolución de llamada?
 - Respuesta: No, el componente secundario puede invocar la devolución de llamada para notificar al componente principal, y el componente principal puede decidir cómo actualizar su estado.
- Pregunta: ¿Es necesario elevar todo el estado al componente principal de nivel superior?
 - Respuesta: No, solo elevar el estado a un nivel donde se necesite compartir entre varios
- componentes.
- Pregunta: ¿Cómo contribuye el estado de elevación a una mejor reutilización de los componentes?
 - Respuesta: El estado de elevación permite que la lógica con estado se concentre en un ancestro común,
- haciendo que los componentes sean más reutilizables.

<https://tinyurl.com/5n77yypj>
<https://tinyurl.com/3brw4d6>
<https://bit.ly/49k1Cf9>
<https://www.robinwieruch.de/react-lift-state/>

Componentes controlados por React

Los elementos HTML tienen un estado interno que no está vinculado a React. Confirme esta tesis usted mismo comprobando cómo se implementa su campo de entrada HTML en el componente de búsqueda. Si bien proporcionamos atributos esenciales como id y type, además de un controlador (en este caso: onChange), no indicamos el valor del elemento. Sin embargo, muestra el valor correcto cuando el usuario escribe en él.

Ahora prueba lo siguiente: Al inicializar el estado de searchTerm en el componente App, usa "React" como estado inicial en lugar de una cadena vacía. Después, abre la aplicación en el navegador. ¿Puedes identificar el problema? Dedica tiempo a averiguar qué sucede y cómo solucionarlo.

Aunque las historias se han filtrado según el nuevo searchTerm inicial en la última sección, el campo de entrada HTML no muestra el valor en el navegador. Solo al escribir en él, vemos los cambios reflejados. Esto se debe a que el campo de entrada desconoce el estado de React (en este caso: searchTerm); solo usa su manejador para comunicar (ver handleSearch()) su estado interno al estado de React. Una vez que el usuario empieza a escribir en el campo de entrada, el elemento HTML registra estos cambios. Sin embargo, para que todo funcione correctamente, el HTML debe conocer el estado de React. Por lo tanto, debemos proporcionarle el estado actual como valor :

src/App.jsx

```
const App = () => { const
  historias = [ ... ];

  const [término de búsqueda, establecertérmino de búsqueda] = React.useState('React');

  ...

  devolver (
    <div>
      Mis historias de hackers

      <Buscar búsqueda={término de búsqueda} onSearch={manejarBúsqueda} />

      ...
    </div>

  ); };

constante Buscar = (props) => (
  <div>
    <label htmlFor="search">Buscar: </label>
    <entrada
```

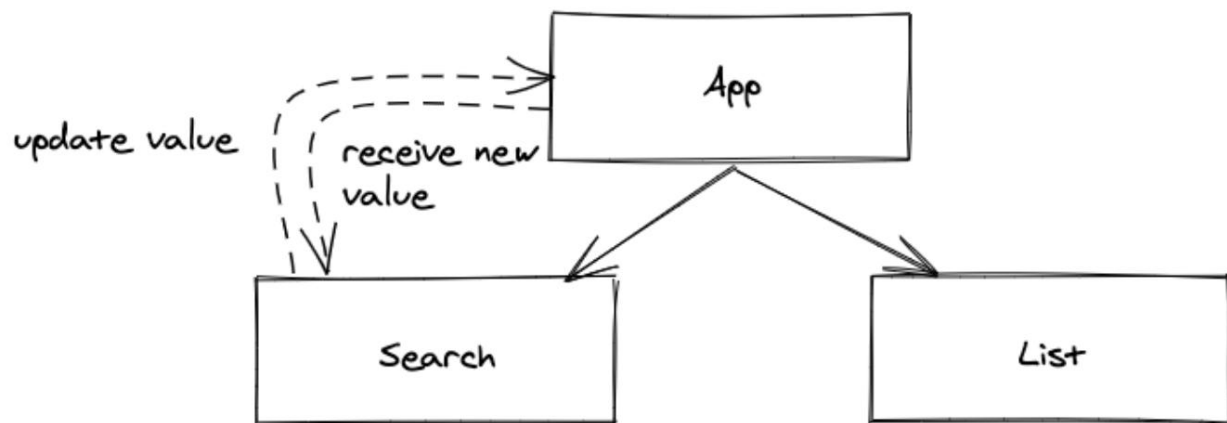
```

    id="buscar"
    tipo="texto"
    valor={props.search}
    onChange={props.onSearch} /> </
  div>
);

```

Ahora ambos estados están sincronizados. En lugar de permitir que el elemento HTML controle su estado interno, utiliza el estado de React aprovechando el atributo `value` del elemento .

Cada vez que el elemento HTML emite un evento de cambio, el nuevo valor se escribe en el estado de React y vuelve a renderizar los componentes. A continuación, el elemento HTML vuelve a usar el estado reciente como valor .



Antes, el elemento HTML funcionaba por sí solo, pero ahora lo controlamos introduciendo el estado de React. Si bien el campo de entrada se convirtió explícitamente en un elemento controlado, el componente de búsqueda se convirtió implícitamente en uno. Como principiante en React, es importante usar componentes controlados, ya que se busca imponer un comportamiento predecible. Sin embargo, más adelante, también podrían darse casos de componentes no controlados.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](#) .
 - Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#) .
- Lea más sobre [los componentes controlados en React](#) ¹.

<https://tinyurl.com/53p67ybt>

<https://tinyurl.com/5auk52n7>

<https://bit.ly/3w3A5Ah>

¹<https://www.robinwieruch.de/componentes-controlados-por-reacción/>

Preguntas de la entrevista:

- Pregunta: ¿Qué es un componente controlado en React?

– Respuesta: Un componente controlado es un componente cuyos elementos de formulario están controlados por Estado de reacción.

- Pregunta: ¿Cómo se crea una entrada controlada en React?

– Respuesta: Establezca el atributo de valor de entrada en una variable de estado y proporcione un controlador onChange para actualizar el estado.

- Pregunta: ¿Cuál es el papel de la propiedad de valor en un elemento de entrada controlado?

– Respuesta: La propiedad value establece el valor actual de la entrada, lo que la convierte en una propiedad controlada. componente. •

Pregunta: ¿Cómo se maneja una casilla de verificación controlada en React?

– Respuesta: Use el atributo verificado y proporcione un controlador onChange para actualizar el estado correspondiente. • Pregunta: ¿Cómo se borra el valor de

un componente controlado?

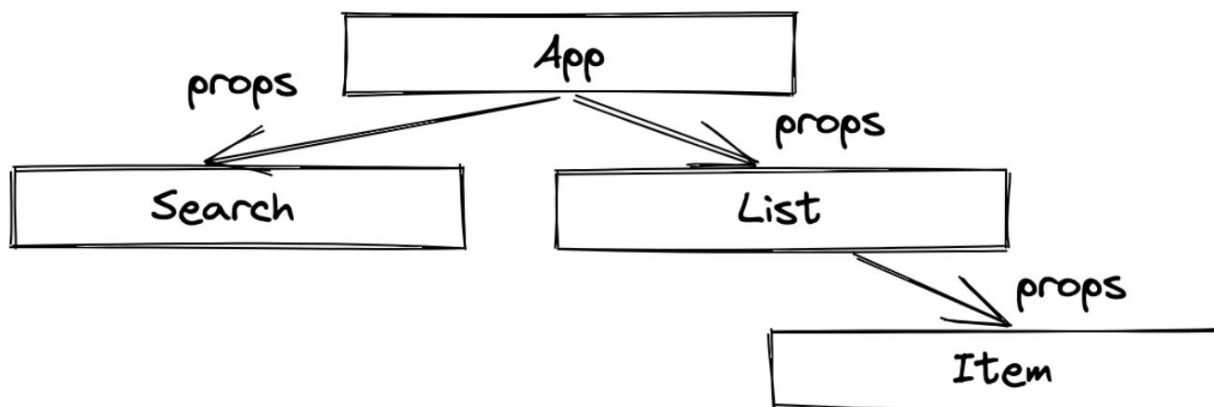
– Respuesta: Establezca la variable de estado en un valor vacío o nulo para borrar el valor de un valor controlado. componente.

- Pregunta: ¿Cuáles son las posibles desventajas de utilizar componentes controlados?

– Respuesta: Los componentes controlados pueden generar un código extenso, especialmente en formularios con muchos elementos de entrada.

Manejo de accesorios (avanzado)

Las propiedades se transmiten de padre a hijo a través del árbol de componentes. Dado que las usamos frecuentemente para transportar información de un componente a otro, y a veces a través de otros componentes intermedios, es útil conocer algunos trucos para facilitar la transmisión de propiedades.



Se recomiendan las siguientes refactorizaciones para aprender diferentes patrones de JavaScript/React, aunque aún es posible crear aplicaciones React completas sin ellos. Considérelas técnicas avanzadas de props que harán que su código React sea más conciso, legible y fácil de mantener en ciertos escenarios.

Desestructuración de accesorios mediante desestructuración de objetos

Las propiedades de React son simplemente un objeto de JavaScript; de lo contrario, no podríamos acceder a `props.list` ni a `props.onSearch` en nuestros componentes de React. Dado que `props` es un objeto que simplemente pasa información de un componente a otro, podemos aplicarle un par de trucos de JavaScript. Por ejemplo, acceder a las propiedades de un objeto con la desestructuración de objetos de JavaScript moderna ²:

Patio de juegos de código

```
const usuario =  
  { nombre: 'Robin', apellido:  
    'Wieruch', };  
  
// sin desestructuración de objetos const  
firstName = user.firstName; const lastName =  
user.lastName;
```

²<https://mzl.la/30KbXTC>

```
// con desestructuración de objetos const  
{ firstName, lastName } = user;
```

Cuando necesitamos acceder a varias propiedades de un objeto, emplear una sola línea de código en lugar de varias suele ser un enfoque más sencillo y elegante. Por eso, la desestructuración de objetos se utiliza comúnmente en JavaScript. Antes de profundizar en el siguiente fragmento de código, intentemos aplicar este conocimiento a las propiedades de React de nuestro componente de búsqueda.

Ahora, exploremos cómo podemos emplear la desestructuración de props. Inicialmente, necesitamos refactorizar la función de flecha del componente Buscar de un cuerpo conciso a un cuerpo de bloque. Posteriormente, podemos implementar la desestructuración del objeto props dentro del cuerpo de la función:

src/App.jsx

```
const Buscar = (props) => { const  
  { buscar, onSearch } = props;  
  
  devolver (  
    <div>  
      <label htmlFor="search">Buscar: </label> <input  
  
        id="buscar"  
        tipo="texto"  
        valor={buscar}  
        onChange={onSearch} />  
  
    </div>  
  
  );  
};
```

Esta es una desestructuración básica del objeto props en un componente de React, para que sus propiedades se puedan usar cómodamente en el componente. Sin embargo, también tuvimos que refactorizar la función de flecha del componente Search, pasando del cuerpo conciso al cuerpo del bloque, para acceder a las propiedades de los props con la desestructuración del objeto en el cuerpo de la función del componente. Esto ocurriría con bastante frecuencia si siguiéramos este patrón y no nos facilitaría las cosas, ya que tendríamos que refactorizar constantemente nuestros componentes. Podemos llevar esto un paso más allá desestructurando el objeto props directamente en la firma de la función de nuestro componente, omitiendo de nuevo el cuerpo del bloque de la función:

src/App.jsx

```
const Buscar = ({ buscar, onSearch }) => (
  <div>
    <label htmlFor="search">Buscar: </label> <input
      id="buscar"
      tipo="texto"
      valor={buscar}
      onChange={onSearch} />

    </div>
  );
```

Las props de React rara vez se usan en componentes por sí solas; en su lugar, se utiliza toda la información contenida en el objeto props. Al desestructurar el objeto props directamente en la firma de función del componente, podemos acceder fácilmente a toda la información sin tener que lidiar con su contenedor. Los componentes List e Item pueden realizar la misma desestructuración de props :

src/App.jsx

```
const Lista = ({ lista }) => (
  <ul>
    {lista.map((elemento) => (
      <Clave del artículo={item.objectID} artículo={item} /> ))} </
    ul>

  );

const Item = ({ item }) => (
  <li>
    <span>
      <a href={item.url}>{item.title}</a> <span>{item.author}

      <span>{item.num_comments}
      <span>{item.points} </li>

  );
```

El uso de la desestructuración de objetos se ajusta a las mejores prácticas de JavaScript y promueve una estructura de componentes React más limpia y eficiente. Permite una extracción más sencilla de las propiedades necesarias, mejorando tanto la claridad del código como la experiencia general de desarrollo en aplicaciones React. Sin embargo, podemos profundizar en este tema con lecciones avanzadas opcionales.

Desestructuración anidada

El parámetro de elemento entrante en el componente Item tiene algo en común con el parámetro props mencionado anteriormente : ambos son objetos JavaScript. Además, aunque el objeto item ya se ha desestructurado de las props en la firma de función del componente Item, no se utiliza directamente en el componente Item. El objeto item solo pasa su información (propiedades del objeto) a los elementos.

La solución actual es correcta, como verán en la discusión en curso. Sin embargo, quiero mostrarles dos variantes más, ya que hay mucho que aprender sobre los objetos JavaScript en React. Comencemos con la desestructuración anidada y su funcionamiento:

Patio de juegos de código

```
usuario const =
  {nombre: 'Robin', mascota:
    {nombre:
      'Trixi', }, }, };

// sin desestructuración de objetos const
firstName = user.firstName; const name =
user.pet.name;

console.log(firstName + ' tiene una mascota llamada ' + nombre);
// "Robin tiene una mascota llamada Trixi"

// con desestructuración de objetos anidados
const
  { firstName,
    pet: {
      nombre,
    }, } = usuario;

console.log(firstName + ' tiene una mascota llamada ' + nombre);
// "Robin tiene una mascota llamada Trixi"
```

La desestructuración anidada nos ayuda a recopilar toda la información necesaria del objeto elemento en la firma de la función para su uso inmediato en los elementos del componente. Aunque no es la opción más legible en este caso, cabe destacar que puede ser útil en otros escenarios:

src/App.jsx

// Variación 1: Desestructuración anidada

```
const Item = ({ item:
  { título, url,

  autor,
  núm_comentarios,
  puntos, }, })

=> (
  <li>
    <span>
      <a href={url}>{título}</a>

      <span>{autor}
      <span>{num_comentarios}
      <span>{puntos} </li> );
```

En resumen, la desestructuración anidada en React demuestra ser una técnica poderosa y eficiente cuando se trabaja con estructuras de datos complejas, especialmente dentro de objetos anidados o matrices en propiedades o estados. Este enfoque simplifica la extracción de valores profundamente anidados, lo que hace que el código sea más conciso y legible. Sin embargo, la desestructuración anidada introduce mucha confusión debido a las sangrías en la firma de la función. Si bien en este caso no es la opción más legible, puede ser útil en otros escenarios.

Operadores de propagación y descanso

Adoptemos otro enfoque con los operadores spread y rest de JavaScript. Para prepararnos, refactorizaremos nuestros componentes List e Item con la siguiente implementación. En lugar de pasar el elemento como un objeto de List al componente Item, pasamos todas las propiedades del objeto item :

src/App.jsx

// Variación 2: Operadores de propagación y reposo // 1.

Paso

```
constante Lista = ({ lista }) => (  
  <ul>  
    {lista.map((elemento) => (  
      <Articulo  
        clave={item.objectID}  
        título={item.title}  
        url={item.url}  
        autor={item.author}  
        num_comments={item.num_comments}  
        puntos={item.points}  
      />  
    ))}  
  </ul>  
);  
  
const Item = ({título, url, autor, num_comentarios, puntos}) => (  
  <li>  
    <span>  
      <a href={url}>{título}</a>  
  
      <span>{autor}  
      <span>{num_comentarios}  
      <span>{puntos} </li>  
  
  );
```

Ahora bien, aunque la firma de la función del componente Item es más concisa, el desorden terminó en el componente List, porque cada propiedad se pasa al componente Item individualmente.

Podemos mejorar este enfoque utilizando el operador de propagación de JavaScript ³:

³<https://mzl.la/3jetlkn>

Patio de juegos de código

```

perfil constante = {
  Nombre: 'Robin',
  apellido: 'Wieruch', };

dirección constante = {
  país: 'Alemania', ciudad:
  'Berlín', };

const usuario =
  { ...perfil,
    género:

'masculino', ...dirección, };

console.log(usuario); //
{ //
nombre: "Robin", // apellido:
"Wieruch", // género: "masculino" //
país: "Alemania", // ciudad:
"Berlín", // }

```

El operador de propagación de JavaScript permite propagar literalmente todos los pares clave-valor de un objeto a otro. Esto también se puede hacer en JSX de React. En lugar de pasar cada propiedad individualmente mediante props del componente List al componente Item, como antes, podemos usar el operador de propagación de JavaScript para pasar todos los pares clave-valor del objeto como pares atributo-valor a un elemento JSX:

src/App.jsx

// Variación 2: Operadores de propagación y reposo // 2.

Paso

```

constante Lista = ({ lista }) => (
  <ul>
    {lista.map((elemento) =>
      ( <Clave del elemento={elemento.objectID} {...elemento} /

    > ))) </
ul> );

```

```
const Item = ({título, url, autor, num_comentarios, puntos}) => (  
  <li>  
    <span>  
      <a href={url}>{título}</a>  
  
      <span>{autor}</span>  
      <span>{número_comentarios}</span>  
      <span>{puntos}</li>  
  
  );
```

Esta refactorización concretó el proceso de pasar la información del componente Lista al componente Elemento.

Finalmente, usaremos la desestructuración de rest de JavaScript como broche de oro. La operación de rest de JavaScript siempre ocurre como la última parte de la desestructuración de un objeto:

Patio de juegos de código

```
constante usuario = {  
  identificación: '1',  
  nombre: 'Robin', apellido:  
    'Wieruch', país: 'Alemania',  
  ciudad: 'Berlín', };  
  
const { id, país, ciudad, ...usuarioSinDirección } = usuario;  
  
console.log(usuarioSinDirección); // { // Nombre:  
  
  "Robin", // Apellido: "Wieruch" // }  
  
console.log(id); // "1"  
  
console.log(ciudad); //  
"Berlín"
```

Aunque ambos tienen la misma sintaxis (tres puntos), el operador de reposo no debe confundirse con el operador de propagación. Mientras que el operador de reposo aparece en el lado izquierdo de una asignación, el operador de propagación...

El operador se encuentra en el lado derecho. El operador rest siempre se utiliza para separar un objeto de algunas de sus propiedades.

Ahora se puede usar en nuestro componente Lista para separar el ID del objeto del elemento, ya que este solo se usa como clave y no en el componente Elemento. Solo el elemento restante (léase: resto) se distribuye como pares atributo/valor en el componente Elemento (como antes):

src/App.jsx //

Variación 2: Operadores de propagación y reposo // Paso

final

```
constante Lista = ({ lista }) => (
  <ul>
    {lista.map(({ objectID, ...item }) => ( <Clave del
      elemento={objectID} {...item} /> ))} </ul> );

const Item = ({título, url, autor, num_comentarios, puntos}) => (
  <li>
    <span>
      <a href={url}>{título}</a>

    <span>{autor}
    <span>{num_comentarios}
    <span>{puntos}
  </li>
);
```

En esta última variación, el operador rest se utiliza para desestructurar el objectID del resto del objeto del elemento . Posteriormente, el elemento se distribuye con sus pares clave/valor en el componente Item. Si bien esta última variación es muy concisa, incluye funciones avanzadas de JavaScript que podrían resultar desconocidas para... alguno.

En esta sección, hemos aprendido sobre la desestructuración de objetos en JavaScript, que se usa comúnmente no solo para el objeto props , sino también para otros objetos como el objeto item , que están anidados dentro de las props. También hemos visto cómo se puede usar la desestructuración anidada (Variación 1), pero también cómo no aportó ninguna ventaja en nuestro caso, ya que simplemente aumentó el tamaño del componente. En el futuro, es más probable que encuentre casos de uso beneficiosos para la desestructuración anidada.

Por último, pero no menos importante, has aprendido sobre los operadores spread y rest de JavaScript, que no deben confundirse entre sí, para realizar operaciones en objetos JavaScript y pasar el objeto props de un componente a otro de la forma más concisa. Finalmente, quiero destacar la versión inicial, que seguiremos usando en las siguientes secciones:

src/App.jsx

```

constante Lista = ({ lista }) => (
  <ul>
    {lista.map((elemento) => (
      <Clave del artículo={item.objectID} artículo={item} /> ))} </
    ul>

);

constante Item = ({ item }) => (
  <li>
    <span>
      <a href={item.url}> {item.title}</a>

    <span>{item.author}
    <span>{item.num_comments}
    <span>{item.points} </li>

);

```

Puede que no sea la más concisa, pero es la más fácil de entender. La Variante 1, con su desestructuración anidada, no aportó muchos beneficios, y la Variante 2 añadió demasiadas funciones avanzadas de JavaScript (operador spread, operador rest) que no son familiares para todos. Al fin y al cabo, todas estas variaciones tienen sus pros y sus contras. Al refactorizar un componente, procure siempre la legibilidad, especialmente si trabaja en equipo, y asegúrese de que todos usen un estilo de código React común.

Reglas generales:

- Casi siempre se usa la desestructuración de objetos para las propiedades en la firma de función de un componente de función, ya que rara vez se usan. **Excepción:** Cuando las propiedades solo se pasan a través del componente al siguiente componente secundario (ver cuándo usar el operador de propagación).
- Use el operador spread cuando desee pasar todos los pares clave-valor de un objeto a un componente secundario en JSX. Por ejemplo, a menudo las propiedades no se usan en un componente, sino que solo se pasan al siguiente. En ese caso, conviene simplemente extender el objeto de propiedades {...props} al siguiente componente.
- Use el operador rest cuando solo desee separar ciertas propiedades de su objeto de propiedades.
- Use la desestructuración anidada solo cuando mejore la legibilidad.

Ceremonias:

- Compare su código fuente con el [código fuente del autor](https://tinyurl.com/mwn986w5).

<https://tinyurl.com/mwn986w5>

- Recapitulación de todos los [cambios en el código fuente](#) de esta sección.
- Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#). • Lea más sobre [cómo usar props en React](#).
- Opcional: Lea más sobre [la tarea de desestructuración de JavaScript](#).
- Lea más sobre [el operador de propagación de JavaScript](#) y [operador de descanso](#)¹.
- Obtenga una buena idea sobre JavaScript (por ejemplo, desestructuración, operador de propagación, desestructuración de descanso) y cómo se relaciona con React (por ejemplo, accesorios) de las últimas lecciones.
- Sigue usando tu método preferido para gestionar las propiedades de React. Si aún no te decides, considera la variación que se usó en la sección anterior.

Preguntas de la entrevista:

- Pregunta: ¿Cómo se desestructuran las propiedades en los parámetros de un componente de función?
 - Respuesta: Puedes desestructurar propiedades directamente en los parámetros de la función, de esta manera: `function MyComponent({prop1, prop2}) {...}`.
 - Pregunta: ¿Puedes proporcionar un valor predeterminado al desestructurar propiedades?
 - Respuesta: Sí, puede proporcionar valores predeterminados durante la desestructuración, como `{prop1 = 'default', prop2 }`.
 - Pregunta: ¿Es necesario desestructurar todos los props o puede elegir algunos específicos?
 - Respuesta: Puedes elegir desestructurar propiedades específicas según las necesidades de tu componente, dejando otras intactas.
 - Pregunta: ¿Cómo se usa el operador de propagación (...) en las propiedades de React?
 - Respuesta: El operador de propagación se utiliza para pasar todas las propiedades de un objeto como propiedades separadas a un componente React, como `<MyComponent {...obj} />`.
 - Pregunta: ¿Puedes usar el operador de propagación para combinar propiedades con otras adicionales?
 - Respuesta: Sí, puedes combinar accesorios existentes con otros adicionales usando la extensión operador, como `<MyComponent {...props} newProp={value} />`.
 - Pregunta: ¿El operador de propagación crea una copia superficial o profunda de un objeto?
 - Respuesta: El operador de propagación crea una copia superficial de un objeto, es decir, objetos anidados. Todavía hay referencias al original.
 - Pregunta: ¿Cuál es el propósito del operador rest (...rest) en React?
 - Respuesta: El operador rest se utiliza para recopilar las propiedades restantes en un nuevo objeto, a menudo utilizado en combinación con desestructuración de accesorios.
 - Pregunta: ¿Por qué se utiliza la desestructuración de matrices para React Hooks como useState y la destrucción de objetos?
 - Pregunta: ¿Turing para accesorios?
 - Respuesta : Un Hook de React como useState devuelve un array, mientras que las propiedades son objetos; por lo tanto, debemos aplicar la operación adecuada para la estructura de datos subyacente. La ventaja de que useState devuelva un array es que los valores pueden tener cualquier nombre en la operación de desestructuración.
 - Pregunta: ¿Qué es el "prop drilling" en React?
 - Respuesta: La perforación de puntales es el proceso de pasar puntales a través de múltiples capas de componentes. componentes para llegar a un componente secundario profundamente anidado.

<https://tinyurl.com/2j2mhtjd>

<https://bit.ly/3SL9uRm>

<https://www.robinwieruch.de/react-pass-props-to-component/>

<https://mzl.la/30KbXTC>

<https://mzl.la/3jetlkn>

¹ <https://mzl.la/3GeJbef>

Efectos secundarios de React

La salida devuelta por un componente React se define por sus propiedades y estado. Los efectos secundarios también pueden afectar esta salida, ya que se utilizan para interactuar con API de terceros (por ejemplo, la API localStorage del navegador, API remotas para la obtención de datos), con elementos HTML para medir el ancho y el alto, o con funciones integradas de JavaScript, como temporizadores o intervalos. Estos son solo algunos ejemplos de efectos secundarios en componentes React; a continuación, aplicaremos uno de ellos.

Actualmente, al buscar un término en nuestra aplicación, obtendrá el resultado. Sin embargo, al cerrar el navegador y volver a abrirlo, el término de búsqueda desaparece. ¿No sería una excelente experiencia de usuario si nuestro componente de búsqueda pudiera recordar la búsqueda más reciente para que la aplicación la mostrara en el navegador al reiniciarse?

Implementemos esta función usando un efecto secundario para almacenar la búsqueda reciente en el almacenamiento local del navegador y recuperarla al inicializar el componente. Primero, use el almacenamiento local para almacenar el término de búsqueda (searchTerm) acompañado de un identificador cada vez que un usuario escriba en el campo de entrada HTML:

src/App.jsx

```
constante Aplicación = () => {
  ...

  const handleSearch = (evento) => {
    setSearchTerm(evento.objetivo.valor);

    localStorage.setItem('buscar', evento.objetivo.valor); };

  ...
};
```

En segundo lugar, use el valor almacenado, si existe, para establecer el estado inicial de searchTerm en el gancho useState de React. De lo contrario, utilice nuestro estado inicial predeterminado (aquí: "React"), como antes:

src/App.jsx

```
constante Aplicación = () => {
  ...

  const [término de búsqueda, setTérmino de búsqueda] =
    React.useState( localStorage.getItem('buscar') ||
    'React' );

  ...
};
```

Información útil: El operador lógico OR¹ de JavaScript devuelve el operando verdadero en esta expresión y se corta si `localStorage.getItem('search')` devuelve un valor verdadero. Se usa como abreviatura para la siguiente implementación para establecer valores predeterminados:

Patio de juegos de código

```

deje que se haya
almacenado; si (localStorage.getItem('buscar')) { se haya
  almacenado = verdadero; }
de lo contrario {
  hasStored = falso;
}

constante estadoinicial = haAlmacenado
  ? localStorage.getItem('buscar')
  : 'Reaccionar';

```

Al usar el campo de entrada y actualizar la pestaña del navegador, este debería recordar el último término de búsqueda. Básicamente, sincronizamos el almacenamiento local del navegador con el estado de React: si bien inicializamos el estado con el valor del almacenamiento local del navegador (o una alternativa), escribimos el nuevo valor al llamar al controlador en el almacenamiento del navegador y en el estado del componente.

La función está completa, pero presenta una falla que podría generar errores a largo plazo: la función del controlador debería encargarse principalmente de actualizar el estado, pero ahora tiene un efecto secundario. La falla: si usamos la función de actualización de estado `setSearchTerm` en otra parte de nuestra aplicación, la función deja de funcionar porque el almacenamiento local no se actualiza, ya que solo se actualiza en el controlador de eventos.

Para solucionar esto, gestionaremos el efecto secundario de forma centralizada, no en un controlador específico. Usaremos el gancho "useEffect" de React para activar el efecto secundario deseado cada vez que cambie el término de búsqueda :

src/App.jsx

```

constante Aplicación = () => {
  ...

  const [término de búsqueda, setTérmino de búsqueda] =
    React.useState( localStorage.getItem('buscar') ||
    'React' );

  React.useEffect(() =>
    { localStorage.setItem('buscar', término de búsqueda); },
    [término de búsqueda]);

  const handleSearch = (evento) => {
    setSearchTerm(evento.objetivo.valor);

```

¹ <https://mzl.la/3aXxyd>

```
};

...

);
```

El gancho `useEffect` de React acepta dos argumentos: el primero es una función que ejecuta nuestro efecto secundario. En nuestro caso, el efecto secundario almacena `searchTerm` en el almacenamiento local del navegador. El segundo argumento es una matriz de dependencia de variables. Si una de estas variables cambia, se llama a la función del efecto secundario. En nuestro caso, la función se llama cada vez que `searchTerm` cambia (por ejemplo, cuando un usuario escribe en el campo de entrada HTML). Además, también se llama inicialmente cuando el componente se renderiza por primera vez.

Omitir el segundo argumento (la matriz de dependencias) haría que la función del efecto secundario se ejecutara en cada renderizado (renderizado inicial y renderizado de actualización) del componente. Si la matriz de dependencias de `useEffect` de React está vacía, la función del efecto secundario solo se llama una vez, cuando el componente se renderiza por primera vez. Al fin y al cabo, el gancho nos permite participar en el ciclo de vida del componente de React al montarlo, actualizarlo y desmontarlo. Puede activarse al montar el componente por primera vez, pero también si se actualiza alguno de sus valores (estado, propiedades, valores derivados de estado/propiedades).

En conclusión, usar el gancho `useEffect` de React en lugar de gestionar el efecto secundario en el controlador de eventos ha mejorado la robustez de la aplicación. Siempre que se actualice el estado de `searchTerm` mediante `setSearchTerm`, el almacenamiento local del navegador estará sincronizado.

Ceremonias:

- Compare su código fuente con el código fuente del autor^{1 2}.
 - Recapitulación de todos los cambios en el código fuente^{1 3} de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor aquí¹. • Lee más sobre el hook `useEffect` de React¹.
 - Asigna un `console.log()` a la función del primer argumento y experimenta con la matriz de dependencias del gancho `useEffect` de React. Revisa también los registros para ver si la matriz de dependencias está vacía.
- Lee más sobre el uso del almacenamiento local con React¹.
- Pruebe el siguiente escenario: En su navegador, presione la tecla de retroceso para seleccionar el término de búsqueda en el campo de entrada hasta que no quede nada. Internamente, debería estar configurado como una cadena vacía. A continuación, actualice el navegador y compruebe lo que muestra. Quizás se pregunte por qué muestra "React" en lugar de "", ya que debería ser la búsqueda reciente. Esto se debe a que el operador lógico OR de JavaScript evalúa... Se convierte en falso y, por lo tanto, toma "React" como valor verdadero. Si desea evitar esto y evaluarlo como verdadero, puede intercambiar el operador lógico OR `||` de JavaScript por el operador de fusión nulo `??`¹.² <https://tinyurl.com/3m7dzk4f>^{1 3} <https://mzl.la/2Z4bsU4>

tinyurl.com/56rcmthz

¹ <https://bit.ly/3HKAvoY>

¹ <https://>

www.robinwieruch.de/react-useeffect-hook/

¹ <https://www.robinwieruch.de/>

[almacenamiento-local-react/](https://mzl.la/2Z4bsU4)¹ <https://mzl.la/2Z4bsU4>

Preguntas de la entrevista:

- Pregunta: ¿Qué es `useEffect` en React?
 - Respuesta: `useEffect` es un gancho en React que permite que los componentes de función realicen acciones secundarias. efectos.
- Pregunta: ¿Se pueden utilizar varios ganchos `useEffect` en un componente?
 - Respuesta: Sí, puedes usar varios ganchos `useEffect` en un solo componente.
- Pregunta: ¿Qué representa el segundo argumento en `useEffect`?

Respuesta : El segundo argumento es una matriz de dependencias. El efecto se ejecuta cuando cualquiera de estas dependencias cambia.
- Pregunta: ¿Cómo ejecutar `useEffect` solo una vez (al montar)?
 - Respuesta: Pase una matriz de dependencia vacía (`[]`) como segundo argumento. • Pregunta:
- ¿Puede `useEffect` devolver una función de limpieza?
 - Respuesta: Sí, la función devuelta por `useEffect` sirve como función de limpieza.
- Pregunta: ¿Cuál es el propósito de la función de limpieza en `useEffect`?
 - Respuesta: Se encarga de la limpieza o desmontaje de recursos cuando se desmonta el componente. o cuando cambian las dependencias.
- Pregunta: ¿Cómo se realiza la limpieza en `useEffect` para cada renderizado?
 - Respuesta: Devuelve una función dentro de `useEffect` con la lógica de limpieza. • Pregunta:
- ¿Puedes ejecutar `useEffect` condicionalmente en función de una determinada condición?
 - Respuesta: Sí, puedes usar declaraciones condicionales dentro de `useEffect` para controlar cuándo debe correr.
- Pregunta: ¿Qué sucede si omite el segundo argumento en `useEffect`?
 - Respuesta: Ejecuta el efecto después de cada renderizado, lo que genera posibles problemas de rendimiento.
- Pregunta: ¿Cómo contribuye `useEffect` a evitar condiciones de carrera en React?
 - Respuesta: Permite manejar operaciones asíncronas y evitar condiciones de carrera gestionar el orden de ejecución.

Ganchos personalizados de React (avanzados)

Hasta ahora, hemos profundizado en dos de los hooks más populares de React: `useState` y `useEffect`. El primero resulta valioso para gestionar valores que cambian, mientras que el segundo facilita la inclusión de efectos secundarios en el ciclo de vida de los componentes de React. Si bien React proporciona hooks adicionales, próximamente nos centraremos en los hooks personalizados de React, lo que implica la creación de nuestros propios hooks adaptados a requisitos específicos.

Para ilustrar este concepto, aprovecharemos nuestra comprensión de `useState` y `useEffect` para crear un nuevo gancho personalizado llamado `useStorageState`. El objetivo principal de este gancho personalizado es asegurar la sincronización del estado de un componente con el almacenamiento local del navegador. Comenzaremos nuestra exploración describiendo cómo pretendemos utilizar este gancho en nuestro componente `App`:

`src/App.jsx`

```
const App = () => { const
  historias = [ ... ];

  const [término de búsqueda, establecertérmino de búsqueda] = useStorageState('React');

  const handleSearch = (evento) => {
    setSearchTerm(evento.objetivo.valor); };

  const searchedStories = historias.filter((historia) =>
    historia.título.toLowerCase().includes(searchTerm.toLowerCase()) );

  devolver (
    ...

  ); };
```

Con este gancho personalizado, podemos usarlo de forma similar al gancho `useState` nativo de React. Proporciona una variable de estado y una función para actualizar el estado, tomando un estado inicial como argumento.

La funcionalidad subyacente de este gancho se diseñará para garantizar la sincronización del estado con el almacenamiento local del navegador. Si observa con atención el componente `App` del fragmento de código anterior, podrá ver que ya no existen las funciones de almacenamiento local introducidas anteriormente.

En su lugar, copiaremos y pegaremos esta funcionalidad en nuestro nuevo gancho personalizado:

src/App.jsx

```
const useStorageState = () => { const
  [término de búsqueda, setTérmino de búsqueda] =
    React.useState( localStorage.getItem('buscar')
    || " ");

  React.useEffect(() =>
    { localStorage.setItem('buscar', término de búsqueda); },
    [término de
    búsqueda]); };

const Aplicación = () => {
  ...
};
```

Hasta ahora, este gancho personalizado es simplemente una función relacionada con los ganchos useState y useEffect que ya usamos en el componente App. Lo que falta es proporcionar un estado inicial y devolver los valores necesarios en nuestro componente App como un array:

src/App.jsx

```
const useStorageState = (estadoinicial) => {
  const [término de búsqueda, establecertérmino de búsqueda] =
    React.useState( localStorage.getItem('buscar') || estadoinicial );

  React.useEffect(() =>
    { localStorage.setItem('buscar', término de búsqueda); },
    [término de búsqueda]);

  devolver [término de búsqueda, establecertérmino de búsqueda];
```

Seguimos dos convenciones de los ganchos integrados de React. Primero, la convención de nomenclatura, que antepone el prefijo "use" al nombre de cada gancho. Segundo, los valores devueltos se devuelven como un array. Otro objetivo de un gancho personalizado es la reutilización. Todos los componentes internos de este gancho personalizado se centran en un dominio de búsqueda específico; sin embargo, para que el gancho personalizado sea reutilizable y, por lo tanto, genérico, debemos ajustar los nombres internos:



src/App.jsx

```
const useStorageState = (initialState) => { const [valor,
  setValue] = React.useState(
    localStorage.getItem('valor') || estadoinicial
  );
```

```
  React.useEffect(() =>
    { localStorage.setItem('valor', valor); }, [valor]);
```

```
  devolver [valor, valorEstablecer]; };
```

Ahora gestionamos un "valor" abstracto dentro del gancho personalizado. Al usarlo en el componente App, podemos asignar al estado actual devuelto y a la función de actualización de estado cualquier nombre relacionado con el dominio (por ejemplo, searchTerm y setSearchTerm) mediante la desestructuración de arrays.

Aún existe un problema con este gancho personalizado. Usarlo más de una vez en una aplicación React provoca la sobrescritura del elemento asignado al valor en el almacenamiento local, ya que usa la misma clave en dicho almacenamiento. Para solucionarlo, necesitamos pasar una clave flexible. Dado que la clave proviene de afuera, el gancho personalizado asume que podría cambiar, por lo que también debe incluirse en la matriz de dependencias del gancho useEffect. Sin ella, el efecto secundario podría ejecutarse con una clave obsoleta (también llamada obsoleta) si esta cambia entre renderizaciones.

src/App.jsx

```
const useStorageState = (clave, estadoinicial) => { const [valor,
  establecerValor] = React.useState(
    localStorage.getItem(clave) || estadoinicial
  );
```

```
  React.useEffect(() => {
    localStorage.setItem(clave, valor);
  }, [valor, clave]);
```

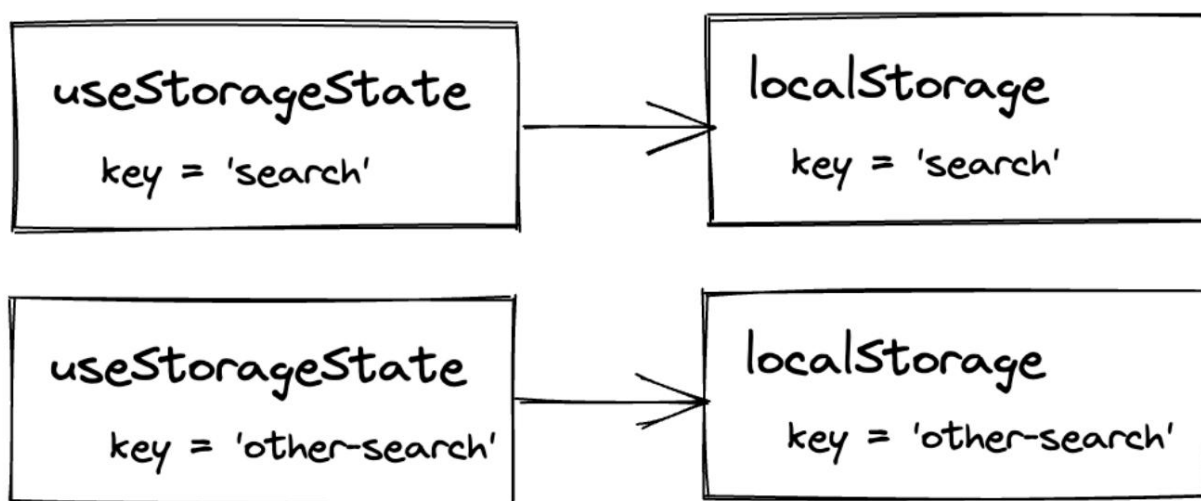
```
  devolver [valor, valorEstablecer]; };
```

```
const App = () => {
  ...
```

```
  const [término de búsqueda, establecertérmino de búsqueda] = useStorageState(
    'buscar',
    'Reaccionar'
```

```
);
...
};
```

Con la clave establecida, puede usar este nuevo gancho personalizado más de una vez en su aplicación. Solo necesita asegurarse de que el primer argumento, la clave que pasa, sea un identificador único que asigne el estado en el almacenamiento local del navegador bajo una clave única. Si usa la misma clave varias veces para varios usos del gancho `useStorageState`, todos estos ganchos funcionarán con el mismo par clave/valor del almacenamiento local.



Acabas de crear tu primer gancho personalizado. Si no te sientes cómodo con los ganchos personalizados, puedes revertir los cambios y usar los ganchos `useState` y `useEffect` como antes en el componente `App`.

Sin embargo, conocer los ganchos personalizados te ofrece muchas opciones nuevas. Un gancho personalizado puede encapsular detalles de implementación importantes que deberían mantenerse fuera de un componente, puede usarse en más de un componente de React, puede ser una combinación de otros ganchos e incluso puede publicarse como biblioteca externa. Si usas tu buscador favorito, verás que hay cientos de ganchos de React que puedes usar en tu aplicación sin preocuparte por los detalles de implementación.



Ceremonias:

- Compare su código fuente con el código fuente del autor¹.
 - Recapitulación de todos los cambios en el código fuente¹ de esta sección.
 - Opcional: Si estás usando TypeScript, consulta el código fuente del autor aquí¹¹.¹ <https://>

tinyurl.com/2j4adp2f

¹ <https://tinyurl.com/3y9sy3b7>

¹¹ <https://bit.ly/485dY9K>

- Leer más sobre [React Hooks¹¹¹](#) y [ganchos de React personalizados¹¹²](#) Para obtener una buena comprensión de ellos, porque son el pan y la mantequilla en los componentes de función de React.

Preguntas de la entrevista:

- Pregunta: ¿Qué son los ganchos personalizados de React?
 - Respuesta: Los ganchos personalizados son funciones de JavaScript que utilizan ganchos de React para encapsular y reutilizar la lógica en los componentes de la función.
- Pregunta: ¿Cómo se crea un gancho personalizado en React?
 - Respuesta: Crea una función que comience con "use" y usa ganchos de React existentes u otros personalizados. ganchos dentro de él.
- Pregunta: ¿Pueden los ganchos personalizados tener estado?
 - Respuesta: Sí, los ganchos personalizados pueden usar ganchos como useState.
- Pregunta: ¿Qué convención de nomenclatura deben seguir los ganchos personalizados?
 - Respuesta: Los ganchos personalizados deben nombrarse con el prefijo "use" para indicar su asociación con ganchos de React.
- Pregunta: ¿Pueden los ganchos personalizados aceptar parámetros?
 - Respuesta: Sí, los ganchos personalizados pueden aceptar parámetros para hacerlos flexibles y personalizables.
- Pregunta: ¿Cómo se comparte la lógica con estado entre componentes mediante ganchos personalizados?
 - Respuesta: Extraiga la lógica compartida en un gancho personalizado y úselo en múltiples componentes.
- Pregunta: ¿Los ganchos personalizados tienen acceso a las propiedades del componente?

Respuesta : No, los ganchos personalizados no tienen acceso directo a las propiedades del componente. Normalmente aceptan los datos necesarios mediante argumentos.

Pregunta: ¿Se pueden usar varios ganchos personalizados en un solo componente?

 - Respuesta: Sí, puedes usar varios ganchos personalizados en un solo componente para aprovechar diferentes piezas de lógica reutilizable.
- Pregunta: ¿Cuál es el beneficio clave de usar ganchos personalizados?
 - Respuesta: Los ganchos personalizados promueven la reutilización de código, la abstracción de lógica compleja y la capacidad de mantenimiento en los componentes de función de React.
- Pregunta: ¿Pueden los ganchos personalizados tener efectos secundarios como la obtención de datos?
 - Respuesta: Sí, los ganchos personalizados pueden encapsular efectos secundarios usando ganchos como useEffect para realizar tareas como la obtención de datos.
- Pregunta: ¿Los ganchos personalizados son solo para la gestión de estados?
 - Respuesta: No, si bien los ganchos personalizados pueden administrar el estado, pueden encapsular cualquier elemento reutilizable. lógica, incluidos efectos secundarios y cálculos.

¹¹¹<https://www.robinwieruch.de/react-hooks/>

¹¹²<https://www.robinwieruch.de/react-custom-hook/>

Fragmentos de React

Es posible que hayas notado que todos nuestros componentes React devuelven JSX con un elemento HTML de nivel superior. Cuando presentamos el componente de búsqueda hace un tiempo, tuvimos que agregar una etiqueta `<div>` (léase: elemento contenedor), porque de lo contrario los elementos de etiqueta y de entrada no podrían devolverse uno al lado del otro sin un elemento de nivel superior envolvente:

src/App.jsx

```
const Buscar = ({ buscar, onSearch }) => (  
  <div>  
    <label htmlFor="search">Buscar: </label>  
    <entrada  
      id="buscar"  
      tipo="texto"  
      valor={buscar}  
      onChange={onSearch} />  
  
  </div> );
```

Sin embargo, existen maneras de representar varios elementos de nivel superior en paralelo. Un enfoque poco común devuelve todos los elementos hermanos como un array. Dado que esto se asemeja a una lista de elementos, tendríamos que asignar a cada elemento de la lista un atributo clave obligatorio :

src/App.jsx

```
const Buscar = ({ buscar, onSearch }) => [  
  <label key="1" htmlFor="buscar">  
    Buscar: { ' ' } </  
  label>, <input  
  
    clave="2"  
    id="buscar"  
    tipo="texto"  
    valor={buscar}  
    onChange={onSearch} />, ];
```

Afortunadamente, existe otra forma de devolver elementos hermanos uno junto al otro sin un elemento de nivel superior, ya que el último enfoque con el array no resulta muy legible y se vuelve complejo con el atributo key adicional. Otra solución es usar un fragmento de React:

src/App.jsx

```
const Buscar = ({ buscar, onSearch }) => (
  <React.Fragment>
    <label htmlFor="search">Buscar: </label>
    <entrada
      id="buscar"
      tipo="texto"
      valor={buscar}
      onChange={onSearch}
    />
  </React.Fragment> );
```

Un fragmento envuelve elementos hermanos en un único elemento de nivel superior sin añadirlos a la salida renderizada. Compruébelo usted mismo inspeccionando los elementos en las herramientas de desarrollo de su navegador después de usar un fragmento en su componente de React. Una alternativa más popular actualmente es usar fragmentos en su versión abreviada:

src/App.jsx

```
const Buscar = ({ buscar, onSearch }) => (
  <>
    <label htmlFor="search">Buscar: </label>
    <entrada
      id="buscar"
      tipo="texto"
      valor={buscar}
      onChange={onSearch}
    />
  </>
);
```

Ambos elementos del componente de búsqueda (campo de entrada y etiqueta) deberían seguir visibles en tu navegador. Al fin y al cabo, si no quieres introducir un elemento intermedio que solo cumple funciones de React, puedes usar fragmentos como elementos auxiliares.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹¹³.
 - Recapitule todos los [cambios del código fuente](#)¹¹ de esta sección.
 - Opcional: Si estás usando TypeScript, consulta el código fuente del autor [aquí](#)¹¹. ¹¹³<https://tinyurl.com/mrf4hc3h>

¹¹ <https://tinyurl.com/3cmv4ump>
¹¹ <https://bit.ly/3HLzrtH>

Preguntas de la entrevista:

- Pregunta: ¿Qué es un fragmento en React?
 - Respuesta: Un Fragmento es una forma de agrupar múltiples elementos React sin introducir un elemento DOM adicional.
- Pregunta: ¿Cómo se utilizan fragmentos en JSX?
 - Respuesta: Envuelva los elementos con `<React.Fragment>` o su sintaxis abreviada `<>...</>`.
- Pregunta: ¿Por qué utilizar fragmentos en React?
 - Respuesta: Los fragmentos permiten agrupar elementos sin agregar nodos adicionales al DOM, lo que resulta útil cuando no se desea un contenedor principal.
- Pregunta: ¿Pueden los fragmentos tener claves?
 - Respuesta: Sí, los fragmentos pueden tener claves al asignarse a una lista de elementos.
- Pregunta: ¿Son necesarios los fragmentos en todos los componentes de React?
 - Respuesta: No, los fragmentos son opcionales y normalmente se utilizan cuando un componente necesita... devuelve múltiples elementos sin un contenedor padre.
- Pregunta: ¿Los fragmentos afectan la estructura HTML renderizada?
 - Respuesta: No, los fragmentos no introducen ningún nodo adicional a la estructura HTML.
- Pregunta: ¿Pueden los fragmentos tener atributos como clase o estilo?
 - Respuesta: No, los fragmentos no pueden tener atributos. Se deben aplicar atributos a los elementos dentro del Fragmento.
- Pregunta: ¿En qué se diferencia el uso de Fragmentos del uso de contenedores `div`?
 - Respuesta: Los fragmentos no crean un nodo DOM adicional, lo que proporciona una estructura HTML más limpia en comparación con el uso de contenedores `div`.

Componente de React reutilizable

Examine el componente de búsqueda con más detenimiento: Cada detalle de su implementación está estrechamente vinculado a la función de búsqueda. Sin embargo, internamente, el componente se compone únicamente de una etiqueta y una entrada. ¿Por qué debería estar tan estrechamente vinculado a un dominio único? Esta estrecha asociación lo hace menos adaptable a otras funcionalidades de la aplicación. En consecuencia, el componente de búsqueda resulta poco práctico para tareas no relacionadas con la búsqueda.

Además, el componente de búsqueda presenta el riesgo de introducir errores. Si se representan varias instancias de este componente en la misma página, su combinación `htmlFor/id` se duplica. Esta duplicación interrumpe el enfoque cuando el usuario hace clic en una de las etiquetas. Para solucionar estos problemas, mejoremos la reutilización del componente de búsqueda.

Dado que el componente de búsqueda carece de funcionalidad de búsqueda propiamente dicha, reutilizarlo para diversas funciones de la aplicación requiere un esfuerzo mínimo. Podemos lograrlo introduciendo propiedades de `id` y `etiqueta` dinámicas en el componente de búsqueda, renombrando el valor específico y el controlador de devolución de llamada con términos más genéricos y, en consecuencia, renombrando el componente en sí:

src/App.jsx

```
const Aplicación = () => {
  ...

  devolver (
    <div>
      Mis historias de hackers

      <EntradaConEtiqueta
        id="buscar"
        etiqueta="Buscar"
        valor={término de
          búsqueda} onChange={manejarBúsqueda}
      />

      ...
    </div>

  ); };

const InputWithLabel = ({ id, etiqueta, valor, onChange }) => (
  <>
    <label htmlFor={id}>{etiqueta}</label>

    <entrada
```

```

      id={id}
      tipo="texto"
      valor={valor}
      onChange={onInputChange} /> </>
    );

```

Si bien es totalmente reutilizable, su aplicabilidad se limita al uso de una entrada con texto. Para ampliar su alcance y admitir tipos de entrada adicionales, como números (number) o números de teléfono (tel), el atributo de tipo del campo de entrada también debe ser accesible desde el exterior:

src/App.jsx

```

const InputWithLabel = ({
  id,
  etiqueta,
  valor,
  tipo = 'texto',
  onInputChange, })
=> (
  <>
    <label htmlFor={id}>{etiqueta}</label> <input id={id}

    tipo={tipo}

    valor={valor}

    onChange={onInputChange} />

  </>
);

```

Debido a que no pasamos una propiedad de tipo del componente App al componente InputWithLabel, el **parámetro predeterminado**¹¹ La firma de la función reemplaza al tipo. Por lo tanto, cada vez que se use el componente InputWithLabel sin una propiedad de tipo, el tipo predeterminado será "texto".

Con solo unos pocos cambios, convertimos un componente de búsqueda especializado en un componente InputWith-Label más reutilizable. Generalizamos la nomenclatura de los detalles de implementación interna y ampliamos la API del nuevo componente para proporcionar toda la información necesaria desde el exterior. Aún no utilizamos el componente en otros entornos, pero aumentamos su capacidad para realizar la tarea si lo hacemos.

¹¹ <https://mzl.la/3aUefkN>

Siempre se trata de un equilibrio entre la generalización y la especialización de los componentes. En este caso, convertimos un componente altamente especializado en uno generalizado. Mientras que un componente generalizado tiene más posibilidades de reutilizarse en la aplicación, un componente especializado implementaría la lógica de negocio para un caso de uso específico y, por lo tanto, no es reutilizable.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹¹.
 - Recapitule todos los [cambios del código fuente](#)¹¹ de esta sección.
 - Opcional: Si usas TypeScript, consulta el código fuente del autor [aquí](#)¹¹. • Lee más sobre [los componentes reutilizables de React](#)¹² y crea algunos de estos componentes tú mismo: – [Botón en React](#)¹²¹, [Botón de opción en React](#)¹²², [Casilla de verificación en React](#)¹²³, [Menú desplegable en React](#)¹², [Lista de arrastrar y soltar en React \(avanzado\)](#)¹², ...
- Antes usábamos el texto "Buscar:" con un ":". ¿Cómo lo manejarías ahora? ¿Lo pasarías con label="Buscar:" como propiedad al componente InputWithLabel o lo codificarías después de usar <label htmlFor={id}>{label}</label> en el componente InputWithLabel? Veremos cómo hacerlo más adelante.

Preguntas de la entrevista:

- Pregunta: ¿Por qué es importante la reutilización en React?
 - Respuesta: La reutilización promueve la eficiencia, la capacidad de mantenimiento y la consistencia del código al permitir que los componentes se utilicen en varias partes de una aplicación.
 - Pregunta: ¿Cómo puedes hacer que un componente React sea reutilizable?
 - Respuesta: Haga que los componentes sean más genéricos mediante el uso de accesorios para un comportamiento personalizable y asegurándose de que no estén estrechamente acoplados a funcionalidades específicas.
 - Pregunta: ¿Cómo contribuyen los accesorios de React a la reutilización?
 - Respuesta: Los accesorios hacen que los componentes sean adaptables y reutilizables al permitir una personalización dinámica.
 - Pregunta: ¿Puede un componente reutilizable tener estado interno?
 - Respuesta: Sí, un componente reutilizable puede tener un estado interno mediante el uso de un gancho como useState.
 - Pregunta: ¿Por qué es importante la abstracción de componentes para la reutilización?
 - Respuesta: La abstracción oculta detalles innecesarios, lo que hace que los componentes sean más versátiles y fáciles de reutilizar sin exponer sus complejidades internas. • Pregunta: ¿Es aconsejable hacer que todos los componentes de una aplicación React sean reutilizables?
- Respuesta : Si bien la reutilización es beneficiosa, no todos los componentes deben ser reutilizables. A menudo, hay componentes que tienen un único propósito y solo se usan una vez en una aplicación React.

¹¹ <https://tinyurl.com/2y5xk95h>

¹¹ <https://tinyurl.com/5n6zzbys>

¹¹ <https://bit.ly/3Oun6hn>

¹² <https://www.robinwieruch.de/react-reusable-components/>

¹²¹<https://www.robinwieruch.de/boton-de-reaccion/>

¹²²<https://www.robinwieruch.de/boton-de-radio-de-reaccion/>

¹²³<https://www.robinwieruch.de/react-checkbox/>

¹² <https://www.robinwieruch.de/react-dropdown/>

¹² <https://www.robinwieruch.de/react-drag-and-drop/>

Composición de componentes de React

En esencia, una aplicación React consiste en un conjunto de componentes React dispuestos en forma de árbol. Al aprender a inicializar componentes como elementos en JSX, habrás visto cómo se usan como cualquier otro elemento HTML en JSX. Sin embargo, hasta ahora solo los hemos usado como etiquetas de cierre automático. ¿Y si también hubiera una etiqueta de apertura y otra de cierre para los elementos de React? Entrando en el concepto de composición de componentes:

src/App.jsx

```
const App = () => {  
  ...  
  
  return (  
    <div>  
      Mis historias de hackers  
  
      <EntradaConEtiqueta  
        id="buscar"  
        valor={término de  
          búsqueda} onChange={manejarBúsqueda}  
      >  
        Buscar:  
      </EntradaConEtiqueta>  
  
      ...  
    </div>  
  
  );  
};
```

La composición de componentes es una de las funciones más potentes de React. En esencia, descubriremos cómo usar un elemento React de la misma forma que un elemento HTML, aprovechando sus etiquetas de apertura y cierre. En el ejemplo anterior, en lugar de usar la propiedad "label", insertamos el texto "Buscar:" entre las etiquetas del componente. En el componente InputWithLabel, ahora se puede acceder a esta información mediante la propiedad "children" de React. En lugar de usar la propiedad "label", use la propiedad "children" para renderizar todo lo que se ha renderizado entre las etiquetas de apertura y cierre <InputWithLabel> :

src/App.jsx

```

constante EntradaConEtiqueta = ({
  identificación,
  valor,
  tipo = 'texto',
  onInputChange,
  hijos, }) => (

  <>
    <label htmlFor={id}>{hijos}</label>
    &nbsp;
    <entrada
      id={id}
      tipo={tipo}
      valor={valor}
      onChange={onInputChange} />

  </>

);

```

Ahora, los elementos del componente React se comportan de forma similar al HTML nativo. Todo lo que se pasa entre los elementos de un componente se puede acceder como elementos secundarios en el componente y renderizarse. A veces, cuando se usa un componente React, se desea tener más libertad desde el exterior con respecto a qué renderizar en el interior de un componente:

src/App.jsx

```

constante Aplicación = () => {
  ...

  devolver (
    <div>
      Mis historias de hackers

      <EntradaConEtiqueta
        id="buscar"
        valor={término de
          búsqueda} onChange={manejarBúsqueda}
      >
        <strong>Buscar:</strong>
      </EntradaConEtiqueta>

    ...
  )
}

```

```
</div>
```

```
); };
```

Con la propiedad `React children`, podemos integrar componentes React entre sí. La hemos usado con una cadena y con una cadena dentro de un elemento HTML `<strong>`, pero no termina aquí. También puedes pasar elementos React a través de `React children`, algo que definitivamente deberías explorar más a fondo. ejercicio.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹².
 - Recapitule todos los [cambios del código fuente](#)¹² de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)¹².
- Lea más sobre [la composición de componentes en React](#)¹².

Preguntas de la entrevista:

- Pregunta: ¿A qué se refiere “niños” en los componentes de React?
 Respuesta: En React, “hijos” se refiere al contenido ubicado entre las etiquetas de apertura y cierre de un componente. Pregunta: ¿Cómo se puede acceder y renderizar a los “hijos” de un componente de React?
 – Respuesta: Utilice la propiedad `props.children` para acceder y representar el contenido ubicado dentro de un componente.
- Pregunta: ¿Puede un componente React tener varios hijos?
 – Respuesta: Sí, pero utilizaría atributos en su lugar, como `<MyComponent slotOne={<em>1</em>} slotTwo={<em>2</em>} />` al que se puede acceder a través de `props.slotOne` y `props.slotTwo` en el componente.
- Pregunta: ¿Se pueden pasar componentes de React como “hijos” a otro componente?
 – Respuesta: Sí, los componentes de React se pueden pasar como “hijos” a otros componentes, lo que permite la componibilidad.
- Pregunta: ¿Cuál es el propósito de la utilidad `React.Children` en React?
 – Respuesta: La utilidad `React.Children` proporciona métodos para trabajar con y manipular los “hijos” de un componente React.
- Pregunta: ¿Cómo se itera y se manipula cada hijo en un componente React?
 – Respuesta: Use `React.Children.map` o `React.Children.forEach` para iterar y realizar operaciones en cada elemento secundario de un componente.
- Pregunta: ¿Es posible tener un componente sin elementos secundarios en React?
 – Respuesta: Sí, un componente puede existir sin ningún “hijo” al no colocar contenido entre sus etiquetas de apertura y cierre.
- Pregunta: ¿Cuál es la diferencia entre “hijos” y otras propiedades en React?
 – Respuesta: “Hijos” se refiere específicamente al contenido entre etiquetas, mientras que otras propiedades son pares clave-valor que se pasan a un componente.

¹² <https://tinyurl.com/4ubprvu6>

¹² <https://tinyurl.com/5h2svtpk>

¹² <https://bit.ly/3uvSg0S>

¹² <https://www.robinwieruch.de/composición-de-componentes-de-react/>

Reaccionar imperativo

La **programación imperativa** implica proporcionar instrucciones explícitas paso a paso que detallan cómo un programa debe realizar una tarea. En cambio, la **programación declarativa** se centra en especificar el resultado deseado sin especificar cada paso del procedimiento. El código declarativo suele considerarse más conciso, legible y fácil de mantener.

React utiliza un enfoque de programación declarativa. En lugar de manipular manualmente el Modelo de Objetos del Documento (DOM) para actualizar la interfaz de usuario, los desarrolladores declaran el estado deseado de la interfaz y React gestiona el proceso de renderizado. Este enfoque declarativo mejora la legibilidad y la escalabilidad del código, simplificando las complejidades de la manipulación del DOM y permitiendo la creación de interfaces de usuario dinámicas con un mayor nivel de abstracción.

Al implementar JSX, le indicas a React qué elementos quieres ver, no cómo crearlos. Al implementar un gancho para el estado, le indicas a React qué quieres gestionar como valor con estado, no cómo hacerlo. Y al implementar un controlador de eventos, no es necesario asignar un oyente imperativamente:

Patio de juegos de código

```
// elemento imperativo JavaScript + API
```

```
DOM.addEventListener('click', () => {  
  // hacer algo });
```

```
// React declarativo const
```

```
App = () => { const  
  handleClick = () => { // hacer algo };
```

```
  devolver (  
    <botón  
      tipo="botón"  
      onClick={handleClick}  
    >  
      Hacer clic  
    </botón>
```

```
  ); };
```

Sin embargo, hay casos en los que no queremos que todo sea declarativo. Por ejemplo, a veces se desea tener acceso imperativo a los elementos renderizados, generalmente como efecto secundario, en casos como estos:

- acceso de lectura/escritura a elementos a través de la API DOM:
 - leer (aquí: medir) el ancho o la altura de un elemento – escribir (aquí: establecer) el estado de foco de un campo de entrada
- implementación de animaciones más complejas: – configuración de transiciones – orquestación de transiciones
- integración de bibliotecas de terceros:
 - [D3¹³](https://d3js.org) es una biblioteca de gráficos imperativos popular

Debido a la verbosidad y la naturaleza contraintuitiva de la programación imperativa en React, solo exploraremos un breve ejemplo que ilustra cómo establecer imperativamente el foco de un campo de entrada. Por el contrario, con el enfoque declarativo, se puede lograr el mismo resultado estableciendo el atributo `autoFocus` del campo de entrada:

src/App.jsx

```
const InputWithLabel = ({ ... }) => (
  <>
    <label htmlFor={id}>{hijos}</label> <input id={id}
      tipo={tipo}
      valor={valor}
      Enfoque automático
      onChange={onInputChange} /> </>
  </>
);
```

// tenga en cuenta que `autoFocus` es una abreviatura de `autoFocus={true}` // cada atributo que se establece en `true` puede usar esta abreviatura



Esto funciona, pero solo si se renderiza uno de los componentes reutilizables. Por ejemplo, si el componente `App` renderiza dos componentes `InputWithLabel`, solo el último componente renderizado recibe la bandera `autoFocus` en su renderizado. Sin embargo, dado que tenemos un componente React reutilizable, podemos pasar una propiedad dedicada que permite al desarrollador decidir si el campo de entrada debe tener un valor activo.

Enfoque automático:

¹³ <https://d3js.org>

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  devolver (  
    <div>  
      Mis historias de hackers  
  
      <EntradaConEtiqueta  
        id="buscar"  
        valor={término de búsqueda}  
        está enfocado  
        onChange={manejarBúsqueda}  
      >  
        <strong>Buscar:</strong>  
      </EntradaConEtiqueta>  
  
      ...  
    </div>  
  
  );  
};
```

Nuevamente, usar solo `isFocused` como atributo equivale a `isFocused={true}`. Dentro del componente, use la propiedad `new` para el atributo `autoFocus` del campo de entrada :

src/App.jsx

```
constante InputWithLabel = ({  
  //identificación  
  valor,  
  tipo = 'texto',  
  onChange,  
  isFocused,  
  hijos, }) => (  
  
  <>  
    <label htmlFor={id}>{hijos}</label>  
  
    <input  
      id={id}  
      tipo={tipo}  
      valor={valor}  
      autoFocus={isFocused}
```

```

      onChange={onInputChange}
    />
  </>
);

```

La función funciona, pero es una implementación declarativa. Le indicamos a React qué hacer, no cómo hacerlo. Aunque es posible hacerlo con el enfoque declarativo (el recomendado), refactoricemos este escenario a un enfoque imperativo. Queremos ejecutar el método `focus()` programáticamente en el elemento del campo de entrada a través de la API del DOM una vez renderizado:

src/App.jsx

```

const EntradaConEtiqueta = ({
  // Identificación
  valor,
  tipo = 'texto',
  onInputChange,
  isFocused,
  hijos, }) =>
{ // Una

  const inputRef = React.useRef();

  // C
  React.useEffect(() => { if
    (isFocused && inputRef.current) {
      // D
      inputRef.current.focus();
    }
  }, [isFocused]);

  devolver (
    <>
      <label htmlFor={id}>{hijos}</label>

      { /* B */ }
      <input
        ref={inputRef} id={id}
        tipo={tipo}
        valor={valor}

        onChange={onInputChange} /> </>
    </>
  );
}

```

```
); };
```

Todos los pasos esenciales están marcados con comentarios que se explican paso a paso:

- (A) Primero, crea una referencia con el [gancho useRef de React](#). Este objeto de referencia es un valor persistente que permanece intacto durante la vida útil de un componente de React. Incluye una propiedad llamada "current" que, a diferencia del objeto de referencia, se puede modificar.
- (B) En segundo lugar, la referencia se pasa al atributo de referencia reservado para JSX del elemento y, por lo tanto, el elemento La instancia se asigna a la propiedad actual modificable.
- (C) En tercer lugar, opte por el ciclo de vida de React con el Hook useEffect de React, realizando el enfoque en el Elemento cuando el componente se renderiza (o sus dependencias cambian).
- (D) Y cuarto, dado que la referencia se pasa al atributo ref del elemento, su propiedad actual da acceso al elemento. Ejecutar su enfoque programáticamente como efecto secundario, pero solo si isFocused está definido y la propiedad actual existe.

Básicamente, este es el ejemplo completo de cómo pasar de la programación declarativa a la imperativa en React. [En este caso, es posible usar tanto el enfoque declarativo como el imperativo, como ya has experimentado. Sin embargo, no siempre es posible usar el enfoque declarativo, por lo que el imperativo puede implementarse siempre que sea necesario.](#) Dado que no hemos abordado ref y useRef con mucho detalle aquí, debido a que es una característica poco utilizada en React, sugiero leer el artículo adicional de los ejercicios para una comprensión más profunda.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹³¹.
 - Recapitular todos los [cambios del código fuente](#)¹³² de esta sección.
 - Opcional: Si usas TypeScript, consulta el código fuente del autor [aquí](#)¹³³. • Lee más sobre [referencias en](#)

[React](#)¹³ y opcionalmente revisa los siguientes tutoriales que son

usando referencias:

- [Crea una imagen desde un componente React con una referencia](#)¹³
- [Crea un componente Slider con una referencia](#)¹³
- [Crea un gancho personalizado con una referencia](#)¹³

- Lea más sobre [por qué son importantes los marcos](#)¹³.

¹³¹<https://tinyurl.com/4hz7a5rv>

¹³²<https://tinyurl.com/y8s3esu2>

¹³³<https://bit.ly/494zUDw>

¹³ <https://www.robinwieruch.de/react-ref/>

¹³ <https://www.robinwieruch.de/react-component-to-image/>

¹³ <https://www.robinwieruch.de/react-slider/>

¹³ <https://www.robinwieruch.de/react-custom-hook-check-if-overflow/>

¹³ <https://www.robinwieruch.de/por-que-importan-los-marcos/>

Preguntas de la entrevista:

- Pregunta: ¿Qué es useRef en React?

- Respuesta: useRef es un gancho en React que proporciona un objeto mutable llamado ref, que puede contener un valor mutable y persiste entre renderizados. •

- Pregunta: ¿En qué se diferencia useRef de useState en React?

- Respuesta: A diferencia de useState, useRef no activa una nueva representación cuando su valor cambia.

- Pregunta: ¿Se puede usar useRef para contener un valor mutable que persista en las representaciones?

- Respuesta: Sí, el propósito principal de useRef es mantener valores mutables que persisten a través de se renderiza sin provocar nuevas

- renderizaciones. • Pregunta: ¿Cuál es el caso de uso común para useRef en React?

- Respuesta: Un caso de uso común es acceder e interactuar con el DOM, ya que useRef puede contener una referencia a un elemento DOM. •

- Pregunta: ¿Se puede usar useRef para activar nuevas representaciones en React?

- Respuesta: No, cambiar el valor de una referencia creada con useRef no activa una nueva representación.

- Pregunta: ¿Se puede usar useRef para conservar valores entre llamadas de función?

- Respuesta: Sí, los valores useRef persisten entre representaciones, lo que los hace adecuados para persistir valores entre llamadas de función sin activar nuevas representaciones.

- Pregunta: ¿Cómo se puede acceder al valor actual de una referencia creada con useRef?

- Respuesta: Utilice myRef.current para acceder al valor actual de una referencia creada con useRef.

Controlador en línea en JSX

En esta sección, aprenderá sobre los controladores en línea como un nuevo bloque de construcción fundamental en React.

Simultáneamente, implementaremos nuestra próxima función que permite la eliminación de elementos de la lista.

Antes de continuar, siéntete libre de intentar esta tarea por tu cuenta, ya que puede resolverse sin conocimientos previos de controladores en línea. A continuación, se proporcionan instrucciones paso a paso. Si encuentras dificultades, consulta esta sección para encontrar la solución. Si encuentras una solución por tu cuenta, compárala con la del libro, que utiliza controladores en línea.

Tarea: La aplicación muestra una lista de elementos y permite a sus usuarios filtrar la lista a través de una función de búsqueda.

A continuación, la aplicación debe mostrar un botón junto a cada elemento de la lista que permita a sus usuarios eliminar el elemento de la lista.

Consejos opcionales:

- La lista de elementos debe convertirse en un valor con estado (en este caso: una matriz con estado) con `useState` para poder manipularla posteriormente (por ejemplo, eliminar un elemento).
- Cada elemento de la lista genera un botón con un controlador de clic. Al hacer clic en el botón, el elemento obtiene... eliminado de la lista manipulando el estado.
- Dado que la lista con estado reside en el componente Aplicación, es necesario utilizar controladores de devolución de llamada para permitir que el componente Elemento se comunice con el componente Aplicación para eliminar un elemento por su identificador.

Ahora queremos ver cómo implementar esta función paso a paso. Actualmente, la lista de elementos (en este caso: historias) que tenemos en nuestro componente App es una variable sin estado. Podemos filtrar la lista renderizada con la función de búsqueda, pero la lista en sí permanece intacta. La lista filtrada es simplemente un estado derivado de un tercero (en este caso: `searchTerm`), pero aún no manipulamos la lista real. Para controlar la lista, conviértala en estado usándola como estado inicial en el gancho `useState` de React. Los valores devueltos por el array son el estado actual (historias) y la función de actualización de estado (`setStories`).

src/App.jsx

```
const HistoriasIniciales = [
  {
    título: 'React', url:
    'https://react.dev', autor: 'Jordan
    Walke', num_comentarios: 3,
    puntos: 4,

    ID de objeto:

  }, {
    título: 'Redux', URL:
    'https://redux.js.org',
```

```

    autor: 'Dan Abramov, Andrew Clark',
    num_comentarios: 2,
    puntos: 5,
    objectID: 1, }, ],
  ];

```

...

```

const App = () => { const
  [término de búsqueda, establecertérmino de búsqueda] = ·

  const [historias, setStories] = React.useState(initialStories);

  ...

};

```

La aplicación se comporta igual porque las historias, ahora devueltas como una lista con estado desde el gancho useState de React , se filtran en searchedStories y se muestran en el componente List. Solo ha cambiado el origen de las historias.

Sin embargo, aún no las estamos modificando.

A continuación, escribiremos un controlador de eventos que elimine un elemento de la lista:

src/App.jsx

```

const Aplicación = () => {
  ...

  const [historias, setStories] = React.useState(initialStories);

  const handleRemoveStory = (item) => { const
    newStories = stories.filter( (story) =>
      item.objectID !== story.objectID );

    setStories(nuevasHistorias); };

  ...

  devolver (
    <div>
      Mis historias de hackers

      ...

```

```

    <hr />

    <Lista lista={Historias buscadas} onRemoveItem={handleRemoveStory} />
  </div>

);

```

El controlador de devolución de llamada del componente App, que se usará eventualmente en los componentes List/Item, recibe como argumento el elemento que debe eliminarse de la lista. Con base en esta información, la función filtra las historias actuales eliminando todos los elementos que no cumplen su condición. Las historias devueltas, donde se ha eliminado el elemento deseado (historia), se configuran con un nuevo estado y se pasan al componente List. Dado que se configura un nuevo estado, el componente App y todos los componentes inferiores (por ejemplo, los componentes List/Item) se renderizarán de nuevo y mostrarán el nuevo estado de las historias.

Sin embargo, falta comprender cómo los componentes Lista/Elemento utilizan esta nueva funcionalidad que modifica el estado en el componente Aplicación. El componente Lista en sí no utiliza este nuevo controlador de devolución de llamada, sino que solo lo pasa al componente Elemento:

src/App.jsx

```

constante Lista = ({ lista, onRemoveItem }) => (
  <ul>
    {lista.map((elemento) => (
      <Articulo
        clave={item.objectID}
        elemento={item}
        onRemoveItem={onRemoveItem} /> ))}
  </ul>
);

```

Finalmente, el componente Item usa el controlador de devolución de llamada entrante como una función en un nuevo controlador. En este controlador, le pasaremos el elemento específico. Además, se necesita un elemento de botón adicional para activar el evento:

src/App.jsx

```
const Item = ({ item, onRemoveItem }) => { const
  handleRemoveItem = () => { onRemoveItem(item); };

  devolver (
    <li>
      <span>
        <a href={item.url}>{item.title}</a>

        <span>{item.author}
        <span>{item.num_comments} <span>{item.points}
        <span>

        <button type="button" onClick={manejarEliminarElemento}>
          Despedir
        </botón> </
      li> ); };

```

Hasta ahora en esta sección, hemos configurado la lista de historias con estado mediante el gancho `useState` de React, hemos transferido las historias que aún se buscan como propiedades al componente `List` e implementado un controlador de devolución de llamada (`handleRemoveStory`) y un controlador (`handleRemoveItem`) para usar en sus respectivos componentes y eliminar una historia al hacer clic en un botón. Para implementar esta función, aplicamos diversas lecciones aprendidas anteriormente: estado, propiedades, controladores y controladores de devolución de llamada. La función funciona y es posible que usted haya encontrado la misma solución o una similar.

Hablemos de los controladores en línea: Quizás hayas notado que tuvimos que añadir un controlador adicional `"handleRemoveItem"` en el componente `Item`, encargado de ejecutar el controlador de devolución de llamada `"onRemoveItem"`. Tuvimos que añadir este controlador de eventos adicional para tomar el elemento como argumento para el controlador de devolución de llamada.

Si desea que esto sea más elegante, puede usar un controlador en línea que le permita ejecutar la función del controlador de devolución de llamada en el componente `Item` directamente en el `JSX`. Hay dos soluciones que utilizan la función entrante `onRemoveItem` en el componente `Item` como controlador en línea.

Primero, usando el método `bind` de JavaScript:

src/App.jsx

```
const Item = ({ item, onRemoveItem }) => (
  <li>
    <span>
      <a href={item.url}>{item.title}</a>

      <span>{item.author}
      <span>{item.num_comments}
      <span>{item.points}
      <span>
        <button type="button" onClick={onRemoveItem.bind(null, item)}>
          Despedir
        </botón>
      </span>
    </li>

  );
```

Usando el método `bind` de JavaScript¹³ En una función, podemos vincular directamente los argumentos que se deben usar al ejecutarla. El método "bind" devuelve una nueva función con el argumento vinculado. Por el contrario, la segunda solución, y la más popular, es usar una función de flecha en línea, que permite introducir argumentos como el elemento:

src/App.jsx

```
const Item = ({ item, onRemoveItem }) => (
  <li>
    <span>
      <a href={item.url}>{item.title}</a>

      <span>{item.author}
      <span>{item.num_comments}
      <span>{item.points}
      <span>
        <button type="button" onClick={() => onRemoveItem(item)}>
          Despedir
        </botón>
      </span>
    </li>

  );
```

Si bien usar un controlador en línea es más conciso que usar un controlador de eventos normal, también puede ser más difícil de depurar porque la lógica de JavaScript puede estar oculta en JSX. Se vuelve aún más complejo si

¹³ <https://mzl.la/3ncEkBu>

La función de flecha en línea **encapsula más de una línea de lógica de implementación** al utilizar un cuerpo de bloque en lugar de un cuerpo conciso:

Patio de juegos de código

```
constante Item = ({ item, onRemoveItem }) => (
  <li>
    ...
    <span>
      <botón
        type="button"
        onClick={() => { //
          hacer otra cosa

          // nota: evite usar lógica compleja en JSX

          onRemoveItem(elemento); }}
      >
      Despedir
    </botón>
  </li>
);
```

Como regla general: se pueden usar controladores en línea si no ocultan detalles críticos de la implementación. Si los controladores en línea necesitan usar un cuerpo de bloque porque se ejecuta más de una línea de código, es hora de extraerlos como controladores de eventos normales. Al fin y al cabo, en este caso todas las versiones de los controladores son legibles y, por lo tanto, aceptables.

Ceremonias:

- Compare su código fuente con el código **fuentes** del autor¹.
 - Recapitule todos los **cambios del código fuente**¹ ¹ de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor **aquí**¹ ².
- Lea más sobre cómo **agregar**¹ ³, **actualizar**¹, **eliminar**¹ elementos en una lista. • Lea más sobre **las propiedades calculadas en React**¹. • Revisar **controladores, controladores de devolución de llamadas y controladores en línea**¹.

¹ <https://tinyurl.com/y55aphew> ¹ <https://tinyurl.com/ymakpvaw> ¹ ²<https://bit.ly/3UwzjWz> ¹ ³<https://www.robinwieruch.de/react-add-item-to-list> ¹ <https://www.robinwieruch.de/react-update-item-in-list/> ¹ <https://www.robinwieruch.de/react-remove-item-from-list> ¹ <https://www.robinwieruch.de/react-computed-properties/> ¹ <https://www.robinwieruch.de/react-event-handler/>

Preguntas de la entrevista:

- Pregunta: ¿Qué es una función en línea en React?
 - Respuesta: Una función en línea en React se usa a menudo como una función definida directamente dentro de la JSX.
- Pregunta: ¿Cuál es la ventaja de utilizar funciones en línea para los controladores de eventos en React?
 - Respuesta: Las funciones en línea permiten pasar parámetros adicionales fácilmente.
- Pregunta: ¿Cuál es la alternativa al uso de funciones en línea para los controladores de eventos en React?
 - Respuesta: Crear funciones de controlador fuera del método de renderizado y pasar referencias a ellas puede ser una alternativa.
- Pregunta: ¿Cuál es la sintaxis para crear una función en línea en un controlador de eventos React JSX?
 - Respuesta: Utilice la sintaxis de la función de flecha directamente dentro del atributo del controlador de eventos, como `onClick={() => miFuncion()}`.

Datos asincrónicos de React

Tenemos dos interacciones en nuestra aplicación: buscar en la lista y eliminar elementos de la lista.

Mientras que la primera interacción es una modificación fluctuante a través de un estado de terceros (searchTerm) aplicado en la lista, la segunda interacción es una eliminación irreversible de un elemento de la lista.

Sin embargo, la lista que manejamos sigue siendo solo datos de muestra. ¿Qué tal si preparamos nuestra aplicación para que gestione datos reales?

Normalmente, los datos de un backend/base de datos remoto llegan de forma asíncrona a aplicaciones del lado del cliente como React. Por lo tanto, a menudo es necesario renderizar un componente antes de iniciar la obtención de datos. A continuación, comenzaremos simulando este tipo de datos asíncronos con nuestros datos de ejemplo en la aplicación. Posteriormente, reemplazaremos los datos de ejemplo con datos reales obtenidos de una API remota real. Comenzamos con una función que devuelve una promesa con datos en su versión abreviada una vez resuelta. El objeto resuelto contiene la lista anterior de historias:

src/App.jsx

```
const initialStories = [ ... ];
```

```
const getAsyncStories = () =>
```

```
  Promise.resolve({ datos: { historias: initialStories } });
```

En el componente de la aplicación, en lugar de utilizar initialStories, utilice una matriz vacía para el estado inicial. Queremos comenzar con una lista vacía de historias y simular la obtención de estas historias de forma asincrónica.

En un nuevo gancho useEffect, llame a la función y resuelva la promesa devuelta como un efecto secundario. Debido a que la matriz de dependencias está vacía, el efecto secundario solo se ejecuta la primera vez que el componente se renderiza:

src/App.jsx

```
const App = () => {
```

```
  ...
```

```
  const [historias, setStories] = React.useState([]);
```

```
  React.useEffect(() =>
```

```
    { getAsyncStories().then(resultado =>
```

```
      { setStories(resultado.data.stories); }); }, []);
```

```
  ...
```

```
};
```

Aunque los datos deberían llegar de forma asíncrona al iniciar la aplicación, parecen llegar de forma síncrona, ya que se procesan inmediatamente. Para cambiar esto, asignemos un pequeño retraso realista, ya que cada solicitud de red a una API remota tendría un retraso. Primero, eliminemos la versión abreviada de la promesa:

src/App.jsx

```
const getAsyncStories = () => nueva
  Promesa((resolver) =>
    resolver({ datos: { historias: initialStories } }));
```

Y segundo, al resolver la promesa, retrasarla 2 segundos:

src/App.jsx

```
const getAsyncStories = () => new
  Promise((resolver) =>
    setTimeout( ()
      => resolve({ datos: { historias: initialStories } })),
    2000

  );
```

Al reiniciar la aplicación, debería ver una renderización diferida de la lista. El estado inicial de las historias es un array vacío, por lo que no se renderiza nada en el componente Lista. Tras renderizar el componente Aplicación, el gancho de efectos secundarios se ejecuta una vez para obtener los datos asíncronos. Tras resolver la promesa y configurar los datos en el estado del componente, este se renderiza de nuevo y muestra la lista de historias cargadas asincrónicamente.

Esta sección fue solo el primer paso hacia los datos asincrónicos en React. En lugar de tener los datos desde el principio, los resolvimos asincrónicamente a partir de una promesa. Sin embargo, solo migramos nuestras historias de datos síncronos a asincrónicos. Sin embargo, siguen siendo datos de muestra y aprenderemos a obtener datos reales con el tiempo.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹.
 - Recapitule todos los [cambios del código fuente](#)¹ de esta sección.
 - Opcional: Si usas TypeScript, consulta el código fuente del autor [aquí](#)¹.
- Opcional: Lee más sobre [las promesas de JavaScript](#)¹. • Lee más sobre [cómo simular una API remota con JavaScript](#)¹ ².
- Lea más sobre [el uso de datos simulados en React](#)¹ ³.

¹ <https://tinyurl.com/jv3xcym3>

¹ <https://tinyurl.com/y4m3j9jb>

¹ <https://bit.ly/3StsfHt>

¹ <https://mzl.la/3aTGuz1> ²[https://](https://www.robinwieruch.de/javascript-fake-api/)

<www.robinwieruch.de/javascript-fake-api/> ¹ ³<https://www.robinwieruch.de/react-mock-data/>

Preguntas de la entrevista:

- Pregunta: ¿Por qué es común el manejo de datos asincrónicos en las aplicaciones React?

– Respuesta: Las aplicaciones React del lado del cliente suelen obtener datos de fuentes remotas. •

Pregunta: ¿Cuál es el enfoque típico para renderizar componentes antes de obtener datos en React?

– Respuesta: Los componentes a menudo se procesan antes de iniciar la obtención de datos y los condicionales se utiliza el contenido renderizado o de marcador de posición hasta que llegan

los datos. • Pregunta: ¿Cómo se puede simular la obtención de datos asincrónica en React usando datos de muestra?

– Respuesta: Simular datos asincrónicos implica el uso de funciones que devuelven promesas, resolver con datos de muestra y luego reemplazarlos con datos reales. • Pregunta:

¿Cuál es el propósito de las promesas en el manejo de datos asincrónicos en React?

– Respuesta: Las promesas se utilizan para gestionar operaciones asincrónicas, lo que permite que los componentes esperen la resolución de los datos antes de

renderizarlos. • Pregunta: ¿Por qué la obtención asincrónica de datos es esencial para las interfaces de usuario responsivas en React?

– Respuesta: La obtención asincrónica de datos evita el bloqueo de la interfaz de usuario, lo que garantiza la capacidad de respuesta y permite mostrar información actualizada cuando esté disponible. •

Pregunta: ¿Se pueden reemplazar los datos de muestra simulados con datos reales obtenidos de una API remota en

¿Reaccionar?

– Respuesta: Sí, después de simular datos asincrónicos con datos de muestra, se pueden reemplazar sin problemas con datos reales obtenidos de una API remota. •

Pregunta: ¿Cuál es la importancia de usar el gancho useState cuando se trabaja con datos asincrónicos?

¿datos en React?

– Respuesta: useState permite que los componentes administren cambios de estado, incluidos los estados de carga y los datos actualizados se recibieron de forma asincrónica.

- Pregunta: ¿Cómo garantiza React que los componentes se vuelvan a renderizar cuando llegan datos asincrónicos?

– Respuesta: La gestión de estado de React garantiza que cuando llegan datos asincrónicos y se actualiza el estado, los componentes se vuelven a renderizar para reflejar los nuevos datos.

Representación condicional de React

Presentamos una nueva función relacionada con el manejo asíncrono de datos, recientemente introducido. En una aplicación genuina, los usuarios suelen recibir retroalimentación, como un indicador de carga, mientras se cargan los datos. En esta sección, nuestro objetivo es implementar este mecanismo de retroalimentación. Siéntase libre de intentar implementarlo de forma independiente y, posteriormente, puede consultar el libro para comparar su solución con la implementación proporcionada.

Tarea: Cargar los datos de muestra de la promesa lleva un tiempo. Durante este tiempo, se debe mostrar al usuario un indicador de carga en su forma más simple (por ejemplo, un texto que diga "Cargando..."). Una vez que los datos lleguen de forma asíncrona, oculte el indicador de carga.

Consejos opcionales:

- Para mostrar un indicador de carga, sería necesario introducir un nuevo valor con estado. A El valor booleano llamado `isLoading` puede ser la mejor solución.
- Cuando se activa el efecto secundario que carga los datos, establezca el booleano con estado en verdadero. Una vez que datos cargados, establezca el booleano con estado en falso nuevamente.
- En JSX, muestra un texto "Cargando..." condicionalmente cuando el booleano `isLoading` se establece en verdadero.

En React, siempre se produce un renderizado condicional si necesitamos renderizar diferentes JSX según la información (p. ej., estado, propiedades). Gestionar datos asíncronos es un buen caso práctico para usar el renderizado condicional. Por ejemplo, cuando la aplicación se inicializa por primera vez, no hay datos con los que empezar. A continuación, cargamos los datos y, finalmente, los tenemos disponibles para mostrarlos. A veces, la obtención de datos falla y recibimos un error. Por lo tanto, como desarrolladores, tenemos muchos aspectos que abordar.

Afortunadamente, algunos aspectos ya están controlados. Por ejemplo, dado que el estado inicial es una lista vacía `[]` en lugar de nulo, se reducen las preocupaciones sobre la interrupción de la aplicación al filtrar o mapear sobre esta lista. Sin embargo, ciertos aspectos aún requieren atención. Considere la ausencia de un estado de carga para proporcionar a los usuarios información sobre las solicitudes de datos pendientes. La introducción de un nuevo valor con estado puede solucionar esto, permitiéndonos establecer el estado según corresponda al recuperar los datos:

src/App.jsx

```
const App = () => {
  ...

  const [historias, establecerHistorias] = React.useState([]); const
  [isLoading, establecerIsLoading] = React.useState(false);

  React.useEffect(() => {
    { setIsLoading(true);

    getAsyncStories().then((resultado) => {
```

```
    setStories(resultado.datos.historias);
    setIsLoading(falso); }); }, []);

...
};
```

El booleano debería estar activado correctamente ahora. Lo que falta es mostrar al usuario el indicador de carga. Una solución sencilla sería usar un retorno anticipado en el componente de la aplicación:

src/App.jsx

```
constante Aplicación = () => {
  ...

  si (estáCargando)
  { devolver <p>Cargando ...</p>;
  }

  devolver (
    <div>
      ...
    </div>
  );
};
```

Sin embargo, de esta forma solo se mostraría el indicador de carga y nada más. En su lugar, **queremos integrar el indicador de carga en el JSX para mostrarlo o el componente Lista.**

Sin embargo, no se recomienda utilizar una declaración if-else incorporada en JSX debido a las limitaciones de JSX en este caso. (Aunque puedes intentarlo como ejercicio). Sin embargo, **puedes utilizar un operador ternario¹ en su lugar y producir una representación condicional en JSX** de esta manera:

¹ <https://mzl.la/3vAPKCL>

src/App.jsx

```

const App = () => {
  ...

  return (
    <div>
      ...

      <hr />

      {isLoading ? (
        <p>Cargando...</p> ) : (

        <Lista
          lista={historiasBuscadas}
          onRemoveItem={handleRemoveStory} /> )}

    </div>

  );
};

```

Eso es todo. Estás renderizando condicionalmente un indicador de carga o el componente Lista según un booleano con estado. A continuación, implementemos también la gestión de errores para los datos asíncronos. En nuestro entorno simulado no se producen errores, pero podrían producirse si empezamos a obtener datos de una API remota. Por lo tanto, introduzca otro estado para la gestión de errores y trátelo en el bloque catch() de la promesa al resolverla:

src/App.jsx

```

const App = () => {
  ...

  const [historias, establecerHistorias] = React.useState([]); const
  [isLoading, establecerIsLoading] = React.useState(false); const [isError,
  establecerIsError] = React.useState(false);

  React.useEffect(() => {
    setIsLoading(true);

    getAsyncStories().then((resultado) => {
      setStories(resultado.datos.historias);
    });
  });
};

```

```

        setIsLoading(falso); }) .catch(()

=> setIsError(verdadero)); }, []);

...
};

```

A continuación, proporcione al usuario retroalimentación en caso de que algo salga mal con otra representación condicional. Esta vez, se representa algo o nada. Por lo tanto, en lugar de usar un operador ternario donde un lado devuelve nulo, use el operador lógico && como abreviatura:

src/App.jsx

```

const App = () => {
  ...

  return (
    <div>
      ...

      <hr />

      {isError && <p>Algo salió mal...</p>}

      {isLoading ?
        ( <p>Cargando ...</p> ) :
        (
          ...
        )}
    </div>

  );
};

```

En JavaScript, un && "Hola Mundo" verdadero siempre se evalúa como "Hola Mundo". Un && "Hola Mundo" falso siempre se evalúa como falso. En React, podemos aprovechar este comportamiento. Si la condición es verdadera, la expresión después del operador lógico && será la salida. Si la condición es falsa, React la ignora y omite la expresión. Usar la expresión && en JSX es más conciso que usar la expresión ? JSX: nulo.

Sin embargo, el renderizado condicional no se limita a datos asíncronos. El ejemplo más simple es un estado booleano que se activa con un botón. Si el indicador booleano es verdadero, se renderiza algo; si es falso, no se renderiza nada. Conocer esta función en React puede ser muy útil.

Porque te permite renderizar JSX condicionalmente. Es una herramienta más de React para hacer tu interfaz de usuario más dinámica. Y, como hemos descubierto, suele ser necesaria para flujos de control más complejos, como datos asíncronos.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹.
 - Recapitulación de todos los [cambios en el código fuente](#)¹ de esta sección.
 - Opcional: si usas TypeScript, consulta el código fuente del autor [aquí](#)¹. • Lee más sobre [la representación condicional en React](#)¹.

Preguntas de la entrevista:

- Pregunta: ¿Por qué no necesitábamos una representación condicional para las historias vacías antes de que se volvieran obsoletas?
¿Obtenido de la API falsa?
 - Respuesta: Las historias se mapean como lista en el componente Lista usando el método `map()`.
Al mapear una lista, el método `map()` devuelve una versión modificada (JSX) para cada elemento. Si no hay elementos en la lista, el método `map()` no devolverá nada.
Por lo tanto, no necesitamos una representación condicional aquí. • Pregunta:
- ¿Qué pasaría si el estado inicial de las historias se estableciera en nulo en lugar de `[]`?
 - Respuesta: Entonces necesitaríamos una representación condicional en el componente Lista, porque
Llamar a `map()` en `null` arrojaría una excepción. • Pregunta: ¿Cómo se puede representar contenido condicionalmente según el estado usando `useState`?
 - Respuesta: Utilice declaraciones condicionales (por ejemplo, `if` o el operador ternario) en JSX según el estado valor.
- Pregunta: ¿Puedes usar `useState` (u otros ganchos) dentro de declaraciones condicionales o bucles?
 - Respuesta: No, los ganchos deben usarse en el nivel superior de un componente de función, no dentro de condiciones o bucles.
- Pregunta: ¿Por qué se utiliza a menudo la representación condicional cuando se manejan datos asíncronos en `React`?
¿Reaccionar?
 - Respuesta: La representación condicional ayuda a administrar la visualización de la interfaz de usuario en función del estado de los datos asíncronos, mostrando indicadores de carga o contenido real según sea necesario.
- Pregunta: ¿Cómo se manejan los estados de carga mientras se esperan datos asíncronos en `React`?
 - Respuesta: Los estados de carga se gestionan mediante renderizado condicional o variables de estado, lo que indica a los usuarios que se están obteniendo datos. • Pregunta: ¿Se pueden gestionar errores durante la obtención de datos asíncrona en `React`?
 - Respuesta: Sí, se pueden implementar mecanismos de manejo de errores, como bloques `try...catch` o `.catch` con promesas, para gestionar errores durante la obtención de datos.

¹ <https://tinyurl.com/4mxfr88j>

¹ <https://tinyurl.com/354u7hrn>

¹ <https://bit.ly/>

3Ss5RP1 ¹ <https://www.robinwieruch.de/conditional-rendering-react/>

Estado avanzado de React

Toda la gestión de estados en esta aplicación hace un uso intensivo del Hook `useState` de React. Por otro lado, el Hook `useReducer` de React permite una gestión de estados más sofisticada para estructuras y transiciones de estados complejas. Dado que el conocimiento sobre reductores en JavaScript divide a la comunidad, no cubriremos los conceptos básicos aquí. Sin embargo, si no conoces los reductores, consulta esta [guía sobre reductores en JavaScript](https://www.robinwieruch.de/javascript-reducer/)¹.

En esta sección, trasladaremos las historias con estado del gancho `useState` de React al gancho `useReducer` de React. Usar `useReducer` en lugar de `useState` tiene sentido cuando varios valores con estado dependen entre sí o están relacionados con un dominio. Por ejemplo, las historias, `isLoading` y `error` están relacionados con la obtención de datos. En una versión más abstracta, las tres podrían ser propiedades de un objeto complejo (p. ej., `data`, `isLoading`, `error`) gestionado por un reductor. Abordaremos esto en una sección posterior. En esta sección, comenzaremos a gestionar las historias y sus transiciones de estado en un reductor.

Primero, introduce una función reductora fuera de tus componentes. Una función reductora siempre recibe un estado y una acción. Con base en estos dos argumentos, un reductor siempre devuelve un nuevo estado:

`src/App.jsx`

```
const getAsyncStories = () => nueva
  Promesa((resolver) => ... );

const storiesReducer = (estado, acción) => { if (acción.type
  === 'SET_STORIES') { return acción.payload; }
  else { throw new Error();

}};
```

Una acción del reductor siempre se asocia a un tipo y, como práctica recomendada, a una carga útil. Si el tipo coincide con una condición del reductor, se devuelve un nuevo estado basado en el estado y la acción entrantes. Si el reductor no lo cubre, se genera un error para recordar que la implementación no está cubierta.

La función `storiesReducer` cubre un tipo y luego devuelve la carga útil de la acción entrante sin usar el estado actual para calcular el nuevo estado. Por lo tanto, el nuevo estado es simplemente la carga útil.

En el componente `App`, intercambia `useState` por `useReducer` para gestionar las historias. El nuevo gancho recibe una función reductora y un estado inicial como argumentos, y devuelve una matriz con dos elementos. El primer elemento es el estado actual y el segundo elemento es la función de actualización del estado (también llamada función de despacho):

¹ <https://www.robinwieruch.de/javascript-reducer/>

src/App.jsx

```
constante Aplicación = () => {
  ...

  const [historias, dispatchStories] = React.useReducer( storiesReducer,
    [] );

  ...
};
```

La nueva función de despacho puede usarse en lugar de la función `setStories` , que anteriormente devolvía `useState`. En lugar de establecer el estado explícitamente con la función de actualización de estado de `useState`, la función de actualización de estado `useReducer` establece el estado implícitamente mediante el envío de una acción para el reductor. La acción incluye un tipo y una carga útil opcional :

src/App.jsx

```
constante Aplicación = () => {
  ...

  React.useEffect(() =>
    { setIsLoading(true);

    getAsyncStories() .then((resultado)
      => { dispatchStories({ tipo:
        'SET_STORIES', carga útil:
        resultado.data.stories, });

        setIsLoading(false); }) .catch(()
          => setIsError(verdadero)); }, []);

  const handleRemoveStory = (item) => { const
    newStories = stories.filter( (story) =>
      item.objectID !== story.objectID );

    dispatchStories({ tipo:
      'SET_STORIES',
```

```
      carga útil:

      nuevasHistorias, }); });

    ...

  };
};
```

La aplicación se ve igual en el navegador, aunque un reductor y el gancho `useReducer` de React gestionan el estado de las historias. Vamos a simplificar el concepto de reductor gestionando más de una transición de estado. Si solo hay una transición de estado, un reductor no tendría sentido.

Hasta ahora, el controlador `handleRemoveStory` calcula las nuevas historias. Es válido trasladar esta lógica a la función del reductor y gestionarlo con una acción, lo cual **constituye otro ejemplo de transición de la programación imperativa a la declarativa**. En lugar de hacerlo nosotros mismos indicando cómo debe hacerse, le indicamos al reductor qué hacer. Todo lo demás está oculto en el reductor:

```
src/App.jsx

const App = () => {
  ...

  const handleRemoveStory = (item) => {
    dispatchStories({ tipo:
      'REMOVE_STORY', carga
      útil: item, }); });

  ...

};
```

Ahora, la función reductora debe cubrir este nuevo caso en una nueva transición de estado condicional. Si se cumple la condición para eliminar una historia, el reductor cuenta con todos los detalles de implementación necesarios para eliminarla. La acción proporciona toda la información necesaria (en este caso, el identificador de un elemento) para eliminar la historia del estado actual y devolver una nueva lista de historias filtradas como estado:

src/App.jsx

```
const storiesReducer = (estado, acción) => { if (acción.type ===  
  'SET_STORIES') { return acción.payload;  
  
  } de lo contrario si (acción.tipo === 'REMOVE_STORY') { devolver  
    estado.filtro( (historia) =>  
      acción.carga útil.objectID !== historia.objectID ); } de lo contrario { lanzar nuevo  
  
  Error();  
  
  } };
```

Todas estas sentencias if-else acabarán congestionando al añadir más transiciones de estado a una función reductora. Refactorizarlas a una sentencia **switch** para todas las transiciones de estado las hace más legibles y se consideran una buena práctica en la comunidad de React:

src/App.jsx

```
const storiesReducer = (estado, acción) => {  
  cambiar (acción.tipo) { caso  
    'SET_STORIES':  
      devolver acción.payload;  
    caso 'REMOVE_STORY':  
      devolver estado.filtro( (historia)  
        => acción.carga útil.objectID !== historia.objectID );  
  
    por defecto:  
      lanzar nuevo Error();  
  
  } };
```

Hemos cubierto una versión mínima de un reductor en JavaScript y su uso en React con la ayuda del gancho `useReducer` de React. El reductor abarca dos transiciones de estado, muestra cómo calcular el estado actual y la acción en un nuevo estado, y utiliza lógica de negocio (eliminación de una historia) para una transición de estado. Ahora podemos establecer una lista de historias como estado para los datos que llegan de forma asíncrona y eliminar una historia de la lista con un solo reductor de gestión de estado y su gancho `useReducer` asociado. Para comprender completamente el concepto de reductores en JavaScript y el uso del gancho `useReducer` de React, consulte los recursos enlazados en los ejercicios. Continuaremos ampliando nuestra implementación de un reductor en la siguiente sección.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹ .
 - Recapitulación de todos los [cambios en el código fuente](#)¹ de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)².
- Lea más sobre [los reductores y el uso de Reducer en React](#)³.

Extraiga los tipos de acción 'SET_STORIES' y 'REMOVE_STORY' como variables y reutilícelos en las funciones reducer y de despacho. De esta forma, evitará errores tipográficos en sus tipos de acción.

Preguntas de la entrevista:

- Pregunta: ¿Qué es useReducer en React?
 - Respuesta: useReducer es un gancho de React que gestiona la lógica de estado compleja en la función componentes mediante el envío de acciones para actualizar el estado.
- Pregunta: ¿En qué se diferencia useReducer de useState en React?
 - Respuesta: Mientras que useState es más sencillo para gestionar variables de estado individuales, useReducer es más adecuado para una lógica de estado compleja donde varios valores dependen unos de otros. • Pregunta: ¿Cuál es la estructura básica del gancho useReducer?
 - Respuesta: Devuelve el estado actual y una función de despacho para activar actualizaciones de estado, tomando una función reductora y un estado inicial como argumentos.
- Pregunta: ¿Qué es una función reductora en useReducer?
 - Respuesta: La función reductora se encarga de especificar cómo debe cambiar el estado en respuesta a las acciones enviadas, en función del estado actual y la acción. • Pregunta: ¿Cómo se actualiza el estado usando useReducer?
 - Respuesta: El estado se actualiza al enviar acciones, y la función reductora determina el nuevo estado según el estado actual y el tipo de acción. • Pregunta: ¿Puede useReducer reemplazar todos los casos de uso de useState en React?
 - Respuesta: Si bien useReducer es potente, no es necesario para todos los escenarios. useState es más simple y más adecuado para administrar variables de estado individuales.

¹ <https://tinyurl.com/5xuuzdj2>

¹ <https://tinyurl.com/bdffdf8>

² <https://bit.ly/3OwtVie>

³ <https://www.robinwieruch.de/react-usereducer-hook/>

Reaccionar estados imposibles

Quizás hayas notado una desconexión entre los estados individuales del componente App al usar varios Hooks `useState` de React. Técnicamente, todos los estados relacionados con los datos asíncronos están relacionados, lo que no solo incluye las historias como datos reales, sino también sus estados de carga y error.

Aquí es donde un reductor y el gancho `useReducer` de React entran en juego para gestionar los estados relacionados con el dominio. Pero ¿por qué debería importarnos?

No hay problema con varios ganchos `useState` en un componente de React. Sin embargo, tenga cuidado si ve varias funciones de actualización de estado seguidas. Estos estados condicionales pueden generar estados imposibles y comportamientos no deseados en la interfaz de usuario. Intente cambiar su pseudofunción de obtención de datos a la siguiente implementación para simular un error y, por lo tanto, nuestra implementación de gestión de errores:

src/App.jsx

```
constante getAsyncStories = () =>
  nueva Promesa((resolver, rechazar) => setTimeout(rechazar, 2000));
```

El estado imposible ocurre cuando se produce un error en los datos asíncronos. El estado del error se establece, pero el del indicador de carga no se revoca. En la interfaz de usuario, esto generaría un indicador de carga infinito y un mensaje de error, aunque sería mejor mostrar solo el mensaje de error y ocultar el indicador de carga. Los estados imposibles no son fáciles de detectar, lo que los hace conocidos por causar errores en la interfaz de usuario. Puedes intentar solucionar este error tú mismo.

Afortunadamente, podemos mejorar nuestras posibilidades de evitar estos errores moviendo estados que pertenecen juntos desde varios ganchos `useState` (y `useReducer`) a un solo gancho `useReducer`. Consideremos los siguientes ganchos:

src/App.jsx

```
constante Aplicación = () => {
  ...

  const [historias, dispatchStories] = React.useReducer( storiesReducer,
    [] ); const [isLoading,

  setIsLoading] = React.useState(false); const [isError, setIsError] =
  React.useState(false);

  ...
};
```

Y fusionarlos en un solo gancho `useReducer` para una gestión de estado unificada que abarca un objeto de estado más complejo y, eventualmente, transiciones de estado más complejas:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  const [historias, dispatchStories] = React.useReducer( storiesReducer,  
    { datos: [], isLoading:  
      false, isError: false } );  
  
  ...  
};
```

Dado que ya no podemos utilizar las funciones de actualización de estado de los hooks `useState` de React, todo lo relacionado con la obtención de datos asincrónicos debe utilizar la nueva función de despacho para las transiciones de estado. El enfoque más sencillo consiste en intercambiar la función de actualización de estado con la función de despacho. Esta última recibe un tipo específico y una carga útil. Esta última se asemeja a la misma carga útil que habríamos usado para actualizar el estado con una función de actualización de estado:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  const [historias, dispatchStories] = React.useReducer( storiesReducer,  
    { datos: [], isLoading:  
      false, isError: false } );  
  
  React.useEffect(() => {  
    dispatchStories({ tipo: 'STORIES_FETCH_INIT' });  
  
    getAsyncStories().then((resultado)  
      => { dispatchStories({ tipo:  
        'STORIES_FETCH_SUCCESS', carga  
        útil: resultado.datos.historias, }); } ).catch(()  
      =>  
  
        dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); }, []);
```

```
...  
};
```

Cambiamos dos aspectos de la función reductora. Primero, introducimos nuevos tipos al llamar a la función de despacho desde el exterior. Por lo tanto, necesitamos añadir nuevos casos para las transiciones de estado. Segundo, cambiamos la estructura de estado de un array a un objeto complejo. Por lo tanto, debemos considerar el nuevo objeto complejo como estado de entrada y estado de retorno:

src/App.jsx

```
const storiesReducer = (estado, acción) => {  
  cambiar (acción.tipo) {  
    caso 'STORIES_FETCH_INIT':  
      devolver {  
        ...estado,  
        isLoading: verdadero,  
        isError: falso, };  
  
    caso 'HISTORIAS_OBTENCIÓN_ÉXITO':  
      devolver {  
        ...estado,  
        isLoading: falso, isError:  
        falso, datos:  
        acción.carga útil, };  
  
    caso 'FALLO DE BÚSQUEDA DE HISTORIAS':  
      devolver {  
        ...estado,  
        isLoading: falso, isError:  
        verdadero, };  
  
    caso 'REMOVE_STORY':  
      return  
      { ...estado,  
        datos: estado.datos.filtro(  
          (historia) => acción.carga útil.objectID !== historia.objectID ), };  
  
    por defecto:  
      lanzar nuevo Error();  
  
  }};
```

Para cada transición de estado, devolvemos un nuevo objeto de estado que contiene todos los pares clave/valor del objeto de estado actual (mediante el operador de propagación de JavaScript) y las nuevas propiedades de sobrescritura. Por ejemplo, `STORIES_FETCH_FAILURE` establece el booleano `isLoading` en falso y el booleano `isError` en verdadero, manteniendo intactos los demás estados (por ejemplo, el alias de datos "historias"). Así solucionamos el error mencionado anteriormente, ya que un error debería establecer el booleano `loading` a falso.

Observe cómo también cambió la acción `REMOVE_STORY`. Opera sobre el archivo `state.data`, y ya no solo sobre el estado simple. El estado es un objeto complejo con datos, `isLoading` y estados de error, en lugar de una simple lista de historias. Esto debe solucionarse en el código restante, direccionando el estado como un objeto y no como un array:

`src/App.jsx`

```
const Aplicación = () => {
  ...

  const [historias, dispatchStories] = React.useReducer( storiesReducer,
    { datos: [], isLoading:
      false, isError: false } );

  ...

  const searchedStories = historias.data.filter((historia) =>
    historia.titulo.toLowerCase().includes(searchTerm.toLowerCase()) );

  devolver (
    <div>
      ...

      {stories.isError && <p>Algo salió mal...</p>}

      {historias.se está cargando?
        ( <p>Cargando ...</p> ) :
        (
          <Lista
            lista={HistoriasBuscadas}
            onRemoveItem={handleRemoveStory} /> ) } </

    <div>

  ); }
```

Intente utilizar nuevamente la función de obtención de datos erróneos y verifique si todo funciona como se espera ahora:

src/App.jsx

```
constante getAsyncStories = () =>
  nueva Promesa((resolver, rechazar) => setTimeout(rechazar, 2000));
```

Pasamos de transiciones de estado poco fiables con múltiples ganchos `useState` a transiciones de estado predecibles con el gancho `useReducer` de React. El objeto de estado gestionado por el reductor encapsula todo lo relacionado con la obtención de historias, incluyendo los estados de carga y error, así como detalles de implementación como la eliminación de una historia. No eliminamos por completo los estados imposibles, ya que aún es posible omitir una bandera booleana crucial como antes, pero nos acercamos a una gestión de estados más predecible.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹.
 - Recapitulación de todos los [cambios en el código fuente](#)¹ de esta sección.
 - Opcional: si usas TypeScript, consulta el código fuente del autor [aquí](#)¹. • Lee más sobre [cuándo usar `useState` o `useReducer` en React](#)¹. • Lea más sobre [cómo derivar el estado a partir de propiedades en React](#)¹.

Preguntas de la entrevista:

- Pregunta: ¿Qué son los "estados imposibles" en React?
 - Respuesta: Los "estados imposibles" se refieren a combinaciones de valores de estado que nunca deberían ocurrir en una aplicación bien diseñada. • Pregunta: ¿Por qué es importante manejar estados imposibles en aplicaciones React?
 - Respuesta: El manejo de estados imposibles garantiza que la aplicación permanezca en un estado válido y estado predecible, evitando comportamientos inesperados. • Pregunta: ¿Cómo pueden los reductores ayudar a gestionar estados imposibles en React?
 - Respuesta: Los reductores proporcionan una forma controlada de actualizar el estado, lo que permite a los desarrolladores aplicar reglas y evitar la transición a estados imposibles.
 - Pregunta: ¿Cómo contribuye la gestión centralizada del Estado a gestionar estados imposibles?
 - Respuesta: La gestión centralizada del estado a través de reductores permite una validación y actualizaciones de estado consistentes, lo que reduce el riesgo de estados imposibles en los componentes.

¹ <https://tinyurl.com/u9dx8a3j>

¹ <https://tinyurl.com/38a5rjnc>

¹ <https://bit.ly/42nPe1>

¹ <https://www.robinwieruch.de/react-usereducer-vs-usestate/>

¹ <https://www.robinwieruch.de/react-derive-state-props/>

Obtención de datos con React

Configuramos todo para la obtención asincrónica de datos en React. Sin embargo, seguimos usando pseudodatos provenientes de una promesa que configuramos nosotros mismos para una API falsa. Aun así, todo lo aprendido hasta ahora sobre React asincrónico y la gestión avanzada de estados nos estaba preparando para obtener datos de una API remota real de terceros. En esta sección, usaremos la API informativa de Hacker News¹ para solicitar noticias tecnológicas populares.

Si ya sabes cómo obtener datos en JavaScript, puedes intentar realizar la siguiente tarea tú mismo y consultar más adelante la implementación del libro. Sin embargo, no dudes en continuar con el libro, ya que es una tarea compleja.

Tarea: La aplicación utiliza datos asíncronos, pero pseudodatos, de una promesa (API falsa). En lugar de usar la función `getAsyncStories()`, utilice la API de Hacker News para obtener los datos.

Consejos opcionales:

- Utilice este punto final de API <https://hn.algolia.com/api/v1/search?query=React> del Hacker API de noticias.
- Elimine la variable `initialStories`, ya que estos datos provendrán de la API. • Utilice la [API de búsqueda nativa](#) del navegador¹ para realizar la solicitud. • Nota: Una solicitud exitosa o errónea utiliza la misma lógica de implementación que ya utilizamos. tener en su lugar.

Comenzamos con una base sólida para la obtención de datos asincrónicos, ya que todo está listo. Lo único que nos impide usar la solución es usar datos de muestra en lugar de datos reales. Por lo tanto, el siguiente fragmento de código muestra todo lo necesario para conectarse a una API remota. En lugar de usar la matriz `initialStories` y la función `getAsyncStories`, que ahora se pueden eliminar, obtendremos los datos directamente de la API:

```
src/App.jsx
// Una
constante API_ENDPOINT = 'https://hn.algolia.com/api/v1/search?query=';

constante Aplicación = () => {
  ...

  React.useEffect(() => {
    dispatchStories({ tipo: 'STORIES_FETCH_INIT' });

    buscar(`${API_ENDPOINT}react`) // B
      .then((respuesta) => respuesta.json()) // C
```

¹ <https://hn.algolia.com/api>

¹ <https://mzl.la/2Z1kyjU>

```

.then((resultado) =>
  { dispatchStories({ tipo:
    'STORIES_FETCH_SUCCESS', carga útil:
    resultado.hits, // D }); }) .catch(() =>

dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); }, []);

...
};

```

En primer lugar, se utiliza `API_ENDPOINT` (A) para obtener historias tecnológicas populares para una determinada consulta (un término de búsqueda). En este caso, recuperamos historias sobre React (B). En segundo lugar, se utiliza la [API de recuperación del navegador nativo](#)¹ para realizar esta solicitud (B). Para la API de recuperación, la respuesta debe traducirse a JSON (C). Finalmente, el resultado devuelto tiene una estructura de datos diferente (D), que enviamos como carga útil al reductor de estado de nuestro componente.

En el ejemplo de código anterior, usamos [los literales de plantilla de JavaScript](#)² para una interpolación de cadenas. Cuando esta función no estaba disponible en JavaScript, habríamos utilizado el operador `+` en las cadenas:

Patio de juegos de código

```

const saludo = 'Hola';

// + operador const
welcome = saludo + console.log(welcome); Reaccionar;
Hola React

// literales de plantilla const
anotherWelcome = `${greeting} React`; console.log(anotherWelcome); //
Hola React

```

Revise su navegador para ver historias relacionadas con la consulta inicial obtenida de la API de Hacker News. Dado que usamos la misma estructura de datos para las historias de ejemplo, no tuvimos que cambiar nada en el componente `Item`. Aún es posible filtrar las historias después de obtenerlas con la función de búsqueda, ya que aún tienen la propiedad `title`. Sin embargo, cambiaremos este comportamiento en una de las siguientes secciones.

¹ <https://mzl.la/2Z1kyjU>

² <https://mzl.la/3jlcVfn>

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹ ³.
 - Recapitule todos los [cambios del código fuente](#)¹ de esta sección.
 - Opcional: Si usas TypeScript, consulta el código fuente del autor [aquí](#)¹. • Lee [Hacker News](#)¹ y su [API](#)¹. • Opcional: Lee más sobre [los literales de plantilla de JavaScript](#)¹.

Preguntas de la entrevista:

- Pregunta: ¿Por qué es común obtener datos de las API en las aplicaciones React?
 - Respuesta: Obtener datos de las API permite que las aplicaciones React recuperen y muestren dinámicamente información de fuentes externas.
- Pregunta: ¿Cuál es el propósito del gancho `useEffect` en React cuando se trabaja con API?

Respuesta : `useEffect` se usa para ejecutar efectos secundarios, como la obtención de datos, en componentes de función. Garantiza que el efecto se ejecute después del renderizado.
- Pregunta: ¿Cómo se gestionan las llamadas asíncronicas a la API en los componentes de React?
 - Respuesta: Las llamadas API asíncronicas generalmente se manejan utilizando la sintaxis `async/await` o promesas dentro del gancho `useEffect`. •
- Pregunta: ¿Puedes realizar operaciones de limpieza después de solicitudes de API usando `useEffect` en React?
 - Respuesta: Sí, `useEffect` permite realizar operaciones de limpieza, como cancelar solicitudes pendientes o borrar suscripciones, al devolver una función de limpieza. • Pregunta:
- ¿Cuál es el propósito del segundo argumento (matriz de dependencia) en `useEffect` cuando se trabaja con API?

Respuesta : La matriz de dependencias controla cuándo se ejecuta el efecto. Especificar dependencias garantiza que el efecto se vuelva a ejecutar solo cuando estas cambien.
- Pregunta: ¿Cómo se manejan los errores durante las solicitudes de API en React?
 - Respuesta: Los errores durante las solicitudes de API se pueden manejar utilizando bloques `try...catch`, `.catch` con promesas o estableciendo variables de estado de error.
- Pregunta: ¿Cuál es la importancia de la gestión del estado cuando se trabaja con datos de API en React?
 - Respuesta: La gestión del estado permite a los componentes de React almacenar y actualizar los datos recuperados desde las API, lo que activa nuevas representaciones cuando es necesario. • Pregunta: ¿Puedes explicar el concepto de rebote de solicitudes de API en React?
- Respuesta: La eliminación de rebotes implica retrasar la ejecución de solicitudes de API para reducir la cantidad de solicitudes realizadas en un período breve, generalmente para mejorar el rendimiento y evitar límites de velocidad.
- Pregunta: ¿Por qué es importante manejar los estados de carga al realizar solicitudes de API en React?
 - Respuesta: El manejo de los estados de carga proporciona retroalimentación a los usuarios mientras se obtienen los datos, lo que mejora la experiencia del usuario e indica los procesos en segundo plano en curso.

¹ ³<https://tinyurl.com/ymsrby6u>

¹ <https://tinyurl.com/bdhrpae4>

¹ <https://bit.ly/3SztLZ0>

¹ <https://news.ycombinator.com>

¹ <https://hn.algolia.com/api>

¹ <https://mzl.la/3jlcVfn>

Recuperación de datos en React

Ahora, tenemos datos recuperados de una API remota, lo que proporciona un entorno más atractivo en comparación con los datos de muestra. Si iniciaste tu aplicación después de la sección anterior, podrías haber notado que le faltaba integridad. Dado que obtenemos los datos con una consulta predefinida (en este caso: "react"), vemos constantemente historias relacionadas con "React". A pesar de tener una función de búsqueda, esta solo puede filtrar historias existentes. Por lo tanto, esta función se denomina búsqueda del lado del cliente porque opera únicamente con los datos disponibles, sin interactuar con la API remota.

Mientras que una búsqueda del lado del cliente solo filtra las historias disponibles en el cliente (tras la obtención inicial de datos), una búsqueda del lado del servidor nos permitiría obtener datos de la API remota según el término de búsqueda. En esencia, la búsqueda del lado del cliente y la del lado del servidor difieren en el lugar donde se realiza la operación. La búsqueda del lado del cliente se realiza en el dispositivo del usuario, lo que proporciona respuestas rápidas, pero puede ser menos adecuada para grandes conjuntos de datos. La búsqueda del lado del servidor se realiza en el servidor, lo que es más adecuado para grandes conjuntos de datos, pero puede resultar en tiempos de respuesta más lentos para el usuario debido a las interrupciones del servidor. La elección depende de factores como el tamaño del conjunto de datos, la complejidad de la búsqueda y el rendimiento. En esta sección, queremos cambiar la búsqueda del lado del cliente a una búsqueda del lado del servidor. Intente realizar la siguiente tarea usted mismo de nuevo antes de continuar leyendo el libro.

Tarea: La función de búsqueda es una búsqueda del lado del cliente, porque filtra solo los datos que ya están allí.

En su lugar, debería ser posible utilizar la búsqueda para obtener datos relacionados con el término de búsqueda.

Consejos opcionales:

- El valor calculado `searchedStories` se puede omitir ya que anticipamos datos filtrados directamente desde la API.
- En el proceso de recuperación de datos, reemplace el 'react' codificado con el `searchTerm` dinámico . • Aborde el caso extremo donde `searchTerm` es una cadena vacía.

Migrar la aplicación de una búsqueda del lado del cliente a una del lado del servidor no requiere muchos pasos. Primero, elimine `searchedStories`, ya que recibiremos las historias filtradas por término de búsqueda desde la API. Pase solo las historias regulares al componente `Lista`:

`src/App.jsx`

```
const App = () => {
  ...

  devolver (
    <div>
      ...

      {historias.se está cargando?
        ( <p>Cargando ...</p> ) :
        (
```

```
<Lista lista={historias.data} onRemoveItem={handleRemoveStory} /> } } </div>
```

```
); };
```

En segundo lugar, en lugar de usar el término de búsqueda predefinido (en este caso: "react"), use el término de búsqueda (searchTerm) del estado del componente. Posteriormente, cada vez que un usuario busque algo mediante el campo de entrada, se usará el término de búsqueda (searchTerm) para solicitar este tipo de historias desde la API remota. Además, debe abordar el caso extremo si searchTerm es una cadena vacía, lo que impide que se ejecute una solicitud:

```
src/App.jsx
```

```
const Aplicación = () => {
  ...

  React.useEffect(() => { if (término
    de búsqueda === "") return;

    dispatchStories({ tipo: 'STORIES_FETCH_INIT' });

    fetch(`${API_ENDPOINT}${searchTerm}
      `) .then((respuesta) => respuesta.json()) .then((resultado)
        => { dispatchStories({ tipo:

          'STORIES_FETCH_SUCCESS', carga útil:
            resultado.hits, }); }) .catch(() =>

            dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); }, []);

    ...
  });
```

Falta una pieza crucial. Si bien la obtención inicial de datos respeta el término de búsqueda (en este caso: "React", que se establece como estado inicial), este no se respeta cuando el usuario lo modifica al escribir en el campo de entrada. Si inspecciona la matriz de dependencias de nuestro gancho useEffect, verá que está vacía. Esto significa que el efecto secundario solo se renderiza durante la renderización inicial del componente App. Si quisiéramos ejecutar el efecto secundario también cuando cambia el término de búsqueda, tendríamos que incluirlo en la matriz de dependencias:

src/App.jsx

```

constante Aplicación = () => {
  ...

  React.useEffect(() => {
    // si `searchTerm` no está presente // p. ej.
    nulo, cadena vacía, indefinido // no hacer nada //
    condición más
    generalizada que searchTerm ===

    si (!searchTerm) retorna;

    dispatchStories({ tipo: 'STORIES_FETCH_INIT' });

    fetch(`${API_ENDPOINT}${searchTerm}
      `) .then((respuesta) => respuesta.json()) .then((resultado)
        => { dispatchStories({ tipo:

          'STORIES_FETCH_SUCCESS', carga útil:
            resultado.hits, }); }) .catch(()

            =>

              dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); },

    [término de búsqueda]);

    ...
  });
};

```

Hemos migrado la función de búsqueda del lado del cliente a búsqueda del lado del servidor. En lugar de filtrar una lista predefinida de historias en el cliente, ahora se utiliza searchTerm para recuperar una lista filtrada del lado del servidor. La búsqueda del lado del servidor se realiza no solo durante la obtención inicial de datos, sino también cuando se modifica searchTerm. La función de búsqueda ahora es completamente del lado del servidor.

Nota: Sin embargo, la recarga de datos con cada pulsación de tecla no es óptima, ya que esta implementación sobrecarga la API con solicitudes frecuentes. Un exceso de solicitudes puede provocar errores de API debido a la limitación de velocidad, una medida que muchas API emplean para protegerse contra un gran volumen de solicitudes (por ejemplo, permitir solo X solicitudes en 1 minuto). Planeamos solucionar este problema pronto.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹ .
 - Recapitulación de todos los [cambios en el código fuente](#)¹ de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)¹ ¹.

Preguntas de la entrevista:

- Pregunta: ¿Qué es la búsqueda del lado del cliente?
 - Respuesta: La búsqueda del lado del cliente implica filtrar y manipular datos del usuario. dispositivo o navegador.
 - Pregunta: ¿Cómo afecta la búsqueda del lado del cliente al rendimiento?
 - Respuesta: Puede ofrecer tiempos de respuesta rápidos, pero puede estar limitado por la cantidad de datos que Debe cargarse desde el servidor.
 - Pregunta: ¿Qué es la búsqueda del lado del servidor?
 - Respuesta: La búsqueda del lado del servidor implica enviar consultas de búsqueda al servidor, donde se almacenan los datos. se filtra y los resultados se devuelven al cliente.
 - Pregunta: ¿Cuándo se prefiere la búsqueda del lado del servidor?
 - Respuesta: La búsqueda del lado del servidor es preferible para conjuntos de datos grandes o cuando se requieren búsquedas complejas. La lógica y los datos residen en el servidor. •
- Pregunta: ¿Cuáles son los posibles inconvenientes de la búsqueda del lado del cliente?
- Respuesta: Pueden surgir limitaciones con grandes conjuntos de datos, cargas de páginas iniciales más lentas y la necesidad para cargar datos extensos al cliente.
- Pregunta: ¿Cuál es el impacto de las solicitudes frecuentes de API en la búsqueda del lado del cliente?
 - Respuesta: Las solicitudes frecuentes pueden estresar la API, lo que podría generar errores, especialmente si la API emplea medidas de limitación de velocidad.
 - Pregunta: ¿Cómo se puede abordar el problema de rendimiento de las solicitudes frecuentes de API?
 - Respuesta: La implementación de técnicas de rebote o limitación puede mitigar el impacto de las solicitudes frecuentes de API y evitar la sobrecarga del servidor.

¹ <https://tinyurl.com/49maj6rm>

¹ <https://tinyurl.com/ysk8cdpw>

¹ <https://bit.ly/492rkVN>

Funciones memorizadas en React (avanzado)

Generalmente, las funciones definidas en componentes de React sirven como controladores de eventos. Sin embargo, dado que un componente de React es esencialmente una función, también se pueden declarar funciones, expresiones de función y expresiones de función de flecha dentro de un componente. Esta sección presenta el concepto de función memorizada mediante el gancho useCallback de React.

Para comenzar, refactorizaremos proactivamente el código para incorporar una función memorizada, seguida de explicaciones detalladas. La refactorización implica transferir toda la lógica de obtención de datos del efecto secundario a una expresión de función de flecha (A). Esta nueva función se encapsula en el gancho useCallback de React (B) y posteriormente se invoca en el gancho useEffect (C).

src/App.jsx

```
const App = () => {
  ...

  // A
  const handleFetchStories = React.useCallback(() => { // B si (!searchTerm) return;

    dispatchStories({ tipo: 'STORIES_FETCH_INIT' });

    fetch(`${API_ENDPOINT}${searchTerm}`)
      .then((respuesta) => respuesta.json()) .then((resultado)
        => { dispatchStories({ tipo:

          'STORIES_FETCH_SUCCESS', carga útil:
            resultado.hits, }); }) .catch(()

          =>

            dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); },

    [termino de búsqueda]); // E

  React.useEffect(() => {
    manejarObtenerHistorias(); // C },
    [manejarObtenerHistorias]); // D

    ...
  };
};
```

En esencia, la aplicación se comporta de la misma manera, porque solo hemos extraído una nueva función de

El gancho `useEffect` de React. En lugar de usar la lógica de obtención de datos directamente en el efecto secundario, la pusimos a disposición como una función para toda la aplicación. La ventaja: la reutilización. La obtención de datos puede ser utilizada por otras partes de la aplicación llamando a esta nueva función. Sin embargo, hemos usado el gancho `useCallback` de React para encapsular la función extraída, así que exploremos por qué es necesario. El gancho `useCallback` de React crea una función memorizada cada vez que cambia su matriz de dependencias (E). Como resultado, el gancho `useEffect` se ejecuta de nuevo (C), ya que depende de la nueva función (D).

Visualización

1. cambio: `searchTerm` (causa: interacción del usuario) 2.
cambio: `handleFetchStories` (causa: se cambió `searchTerm`) 3. ejecución:
efecto secundario (causa: se cambió `handleFetchStories`)

Si omitiéramos el gancho `useCallback` de React y solo definiéramos el nuevo controlador de eventos `handleFetchStories` sin él, se crearía una nueva función `handleFetchStories` cada vez que el componente App se renderizase de nuevo y se ejecutaría en el gancho `useEffect` para obtener los datos. Los datos obtenidos se almacenan como estado en el componente. Luego, al cambiar el estado del componente, este se renderiza de nuevo y crea una nueva función `handleFetchStories`. El efecto secundario se activaría para obtener los datos, y quedaríamos atrapados en un bucle infinito:

Visualización

1. definir: `handleFetchStories`
2. correr: efecto secundario
3. actualización: estado
4. volver a renderizar: componente
5. redefinir: `handleFetchStories`
6. correr: efecto secundario

...

Podrías probar este bucle infinito tú mismo eliminando el gancho `useCallback` de React, pero prepárate para un bloqueo del navegador. Al fin y al cabo, el gancho `useCallback` de React modifica la función solo cuando cambia uno de sus valores en la matriz de dependencias. En ese momento, queremos que se vuelva a obtener la información, ya que el campo de entrada tiene una nueva entrada y queremos que los nuevos datos se muestren en nuestra lista.

Al mover la función de obtención de datos fuera del gancho `useEffect` de React, se vuelve reutilizable para otras partes de la aplicación. No la usaremos por ahora, pero es un buen caso de uso para comprender las funciones memorizadas en React. Ahora, el gancho `useEffect` se ejecuta implícitamente al cambiar `searchTerm`, ya que `handleFetchStories` se redefine con cada cambio. Dado que el gancho `useEffect` depende de `handleFetchStories`, el efecto secundario para la obtención de datos se ejecuta de nuevo.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹ ².

¹ ²<https://tinyurl.com/2s4chmah>

- Recapitulación de todos los [cambios en el código fuente](#)¹ ³ de esta sección.
- Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)¹ . • Lee más sobre el hook [useCallback de React](#)¹ .

Preguntas de la entrevista:

- Pregunta: ¿Cuál es el propósito del gancho useCallback en React?
 - Respuesta: useCallback se utiliza para memorizar funciones en React, evitando así repeticiones innecesarias. creaciones de funciones en re-renders.
- Pregunta: ¿Cuándo deberías usar useCallback en un componente React?
 - Respuesta: Use useCallback cuando desee memorizar una función para optimizar el rendimiento, especialmente en escenarios que involucran funciones de devolución de llamada pasadas a componentes secundarios. • Pregunta: ¿Cuáles son los argumentos del gancho useCallback?
 - Respuesta: El primer argumento es la función que se va a memorizar y el segundo argumento es una matriz de dependencias que, cuando se modifican, desencadenan la creación de una nueva función memorizada.
- Pregunta: ¿Qué sucede si la matriz de dependencias en useCallback está vacía?
 - Respuesta: Si la matriz de dependencias está vacía, la función memorizada se crea solo una vez y permanece igual durante todo el ciclo de vida del componente.

¹ ³<https://tinyurl.com/4e6z282z>

¹ <https://bit.ly/>

49Fo6Yj ¹ <https://www.robinwieruch.de/react-usecallback-hook/>

Obtención explícita de datos con React

Volver a obtener todos los datos cada vez que alguien escribe en el campo de entrada no es óptimo. Dado que usamos una API de terceros para obtener los datos, sus funciones internas están fuera de nuestro alcance. Eventualmente, nos enfrentaremos a una limitación de velocidad¹ que devuelve un error en lugar de datos. Para solucionar este problema, cambiaremos los detalles de implementación de la (re)obtención de datos implícita a explícita. En otras palabras, la aplicación solo volverá a obtener datos si alguien hace clic en un botón de confirmación.

Tarea: La búsqueda del lado del servidor se ejecuta cada vez que un usuario escribe en el campo de entrada. La nueva implementación solo debería ejecutar una búsqueda cuando el usuario haga clic en un botón de confirmación. Mientras no se haga clic en el botón, el término de búsqueda puede cambiar, pero no se ejecuta como una solicitud de API.

Consejos opcionales:

- Agregue un elemento de botón para confirmar la solicitud de búsqueda.
- Cree un valor con estado para la búsqueda confirmada.
- El controlador de eventos del botón establece la búsqueda confirmada como estado utilizando el término de búsqueda actual.
- Solo cuando la nueva búsqueda confirmada se establece como estado, ejecute el efecto secundario para realizar una... búsqueda lateral.

Lo importante de esta función es que necesitamos un estado para el término de búsqueda fluctuante y un nuevo estado para la búsqueda confirmada. Pero primero, crea un nuevo elemento de botón que confirme la búsqueda y ejecute la solicitud de datos:

src/App.jsx

```
const App = () => {
  ...

  return (
    <div>
      Mis historias de hackers

      <EntradaConEtiqueta
        id="buscar"
        valor={término de búsqueda}
        está enfocado
        onChange={manejarEntradaDeBúsqueda}
      />

      <strong>Buscar:</strong>
    </EntradaConEtiqueta>

    <botón
```

¹ <https://bit.ly/2ZaJX18>

```

      tipo="botón"
      deshabilitado={!searchTerm} al
      hacer clic={handleSearchSubmit}
    >
      Entregar
    </botón>

    ...
  </

```

```
div> ); };
```

En segundo lugar, distinguimos entre el controlador del campo de entrada y el botón. Mientras que el controlador renombrado del campo de entrada aún establece el término de búsqueda con estado, el nuevo controlador del botón establece el nuevo valor con estado llamado URL , que se deriva del término de búsqueda actual y del punto final estático de la API.

como un nuevo estado:

src/App.jsx

```

const App = () => { const
  [término de búsqueda, establecertérmino de búsqueda] = useStorageState(
    'buscar',
    'Reaccionar'
  );

  constante [url, setUrl] = React.useState( `
    ${API_ENDPOINT}${searchTerm}` );

  ...

  const handleSearchInput = (evento) =>
    { setSearchTerm(evento.objetivo.valor); };

  constante handleSearchSubmit = () => {
    setUrl(`${API_ENDPOINT}${término de búsqueda}`);
  };

  ...
};

```

En tercer lugar, en lugar de ejecutar el efecto secundario de obtención de datos en cada cambio de searchTerm (lo que ocurre

cada vez que el valor del campo de entrada cambia como hemos visto antes), la nueva URL con estado se utiliza siempre que un usuario la cambia al confirmar una solicitud de búsqueda al hacer clic en el botón:

src/App.jsx

```
const App = () => {
  ...

  const handleFetchStories = React.useCallback(() => { dispatchStories({ tipo:
    'STORIES_FETCH_INIT' });

    fetch(url) .then((respuesta) =>

      respuesta.json() ) .then((resultado) =>
        { dispatchStories({ tipo:

          'STORIES_FETCH_SUCCESS', carga útil: resultado.hits, }); }) .catch(() =>
            dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); },

    [url]);

  React.useEffect(() =>
    { handleFetchStories(); },
    [handleFetchStories]);

  ...
};
```

Anteriormente, `searchTerm` se usaba para dos cosas: actualizar el estado del campo de entrada y activar el efecto secundario para obtener datos. Ahora solo se usa para el primero. Se introdujo un segundo estado llamado `url` para activar el efecto secundario que obtiene los datos, lo cual solo ocurre cuando el usuario hace clic en el botón de confirmación.

Ceremonias:

- Compare su código fuente con el código fuente del autor¹ .
 - Recapitulación de todos los cambios en el código fuente¹ de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor aquí¹ .¹ <https://tinyurl.com/8k7fmj6p>

¹ <https://tinyurl.com/4we2j96f> ¹ <https://bit.ly/42tCHUa>

Preguntas de la entrevista:

- Pregunta: ¿Por qué se utiliza useState en lugar de useStorageState para la gestión del estado de la URL ?
 - Respuesta: No queremos recordar la URL en el almacenamiento local del navegador, porque ya se deriva de una cadena estática (aquí: API_ENDPOINT) y el searchTerm que ya proviene del almacenamiento local del navegador.
- Pregunta: ¿Por qué no se verifica si hay un término de búsqueda vacío en la función handleFetchStories?
¿ya no?
 - Respuesta: La prevención de una búsqueda del lado del servidor se produce en el botón nuevo, porque se deshabilita siempre que no hay un término de búsqueda.

Bibliotecas de terceros en React

Anteriormente introdujimos la API de búsqueda nativa (que proporciona el navegador) para realizar solicitudes a la API de Hacker News. Sin embargo, no todos los navegadores la admiten, especialmente los más antiguos. Además, una vez que comiences a probar tu aplicación en un entorno de navegador sin interfaz gráfica¹, Pueden surgir problemas con la API de búsqueda porque el navegador no está disponible. Hay un par de maneras de hacer que la búsqueda funcione en navegadores antiguos (polyfills¹ ¹). y en pruebas (búsqueda isomórfica), pero estos conceptos están un poco fuera del tema de esta experiencia de aprendizaje.

Una alternativa es sustituir la API de búsqueda nativa con una biblioteca estable como axios¹ ², que realiza solicitudes asincrónicas a API remotas. En esta sección, descubriremos cómo sustituir una biblioteca (en este caso, una API nativa del navegador) por otra biblioteca del registro de npm.

Si sabe cómo instalar axios mediante npm y usarlo como reemplazo de la API de búsqueda del navegador, impleméntelo usted mismo. De lo contrario, el libro le guiará en la implementación.

Primero, instale axios desde la línea de comandos:

Línea de comandos

```
npm instalar axios
```

En segundo lugar, importe axios en el archivo del componente de su aplicación:

src/App.jsx

```
importar * como React desde 'react';  
importar axios desde 'axios';
```

...

Puedes usar axios en lugar de fetch. Su uso es casi idéntico al de la API nativa de fetch: toma la URL como argumento y devuelve una promesa. Ya no tienes que convertir la respuesta devuelta a JSON, ya que axios encapsula el resultado en un objeto de datos en JavaScript. Solo asegúrate de adaptar tu código a la estructura de datos devuelta:

¹ <https://bit.ly/3ncFfSs>

¹ <https://bit.ly/3ASC86Y>

¹ <https://bit.ly/3jjEupg>

src/App.jsx

```

const App = () => {
  ...

  const handleFetchStories = React.useCallback(() => { dispatchStories({ tipo:
    'STORIES_FETCH_INIT' });

    axios

    .get(url) .then((resultado)
      => { dispatchStories({ tipo:
        'STORIES_FETCH_SUCCESS', carga
        útil: resultado.data.hits, }); }) .catch(()
      =>

        dispatchStories({ tipo: 'FALLO DE BÚSQUEDA DE HISTORIAS' }) ); },

    [url]);

    ...
  };

```

En este código, llamamos a `axios.get()` para una solicitud HTTP GET explícita³. Este es el mismo método HTTP que usamos por defecto con la API de búsqueda nativa del navegador. También puedes usar otros métodos HTTP, como HTTP POST, con `axios.post()`. Con estos ejemplos, podemos ver que `axios` es una biblioteca potente para realizar solicitudes a API remotas. La recomiendo en lugar de la API de búsqueda nativa cuando las solicitudes se vuelven complejas, al trabajar con navegadores antiguos o para realizar pruebas.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor¹.
 - Recapitulación de todos los [cambios del código fuente](#)¹ de esta sección.
 - Opcional: si usas TypeScript, consulta el código fuente del autor [aquí](#)¹. • Lee más sobre [las bibliotecas populares en React](#)¹. • Opcional: Lea más sobre [axios](#)¹.

¹ <https://mzl.la/3n5kUyi>

¹ <https://tinyurl.com/34bm9s62>

¹ <https://tinyurl.com/3hvu5r85>

¹ <https://bit.ly/3Ss6Yyb>

¹ <https://www.robinwieruch.de/react-libraries/>

¹ <https://bit.ly/3jjEupg>

Preguntas de la entrevista:

- Pregunta: ¿Qué es Axios en el contexto de React?

- Respuesta: Axios es una biblioteca de JavaScript popular para realizar solicitudes HTTP, comúnmente utilizada en aplicaciones React para la obtención de datos. •

- Pregunta: ¿En qué se diferencia Axios de la API Fetch en React?

- Respuesta: Axios es una biblioteca de terceros que ofrece funciones adicionales y una API más conveniente en comparación con la API nativa de Fetch. •

- Pregunta: ¿Por qué elegirías Axios en lugar de Fetch en un proyecto de React?

- Respuesta: Axios ofrece funciones como análisis automático de JSON, interceptores de solicitud/respuesta y mejor compatibilidad con navegadores, lo que lo convierte en la opción preferida de muchos desarrolladores.

- Pregunta: ¿Cómo se realiza una solicitud GET usando Axios en React?

- Respuesta: Utilice `axios.get(url)` para realizar una solicitud GET en React con Axios.

- Pregunta: ¿Cómo se realiza el manejo de errores en las solicitudes de Axios en React?

- Respuesta: Axios proporciona un método `.catch()` para manejar errores en la solicitud.

- Pregunta: ¿Cuál es la principal ventaja de usar Fetch API en React?

- Respuesta: La API Fetch está integrada en los navegadores modernos, lo que elimina la necesidad de dependencias y lo convierte en una opción liviana para escenarios simples.

Async/Await en React

No hay forma de evitar los datos asíncronos al trabajar en aplicaciones del mundo real. Siempre habrá una API remota que proporcione datos, ya sea en el frontend o en el backend, por lo que es necesario comprender cómo trabajar con estos datos de forma asíncrona. En nuestra aplicación React, hemos empezado a resolver promesas con bloques then/catch. Sin embargo, en JavaScript moderno (y, por lo tanto, en React), una solución más popular es usar async/await.

Si ya está familiarizado con async/await o desea explorar [su uso](https://mzl.la/3AWyWaw)¹ Si lo deseas, cambia el código de then/catch a async/await. Si has llegado hasta aquí, también podrías considerar compensar la eliminación del bloque catch para la gestión de errores con un bloque try/catch.

Continuemos con la tarea. Primero, deberá reemplazar la sintaxis then/catch por la sintaxis async/await. La siguiente refactorización de la función handleFetchStories() muestra cómo lograrlo sin gestión de errores:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  const handleFetchStories = React.useCallback(async () => { dispatchStories({ tipo:  
    'STORIES_FETCH_INIT' });  
  
    constante resultado = await axios.get(url);  
  
    dispatchStories({ tipo:  
      'STORIES_FETCH_SUCCESS', carga  
      útil: result.data.hits, }); }, [url]);  
  
  ...  
};
```

Para usar async/await, nuestra función requiere la palabra clave async . Una vez que se empieza a usar la palabra clave await en las promesas devueltas, todo se lee como código síncrono. Las acciones posteriores a la palabra clave await no se ejecutan hasta que se resuelve la promesa, lo que significa que el código esperará. Para incluir la gestión de errores como antes, los bloques try y catch son de ayuda. Si algo falla en el bloque try , el código saltará al bloque catch para gestionar el error:

¹ <https://mzl.la/3AWyWaw>

src/App.jsx

```

const App = () => {
  ...

  const handleFetchStories = React.useCallback(async () => {
    dispatchStories({ tipo:
      'STORIES_FETCH_INIT' });

    intenta
    { const resultado = await axios.get(url);

    dispatchStories({ tipo:
      'STORIES_FETCH_SUCCESS', carga
      útil: result.data.hits, }); } catch

    { dispatchStories({ tipo: 'STORIES_FETCH_FAILURE' });

    } }, [url]);

    ...
  });

```

Después de todo, usar `async/await` con `try/catch` en lugar de `then/catch` suele ser más legible, ya que evitamos usar funciones de devolución de llamada y, en su lugar, intentamos que nuestro código sea más legible de forma síncrona. Sin embargo, usar `then/catch` también es adecuado. Al final, todo el equipo que trabaja en un proyecto debería estar de acuerdo en una sintaxis.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
 - Recapitular todos los [cambios del código fuente](#)² ¹ de esta sección.
 - Opcional: si usas TypeScript, consulta el código fuente del autor [aquí](#)² ². • Lee más sobre [la obtención de datos en React](#)² ³. • Lea más sobre [la obtención de datos en React con Hooks](#)² .

² <https://tinyurl.com/32shkr5e>

² ¹<https://tinyurl.com/2fvjcpak>

² ²<https://bit.ly/3wdPFJq>

² ³<https://www.robinwieruch.de/react-fetching-data/>

² <https://www.robinwieruch.de/react-hooks-fetch-data/>

Preguntas de la entrevista:

- Pregunta: ¿Qué es `async/await`?
 - Respuesta: `async` y `await` son palabras clave en JavaScript que se utilizan para manejar operaciones asincrónicas. operaciones de manera sincrónica, lo que hace que el código sea más legible.
- Pregunta: ¿Cómo se usa `async/await` con una función en React?
 - Respuesta: Declare la función con la palabra clave `async` y use `await` dentro de la función para manejar promesas.
 - Pregunta: ¿Cuál es el propósito de las funciones asincrónicas en React?
 - Respuesta: Las funciones asíncronas permiten trabajar con código asíncrono de forma más legible. y de manera secuencial, mejorando el manejo de promesas.
 - Pregunta: ¿Cómo se manejan los errores con `async/await` en React?
 - Respuesta: Use un bloque `try/catch` para capturar y manejar errores en una función asíncrona.
 - Pregunta: ¿En qué se diferencia `async/await` de usar `.then()` con promesas en React?
 - Respuesta: `async/await` proporciona una sintaxis más concisa, lo que hace que el código asíncrono se parezca al código sincrónico, en comparación con el encadenamiento de `.then()`.
 - Pregunta: ¿Puedes usar `async/await` con la API `Fetch` en React?
 - Respuesta: Sí, `async/await` se usa comúnmente con la API `Fetch` para datos asincrónicos. Recuperación en React.
 - Pregunta: ¿Cómo se manejan múltiples operaciones asincrónicas con `async/await` en React?
 - Respuesta: Use `Promise.all()` para manejar múltiples operaciones asincrónicas simultáneamente en una función asíncrona.

Formularios en React

No existe ninguna aplicación moderna que no utilice formularios. Un formulario es simplemente un vehículo adecuado para enviar datos mediante un botón desde diversos controles de entrada (p. ej., campo de entrada, casilla de verificación, botón de opción, control deslizante). Anteriormente, presentamos un nuevo botón para obtener datos explícitamente con un clic. Desarrollaremos su uso con un formulario HTML adecuado, que encapsula el botón y el campo de entrada para el término de búsqueda con su etiqueta.

Los formularios no difieren mucho en JSX de React que en HTML. Lo implementaremos en dos pasos de refactorización con HTML/JavaScript. Primero, integraremos el campo de entrada y el botón en un elemento de formulario HTML:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  devolver (  
    <div>  
      Mis historias de hackers  
  
      <formulario onSubmit={manejarBúsquedaEnviada}>  
        <EntradaConEtiqueta  
          id="buscar"  
          valor={término de búsqueda}  
          está enfocado  
          onChange={manejarEntradaDeBúsqueda}  
        >  
          <strong>Buscar:</strong>  
        </EntradaConEtiqueta>  
  
        <button type="submit" deshabilitado={!searchTerm}>  
          Entregar  
        </botón>  
      </formulario>  
  
      <hr />  
  
      ...  
    </div>  
  
  );  
};
```

En lugar de pasar el controlador handleSearchSubmit() al botón, se usa en el atributo onSubmit del nuevo elemento de formulario . El botón recibe un nuevo atributo de tipo llamado "submit", que indica

Que el elemento `onSubmit` del formulario gestiona el clic, no el botón. Además, **dado que el controlador se usa para el evento del formulario, ejecuta `preventDefault()` adicionalmente en el evento sintético de React. Esto evita el comportamiento nativo del formulario HTML, que provocaría la recarga del navegador.**

src/App.jsx

```
const Aplicación = () => {
  ...

  const handleSearchSubmit = (evento) => {
    setUrl(`${API_ENDPOINT}${término de búsqueda}`);

    evento.preventDefault(); };

  ...
};
```

Ahora podemos ejecutar la función de búsqueda con la tecla "Intro" del teclado, ya que usamos un formulario en lugar de un botón independiente. En los dos siguientes pasos, separaremos todo el formulario en un nuevo componente `SearchForm`. Si desea hacerlo usted mismo, no dude en hacerlo. Así es como se puede extraer el formulario en su propio componente:

src/App.jsx

```
const FormularioDeBúsqueda =
  ({TérminoDeBúsqueda,
    onSearchInput,
    onSearchSubmit, })
=>
  ( <form onSubmit={onSearchSubmit}>
    <EntradaConEtiqueta
      id="buscar"
      valor={término de búsqueda}
      está enfocado
      onChange={onSearchInput}
    >
      <strong>Buscar:</strong>
    </EntradaConEtiqueta>

    <button type="submit" deshabilitado={!searchTerm}>
      Entregar
    </botón>
  </form>
);
```

El nuevo componente se instancia en el componente App. Sin embargo, este componente sigue gestionando el estado del formulario, ya que este activa la solicitud de datos en el componente App, donde los datos solicitados se pasarán como propiedades (en este caso: `stories.data`) al componente List.

src/App.jsx

```
const App = () => {
  ...

  return (
    <div>
      Mis historias de hackers

      <Formulario de búsqueda
        searchTerm={término de
        búsqueda} onSearchInput={manejarSearchInput}
        onSearchSubmit={manejarSearchSubmit} />

      <hr />

      {stories.isError && <p>Algo salió mal...</p>}

      {historias.isLoading ?
        ( <p>Cargando ...</p> ) :
        (
          <List list={historias.data} onRemoveItem={handleRemoveStory} /> )} </div>

    );
  }
};
```

Los formularios en React no difieren mucho de los de HTML simple. Cuando tenemos campos de entrada y un botón para enviar datos desde ellos, podemos estructurar mejor nuestro HTML envolviéndolo en un elemento de formulario con el atributo "onSubmit". Por lo tanto, el botón que ejecuta el envío necesita el tipo "submit" para remitir el proceso al controlador del elemento de formulario. Al fin y al cabo, también lo hace más accesible para los usuarios de teclado.

Ceremonias:

- Compare su código fuente con el código [fuente](https://tinyurl.com/mra85b3s) del autor².

² <https://tinyurl.com/mra85b3s>

- Recapitular todos los [cambios del código fuente](#)² de esta sección.
 - Opcional: si estás usando TypeScript, consulta el código fuente del autor [aquí](#)².
 - Lea más sobre [los formularios en React](#)².
- Pruebe lo que sucede sin utilizar preventDefault.
- Obtenga más información sobre [preventDefault para eventos en React](#)².

Preguntas de la entrevista:

- Preguntas: ¿Cómo se maneja la entrada del formulario en React?
 - Respuestas: En React, la entrada del formulario se gestiona normalmente mediante el estado. Cada campo de entrada tiene un variable de estado correspondiente y el valor de la entrada se establece en el estado.
 - Preguntas: ¿Cuál es el propósito del evento onChange en los formularios React?
 - Respuestas: El evento onChange se utiliza para capturar la entrada del usuario en tiempo real y actualizar la Indique en consecuencia, asegurándose de que el formulario refleje la última entrada. •
- Preguntas: ¿Cómo se puede evitar el comportamiento de envío de formulario predeterminado en React?
- Respuestas: Utilice el método e.preventDefault() dentro del controlador de envío del formulario para evitar El comportamiento de envío de formulario predeterminado.
- Preguntas: ¿Qué es la entrada de formulario controlada y no controlada en React?

Respuestas : La entrada controlada de un formulario se produce cuando el estado de React gestiona el valor de entrada. La entrada no controlada se produce cuando el DOM gestiona la entrada y React no registra su estado. • Preguntas: ¿Cómo se realiza la validación de formularios en React?

Respuestas : La validación de formularios en React se realiza generalmente comprobando los valores de entrada con ciertas condiciones o usando bibliotecas de validación. El controlador onSubmit es un lugar común para implementar la validación. Preguntas: ¿Cuál es el propósito del atributo value en las entradas de formulario?

 - Respuestas: El atributo de valor establece el valor inicial de una entrada de formulario y garantiza que La entrada está controlada por el estado de React. • Preguntas: ¿Cómo se pueden manejar múltiples entradas de formulario con un único controlador onChange en React?
 - Respuestas: Utilice el atributo de nombre en cada campo de entrada y acceda al valor correspondiente utilizando event.target.name en el controlador onChange.
 - Preguntas: ¿Cuál es el rol del evento onSubmit en los formularios React?

Respuestas : El evento onSubmit se activa al enviar el formulario. Es donde se gestiona la validación del formulario, el procesamiento de datos o cualquier otra acción relacionada con el envío.

² <https://tinyurl.com/3ecdcpje>

² <https://bit.ly/>

42qnJ18 ² <https://www.robinwieruch.de/>

react-form/ ² <https://www.robinwieruch.de/react-preventdefault/>

Formularios con acciones

Los formularios de React son una herramienta potente para enviar datos. En la sección anterior, presentamos un formulario para enviar un término de búsqueda y obtener datos de una API. Usamos el controlador de eventos "onSubmit" para activar el proceso de obtención de datos. En esta sección, presentaremos un nuevo concepto para el formulario: el atributo "action". El atributo de acción es un atributo HTML estándar que especifica la URL donde deben enviarse los datos del formulario. Al usar React, se puede pasar una función de acción al componente de formulario, que se ejecutará al enviar el formulario:

src/App.jsx

```
const SearchForm = ({término de búsqueda, onSearchInput, acción de búsqueda}) => (  
  <form action={AcciónDeBúsqueda}>  
    <EntradaConEtiqueta  
      id="buscar"  
      valor={término de búsqueda}  
      está enfocado  
      onChange={onSearchInput}  
    >  
      <strong>Buscar:</strong>  
    </EntradaConEtiqueta>  
  
    <button type="submit" deshabilitado={!searchTerm}>  
      Entregar  
    </botón>  
  </form>  
)
```

En lugar de pasar el controlador de envío al atributo onSubmit del formulario, pasamos una nueva función searchAction al atributo action del formulario:

src/App.jsx

```
<Formulario de búsqueda  
  searchTerm={término de  
  búsqueda} onSearchInput={handleSearchInput}  
  searchAction={searchAction} />
```

Dado que estamos más cerca del comportamiento del formulario nativo, podemos eliminar la llamada preventDefault() del controlador de envío:

src/App.jsx

```
const searchAction = () =>
  { setUrl(`${API_ENDPOINT}${searchTerm}`);

  // event.preventDefault(); <--- ya no necesitamos esto };
```

A partir de React 19, el atributo `action` se usará en lugar del atributo `onSubmit` para enviar datos del formulario, ya que aprovecha mejor el comportamiento nativo del formulario. Opcionalmente, la firma de la función de la acción del formulario otorgará acceso a los datos del formulario²¹, lo cual puede ser útil para la validación de formularios u otras tareas relacionadas con ellos.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor²¹¹.
 - Recapitular todos los [cambios del código fuente](#)²¹² de esta sección.
- Lea más sobre [React y FormData](#)²¹³.
- Lea más sobre [los formularios y su estado de carga](#)²¹.

²¹ <https://tinyurl.com/bddjd59s>

²¹¹<https://tinyurl.com/4nchwt46>

²¹²<https://tinyurl.com/2vnwf7ty>

²¹³<https://www.robinwieruch.de/react-form-data/>

²¹ <https://www.robinwieruch.de/react-form-loading-pending-action/>

Una hoja de ruta para React

Hasta ahora, has aprendido todo lo necesario sobre los fundamentos de React. Con este conocimiento, puedes tomarte un descanso de este libro y crear tu propia aplicación React. Es inevitable que surjan preguntas, pero siempre puedes encontrar respuestas repasando los fundamentos que se tratan en este libro. Después de todo, aprender React.js (y cualquier otra cosa) se aprende mejor con práctica. Mi enfoque favorito: aprender los fundamentos, aprender a conectarte a una API, encontrar una API que proporcione datos alineados con tus intereses personales y ¡crear algo con ella! Fortalecer tus conocimientos de los fundamentos es clave para lo que viene después.

Desde aquí, puedes seguir varios caminos en tu aprendizaje. Primero, puedes continuar leyendo este libro. Las siguientes secciones cubren principalmente tres aspectos: funciones avanzadas de React, temas organizativos para cada proyecto de React y el ecosistema de React. Aprenderás sobre temas como optimizaciones de rendimiento, estructuras de carpetas y archivos en proyectos de React, tipado estático con TypeScript y estilos en React. Sin embargo, he mantenido la selección al mínimo para no abrumarte; de lo contrario, este libro se volvería interminable. Si deseas profundizar en ciertos temas, te recomiendo los siguientes recursos.

El ecosistema de React es vasto. Cada año, resumo [todas las bibliotecas esenciales y populares](#)²¹ que se pueden usar en React para diversos fines. Puedes explorar la lista y experimentar con bibliotecas que podrían mejorar tu proyecto. También he escrito tutoriales específicos para algunas de ellas.

Hay una característica avanzada de React que he excluido intencionalmente de este libro: React Context. La razón es simple: se trata de una función avanzada que complicaría innecesariamente nuestra aplicación mínima sin demostrar claramente el problema que resuelve. Para un desarrollador de nivel intermedio, podría parecer una optimización prematura; para un principiante, un obstáculo. Sin embargo, es probable que la encuentres en algún momento, así que no quiero que la pases por alto. Si quieres aprender más sobre ella, te recomiendo leer más sobre [React Context](#)²¹ y [el useContext Hook](#)²¹ de React, junto con una [guía sobre cómo combinar múltiples ganchos en una aplicación real](#)²¹ (una lectura recomendada).

React también ofrece varios patrones de componentes. Ya has aprendido sobre [la composición de componentes](#)²¹ y [componentes reutilizables](#)²². Si aún no has explorado los tutoriales de referencia sobre estos patrones, te animo a que lo hagas. Hay patrones adicionales, pero abarcarlos todos en una aplicación pequeña no les haría justicia. Por eso he escrito sobre ellos por separado; por ejemplo, [React Render Prop Components](#)²²¹ y [componentes de orden superior de React](#)²²². Sin embargo, con el aumento

²¹ <https://www.robinwieruch.de/react-libraries>

²¹ <https://www.robinwieruch.de/react-context/>

²¹ <https://www.robinwieruch.de/react-usecontext-hook/>

²¹ <https://www.robinwieruch.de/react-state-usereducer-usestate-usecontext/>

²¹ <https://www.robinwieruch.de/composición-de-componentes-de-react/>

²² <https://www.robinwieruch.de/componentes-reutilizables-react/>

²²¹ <https://www.robinwieruch.de/react-render-prop-components/>

²²² <https://www.robinwieruch.de/react-higher-order-components/>

En el caso de React Hooks, estos patrones se utilizan con menos frecuencia hoy en día, aunque aún es posible encontrarlos en aplicaciones React más grandes.

Además, en este libro, usaste Vite para iniciar tu proyecto. Si te interesa [crear un proyecto React desde cero](#)²²³ Usando herramientas como Webpack y Babel (que impulsan las canalizaciones de compilación de JavaScript), te animo a que sigas el proceso. Aunque Webpack ya no es tan popular, comprender su funcionamiento proporciona información valiosa sobre lo que ocurre en herramientas de terceros como Vite.

Por último, pero no menos importante, te animo a que consultes mi curso, "[El Camino a Next](#)"²² ". Mientras que React y Vite siguen un enfoque de biblioteca del lado del cliente, Next.js es un framework completo para React. Incluye numerosas funciones integradas, como renderizado del lado del servidor, enrutamiento y más. El curso te guiará a través de los fundamentos de Next.js, ayudándote a crear una aplicación práctica. Es una excelente manera de profundizar en tu comprensión de React y su ecosistema, a la vez que adquieres experiencia en desarrollo full-stack.

Además, React incluye características como [Componentes de servidor](#)²² y [funciones del servidor](#)²² , que (al momento de escribir esto) solo se puede usar en un framework full-stack como Next.js. Por lo tanto, recomiendo explorar Next.js y sus características para comprender mejor el ecosistema de React y [cómo se está convirtiendo en un framework full-stack](#)²² .

Así es como priorizaría los recursos mencionados anteriormente:

- continuar con The Road to React • explorar el
- ecosistema de React • consultar The Road
- to Next para el desarrollo de pila completa , que incluye: componentes del servidor
- y funciones del servidor

Opcionalmente, puedes:

- recapitular patrones de componentes
- configurar un proyecto React desde cero • aprender
- sobre React Context y useContext

Ya hemos llegado a la mitad de "El Camino a React", y espero que lo hayan disfrutado hasta ahora. Si les gustó el libro, sería muy importante para mí que lo compartieran con amigos interesados en aprender React. También les dejo una reseña en [Amazon](#)²² o [Goodreads](#)²² Sería muy apreciado.

A partir de este punto, puede seguir leyendo para conocer una selección basada en opiniones del ecosistema de React, recomendaciones organizacionales y más funciones integradas de React (por ejemplo, rendimiento).

²²³<https://www.robinwieruch.de/minimal-react-webpack-babel-setup/>

²² <https://www.road-to-next.com/>

²² <https://tinyurl.com/283wewxw>

²² <https://tinyurl.com/4en9pmj9>

²² <https://www.robinwieruch.de/react-full-stack-framework/>

²² <https://amzn.to/>

2JHIP42 ²² <https://tinyurl.com/4bhcusu7>

Optimizaciones). Hacia el final del libro, encontrarás secciones adicionales que te ayudarán a implementar funciones avanzadas en tu aplicación React. En resumen, espero que los conocimientos adquiridos hasta ahora, junto con los materiales de referencia y los próximos capítulos, te ayuden a convertirte en un excelente desarrollador de React.

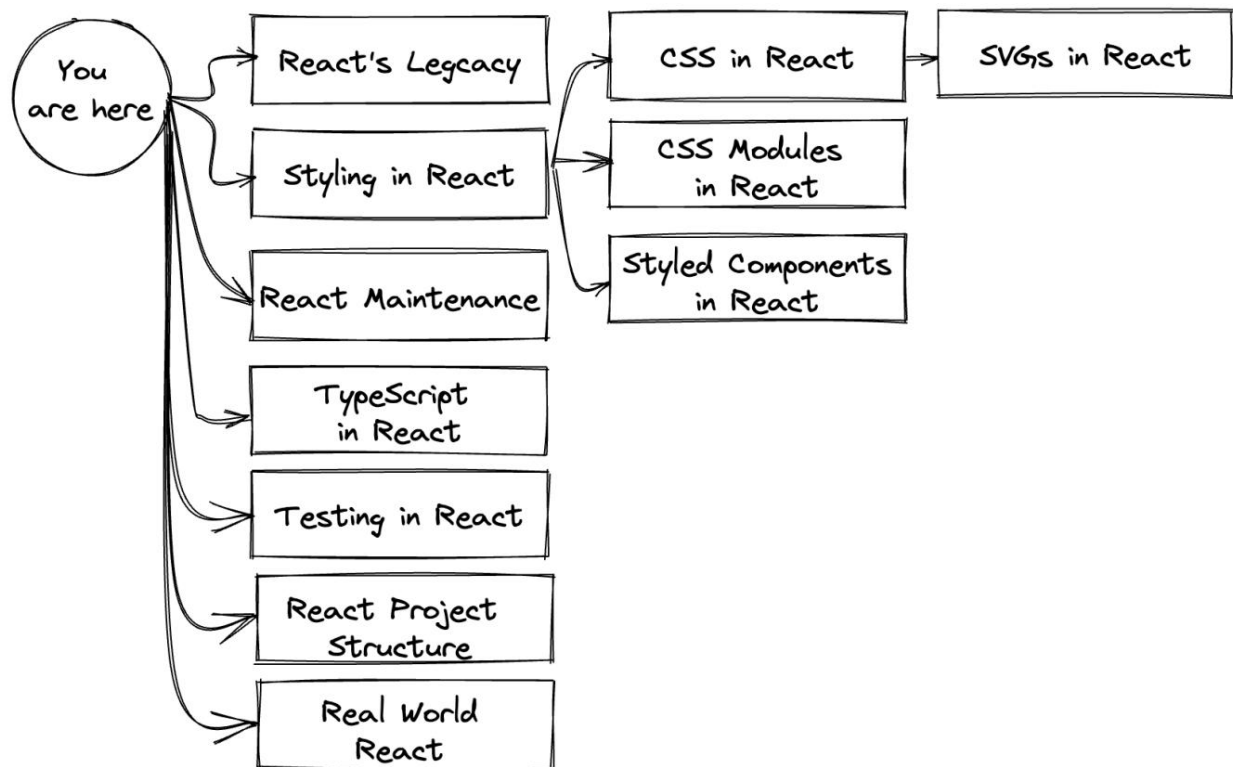
Importante: Los siguientes capítulos no siguen una ruta estrictamente lineal. Si bien todos se basan en la aplicación que has creado, divergirán en diferentes direcciones. Puedes intentar integrarlos en tu proyecto actual, lo cual suele funcionar, excepto en las secciones de estilo, donde tendrás que elegir un enfoque. Si esto te resulta abrumador, considera estas dos alternativas:

1. Copie y pegue su aplicación actual y use copias separadas para diferentes rutas.
2. Lea un capítulo (ruta), aplique los cambios y, opcionalmente, reviértalos después para comenzar de nuevo con el siguiente capítulo.

A continuación se muestra un resumen de las rutas disponibles en este libro:

- Estilo en React: Cada sección de este capítulo presenta una ruta alternativa. •
- Mantenimiento de React: Este capítulo sigue una ruta lineal a través de sus secciones.
- TypeScript en React: Este capítulo sigue una ruta lineal a través de sus secciones. •
- Pruebas en React: Este capítulo sigue una ruta lineal a través de sus secciones. •
- Estructura del proyecto React: Este capítulo sigue una ruta lineal a través de sus secciones.
- React en el mundo real (avanzado): Este capítulo sigue una ruta lineal a través de sus secciones.

Cada capítulo se basa en su aplicación actual, pero ningún capítulo hereda los cambios de otros.



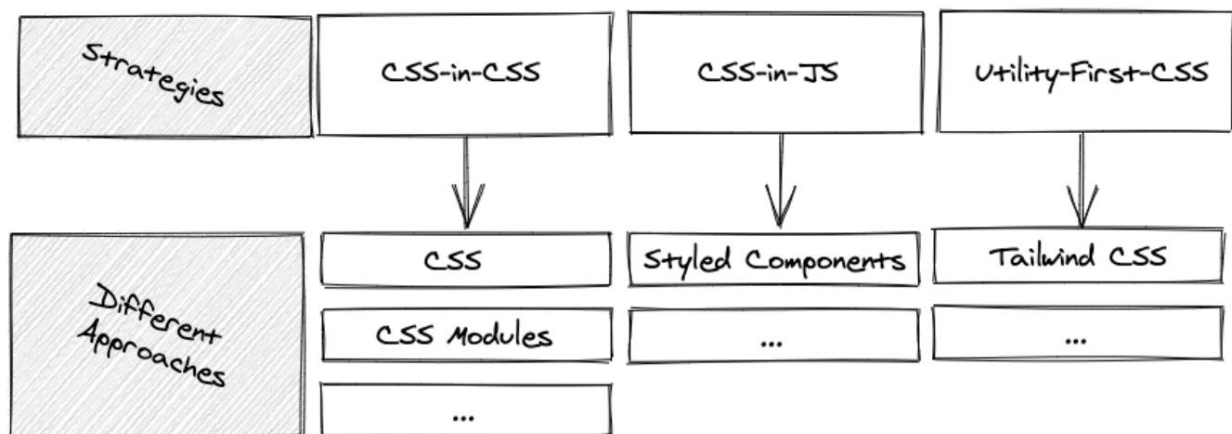
Estilo en React

Hay muchas maneras de diseñar una aplicación React, y existen largos debates sobre la mejor estrategia y enfoque de estilo. Repasaremos algunas de estas estrategias, cada una representando un enfoque sin darles demasiada importancia. Habrá argumentos a favor y en contra, pero se trata principalmente de qué se adapta mejor a los desarrolladores y sus equipos.

Comenzaremos a aplicar estilo a React con CSS común en React, pero luego exploraremos dos alternativas para estrategias más avanzadas de CSS-in-CSS (con módulos CSS) y CSS-in-JS (con componentes sStyled).

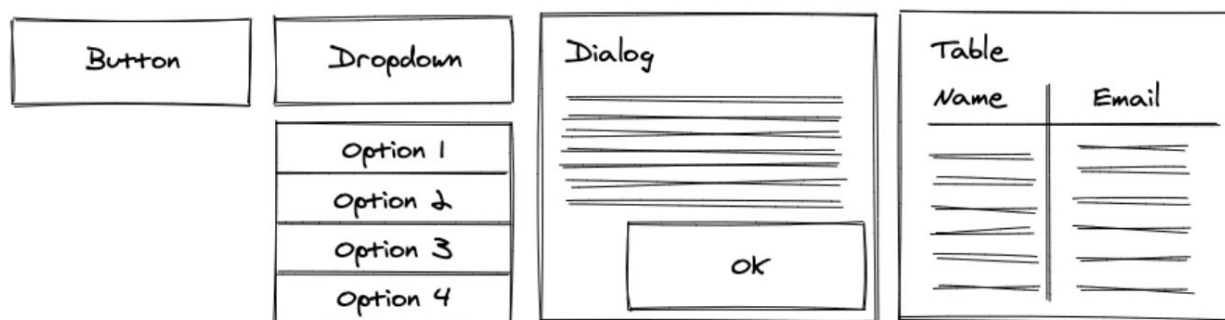
Los módulos CSS y los componentes con estilo son solo dos de los muchos enfoques disponibles en ambos grupos de estrategias.

También explicaremos cómo incluir gráficos vectoriales escalables (SVG), como logotipos o iconos, en nuestra aplicación React.



Si no desea crear componentes de UI comunes (por ejemplo, botón, cuadro de diálogo, menú desplegable) desde cero, siempre puede elegir una biblioteca de UI popular adecuada para React²³, que proporciona estos componentes por defecto. Sin embargo, para aprender React, es mejor intentar compilarlos antes de usar una solución predefinida. Por lo tanto, no usaremos ninguna de las bibliotecas de componentes de interfaz de usuario.

²³ <https://www.robinwieruch.de/react-libraries/>



Los siguientes enfoques de estilo y SVG vienen preconfigurados en su mayoría en Vite. Si controlas las herramientas de compilación (por ejemplo, Webpack) con una configuración personalizada, es posible que debas configurarlas para permitir la importación de archivos CSS o SVG. Dado que usamos Vite, podemos usar estos archivos de inmediato. Por ejemplo, en tu archivo `src/main.jsx`, asegúrate de importar el archivo `src/index.css`:

`src/main.jsx`

```
importar { StrictMode } desde 'react'; importar
{ createRoot } desde 'react-dom/client'; importar './index.css';
importar App desde './
App.jsx';

createRoot(document.getElementById('root')).render(
  <Modo estricto>
    <Aplicación />
  </Modo estricto>
);
```

Utilice el siguiente CSS en el archivo `src/index.css` para eliminar el margen y utilizar una fuente estandarizada con alternativas:

`src/index.css`

`cuerpo`

```
{ margen: 0;
  familia de fuentes: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue', sans-serif;

  -webkit-font-smoothing: suavizado de fuentes;
  -moz-osx-font-smoothing: escala de grises;
}
```

Básicamente, puedes declarar todo el CSS que debería aplicarse globalmente para tu proyecto en este archivo.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor²³¹.
 - Recapitular todos los [cambios del código fuente](#)²³² de esta sección.
- Lea más sobre [las diferentes estrategias y enfoques de estilo en React](#)²³³.

²³¹<https://tinyurl.com/38kw8tkn>

²³²<https://tinyurl.com/mr2fc836>

²³³<https://www.robinwieruch.de/react-css-styling/>

CSS en React

El CSS común en React es similar al CSS estándar que quizás ya conozcas. Cada aplicación web asigna a los elementos HTML un atributo de clase (en React, `className`), cuyo estilo se define mediante un archivo CSS:

src/App.jsx

```
const App = () => {  
  ...  
  
  return (  
    <div className="contenedor">  
      <h1 className="headline-primary">Mis historias de hackers</h1>  
  
      <Formulario de búsqueda  
        searchTerm={término de  
        búsqueda} onSearchInput={handleSearchInput}  
        searchAction={searchAction} />  
  
      {stories.isError && <p>Algo salió mal...</p>}  
  
      {historias.isLoading ?  
        ( <p>Cargando ...</p> ) :  
        (  
          <List list={historias.data} onRemoveItem={handleRemoveStory} /> ) } </div>  
  
    );  
  };  
};
```

Se eliminó el `<hr />` porque el CSS gestiona el borde en los siguientes pasos. Importaremos el archivo CSS, lo cual se realiza mediante la resolución de importaciones de Vite:

src/App.jsx

```
import * as React from 'react'; import  
  axios from 'axios'; import './  
  App.css';
```

Este archivo CSS definirá las dos (y más) clases CSS que usamos (y usaremos) en el componente `App`. En su archivo `src/App.css`, defínalos de la siguiente manera:

src/App.css

```
.container { altura:
  100vw; relleno: 20px;

  fondo: #83a4d4; /* respaldo para navegadores antiguos */ fondo: gradiente lineal
  (a la izquierda, #b6fbff, #83a4d4);

  color: #171212;
}

.headline-primary { tamaño
  de fuente: 48px; peso
  de fuente: 300; espaciado
  entre letras: 2px;
}
```

Deberías ver los primeros estilos surtiendo efecto en tu aplicación al reiniciarla. A continuación, nos dirigiremos al componente Item. Algunos de sus elementos también reciben el atributo className ; sin embargo, aquí también usamos una nueva técnica de estilo:

src/App.jsx

```
constante Item = ({ item, onRemoveItem }) => (
  <li className="elemento">
    <span style={{ ancho: '40%' }}>
      <a href={item.url}>{item.title}</a>
    <span
      style={{ ancho: '30%' }}>{item.author} <span style={{ ancho: '10%' }}
    >{item.num_comments} <span style={{ ancho: '10%' }}>{item.points} <span style={{ ancho:
    '10%' }}>

    <botón
      tipo="botón"
      onClick={() => onRemoveItem(elemento)}
      nombreDeClase="botón botón_pequeño"
    >
      Despedir
    </botón>

  </li>
);
```

Como puede ver, también podemos usar el atributo de estilo para elementos HTML. En JSX, el estilo se puede pasar como un objeto JavaScript en línea a estos atributos. De esta forma, podemos definir propiedades de estilo dinámicas en archivos JavaScript, en lugar de archivos CSS estáticos. Este enfoque se denomina estilo en línea y resulta útil para la creación rápida de prototipos y la definición de estilos dinámicos. Sin embargo, el estilo en línea debe usarse con moderación, ya que una definición de estilo independiente con un archivo CSS permite que el JSX sea más conciso.

En el archivo `src/App.css`, defina las nuevas clases CSS. Aquí se utilizan las funciones CSS básicas, ya que las funciones CSS avanzadas (p. ej., anidación) de las extensiones CSS (p. ej., Sass) no se incluyen en este ejemplo, ya que son configuraciones opcionales²³ :

`src/App.css`

```
.item
{ display: flex; align-
  items: center; padding-
  bottom: 5px;
}

.item > span {
  relleno: 0 5px;
  espacio en blanco: nowrap;
  desbordamiento: oculto;
  espacio en blanco: nowrap;
  desbordamiento de texto: elipsis;
}

.item > span > a {
  color: heredar;
}
```

Aún falta el estilo de botón del componente anterior, por lo que definiremos un estilo de botón base y dos más específicos (pequeño y grande). Una de las especificaciones de botón ya se ha utilizado; la otra se usará en los siguientes pasos:

²³ <https://bit.ly/3E1a2bM>

src/App.css

```

.button
{ fondo: transparente; borde: 1px
sólido #171212;
relleno: 5px;
cursor: puntero;

transición: todos los 0,1 s entran lentamente;
}

.botón:hover {
fondo: #171212; color: #ffffff;
}

.button_small
{ relleno: 5px;
}

.botón_grande {
relleno: 10px;
}

```

Además de los enfoques de estilo en React, las convenciones de nomenclatura ([directrices CSS²³](#)) Son un tema completamente distinto. El último fragmento de CSS siguió las reglas BEM al definir las modificaciones de una clase con un guion bajo (_). Elija la convención de nomenclatura que mejor se adapte a usted y a su equipo. Sin más preámbulos, daremos estilo al siguiente componente de React:

src/App.jsx

```

const FormularioDeBúsqueda = ({ ... }) => (
  <form action={searchAction} className="formulario-de-búsqueda">
    <InputWithLabel ... >
      <strong>Buscar:</strong> </
    InputWithLabel>

    <botón
      tipo="enviar"
      deshabilitado={!searchTerm}
      nombreDeClase="botón botón_grande"
    >
      Entregar

```

²³ <https://mzl.la/3m5avnb>

```
        </botón>
      </form>
    );
```

También podemos pasar el atributo `className` como propiedad a los componentes de React. Por ejemplo, podemos usar esta opción para pasar al componente `SearchForm` un estilo flexible con una propiedad `className` de varias clases predefinidas (por ejemplo, `button_large` o `button_small`) desde un archivo CSS. Por último, aplicar estilo al componente `InputWithLabel`:

`src/App.jsx`

```
const InputWithLabel = ({ ... }) => {
  ...

  devolver (
    <>
      <label htmlFor={id} className="label"> {hijos}

      </etiqueta>
      &nbsp;
      <input
        ref={inputRef}
        id={id}
        tipo={tipo}
        valor={valor}
        onChange={onInputChange}
        className="input"
      />
    </>
  );
};
```

En su archivo `src/App.css`, agregue las clases restantes:

src/App.css

```
.search-form
{
  relleno: 10px 0 20px 0;
  visualización: flex;
  alinear elementos: línea base;
}

.label
{
  borde superior: 1px sólido #171212;
  borde izquierdo: 1px sólido #171212;
  relleno izquierdo: 5px;
  tamaño de fuente: 24px;
}

.input
{
  borde: ninguno;
  borde inferior: 1px sólido #171212; color de
  fondo: transparente;

  tamaño de fuente: 24px;
}
```

Para simplificar, aplicamos estilos a elementos como la etiqueta y la entrada individualmente en el archivo `src/App.css`. Sin embargo, en una aplicación real, puede ser mejor definir estos elementos una sola vez en el archivo `src/index.css` de forma global. Como los componentes de React se dividen en varios archivos, compartir estilos se vuelve necesario. Al fin y al cabo, este es el uso básico de CSS en React. Sin embargo, sin extensiones CSS como Sass (Hojas de Estilo Sintácticamente Impresionantes), aplicar estilos puede resultar más complicado, ya que funciones como la anidación de CSS no están disponibles en CSS nativo.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor²³ .
 - Recapitular todos los [cambios del código fuente](#)²³ de esta sección. •

Intente pasar la propiedad `className` de la aplicación al componente `SearchForm`, ya sea con el valor `button_pequeño` o `botón_grande`, y use esto como nombre de clase para el elemento del botón.

²³ <https://tinyurl.com/3uncn45p>

²³ <https://tinyurl.com/2588rxvc>

Módulos CSS en React

Los módulos CSS son un enfoque avanzado de CSS dentro de CSS . El archivo CSS se mantiene igual, donde se pueden aplicar extensiones CSS como Sass, pero su uso en los componentes de React cambia. Para habilitar los módulos CSS en Vite, cambie el nombre del archivo src/App.css a src/App.module.css. Esta acción se realiza en la línea de comandos desde el directorio de su proyecto:

Línea de comandos

```
mv src/App.css src/App.módulo.css
```

En el archivo src/App.module.css renombrado, comience con las primeras definiciones de clase CSS, como antes:

src/App.module.css

```
.container
{ altura: 100vw;
relleno: 20px;

fondo: #83a4d4; /* respaldo para navegadores antiguos */ fondo:
gradiente lineal (a la izquierda, #b6fbff, #83a4d4);

color: #171212;
}

.headlinePrimary
{ tamaño de fuente:
48px; peso de fuente:
300; espaciado entre letras: 2px;
}
```

Importa de nuevo el archivo src/App.module.css con una ruta relativa. Esta vez, impórtalo como un objeto JavaScript, donde el nombre del objeto (en este caso: estilos) lo decides tú:

src/App.jsx

```
importar * como React desde 'react';
importar axios desde 'axios';
importar estilos desde './App.module.css';
```

En lugar de definir className como una cadena asignada a un archivo CSS, acceda a la clase CSS directamente desde el objeto de estilos y asígnelo con una expresión JavaScript en JSX a sus elementos.

src/App.jsx

```

constante Aplicación = () => {
  ...

  devolver
    ( <div className={styles.container}> <h1
      className={styles.headlinePrimary}>Mis historias de hackers</h1>

      <Formulario de búsqueda
        searchTerm={término de
          búsqueda} onSearchInput={handleSearchInput}
          searchAction={searchAction} />

      {stories.isError && <p>Algo salió mal...</p>}

      {historias.isLoading ?
        ( <p>Cargando ...</p> ) :
        (
          <List list={historias.data} onRemoveItem={handleRemoveStory} /> ) } </div>

    );
};

```

Hay varias maneras de agregar varias clases CSS mediante el objeto de estilos al atributo className del elemento . Aquí, usamos literales de plantilla de JavaScript:

src/App.jsx

```

constante Item = ({ item, onRemoveItem }) => (
  <li className={estilos.item}>
    <span style={{ ancho: '40%' }}>
      <a href={item.url}>{item.title}</a>
    <span
      style={{ ancho: '30%' }}>{item.author} <span style={{ ancho: '10%' }}
      >{item.num_comments} <span style={{ ancho: '10%' }}>{item.points} <span
      style={{ ancho: '10%' }}>

      <botón
        tipo="botón"
        onClick={() => onRemoveItem(elemento)}
        nombreDeClase={` ${styles.button} ${styles.buttonSmall}`}

```

```

      >
        Despedir
      </botón> </
    li>
  );

```

También podemos añadir estilos en línea como estilos más dinámicos en JSX. También es posible añadir una extensión CSS como Sass para habilitar funciones avanzadas como la anidación de CSS (véase la sección anterior). Sin embargo, nos ceñiremos a las funciones nativas de CSS:

src/App.module.css

```

.item
{
  pantalla: flex;
  alinear elementos: centro;
  relleno inferior: 5px;
}

.item > span {
  relleno: 0 5px;
  espacio en blanco: nowrap;
  desbordamiento: oculto;
  espacio en blanco: nowrap;
  desbordamiento de texto: elipsis;
}

.item > span > a { color:
  heredar;
}

```

Luego, las clases CSS del botón en el archivo src/App.module.css:

src/App.module.css

```

.button
{
  fondo: transparente; borde: 1px
  sólido #171212; relleno: 5px; cursor:
  puntero;

  transición: todos los 0,1 s entran lentamente;
}

```

```

.botón: hover {
  fondo: #171212; color: #ffffff;
}

.botónPequeño {
  relleno: 5px;
}

.botónGrande {
  relleno: 10px;
}

```

Aquí se observa un cambio hacia convenciones de nomenclatura pseudo BEM, a diferencia de `button_small` y `button_large` de la sección anterior. Si la convención de nomenclatura anterior se cumple, solo podemos acceder al estilo con `styles["button_small"]`, lo que lo hace más detallado debido a la limitación de JavaScript con los guiones bajos de objeto. Las mismas deficiencias se aplicarían a las clases definidas con un guion (-). En cambio, ahora podemos usar `styles.buttonSmall` (véase: Componente Item).

src/App.jsx

```

const SearchForm = ({ ... }) => ( <form
  acción={searchAction} nombreClase={styles.searchForm}>
    <InputWithLabel ... >
      <strong>Buscar:</strong> </
    InputWithLabel>

    <botón
      tipo="enviar"
      deshabilitado={!searchTerm}
      nombreDeClase={` ${styles.button} ${styles.buttonLarge}`}
    >
      Entregar
    </botón>
  </form>
);

```

El componente `SearchForm` también recibe los estilos. Debe usar la interpolación de cadenas para usar dos estilos en un elemento mediante los literales de plantilla de JavaScript. Una alternativa es `clsx`²³ biblioteca, que se instala a través de la línea de comandos como una dependencia del proyecto:

²³ <https://bit.ly/3DNEA3R>

src/App.jsx

```
importar clsx desde 'clsx';
```

```
...
```

```
// en algún lugar de un atributo className
```

```
nombreDeClase={clsx(estilos.botón, estilos.botónGrande)}
```

La biblioteca también ofrece estilo condicional; mientras que el lado izquierdo de la propiedad del objeto debe usarse como un nombre de propiedad calculado²³ y solo se aplica si el lado derecho se evalúa como verdadero:

src/App.jsx

```
importar clsx desde 'clsx';
```

```
...
```

```
// En algún lugar de un atributo className
```

```
className={clsx(styles.button, { [styles.buttonLarge]: isLarge })}
```

Finalmente, continuamos con el componente InputWithLabel:

src/App.jsx

```
constante InputWithLabel = ({ ... }) => {
```

```
...
```

```
devolver (
```

```
<>
```

```
<label htmlFor={id} className={styles.label}> {hijos} </label>
```

```
&nbsp;
```

```
<entrada
```

```
  ref={inputRef} id={id}
```

```
  tipo={tipo}
```

```
  valor={valor}
```

```
  onChange={onInputChange}
```

```
  nombreClase={styles.input} />
```

```
</>
```

²³ <https://mzl.la/2XuN651>

```
); };
```

Y termina el estilo restante en el archivo `src/App.module.css`:

`src/App.module.css`

```
.formulario de búsqueda {
  relleno: 10px 0 20px 0; visualización:
  flex; alinear
  elementos: línea base;
}

.label
{ borde superior: 1px sólido #171212; borde
  izquierdo: 1px sólido #171212; relleno izquierdo:
  5px; tamaño de fuente:
  24px;
}

.input
{ borde: ninguno;
  borde inferior: 1px sólido #171212; color de fondo:
  transparente;

  tamaño de fuente: 24px;
}
```

Se aplica la misma precaución que en la sección anterior: algunos de estos estilos, como entrada y etiqueta, podrían ser más eficientes en un archivo `src/index.css` global sin módulos CSS.

De nuevo, los módulos CSS, como cualquier otro enfoque CSS dentro de CSS, pueden usar Sass para funciones CSS más avanzadas, como la anidación. La ventaja de los módulos CSS es que recibimos un error en JavaScript cada vez que no se define un estilo. Con el enfoque CSS estándar, los estilos no coincidentes en archivos JavaScript y CSS podrían pasar desapercibidos.

Ceremonias:

- Compare su código fuente con el código [fuente](https://tinyurl.com/3h4j6uhr) del autor² .
 - Recapitular todos los [cambios del código fuente](https://tinyurl.com/2m4apv8m)² ¹ de esta sección.

² <https://tinyurl.com/3h4j6uhr>

² ¹<https://tinyurl.com/2m4apv8m>

Componentes con estilo en React

Con los enfoques anteriores de CSS-in-CSS, los componentes con estilo son uno de los varios enfoques para CSS-in-JS. Elegí los componentes con estilo porque son los más populares. Vienen como una dependencia de JavaScript, por lo que debemos instalarlos en la línea de comandos:

Línea de comandos

```
npm install componentes con estilo
```

Luego impórtalo en tu archivo src/App.jsx:

src/App.jsx

```
importar * como React desde 'react';  
importar axios desde 'axios';  
importar con estilo desde 'styled-components';
```

Como su nombre indica, CSS-in-JS se ejecuta en el archivo JavaScript. En el archivo src/App.jsx, define los primeros componentes con estilo:

src/App.jsx

```
const StyledContainer = styled.div` altura:  
  100vw; relleno:  
  20px;  
  
  fondo: #83a4d4; fondo:  
  gradiente-lineal(a la izquierda, #b6fbff, #83a4d4);  
  
  color: #171212; `;  
  
const StyledHeadlinePrimary = styled.h1` tamaño de  
  fuente: 48px; peso  
  de fuente: 300;  
  espaciado entre letras: 2px;  
  `;
```

Al usar componentes con estilo, se utilizan los literales de plantilla de JavaScript de la misma forma que las funciones de JavaScript. Todo lo que está entre comillas invertidas se considera un argumento, y el objeto con estilo permite acceder a todos los elementos HTML necesarios (p. ej., div, h1) como funciones. Al llamar a una función con el estilo, esta devuelve un componente de React que puede usarse en el componente de la aplicación:

src/App.jsx

```

const App = () => {
  ...

  return (
    <Contenedor con estilo>
      <StyledHeadlinePrimary>Mis historias de hackers</StyledHeadlinePrimary>

      <Formulario de búsqueda
        searchTerm={término de
        búsqueda} onSearchInput={handleSearchInput}
        searchAction={searchAction} />

      {stories.isError && <p>Algo salió mal...</p>}

      {historias.isLoading ?
        ( <p>Cargando ...</p> ) :
        (
          <List list={historias.data} onRemoveItem={handleRemoveStory} /> )}

    </Contenedor con
estilo> );
  };

```

Este tipo de componente de React sigue las mismas reglas que un componente de React común. Todo lo que se pasa entre sus etiquetas de elemento se pasa automáticamente como propiedad secundaria de React. Para el componente `Item`, esta vez no usamos estilos en línea, sino que definimos un componente con estilo específico para él. `StyledColumn` recibe sus estilos dinámicamente mediante una propiedad de React:

src/App.jsx

```

const Item = ({ item, onRemoveItem }) => ( <StyledItem>
  <StyledColumn
    width="40%"> <a href={item.url}
      >{item.title}</a> </StyledColumn> <StyledColumn
    width="30%">{item.author}
  </StyledColumn> <StyledColumn width="10%">{item.num_comments}</
    StyledColumn> <StyledColumn width="10%">{item.points}</StyledColumn>
    <StyledColumn width="10%"> <StyledButtonSmall
      tipo="botón"

```



```

      onClick={() => onRemoveItem(elemento)}
    >
      Despedir
    </BotónEstilizadoPequeño>
  </ColumnaEstilizada>
</ElementoEstilizado> );

```

La propiedad de ancho flexible es accesible en el literal de plantilla del componente con estilo como argumento de una función en línea. El valor de retorno de la función se aplica como una cadena. Dado que podemos usar retornos inmediatos al omitir el cuerpo de la función de flecha, esta se convierte en una función en línea concisa:

src/App.jsx

```
constante StyledItem = styled.li`
```

```
  pantalla: flex; aligner
```

```
  elementos: centro; relleno
```

```
  inferior: 5px; `;
```

```
const StyledColumn = styled.span` relleno: 0 5px;
```

```
  espacio en blanco: nowrap;
```

```
  desbordamiento: oculto;
```

```
  espacio en blanco: nowrap;
```

```
  desbordamiento de texto: elipsis;
```

```
  a
```

```
    { color: heredar;
```

```
  }
```

```
  ancho: ${({props}) => props.width}; `;
```

Funciones avanzadas como la anidación de CSS están disponibles de forma predeterminada en los componentes con estilo. Se puede acceder a los elementos anidados y el elemento actual se puede seleccionar con el operador CSS `&` .

src/App.jsx

```
const StyledButton = styled.button` fondo:
  transparente; borde: 1px sólido
  #171212; relleno: 5px; cursor:
  puntero;

  transición: todos los 0,1 s entran lentamente;

  &:hover
    { fondo: #171212; color:
      #ffffff;

} `;
```

También puedes crear versiones especializadas de componentes con estilo pasando otro componente a la función de la biblioteca. El botón especializado recibe todos los estilos base del componente StyledButton definido previamente:

src/App.jsx

```
const StyledButtonSmall = styled(StyledButton)` relleno: 5px;

`;

const StyledButtonLarge = styled(StyledButton)` relleno: 10px;

`;

const StyledSearchForm = styled.form`
  relleno: 10px 0 20px 0;
  visualización: flex;
  alinear elementos: línea base;

`;
```

Al usar un componente con estilo como StyledSearchForm, su elemento de formulario subyacente se usa en la salida HTML real. Podemos seguir usando los atributos HTML nativos (onSubmit, type, disabled) allí:

src/App.jsx

```

const FormularioDeBúsqueda = ({ ... }) => (
  <Acción StyledSearchForm={AcciónDeBúsqueda}>
    <EntradaConEtiqueta
      id="buscar"
      valor={término de búsqueda}
      está enfocado
      onChange={onSearchInput}
    >

    <strong>Buscar:</strong>
  </EntradaConEtiqueta>

  <StyledButtonLarge type="enviar" deshabilitado={!searchTerm}>
    Entregar
  </BotónEstilizadoGrande>
</Forma de búsqueda con
estilo> );

```

Finalmente, el InputWithLabel decorado con sus componentes con estilo aún no definidos:

src/App.jsx

```

const InputWithLabel = ({ ... }) => {
  ...

  devolver (
    <>
      <StyledLabel htmlFor={id}>{hijos} </StyledLabel>

      <StyledInput
        ref={inputRef} id={id}
        tipo={tipo}
        valor={valor}

        onChange={onInputChange} />

    </>

  > );
};

```

Y sus componentes de estilo correspondientes se definen en el mismo archivo:

src/App.jsx

```
const StyledLabel = styled.label` borde
  superior: 1px sólido #171212; borde
  izquierdo: 1px sólido #171212; relleno
  izquierdo: 5px; tamaño
  de fuente: 24px;
`;

const StyledInput = styled.input`
  borde: ninguno;
  borde inferior: 1px sólido #171212; color de
  fondo: transparente;

  tamaño de fuente: 24px;
`;
```

CSS-in-JS con componentes con estilo cambia el foco de la definición de estilos a los componentes React reales. Los componentes con estilo son estilos definidos como componentes de React sin el archivo CSS intermedio. Si un componente con estilo no se usa en JavaScript, tu IDE/editor te lo indicará. Los componentes con estilo se agrupan junto con otros recursos JavaScript en archivos JavaScript para una aplicación lista para producción. No hay archivos CSS adicionales, solo JavaScript cuando se usa la estrategia CSS-in-JS. Ambas estrategias, CSS-in-JS y CSS-in-CSS, y sus enfoques (por ejemplo, componentes con estilo y módulos CSS) son populares entre los desarrolladores de React. Usa la que mejor se adapte a ti y a tu equipo.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² ².
 - Recapitular todos los [cambios del código fuente](#)² ³ de esta sección.
- Lea más sobre [las mejores prácticas para componentes con estilo en React](#)² ².
- Normalmente no hay un archivo src/index.css para estilos globales al usar componentes con estilo. Descubre cómo usar estilos globales al usar componentes con estilo con tu buscador favorito.

² <https://tinyurl.com/2kt94677>

² <https://tinyurl.com/4b49zjac>

² <https://www.robinwieruch.de/componentes-con-estilo/>

SVG en React

Para crear una aplicación React moderna, probablemente necesitemos usar SVG. En lugar de asignar texto a cada botón, por ejemplo, podríamos simplificarlo con un icono. En esta sección, usaremos un gráfico vectorial escalable (SVG) como icono en uno de nuestros componentes React.

Importante: Esta sección se basa en el tema "CSS en React" que abordamos anteriormente, lo que nos ayuda a darle al icono SVG una buena apariencia desde el principio. Es aceptable usar un enfoque de estilo diferente (por ejemplo, Módulos CSS, componentes con estilo) o ningún estilo en absoluto, aunque el SVG podría verse extraño sin él.

Vite no es compatible con SVG. Para permitir SVG en Vite, debemos instalar uno de sus plugins mediante la línea de comandos:

Línea de comandos

```
npm instalar vite-plugin-svgr --save-dev
```

A continuación, el nuevo complemento para SVG se puede utilizar para la configuración de Vite:

vite.config.js

```
importar { defineConfig } de 'vite'; importar react  
de '@vitejs/plugin-react'; importar svgr de 'vite-plugin-  
svgr';
```

```
// https://vitejs.dev/config/ exportar  
predeterminado defineConfig({ plugins:  
  [react(), svgr()], });
```

Eso es todo por la configuración general. Usaremos el siguiente SVG² de Heroicons² y crea un nuevo archivo src/check.svg:

src/check.svg

```
<svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" ancho de trazo\  
="1.5" trazo="colorActual" clase="tamaño-6">  
  <path stroke-linecap="redondo" stroke-linejoin="redondo" d="m4.5 12.75 6 6 9-13.5" />  
</svg>
```

Ahora podemos importar SVG (similar a CSS) como componentes de React inmediatamente. En src/App.jsx, use la siguiente sintaxis para importar el SVG:

² <https://bit.ly/3w4xNRz>

² <https://heroicons.com/>

src/App.jsx

```
importar * como React desde 'react'; importar
axios desde 'axios'; importar './App.css';
importar Check desde './
check.svg?react";
```

Aquí importamos un SVG para usarlo como icono. Sin embargo, esto funciona para muchos usos diferentes, como logotipos y fondos. Ahora, en lugar del texto del botón "Descartar", pase el componente SVG con los atributos de altura y anchura :

src/App.jsx

```
constante Item = ({ item, onRemovelItem }) => (
  <li className="elemento">
    <span style={{ ancho: '40%' }}>
      <a href={item.url}>{item.title}</a>
    <span
      style={{ ancho: '30%' }}>{item.author} <span style={{ ancho: '10%' }}
      >{item.num_comments} <span style={{ ancho: '10%' }}>{item.points} <span style={{ ancho:
      '10%' }}>

      <botón
        tipo="botón"
        onClick={() => onRemovelItem(elemento)}
        nombreDeClase="botón botón_pequeño"
      >

      <Comprobar altura="18px" ancho="18px" /> </botón>
    </li> );
```

El plugin Vite facilita el uso de SVG, sin necesidad de mucha configuración adicional. Esto es diferente si creas un proyecto de React desde cero con herramientas de compilación como Webpack, ya que debes encargarte tú mismo. En cualquier caso, los SVG hacen que tu aplicación sea más accesible, así que úsalos cuando te convenga.

Ceremonias:

- Compare su código fuente con el código [fuente](https://tinyurl.com/3nf39wy3) del autor² .

² <https://tinyurl.com/3nf39wy3>

– Recapitular todos los [cambios del código fuente](#)² de esta
sección. • Integrar la biblioteca de terceros [react-icons](#)² en su aplicación y utilice sus símbolos SVG
importándolos como componentes React de inmediato.

² <https://tinyurl.com/mrsfx32y>

² <https://bit.ly/3nayoJ7>

Mantenimiento de React

A medida que una aplicación React crece, el mantenimiento se convierte en una prioridad. Para prepararnos para esta eventualidad, abordaremos la optimización del rendimiento, la seguridad de tipos, las pruebas y la estructura del proyecto. Cada uno de estos temas fortalecerá tu aplicación para que pueda incorporar más funcionalidad sin perder calidad.

La optimización del rendimiento evita que las aplicaciones se ralenticen al garantizar el uso eficiente de los recursos disponibles. Los lenguajes de programación tipificados, como TypeScript, detectan errores en una etapa temprana del ciclo de retroalimentación. Las pruebas nos brindan retroalimentación más explícita que la programación tipificada y nos permiten comprender qué acciones pueden afectar la aplicación. Por último, la estructura de un proyecto facilita la gestión organizada de recursos en carpetas y archivos, lo cual resulta especialmente útil en escenarios donde los miembros del equipo trabajan en diferentes dominios.

Rendimiento en React (Avanzado)

Esta sección está aquí únicamente para aprender sobre las mejoras de rendimiento en React. No necesitaríamos optimizaciones en la mayoría de las aplicaciones React, ya que React es rápido desde el primer momento. Aunque existen herramientas más sofisticadas para medir el rendimiento en JavaScript y React, nos ceñiremos a un simple `console.log()` y a las herramientas de desarrollo de nuestro navegador para la salida del registro.

Modo estricto

Antes de que podamos aprender sobre el rendimiento en React, veremos brevemente el **modo estricto de React** que se habilita en el archivo `src/main.jsx`:

`src/main.jsx`

```
createRoot(document.getElementById('root')).render(  
  <Modo estricto>  
    <Aplicación />  
  </Modo estricto>  
);
```

Modo estricto de React² Es un componente auxiliar que notifica a los desarrolladores en caso de que algo falle en nuestra implementación. Por ejemplo, al usar un **obsoleto²** ¹ La API de React (por ejemplo, usando un gancho de React heredado) nos mostraría una advertencia en las herramientas de desarrollo del navegador. Sin embargo, también garantiza que el desarrollador implemente correctamente el estado y los efectos secundarios. Veamos qué significa esto en nuestro código.

El componente App obtiene inicialmente datos de una API remota, que se muestran como una lista. Usamos el gancho `useEffect` de React para inicializar la obtención de datos. Ahora, te recomiendo agregar un objeto `console.log()` que registre cada vez que se ejecuta este gancho:

`src/App.jsx`

```
constante Aplicación = () => {  
  ...  
  
  React.useEffect(() =>  
    { console.log('¿Cuántas veces debo registrar?');  
      handleFetchStories(); },  
    [handleFetchStories]);  
  
  ...  
};
```

² <https://bit.ly/48TUA0k>

² ¹<https://bit.ly/3R8ycam>

Muchos esperarían ver el registro solo una vez en las herramientas de desarrollo del navegador, ya que este efecto secundario solo debería ejecutarse una vez (o si cambia la función `handleFetchStories`). Sin embargo, verá el registro dos veces durante el renderizado inicial del componente `App`. Siendo honestos, este es un comportamiento muy inesperado (incluso para desarrolladores experimentados de React), lo que dificulta su comprensión para principiantes. No obstante, el equipo principal de React decidió que este comportamiento es necesario para detectar errores relacionados con el uso incorrecto de efectos secundarios en la aplicación.

El modo estricto de React ejecuta los ganchos `useEffect` de React dos veces para el renderizado inicial. Dado que esto resulta en la obtención de los mismos datos dos veces, no supone un problema. Esta operación se denomina idempotente, lo que significa que el resultado de una solicitud realizada correctamente es independiente del número de veces que se ejecuta. Al fin y al cabo, solo representa un problema de rendimiento, ya que hay dos solicitudes de red, pero no genera errores en la aplicación. Además de toda esta incertidumbre, el modo estricto solo se aplica al entorno de desarrollo, por lo que, al compilar la aplicación para producción, se elimina automáticamente.

Ambos comportamientos, ejecutar el hook `useEffect` de React dos veces para la renderización inicial y tener diferentes resultados entre el desarrollo y la producción, generan muchas discusiones justificadas en torno al modo estricto de React.

Para las siguientes secciones de rendimiento, les recomiendo deshabilitar el modo estricto simplemente eliminándolo. De esta manera, podemos seguir el registro que se generaría para esta aplicación una vez compilada para un entorno de producción:

```
src/main.jsx
```

```
createRoot(document.getElementById('root')).render(<App />);
```

Sin embargo, al final de las secciones de rendimiento, te animo a que vuelvas a agregar el Modo Estricto, porque después de todo, está ahí para ayudarte.

No ejecutar en el primer render

Anteriormente, vimos el gancho `useEffect` de React, que se usa para efectos secundarios. Se ejecuta la primera vez que un componente se renderiza (montaje) y luego en cada nuevo renderizado (actualización). Al pasarle una matriz de dependencias vacía como segundo argumento, podemos indicarle que se ejecute solo en el primer renderizado. De fábrica, no hay forma de indicarle al gancho que se ejecute solo en cada re-renderizado (actualización) y no en el primer renderizado (montaje). Por ejemplo, examine nuestro gancho personalizado para la gestión de estados con el gancho `useState` de React y su estado semipersistente con almacenamiento local usando el gancho `useEffect` de React:

src/App.jsx

```
const useStorageState = (clave, estadoinicial) => { const [valor,
  establecerValor] = React.useState(
    localStorage.getItem(clave) || estadoinicial );
```

```
  React.useEffect(() =>
    { console.log('A');
      localStorage.setItem(clave, valor);
    }, [valor, clave]);
```

```
  devolver [valor, valorEstablecer]; };
```

Al examinar las herramientas de desarrollo con más detalle, podemos ver el registro por primera vez cuando el componente se renderiza usando este gancho personalizado. Sin embargo, no tiene sentido ejecutar el efecto secundario para el renderizado inicial del componente, ya que no hay nada que almacenar en el almacenamiento local excepto el valor inicial. Es una invocación de función redundante y solo debería ejecutarse para cada actualización (re-renderizado) del componente.

Como se mencionó, no hay ningún Hook de React que se ejecute en cada re-renderizado, y no hay forma de indicarle al hook useEffect, de forma idiomática, que llame a su función solo en cada re-renderizado. Sin embargo, al usar el Hook useRef de React, que mantiene su propiedad ref.current intacta durante los re-renderizados, podemos mantener un estado predefinido (sin re-renderizar el componente al actualizarlo) con una variable de instancia del ciclo de vida de nuestro componente:

src/App.jsx

```
const useStorageState = (clave, estadoinicial) => { const isMounted
  = React.useRef(false);
```

```
  const [valor, establecerValor] = React.useState(
    localStorage.getItem(clave) || estadoinicial );
```

```
  React.useEffect(() => {
    si (!estáMontado.actual) {
      isMounted.current = true; } else

    { console.log('A');
      localStorage.setItem(clave, valor);
    }
  }, [valor, clave]);
```

```
devolver [valor, valorEstablecer]; };
```

Estamos aprovechando la referencia y su propiedad actual mutable para la gestión imperativa del estado, sin que esto active un nuevo renderizado. Una vez que se llama al gancho desde su componente por primera vez (renderizado del componente), la propiedad actual de la referencia se inicializa con un booleano falso llamado `isMounted`. Como resultado, no se llama a la función de efecto secundario en `useEffect`; solo el indicador booleano de `isMounted` se cambia a verdadero en el efecto secundario. Cada vez que el gancho se ejecuta de nuevo (renderizado del componente), el indicador booleano se evalúa en el efecto secundario. Al ser verdadero, la función de efecto secundario se ejecuta. Durante la vida útil del componente, el booleano `isMounted` se mantendrá verdadero. Esto se hizo para evitar llamar a la función de efecto secundario por primera vez en un renderizado que utiliza nuestro gancho personalizado.

Lo anterior solo trataba sobre evitar la invocación de una función simple para la renderización de un componente por primera vez. Pero imagine que tiene un cálculo costoso en su efecto secundario o que el gancho personalizado se usa con frecuencia en la aplicación. Es más práctico implementar esta técnica para evitar invocaciones innecesarias de funciones.

Ceremonias:

- Lea más sobre [cómo ejecutar `useEffect` solo en `update`](#)².
- Lea más sobre [cómo ejecutar `useEffect` solo una vez](#)³.

No vuelva a renderizar si no es necesario

Anteriormente, exploramos el mecanismo de re-renderizado de React. Repetiremos este ejercicio para los componentes `App` y `List`. Para ambos componentes, agregue una declaración de registro:

```
src/App.jsx
```

```
const App = () => {
```

```
  ...
```

```
  console.log('B:App');
```

```
  return ( ... );
```

```
const List = ({ lista, onRemoveItem }) =>
```

```
  console.log('B:List') || (
```

```
    <ul>
```

```
      {lista.map((elemento) => (
```

² <https://www.robinwieruch.de/react-useeffect-only-on-update/>

³ <https://www.robinwieruch.de/react-useeffect-only-once/>

```

      <Articulo
        clave={item.objectID}
        elemento={item}
        onRemoveItem={onRemoveItem} /> ))}
    </
  ul>

);

```

Dado que el componente Lista no tiene cuerpo de función y los desarrolladores son perezosos y no quieren refactorizarlo para una simple sentencia de registro, el componente Lista usa el operador `||` en su lugar. Este es un truco útil para añadir una sentencia de registro a un componente de función sin cuerpo de función. Dado que `console.log()`, a la izquierda del operador, siempre evalúa como falso, el lado derecho del operador siempre se ejecuta.

Patio de juegos de código

```

función getTheTruth() { if
  (console.log('B:Lista')) { devolver
    verdadero; } de
  lo contrario
    { devolver falso;
  }
}

console.log(getTheTruth()); // B:Lista //
falso

```

Centrémonos en el registro en las herramientas de desarrollo del navegador al actualizar la página. Deberías ver un resultado similar. Primero se renderiza el componente App, seguido de sus componentes secundarios (p. ej. Componente de lista).

Visualización

B:Aplicación

Lista B

B:Aplicación

B:Aplicación

Lista B

Nuevamente: Si observa más registros, verifique si su archivo `*src/main.jsx` usa `<React.StrictMode>` como contenedor para su componente de aplicación. De ser así, elimine el modo estricto y revise sus registros nuevamente. Explicación: En el modo de desarrollo, el modo estricto de React renderiza un

componente dos veces para detectar problemas con su implementación con el fin de advertirle sobre ellos.

Este modo estricto se excluye automáticamente en aplicaciones en producción. Sin embargo, si no quiere confundirse con los múltiples renderizados, elimine el modo estricto del archivo `src/main.jsx`.*

Dado que un efecto secundario activa la obtención de datos después del primer renderizado, solo se renderiza el componente App, ya que el componente List se reemplaza por un indicador de carga en un renderizado condicional. Una vez que llegan los datos, ambos componentes se renderizan de nuevo.

Visualización

```
// renderizado inicial
```

B:Aplicación

Lista B

```
// Obtención de datos con carga en lugar del componente Lista
```

B:Aplicación

```
// re-renderizado con datos
```

B:Aplicación

Lista B

Hasta ahora, este comportamiento es aceptable, ya que todo se renderiza a tiempo. Ahora, profundizaremos en este experimento escribiendo en el campo de entrada del componente SearchForm. Deberías ver los cambios con cada carácter introducido en el elemento:

Visualización

B:Aplicación

Lista B

Lo sorprendente es que el componente Lista no debería volver a renderizarse, pero lo hace. La función de búsqueda no se ejecuta mediante su botón, por lo que la lista que se pasa al componente Lista a través del componente Aplicación permanece igual con cada pulsación de tecla. Este es el comportamiento predeterminado de React: volver a renderizar todo lo que se encuentra debajo de un componente (en este caso, el componente Aplicación) con un cambio de estado, lo cual sorprende a muchos. En otras palabras, si un componente padre se vuelve a renderizar, sus componentes descendientes también lo hacen. React lo hace por defecto, ya que impedir que los componentes secundarios vuelvan a renderizarse podría provocar errores. Dado que el mecanismo de rerenderizado de React suele ser bastante rápido por defecto, React fomenta la rerenderización automática de los componentes descendientes.

Sin embargo, a veces queremos evitar que se vuelva a renderizar. Por ejemplo, los conjuntos de datos grandes que se muestran en una tabla (p. ej., el componente Lista) no deberían volver a renderizarse si no se ven afectados por una actualización (p. ej., Componente de búsqueda). Es más eficiente realizar una comprobación de igualdad si algo cambia en el componente. Por lo tanto, podemos usar la API memo de React para realizar esta comprobación de igualdad para las propiedades:

src/App.jsx

```

constante Lista = React.memo(
  ({ lista, onRemoveItem }) =>
    console.log('B:Lista') || (
      <ul>
        {lista.map((elemento) => (
          <Artículo
            clave={item.objectID}
            elemento={item}
            onRemoveItem={onRemoveItem} /> ))}
        </
      <ul>

    )
  );

```

La API memo de React comprueba si las propiedades de un componente han cambiado. De lo contrario, no se vuelve a renderizar, aunque su componente principal sí lo haya hecho. Sin embargo, la salida permanece igual al escribir en el campo de entrada del formulario de búsqueda:

Visualización

B:Aplicación

Lista B

La lista pasada al componente List es la misma, pero el controlador de devolución de llamada onRemoveItem no. Si el componente App se vuelve a renderizar, siempre crea una nueva versión de este controlador de devolución de llamada como una nueva función. Anteriormente, usamos el gancho useCallback de React para evitar este comportamiento, creando una función solo en el renderizado inicial (o si una de sus dependencias ha cambiado):

src/App.jsx

```

constante Aplicación = () => {
  ...

  const handleRemoveStory = React.useCallback((item) => { dispatchStories({ tipo:
    'REMOVE_STORY',
    carga útil: item, }); }, []);

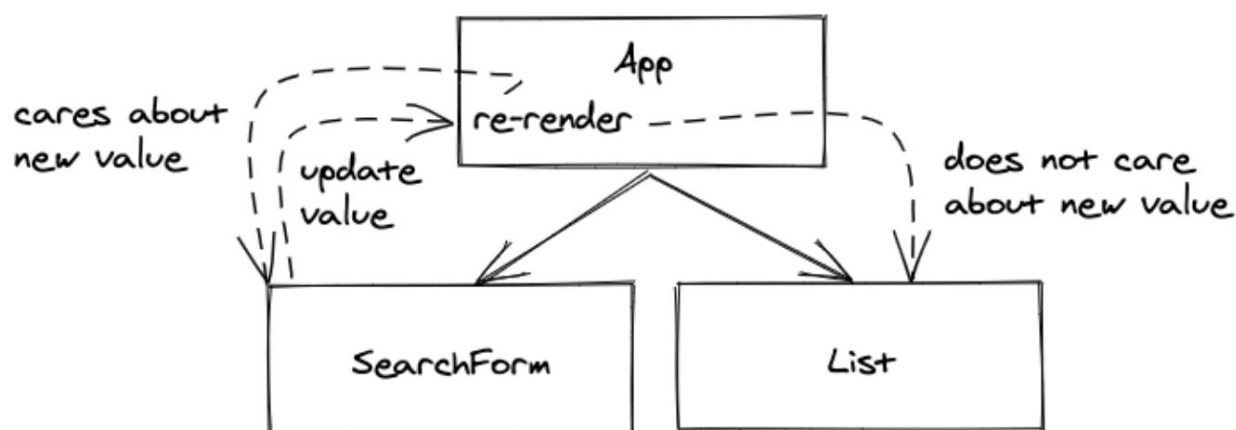
  ...

```

```
console.log('B:App');
```

```
devolver (...);
```

Dado que el controlador de devolución de llamada recibe el elemento pasado como argumento en la firma de su función, no tiene dependencias y solo se declara una vez cuando el componente App se renderiza inicialmente. Ninguna de las propiedades pasadas al componente List debería cambiar ahora. Pruébalo con la combinación de React memo y useCallback para buscar mediante el campo de entrada del SearchForm. El registro "B:List" desaparece y solo el componente App se vuelve a renderizar con su registro "B:App".



Aunque todas las propiedades pasadas a un componente permanecen iguales, el componente se renderiza de nuevo si su componente padre se ve obligado a volver a renderizarse. Este es el comportamiento predeterminado de React, que funciona la mayoría de las veces gracias a la rapidez del mecanismo de rerenderizado. Sin embargo, si el rerenderizado disminuye el rendimiento de una aplicación React, la API memo de React ayuda a evitarlo. Como hemos visto, a veces memo por sí solo no es suficiente. Los controladores de devolución de llamada se redefinen cada vez en el componente padre y se pasan como propiedades modificadas al componente, lo que provoca otro rerenderizado. En ese caso, se usa useCallback para que el controlador de devolución de llamada solo cambie cuando cambien sus dependencias.

Ceremonias:

- Lea más sobre la API [de notas de React²](#).
- Lea más sobre el gancho [useCallback de React²](#).

No vuelva a ejecutar cálculos costosos

A veces, tendremos cálculos de alto rendimiento en nuestros componentes de React (entre la firma de función de un componente y el bloque de retorno) que se ejecutan en cada renderizado. En este caso,

² <https://www.robinwieruch.de/react-memo/>

² <https://www.robinwieruch.de/react-usecallback-hook/>

Primero debemos crear un caso de uso en nuestra aplicación actual:

src/App.jsx

```
const getSumComments = (historias) =>
  { console.log('C');

    devolver historias.datos.reduce((resultado,
      valor) => resultado + valor.num_comentarios,
      0

    ); };

const App = () => {
  ...

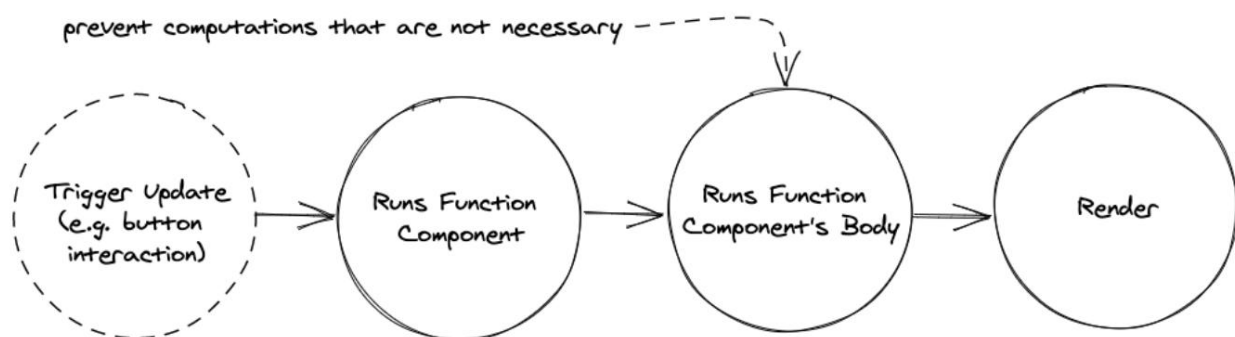
  const sumaComentarios = obtenerSumComentarios(historias);

  devolver (
    <div>
      Mis historias de hackers con {sumComments} comentarios.

      ...
    </div>

  ); };
```

Si se pasan todos los argumentos a una función, es aceptable tenerla fuera del componente, ya que no requiere ninguna dependencia adicional dentro del componente. Esto evita crear la función en cada renderizado, por lo que el gancho `useCallback` se vuelve innecesario. Sin embargo, la función sigue calculando el valor de los comentarios sumados en cada renderizado, lo que supone un problema para cálculos más costosos.



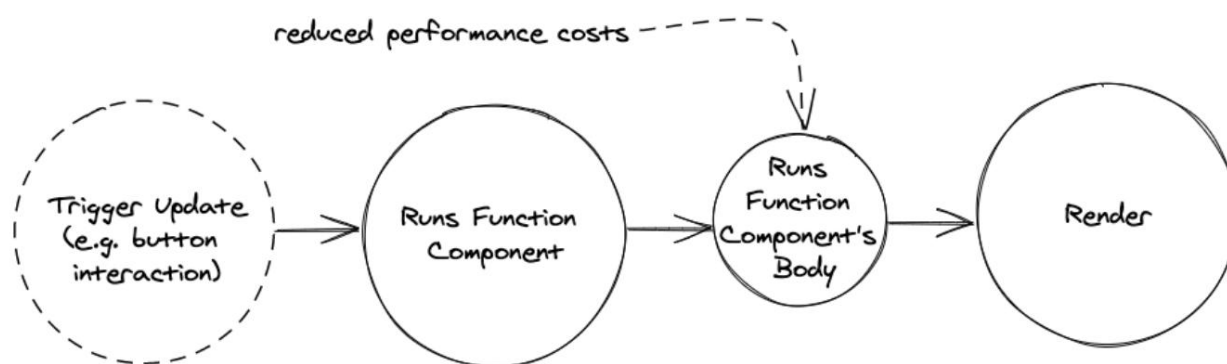
Cada vez que se escribe texto en el campo de entrada del componente SearchForm, este cálculo se ejecuta de nuevo con un resultado de "C". Esto puede ser adecuado para un cálculo sencillo como este, pero imaginemos que este cálculo tarda más de 500 ms. Esto provocaría un retraso en la renderización, ya que todo el componente debe esperar a este cálculo. Podemos indicar a React que solo ejecute una función si una de sus dependencias ha cambiado. Si ninguna dependencia ha cambiado, el resultado de la función permanece igual.

El hook `useMemo` de React nos ayuda aquí:

```
src/App.jsx
```

```
const Aplicación = () => {  
  ...  
  
  const sumaComentarios = React.useMemo( ()  
    => getSumComments(historias),  
    [historias] );  
  
  devolver ( ... );  
};
```

Cada vez que alguien escribe en el formulario de búsqueda, el cálculo no debería volver a ejecutarse. Solo se ejecuta si la matriz de dependencias (en este caso, "historias") ha cambiado. Al fin y al cabo, esto solo debería usarse para cálculos costosos que podrían retrasar la (re)renderización de un componente.



Ahora bien, después de analizar estos escenarios para `useMemo`, `useCallback` y `memo`, recuerda que no deberían usarse necesariamente por defecto. Aplica estas optimizaciones de rendimiento solo si encuentras cuellos de botella. En la mayoría de los casos, esto no debería ocurrir, ya que el mecanismo de renderizado de React es bastante eficiente por defecto. A veces, la comprobación de utilidades como `memo` puede ser más costosa que el propio re-renderizado.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor².

² <https://tinyurl.com/yz2jhjka>

- Recapitular todos los [cambios del código fuente](#)² de esta sección. • Lea más sobre el Hook [useMemo](#)² de React .
 - Descarga React Developer Tools como extensión para tu navegador. Ábrela para tu aplicación en el navegador mediante las herramientas de desarrollo y prueba sus diversas funciones. Por ejemplo, puedes usarla para visualizar el árbol de componentes de React y sus componentes en proceso de actualización.
- ¿Se vuelve a renderizar el formulario de búsqueda al eliminar un elemento de la lista con el botón "Descartar"? De ser así, aplique técnicas de optimización del rendimiento (usando `useCallback` y `memo`) para evitar que se vuelva a renderizar. ¿Se vuelve a renderizar cada elemento al eliminar un elemento de la lista con el botón "Descartar"? De ser así, aplique técnicas de optimización del rendimiento para evitar que se vuelva a renderizar.
- Elimine todas las optimizaciones de rendimiento para simplificar la aplicación. Nuestra aplicación actual no presenta cuellos de botella de rendimiento. [Evite optimizaciones prematuras](#)² . Utilice esta sección y el material de lectura adicional como referencia, en caso de que surjan problemas de rendimiento.

² <https://tinyurl.com/3423xu4v>

² <https://www.robinwieruch.de/react-usememo-hook/>

² <https://bit.ly/3AYktL8>

TypeScript en React

TypeScript para JavaScript y React ofrece numerosas ventajas para desarrollar aplicaciones robustas. En lugar de obtener errores de tipo en tiempo de ejecución en la línea de comandos o el navegador, la integración con TypeScript los presenta durante la compilación dentro del IDE. Esto acorta el ciclo de retroalimentación para el desarrollador y mejora su experiencia. Además, el código se vuelve más autodocumentado y legible, ya que cada variable está definida con un tipo. Asimismo, mover bloques de código o realizar una refactorización más extensa del código base se vuelve mucho más eficiente. Los lenguajes de tipado estático como TypeScript son tendencia debido a sus ventajas sobre los lenguajes de tipado dinámico como JavaScript.

Es útil aprender más [sobre TypeScript²](#) siempre que es posible.

Configuración de TypeScript

Para usar TypeScript en React (con Vite), instale TypeScript y sus dependencias en su aplicación usando la línea de comando:

Línea de comandos

```
npm instalar typescript @types/react @types/react-dom --save-dev npm instalar @typescript-  
eslint/eslint-plugin --save-dev npm instalar @typescript-eslint/parser --save-dev
```

Agregue tres archivos de configuración de TypeScript: uno para el entorno del navegador, uno para el entorno de Node y uno para fusionar ambas configuraciones:

Línea de comandos

```
toque tsconfig.json tsconfig.app.json tsconfig.node.json
```

En el archivo TypeScript para el entorno del navegador incluya la siguiente configuración:

tsconfig.app.json

```
{  
  "opcionesdelcompilador": {  
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo", "target": "ES2020",  
    "useDefineForClassFields":  
verdadero, "lib": ["ES2020", "DOM",  
    "DOM.Iterable"],  
    "módulo": "ESNext",  
    "skipLibCheck": verdadero,  
  
    /* Modo empaquetador */
```

² <https://bit.ly/3G0I3vL>

```
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": verdadero,
    "isolatedModules": verdadero,
    "moduleDetection": "forzar", "noEmit":
verdadero,
    "jsx": "react-jsx",

    /* Linting */ "strict":
verdadero,
    "noUnusedLocals": verdadero,
    "noUnusedParameters": verdadero,
    "noFallthroughCasesInSwitch": verdadero,
    "noUncheckedSideEffectImports": verdadero },

    "include": ["src"]
}
```

Luego, en el archivo TypeScript para el entorno Node, incluya más configuraciones:

tsconfig.node.json

```
{
  "opcionesdelcompilador": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo", "target": "ES2022", "lib":
["ES2023"], "module":
    "ESNext", "skipLibCheck":
verdadero,

    /* Modo empaquetador */
    "moduleResolution": "agrupador",
    "allowImportingTsExtensions": verdadero,
    "aisladoMódulos": verdadero,
    "móduloDetección": "forzar", "noEmit":
verdadero,

    /* Pelusa */
    "estricto": verdadero,
    "noUnusedLocals": verdadero,
    "noUnusedParameters": verdadero,
    "noFallthroughCasesInSwitch": verdadero,
    "noUncheckedSideEffectImports": verdadero
  },
}
```

```
"incluir": ["vite.config.ts"]
}
```

Finalmente, fusiona ambas configuraciones en el archivo de configuración principal de TypeScript:

tsconfig.json

```
{
  "archivos": [],
  "referencias":
    [ { "ruta": "./tsconfig.app.json" }, { "ruta": "./
    tsconfig.node.json" }
  ]
}
```

Si tienes una configuración ESLint, también debes adaptarla a TypeScript:

eslint.config.js

```
importar js desde "@eslint/js"; importar
globals desde "globals"; importar
reactHooks desde "eslint-plugin-react-hooks"; importar reactRefresh
desde "eslint-plugin-react-refresh"; importar tseslint desde "typescript-eslint";

exportar predeterminado
tseslint.config( { ignora:
  ["dist"] }, {
  extiende: [js.configs.recommended, ...tseslint.configs.recommended], archivos: ["**/*.ts,tsx"],
  opciones de idioma: {

    ecmaVersión: 2020,
    globales: globals.browser, },

  complementos:
    { "react-hooks": reactHooks, "react-
    refresh": reactRefresh, }, reglas: {

    ...reactHooks.configs.recommended.rules, "react/prop-
    types": "desactivado", "react-refresh/
    only-export-components": [
      "advertir",
      { allowConstantExport: true },
```

```
    }, },  
  
  });  
};
```

A continuación, cambie el nombre de todos los archivos JavaScript (.jsx) a archivos TypeScript (.tsx).

Línea de comandos

```
mv src/main.jsx src/main.tsx mv src/  
App.jsx src/App.tsx
```

Y en su archivo index.html, haga referencia al nuevo archivo TypeScript en lugar de a un archivo JavaScript:

índice.html

```
<!doctype html> <html  
lang="es">  
  <cabeza>  
    <meta charset="UTF-8" />  
    <link rel="icono" tipo="imagen/svg+xml" href="/vite.svg" /> <meta nombre="ventana  
gráfica" contenido="ancho=ancho-del-dispositivo, escala-inicial=1.0" /> <título>Vite + React</título> </  
cabeza>  
  
  <cuerpo>  
    <div id="raíz"></div>  
    <script type="module" src="/src/main.tsx"></script> </body> </html>
```

Es posible que también necesite un nuevo archivo vite-env.d.ts en la raíz de su proyecto con el siguiente contenido:

índice.html

```
/// <tipos de referencia="vite/cliente" />
```

Reinicie su servidor de desarrollo desde la línea de comandos. Podría encontrar errores de compilación en el navegador y el editor/IDE. Si no ve ningún error en su editor/IDE al abrir los archivos TypeScript renombrados (p. ej., src/App.tsx), intente instalar un complemento de TypeScript para su editor o una extensión de TypeScript para su IDE. Normalmente, debería ver líneas rojas debajo de todos los valores donde faltan definiciones de TypeScript.

Seguridad de tipos para funciones y componentes

La aplicación debería iniciarse; sin embargo, faltan las definiciones de tipo en los archivos src/main.tsx y src/App.tsx. Empecemos con la primera, ya que solo representa un pequeño cambio:

src/main.tsx

```
createRoot(document.getElementById("root!")).render(
  <Modo estricto>
    <Aplicación />
  </Modo Estricto> );
```

Sin este cambio, TypeScript debería mostrarnos el siguiente error: El argumento de tipo 'HTMLEle-ment | null' no es asignable al parámetro de tipo 'Element | DocumentFragment'. Se puede traducir como: "El elemento HTML devuelto de getElementById() podría ser nulo si no existe dicho elemento HTML, pero createRoot() espera que sea un elemento". Como sabemos con certeza que hay un elemento HTML con este identificador específico en el archivo index.html, estamos respondiendo a TypeScript con "Yo sé mejor" mediante el uso de una llamada aserción de tipo (aquí: como palabra clave) en TypeScript.

A continuación, agregaremos [seguridad de tipos](#)² ¹ Para todo el archivo src/App.tsx. Desde la perspectiva de un lenguaje de programación, un gancho personalizado de React es simplemente otra función. Sin embargo, en TypeScript, la entrada (y opcionalmente la salida) de una función debe ser de tipo seguro. Comencemos por configurar nuestro gancho useStorageState() como de tipo seguro, indicando a la función que espere dos argumentos como primitivas de cadena:

src/App.tsx

```
const useStorageState = (clave: cadena, initialState: cadena) => {
  ...
};
```

También podemos decirle a la función que devuelva una matriz ([]) con un primer valor (estado actual) de tipo cadena y un segundo valor (función de actualización de estado) que toma un nuevo valor (nuevo estado) de tipo cadena para no devolver nada (vacío):

src/App.tsx

```
const useStorageState = (
  clave: cadena,
  estadoinicial: cadena ):
[cadena, (nuevoValor: cadena) => void] => {
  ...

  devolver [valor, valorEstablecer]; };
```

Dado que TypeScript ya podía inferir este tipo del gancho useState de React, podríamos simplemente eliminar el tipo de retorno. Sin embargo, debemos declarar el array devuelto como constante de TypeScript, ya que, de lo contrario, el orden de las entradas en el array no sería conocido por otras partes de la aplicación.

² ¹<https://bit.ly/3jhm6xi>

origen/App.tsx

```
const useStorageState = (clave: cadena, initialState: cadena) => { const [valor,
  setValue] = React.useState(
    localStorage.getItem(clave) || estadoinicial );

  React.useEffect(() => {
    localStorage.setItem(clave, valor);
  }, [valor, clave]);

  devuelve [valor, setValue] como const; };
```

Sin embargo, en relación con React, considerando las mejoras previas en la seguridad de tipos para el gancho personalizado, no tuvimos que añadir tipos a los ganchos internos de React en el cuerpo de la función. Esto se debe a que la inferencia de tipos funciona la mayoría de las veces con los ganchos de React de fábrica. Si el estado inicial de un gancho useState de React es una cadena primitiva de JavaScript, el estado actual devuelto se inferirá como una cadena y la función de actualización de estado devuelta solo tomará una cadena como argumento y no devolverá nada.

Patio de juegos de código

```
const [value, setValue] = React.useState('React'); // Se infiere que
value es una cadena // setValue solo toma una
cadena como argumento
```

Sin embargo, si el estado inicial fuera nulo, tendríamos que indicarle a TypeScript todos los tipos potenciales del Hook useState de React (aquí con un tipo de unión en TypeScript, donde | realiza una unión de dos o más tipos). Un [genérico de TypeScript²](#) se utiliza para informar a la función (aquí: un gancho de React) al respecto:

Patio de juegos de código

```
const [value, setValue] = React.useState<string | null>(null); // value debe ser una
cadena o un valor nulo // setValue solo acepta una
cadena o un valor nulo como argumento
```

Si añadir seguridad de tipos se convierte en una consecuencia para una aplicación React y sus componentes, como en nuestro caso, existen varias maneras de abordarlo. Comenzaremos con las propiedades y el estado de los componentes hoja de nuestra aplicación. Por ejemplo, el componente Item recibe una historia (en este caso: item) y una función de control de devolución de llamada (en este caso: onRemoveItem). Empezando con un poco de detalle, podríamos añadir los tipos en línea para ambos argumentos de la función, como hicimos antes:

² <https://www.robinwieruch.de/typescript-generics/>

origen/App.tsx

```
constante Artículo =
  ({ artículo,
    onRemovelItem, }):

{ item:
  { objectID: cadena; url:
    cadena; título:
    cadena; autor:
    cadena;
    num_comentarios: número;
    puntos: número; };

  onRemovelItem: (item:
    { objectID: cadena; url:
    cadena; título:
    cadena; autor:
    cadena;
    num_comentarios: número;
    puntos: número; })
    => void; }) => (

  <li>
    ...
  </li>

);
```

Hay dos problemas: el código es extenso y tiene duplicados (ver: elemento). Solucionemos ambos problemas definiendo un tipo de historia personalizado fuera del componente, en la parte superior de src/App.jsx:

origen/App.tsx

```
tipo Historia =
  { objectID: cadena; url:
    cadena; título:
    cadena; autor:
    cadena;
    num_comentarios: número;
    puntos: numero; };

...
```

```
const Item = ({ item,
  onRemoveItem, }:
  { item: Historia;
    onRemoveItem: (item: Historia) => void; }) => (
  <li>
    ...
  </li>
);
```

El elemento es de tipo Story y la función onRemoveItem toma un elemento de tipo Story como argumento y no devuelve nada. A continuación, limpie el código definiendo las propiedades del componente Item como tipo externo:

origen/App.tsx

```
tipo ItemProps =
  { elemento:
    Historia; onRemoveItem: (elemento: Historia) =>
    void; };

constante Item = ({ item, onRemoveItem }: ItemProps) => (
  <li>
    ...
  </li>
);
```

Desde aquí, podemos navegar por el árbol de componentes hasta el componente Lista y aplicar las mismas definiciones de tipo a las propiedades. Pruébalo primero y luego revise la siguiente implementación:

origen/App.tsx

```
tipo ListProps = { lista:
  Historia[];
  onRemoveItem: (elemento: Historia) =>
  void; };

constante Lista = ({ lista, onRemoveItem }: ListProps) => (
  <ul>
    ...
  </ul>
);
```

La función `onRemoveItem` ahora se escribe dos veces para `ItemProps` y `ListProps`. Para mayor precisión, podría extraer esto a un tipo `OnRemoveItem` de TypeScript independiente y reutilizarlo para ambas definiciones de tipo de propiedad `onRemoveItem`. Sin embargo, tenga en cuenta que el desarrollo se vuelve cada vez más difícil a medida que los componentes se dividen en archivos diferentes. Por eso, mantendremos la duplicación.

A continuación, podemos reutilizar el tipo `Historia` para otros componentes. Por ejemplo, añadir el tipo `Historia` al controlador de devolución de llamada del componente `Aplicación`:

`src/App.tsx`

```
constante Aplicación = () => {
  ...

  const handleRemoveStory = (item: Historia) =>
    { dispatchStories({ tipo:
      'REMOVE_STORY', carga
      útil: item, }); };

  ...
};
```

La función `reductora` también gestiona el tipo de historia, sin afectarlo realmente debido a los tipos de estado y acción. Como desarrolladores de la aplicación, conocemos los objetos y sus formas que se pasan a esta función `reductora`:

`src/App.tsx`

```
tipo StoriesState = { datos:
  Story[]; isLoading:
  booleano; isError: booleano; };

tipo StoriesAction = { tipo:
  cadena; carga
  útil: cualquiera; };

const storiesReducer = (estado:
  StoriesState,
  acción: HistoriasAcción
) => {
  ...
};
```

El tipo Acción , con su cadena y cualquier definición de tipo (comodín de TypeScript) , sigue siendo demasiado amplio; y no se obtiene una seguridad de tipos real, ya que las acciones no son distinguibles. Podemos mejorarlo especificando cada acción como un tipo de TypeScript y usando un tipo de unión (en este caso: StoriesAction) para la seguridad de tipos final:

origen/App.tsx

```

tipo StoriesFetchInitAction = { tipo:
  'STORIES_FETCH_INIT';
}

tipo StoriesFetchSuccessAction = { tipo:
  'STORIES_FETCH_SUCCESS'; carga útil: Story[];
}

tipo StoriesFetchFailureAction = { tipo:
  'STORIES_FETCH_FAILURE';
}

tipo StoriesRemoveAction = { tipo:
  'REMOVE_STORY'; carga útil:
  Historia;
}

tipo HistoriasAcción =
  HistoriasFetchInitAction
  | HistoriasObtenerÉxitoAcción
  | HistoriasObtenerFracasoAcción
  | HistoriasEliminarAcción;

const storiesReducer = (estado:
  StoriesState,
  acción: HistoriasAcción
) => {
  ...
};

```

El estado actual, la acción y el estado devuelto (inferido) del reductor ahora son seguros para tipos. Por ejemplo, si se envía una acción al reductor con un tipo de acción no definido, se obtendrá un error de TypeScript. O si se pasa algo distinto a una historia a la acción que la elimina, también se obtendrá un error de tipo. Ahora, centrémonos en el componente SearchForm, que tiene controladores de devolución de llamada con eventos:

origen/App.tsx

```

tipo SearchFormProps =
  { searchTerm: cadena;
    onSearchInput: (evento: React.ChangeEvent<HTMLInputElement>) => vacío; searchAction:
    (formData: FormData) => vacío; };

```

```

constante FormularioDeBúsqueda = ({
  término de
  búsqueda, en entrada de
  búsqueda, acción de búsqueda,
}: Propiedades del formulario de búsqueda) => (
  ...
);

```

Usar `React.SyntheticEvent` en lugar de `React.ChangeEvent` o `React.FormEvent` suele ser suficiente. Sin embargo, la mayoría de las veces, las aplicaciones requieren un tipo más específico. A continuación, volviendo al componente `App`, aplicamos el mismo tipo para el controlador de devolución de llamada:

origen/App.tsx

```

constante Aplicación = () => {
  ...

  constante handleSearchInput = (
    evento: React.ChangeEvent<HTMLInputElement>
  ) =>
    { setSearchTerm(evento.objetivo.valor); };

  ...
};

```

Solo queda el componente `InputWithLabel`. Antes de gestionar las propiedades de este componente, revisemos la referencia del gancho `useRef` de React. Lamentablemente, el valor de retorno no se infiere:

origen/App.tsx

```
constante InputWithLabel = ({ ... }) => {
  constante inputRef = React.useRef<HTMLInputElement>(null);

  React.useEffect(() => { if
    (isFocused && inputRef.current)
      { inputRef.current.focus();
    }
  }, [está enfocado]);
```

Hemos protegido el tipo de referencia devuelto y lo hemos tipificado como de solo lectura, ya que solo ejecutamos el método de enfoque (lectura). React toma el control, asignando la propiedad actual al elemento DOM.

Por último, aplicaremos comprobaciones de seguridad de tipos a las propiedades del componente InputWithLabel. Observe que la propiedad secundaria tiene su tipo específico de React y que los tipos opcionales se indican con un signo de interrogación:

origen/App.tsx

```
tipo InputWithLabelProps = { id: cadena;
  valor: cadena;
  tipo?: cadena;
  onChange:
    (evento: React.ChangeEvent<HTMLInputElement>) => void; isFocused?: boolean; hijos:
    React.ReactNode; };
```

```
constante EntradaConEtiqueta = ({
  identificación,
  valor,
  tipo = 'texto',
  onChange,
  isFocused,
  hijos,
}: EntradaConEtiquetasProps) => {
  ...
};
```

Tanto las propiedades "type" como "isFocused" son opcionales. Con TypeScript, puedes indicar al compilador que no es necesario pasarlas al componente como props. La propiedad "children" tiene muchas definiciones de tipos de TypeScript que podrían aplicarse a este concepto, siendo la más universal React.ReactNode de la biblioteca de React.

Nuestra aplicación React completa está finalmente tipificada con TypeScript, lo que facilita la detección de errores de tipo en tiempo de compilación. Al añadir TypeScript a tu aplicación React, empieza añadiendo definiciones de tipo a los argumentos de tus funciones. Estas funciones pueden ser funciones JavaScript estándar, ganchos personalizados de React o componentes de función de React. Solo al usar React es importante conocer los tipos específicos de elementos de formulario, eventos y JSX.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² ³ .
 - Recapitular todos los [cambios del código fuente](#)² De esta sección.
- Mientras continúa con el aprendizaje en las siguientes secciones, elimine o conserve sus tipos con TypeScript. Si opta por esto último, agregue nuevos tipos cada vez que obtenga una compilación. error.

² ³<https://tinyurl.com/4v7ecp5w>

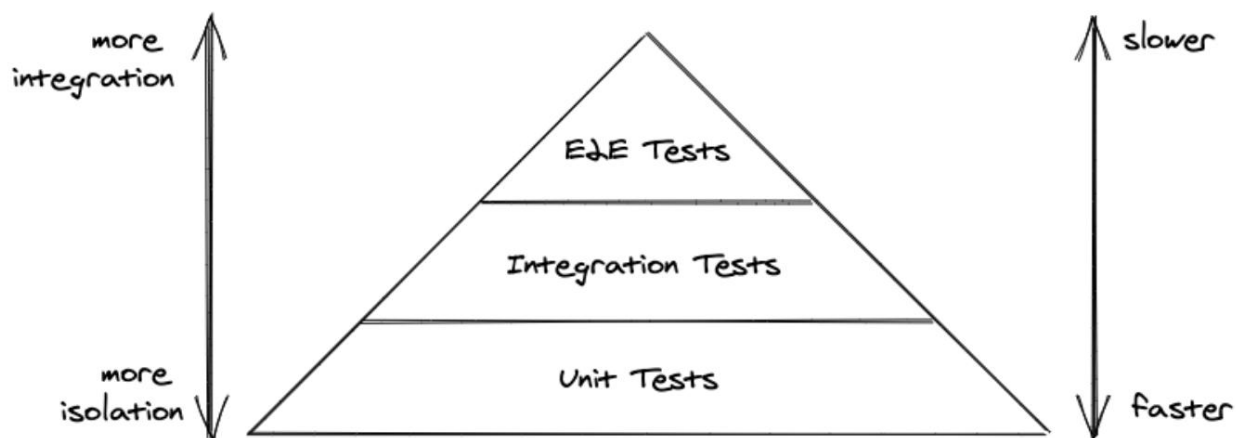
² <https://tinyurl.com/8umbbynt>

Pruebas en React

Probar el código fuente es una parte esencial de la programación y debería considerarse una actividad obligatoria para los desarrolladores profesionales. El objetivo es verificar la calidad y funcionalidad del código fuente antes de usarlo en producción. La [pirámide de pruebas](#)² servirá como nuestra guía.

La pirámide de pruebas incluye pruebas integrales, pruebas de integración y pruebas unitarias. Las pruebas unitarias se utilizan para bloques de código pequeños y aislados, como una sola función o componente. Las pruebas de integración nos ayudan a determinar el correcto funcionamiento conjunto de estos bloques de código. Las pruebas integrales simulan un escenario real, como un usuario que inicia sesión en una aplicación web. Si bien las pruebas unitarias son rápidas y fáciles de escribir y mantener, las pruebas integrales son todo lo contrario.

Se requieren numerosas pruebas unitarias para cubrir todas las funciones y componentes de una aplicación funcional. Tras ellas, varias pruebas de integración garantizan el funcionamiento conjunto de las unidades más importantes. El toque final se consigue con pruebas de extremo a extremo para simular escenarios críticos de usuario. En esta sección, abordaremos las pruebas unitarias y de integración, además de una técnica útil de pruebas específicas para cada componente: las pruebas instantáneas. Las pruebas de extremo a extremo (E2E) formarán parte del ejercicio.



Elegir una biblioteca de pruebas puede ser un desafío para quienes se inician en React, ya que existen muchas opciones. Para simplificar, usaremos las herramientas más populares: [Vitest](#)² y [biblioteca de pruebas de React](#)² (RTL).

Vitest es un completo framework de pruebas con ejecutores de pruebas, suites de pruebas, casos de prueba y aserciones. RTL se utiliza para renderizar componentes de React, activar eventos como clics del ratón y seleccionar elementos HTML del DOM para ejecutar aserciones. Exploraremos ambas herramientas paso a paso, desde la configuración hasta las pruebas unitarias y las pruebas de integración.

Antes de escribir nuestra primera prueba, debemos instalar Vitest y configurarlo. Para empezar, escriba la siguiente instrucción en la línea de comandos:

² <https://bit.ly/3BYEra1>

² <https://vitest.dev/>

² <https://testing-library.com>

Línea de comandos

```
npm instalar vitest --save-dev
```

Luego, en su archivo package.json, agregue otro script que ejecutará las pruebas eventualmente:

paquete.json

```
"dev": "vite",  
"construir": "vite construir",  
"probar": "vitest",  
"lint": "eslint.", "preview":  
"vista previa de vite"
```

Por último, cree un nuevo archivo para probar funciones y componentes:

Línea de comandos

```
toque src/App.test.jsx
```

Desde allí, comenzaremos a escribir pruebas para las características que provienen del archivo src/App.jsx que se encuentra al lado.

Conjuntos de pruebas, casos de prueba y afirmaciones

Las suites de pruebas y los casos de prueba se utilizan comúnmente en JavaScript y muchos otros lenguajes de programación.

Una suite de pruebas agrupa los casos de prueba individuales en un solo sujeto. Veamos cómo se ve esto con Vitest en nuestro archivo src/App.test.jsx:

src/App.test.jsx

```
importar { describe, eso, espera } de 'vitest';  
  
describe('algo verdadero y falso', () => { it('verdadero para ser  
verdad', () => { expect(true).toBe(true); });  
  
it('false ser falso', () =>  
  { expect(false).toBe(false); }); });
```

El bloque "describe" representa nuestra suite de pruebas, y los bloques "it" representan nuestros casos de prueba. Tenga en cuenta que los casos de prueba pueden usarse sin suites de pruebas:

Patio de juegos de código

```
import { it, esperar } de 'vitest';
```

```
it('es cierto que es cierto', () =>
  { expect(true).toBe(true); });
```

```
it('false ser falso', () =>
  { expect(false).toBe(false); });
```

Los temas grandes, como funciones o componentes, a menudo requieren múltiples casos de prueba, por lo que tiene sentido usarlos con conjuntos de pruebas:

Patio de juegos de código

```
describe('Componente de la aplicación', ()
=> { it('elimina un elemento al hacer clic en el botón Descartar', () => {

  });
```

```
  it('solicita algunas historias iniciales de una API', () => {

  }); });
```

Finalmente, puedes ejecutar pruebas usando el script de prueba de tu package.json en la línea de comando con npm run test para producir el siguiente resultado:

Línea de comandos

```
src/App.test.jsx (2)  algo
verdadero y falso (2)
  cierto para ser verdad
  falso ser falso
```

```
Archivos de prueba 1 aprobados (1)
  Pruebas 2 aprobadas (2)
Comienza en 13:19:38
Duración 122ms
```

Al ejecutar el comando de prueba, el ejecutor de pruebas compara todos los archivos con el sufijo test.jsx. Las pruebas exitosas se muestran en verde y las fallidas en rojo. El script de prueba interactivo monitorea las pruebas y el código fuente.

código y ejecuta pruebas cuando los archivos cambian. Vitest también ofrece algunos comandos interactivos (presione "h" para verlos todos), como presionar "f" para ejecutar pruebas fallidas y "a" para ejecutar todas las pruebas. Veamos cómo se ve esto para una prueba fallida:

```
src/App.test.jsx
```

```
importar { describe, eso, espera } de 'vitest';
```

```
describe('algo verdadero y falso', () => { it('verdadero para ser
  verdad', () => { expect(true).toBe(true); });
```

```
  it('false ser falso', () =>
    { expect(false).toBe(true); }); });
```

Las pruebas se ejecutan nuevamente y la salida de la línea de comando muestra una prueba fallida en rojo:

```
Línea de comandos
```

```
AssertionError: se esperaba que falso fuera verdadero // Object.is igualdad
```

```
- Esperado
+ Recibido
```

```
- verdadero
+ falso
```

```
src/App.test.jsx:9:19
```

```
7|
8| it('falso es falso', () => { 9| | 10| }); 11| });
   esperar(falso).ser(verdadero);
                        ^
```

```
Archivos de prueba 1 fallaron (1)
```

```
  Pruebas 1 fallida | 1 aprobada (2)
```

```
  Comienza en 13:20:44
```

```
  Duración 124ms
```

```
Pruebas fallidas. Buscando cambios en el archivo...
```

```
  Presione h para mostrar ayuda, presione q para salir
```

Familiarícese con esta salida de prueba, ya que muestra todas las pruebas fallidas, así como información sobre sus causas. Con esta información, puede corregir ciertas partes de su código hasta que todas las pruebas se ejecuten en verde. A continuación, abordaremos las aserciones de prueba, dos de las cuales ya hemos usado con la función `expect` de Vitest. Una aserción funciona esperando que el valor del lado izquierdo (`expect`) coincida con el valor del lado derecho (`toBe`). `toBe` es solo una de las muchas funciones asertivas disponibles que ofrece Vitest.

`src/App.test.jsx`

```
importar { describe, eso, espera } de 'vitest';

describe('algo verdadero y falso', () => { it('verdadero para
  ser verdad', () =>
    { expect(true).toBeTruthy(); });

  it('false es falso', () =>
    { expect(false).toBeFalsy(); }); });
```

Una vez que comiences a probar, es recomendable mantener abiertas dos interfaces de línea de comandos: una para supervisar tus pruebas (`npm run test`) y otra para desarrollar tu aplicación (`npm run dev`). Si usas control de código fuente como Git, quizás quieras tener una interfaz de línea de comandos adicional para agregar tu código fuente al repositorio.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
 - Recapitular todos los [cambios del código fuente](#)² de esta sección.
- Lea más sobre [Vitest](#)² .

Pruebas unitarias: funciones

Una prueba unitaria se usa generalmente para probar componentes o funciones de forma aislada. En el caso de las funciones, las pruebas unitarias se utilizan para la entrada y la salida; en el caso de los componentes, probamos las propiedades, los controladores de devolución de llamada que se comunican con el exterior o la salida de los componentes. Antes de realizar una prueba unitaria en nuestro archivo `src/App.jsx`, debemos exportar componentes y funciones como el reductor desde nuestro archivo `src/App.jsx` con una exportación con nombre:

² <https://tinyurl.com/bdfyt6bd>
² <https://tinyurl.com/2zbz4jdh>
² <https://vitest.dev/>

src/App.jsx

...

exportar aplicación predeterminada;

exportar { storiesReducer, Formulario de búsqueda, Entrada con etiqueta, Lista, Elemento };

Los ejercicios al final de este capítulo cubrirán todas las pruebas restantes que debería considerar realizar. Por ahora, podemos importar todos los componentes y reductores en nuestro archivo src/App.test.jsx y nos centraremos primero en la prueba del reductor. También importamos React, ya que debemos incluirlo siempre que probemos componentes de React:

src/App.test.jsx

importar { describe, eso, espera } de 'vitest';

importar aplicación, {
 historiasReductor,
 Artículo,
 Lista,
 Formulario de búsqueda,
 Entrada con etiqueta,
} de './App';

Antes de realizar la prueba unitaria de nuestro primer componente React, veremos cómo probar solo una función de JavaScript. El mejor candidato para este caso de uso de prueba es la función storiesReducer y una de sus acciones. Definamos algunos datos de prueba y el conjunto de pruebas para la prueba del reductor:

src/App.test.jsx

...

```
const storyOne = { título:  
  'React', url: 'https://  
  react.dev/', autor: 'Jordan Walke',  
  num_comentarios: 3, puntos: 4,  
  objectID: 0, };
```

```
const storyTwo = { título:  
  'Redux',
```

```
URL: 'https://redux.js.org/', autor: 'Dan  
Abramov, Andrew Clark', número de comentarios:  
2, puntos: 5, objectID:  
1, };
```

```
const historias = [historiaUno, historiaDos];  
  
describe('storiesReducer', () => { it('elimina una  
historia de todas las historias', () => {  
  
  
}); });
```

Si extrapolamos los casos de prueba, debería haber un caso de prueba por cada acción del reductor. Nos centraremos en una sola acción, que puede usar para realizar el resto como ejercicio. La función del reductor acepta un estado y una acción, y luego devuelve un nuevo estado, por lo que todas las pruebas del reductor siguen básicamente el mismo patrón:

```
src/App.test.jsx
```

```
...
```

```
describe('storiesReducer', () => { it('elimina una  
historia de todas las historias', () => {  
  const action = // TODO: alguna acción const state  
  = // TODO: algún estado actual  
  
  const newState = storiesReducer(estados, acción);  
  
  const expectedState = // TODO: el estado esperado  
  
  esperar(nuevoEstado).ser(estadosesperado); }); });
```

En nuestro caso, definimos la acción, el estado y el estado esperado según nuestro reductor. El estado esperado tendrá una historia menos, que se eliminó al pasarse al reductor como acción:

src/App.test.jsx

```
describe('storiesReducer', () => { it('elimina una
  historia de todas las historias', () => { const action = { type:
    'REMOVE_STORY', payload: storyOne }; const state = { data: stories, isLoading:
    false, isError: false };

    const newState = storiesReducer(estados, acción);

    const expectedState = { datos:
      [storyTwo], isLoading:
      falso, isError: falso, };

    esperar(nuevoEstado).ser(estadosesperado); }); });
```

Esta prueba sigue fallando porque usamos `toBe` en lugar de `toBeStrictEqual`. La función asertiva `toBe` realiza una comparación estricta como `newState === expectedState`. El contenido de los objetos es el mismo, pero sus referencias a objetos no lo son. Usamos `toBeStrictEqual` en lugar de `toBe` para limitar la comparación al contenido del objeto:

src/App.test.jsx

```
esperar(nuevoEstado).toBeStrictEqual(estadosesperado); //
esperar(nuevoEstado).toBe(estadosesperado);
```

Siempre se debe decidir si se desea una comparación estricta o solo una comparación de contenido para los objetos JavaScript. Normalmente, solo se desea una comparación de contenido, por lo que se debe usar `toBeStrictEqual`. Sin embargo, para primitivas de JavaScript, como cadenas o booleanos, se puede usar `toBe`. También hay que tener en cuenta que existe una función `toEqual` [que funciona de forma ligeramente diferente](https://bit.ly/3jIPpii)² ¹ que a `StrictEqual`.

Seguimos realizando ajustes hasta que la prueba del reductor se vuelva verde, lo que en realidad prueba una función de JavaScript con una entrada específica y espera una salida específica. Aún no hemos realizado pruebas con componentes de React.

Recuerde que una función reductora siempre seguirá el mismo patrón de prueba: dados un estado y una acción, esperamos el siguiente nuevo estado. Cada acción del reductor podría ser otro caso de prueba en su conjunto de pruebas, así que considere usar los ejercicios para revisar todo su código fuente.

² ¹<https://bit.ly/3jIPpii>

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² ².
- Recapitular todos los [cambios del código fuente](#)² ³ De esta sección.
- Continúe escribiendo un caso de prueba para cada acción del reductor y su transición de estado.
- Lea más sobre [las funciones asertivas de Vitest](#)² como `toBe` y `toBeStrictEqual`.

Pruebas unitarias: componentes

En la sección anterior, probamos nuestra primera función en JavaScript con Vitest. A continuación, probaremos nuestro primer componente de React de forma aislada mediante una prueba unitaria. Por lo tanto, debemos indicar a Vitest el entorno del navegador sin interfaz gráfica donde queremos renderizar los componentes de React, ya que la prueba no iniciará un navegador real. Sin embargo, el HTML que se renderiza con un componente de React debe terminar en algún lugar (por ejemplo, un navegador sin interfaz gráfica) para que sea accesible para las pruebas. La forma más común de realizar esta tarea es instalar [jsdom](#)² que actúa como un navegador sin cabeza para nosotros:

Línea de comandos

```
npm instalar jsdom --save-dev
```

Luego podemos incluirlo en el archivo de configuración de Vite:

vite.config.js

```
importar { defineConfig } de 'vite'; importar react
de '@vitejs/plugin-react';
```

```
// https://vitejs.dev/config/ exportar
predeterminado defineConfig({ plugins:
  [react()], prueba: { entorno:

  'jsdom', }, });
```

Además, renderizaremos componentes de React en pruebas con una biblioteca llamada `react-testing-library` (RTL). También necesitamos instalarla:

² <https://tinyurl.com/yv78dcyr>

² <https://tinyurl.com/4vpp7sbe>

² <https://bit.ly/3xVJbwZ>

² <https://bit.ly/3LJrExK>

Línea de comandos

```
npm install @biblioteca-de-pruebas/react @biblioteca-de-pruebas/jest-dom --save-dev
```

Luego, creamos un nuevo archivo para una configuración de prueba general:

Línea de comandos

```
pruebas mkdir
```

```
pruebas de cd
```

```
toque setup.js
```

Y referenciarlo en el archivo de configuración de Vite:

vite.config.js

```
importar { defineConfig } de 'vite'; importar react de '@vitejs/  
plugin-react';
```

```
// https://vitejs.dev/config/ exportar predeterminado
```

```
defineConfig({ plugins: [react()], prueba: { entorno:  
  'jsdom', setupFiles: './tests/  
  setup.js', }, });
```

Por último, incluya los siguientes detalles de implementación en el nuevo archivo de configuración para nuestras pruebas:

pruebas/setup.js

```
importar { esperar, después de cada } de 'vitest'; importar { limpieza } de  
'@testing-library/react'; importar * como comparadores de "@testing-library/jest-dom/  
matchers";
```

```
esperar.extender(coincidentes);
```

```
después de cada(() =>  
  { limpieza(); });
```

Básicamente, el método expect de Vitest se extiende con más métodos proporcionados por RTL. Usaremos estos métodos (por ejemplo, toBeInTheDocument) en nuestras pruebas próximamente. Ahora finalmente podemos importar las siguientes funciones de la biblioteca de pruebas de React, que se usan para las pruebas de componentes:

src/App.test.jsx

```
importar { describir, eso, esperar } de 'vitest'; importar { renderizar,
```

```
  pantalla,  
  fireEvent,  
  esperar, }  
de '@testing-library/react';
```

```
...
```

Comenzamos con el componente Item, donde verificamos si representa todas las propiedades esperadas según sus propiedades. Con base en la entrada (léase: propiedades), verificamos una salida (HTML representado). Usaremos la función de renderizado de RTL en cada prueba para renderizar un componente de React. En este caso, renderizamos el componente Item como un elemento y le pasamos un objeto item (una de nuestras historias definidas previamente) como propiedad:

src/App.test.jsx

```
...
```

```
constante storyOne = { ... };
```

```
constante storyTwo = { ... };
```

```
const historias = [historiaUno, historiaDos];
```

```
describe('storiesReducer', () => {  
  ...  
});
```

```
describe('Item', () => { it('representa  
  todas las propiedades', () => { render(<Item  
    item={storyOne} />); }); });
```

Después de renderizarlo, aún no hicimos ninguna prueba (las pruebas se están volviendo verdes, porque no había ninguna prueba fallida en el archivo de prueba), por lo que podemos usar la función de depuración del objeto de pantalla de RTL para generar en la línea de comando lo que se ha renderizado en el entorno de jsdom:

src/App.test.jsx

```
describe('Item', () =>
  { it('representa todas las propiedades', () =>
    { render(<Item item={storyOne} />);

    pantalla.debug(); }); });
```

Ejecute las pruebas con `npm run test` y verá el resultado de la función de depuración . Imprime todos los elementos HTML de su componente y de sus componentes secundarios. El resultado debería ser similar al siguiente:

Línea de comandos

```
<cuero>
  <div>
    <li>
      <span>
        <a
          href="https://reactjs.org/"
        >
          Reaccionar
        </a>

      <span>
        Jordan Walke

      <span>
        3
      <span>

        4

      <span>
        <botón
          tipo="botón"
        >
          Despedir
        </botón>
      </li> </
    <div>

  </cuero>
```

Aquí deberías acostumbrarte a usar la función de depuración de RTL al renderizar un nuevo componente en una prueba de componentes de React. Esta función ofrece una visión general útil de lo que se renderiza e indica la mejor manera de proceder con las pruebas. Con base en la salida actual, podemos comenzar con nuestra primera aserción. El objeto de pantalla de RTL proporciona una función llamada `getByText`, una de las muchas funciones de búsqueda:

src/App.test.jsx

```
describe('Item', () =>
  { it('representa todas las propiedades', () =>
    { render(<Item item={storyOne} />);

    esperar(pantalla.getByText('Jordan Walke')).toBeInTheDocument();
    esperar(pantalla.getByText('React')).toHaveAttribute(
      'href',
      'https://react.dev/' ); }); });
```

Para las dos aserciones, utilizamos las funciones asertivas `toBeInTheDocument` y `toHaveAttribute` (ambas requieren la extensión `expect` del archivo `tests/setup.js`). Estas funciones verifican la presencia de un elemento con el texto "Jordan Walke" en el documento y la presencia de un elemento con el texto "React" con un valor de atributo `href` específico. Con el tiempo, se observará un mayor uso de estas funciones asertivas.

La función de búsqueda `getByText` de RTL encuentra el elemento con los textos visibles "Jordan Walke" y "React". Podemos usar el equivalente `getAllByText` para encontrar más de un elemento. Existen equivalentes similares para otras funciones de búsqueda.

La función `getByText` devuelve el elemento con un texto visible para los usuarios, relacionado con el uso real de la aplicación. Tenga en cuenta que `getByText` no es la única función de búsqueda. Otra función de búsqueda muy utilizada es `getByRole` o `getAllByRole`:

src/App.test.jsx

```
describe('Item', () =>
  { it('representa todas las propiedades', () => {
    ...
  });

  it('muestra un botón de desestimación en el que se puede hacer clic', () => {
    render(<Item item={storyOne} />);

    esperar(pantalla.getByRole('botón')).toBeInTheDocument();
```

```
}); });
```

La función `getByRole` se utiliza generalmente para recuperar elementos por [atributos aria-label²](#). Sin embargo, también existen [roles implícitos en los elementos HTML²](#) Como "botón" para un elemento HTML de botón. De esta forma, puedes seleccionar elementos no solo por el texto visible, sino también por su rol de accesibilidad (implícito) con la biblioteca de pruebas de React. Una característica interesante de `getRoleBy` es que [sugiere roles si proporcionas uno que no está disponible²](#).

```
src/App.test.jsx
```

```
describe('Item', () => {
  it('representa todas las propiedades', () => {
    ...
  });

  it('muestra un botón de desestimación en el que se puede hacer clic', () => {
    render(<Item item={storyOne} />);

    pantalla.getByRole("");

    esperar(pantalla.getByRole('botón')).toBeInTheDocument(); }); });
```

Lo que debería generar algo similar a lo siguiente en la línea de comando:

```
Línea de comandos
```

Error de elemento de biblioteca de prueba:

No se puede encontrar un elemento accesible con el rol ""

Aquí están los roles accesibles:

```
elemento de lista:
```

```
Nombre "":
```

```
<li />
```

```
enlace:
```

```
Nombre "React":
```

```
<a
```

```
href="https://reactjs.org/"
```

² <https://mzl.la/49oo8U0>

² <https://mzl.la/3n7SgN7>

² <https://bit.ly/3pnPXrQ>

```

/>

```

botón:

Nombre "Despedir":

```

<botón
  tipo="botón" />

```

Tanto `getByText` como `getByRole` son las funciones de búsqueda RTL más utilizadas. Podemos continuar verificando no solo que todo esté en el documento, sino también que nuestros eventos funcionen correctamente. Por ejemplo, se puede hacer clic en el botón del componente `Item` y queremos verificar que se llama al controlador de devolución de llamada. Por lo tanto, usamos `Vitest` para crear una función simulada que proporcionamos como controlador de devolución de llamada al componente `Item`. Después de ejecutar un evento de clic con la biblioteca de pruebas de React en el botón, queremos verificar que se ha llamado a la función del controlador de devolución de llamada:

`src/App.test.jsx`

```

importar { describe, it, esperar, vi } de 'vitest';

```

```

...

```

```

describe('Artículo', () => {

```

```

  it('representa todas las propiedades', () => {

```

```

    ...

```

```

  });

```

```

  it('muestra un botón de desestimación en el que se puede hacer clic', () => {

```

```

    ...

```

```

  });

```

```

  it('al hacer clic en el botón descartar se llama al controlador de devolución de llamada',

```

```

    () => { const handleRemoveItem = vi.fn();

```

```

      render(<Item item={storyOne} onRemoveItem={handleRemoveItem} />);

```

```

      fireEvent.click(pantalla.getByRole('botón'));

```

```

      esperar(manejarEliminarElemento).haberSidoLlamadoTimes(1); }); });

```

`Vitest` nos permite pasar una función específica de la prueba al componente `Item` como una propiedad. Estas funciones específicas de la prueba se llaman `spy`, `stub` o `mock`; cada una se utiliza para diferentes escenarios de prueba. `vi.fn()` devuelve

Nos proporciona una simulación de la función real, lo que nos permite capturar cuándo se llama. Como resultado, podemos usar aserciones de Vitest como "toHaveBeenCalledTimes", que nos permite afirmar el número de veces que se ha llamado a la función; y "toHaveBeenCalledWith", para verificar los argumentos que se le pasan.

Cada vez que queramos simular una función JavaScript, ya sea que haya sido llamada o que haya recibido ciertos argumentos, podemos usar la función auxiliar de Vitest para crear una función simulada. Después de invocar esta función con la función del objeto fireEvent de RTL, podemos confirmar que el controlador de devolución de llamada proporcionado (que es la función simulada) se ha llamado una vez.

En el último ejercicio, probamos la entrada y la salida del componente Item mediante aserciones de renderizado y aserciones del controlador de devolución de llamada. Sin embargo, aún no estamos probando cambios de estado reales, ya que no se elimina ningún elemento del DOM tras hacer clic en el botón "Descartar". La lógica para eliminar el elemento de la lista se encuentra en el componente App, pero solo estamos probando el componente Item de forma aislada. A veces es útil comprobar si un solo bloque funciona antes de probarlo todo en conjunto. Probaremos la lógica de implementación para eliminar un Item cuando tratemos el componente App más adelante.

Por ahora, el componente SearchForm usa el componente InputWithLabel como componente secundario. Lo haremos en nuestro siguiente caso de prueba. Como antes, comenzaremos renderizando el componente (en este caso, el componente principal) y proporcionando todas las propiedades esenciales:

```
src/App.test.jsx
```

```
...
```

```
describe('Formulario de búsqueda', ()
  => { const searchFormProps = {
    searchTerm: 'Reaccionar',
    onSearchInput: vi.fn(),
    searchAction: vi.fn(), };

    it('representa el campo de entrada con su valor', () => { render(<SearchForm
      {...searchFormProps} />);

      pantalla.debug(); }); });
```

De nuevo, comenzamos con la depuración. Tras evaluar lo que se muestra en la línea de comandos, podemos realizar la primera aserción para el componente SearchForm. Con los campos de entrada configurados, la función de búsqueda getByDisplayValue de RTL es la opción ideal para devolver el campo de entrada como un elemento:

src/App.test.jsx

```
describe('Formulario de búsqueda', () => {
  constante searchFormProps = { ... };

  it('representa el campo de entrada con su valor', () =>
    { render(<SearchForm {...searchFormProps} />);

    expect(screen.getByDisplayValue('React')).toBeInTheDocument(); }); });
```

Dado que el elemento de entrada HTML se representa con un valor predeterminado, podemos usar dicho valor (en este caso: "React"), que es el valor mostrado en nuestra aserción de prueba. Si el elemento de entrada no tiene un valor predeterminado, la aplicación podría mostrar un marcador de posición con el atributo HTML correspondiente en el campo de entrada. A continuación, usaríamos otra función de RTL llamada `getByPlaceholderText`, que permite buscar un elemento con un texto de marcador de posición.

Debido a que la información de depuración presentó múltiples opciones para consultar el HTML, podríamos continuar con una prueba más para confirmar la etiqueta renderizada:

src/App.test.jsx

```
describe('Formulario de búsqueda', () => {
  constante searchFormProps = { ... };

  it('representa el campo de entrada con su valor', () => {
    ...
  });

  it('representa la etiqueta correcta', () => {
    render(<Formulario de búsqueda {...Propiedades del formulario de búsqueda} />);

    esperar(pantalla.getByLabelText(/Buscar/)).toBeInTheDocument(); }); });
```

La función de búsqueda `getByLabelText` permite encontrar un elemento por etiqueta en un formulario. Esto es útil para componentes que representan múltiples etiquetas y controles HTML. Sin embargo, quizás hayas notado que usamos una [expresión regular](https://mzl.la/3CdDjiZ)² Aquí. Si usáramos una cadena, se deben incluir los dos puntos para "Buscar:". Al usar una expresión regular, buscamos cadenas que incluyan la cadena "Buscar", lo que hace que la búsqueda de elementos sea mucho más eficiente. Por esta razón, es posible que a menudo uses expresiones regulares en lugar de cadenas.

² <https://mzl.la/3CdDjiZ>

De todos modos, quizás sería más interesante probar las partes interactivas del componente SearchForm. Dado que nuestros manejadores de devolución de llamada, que se pasan como props al componente SearchForm, ya están simulados con Vitest, podemos comprobar si estas funciones se invocan correctamente:

src/App.test.jsx

```
describe('Formulario de búsqueda', () =>
  { const searchFormProps =
    { searchTerm: 'React',
      onSearchInput: vi.fn(),
      searchAction: vi.fn(), };

    ...

    it('llama a onSearchInput cuando cambia el campo de entrada', () =>
      { render(<SearchForm {...searchFormProps} />);

        fireEvent.change(pantalla.getByDisplayValue('Reaccionar'), {
          objetivo: { valor: 'Redux' }, });

        esperar(searchFormProps.onSearchInput).toHaveBeenCalledTimes(1); });

    it('llama a searchAction al hacer clic en el botón enviar', () =>
      { render(<SearchForm {...searchFormProps} />);

        fireEvent.click(pantalla.getByRole('botón'));

        esperar(searchFormProps.searchAction).toHaveBeenCalledTimes(1); }); });
```

Al igual que con el componente Item, probamos la entrada (props) y la salida (controlador de devolución de llamada) del componente Search-Form. La diferencia radica en que el componente SearchForm renderiza un componente secundario llamado InputWithLabel. Si revisas la salida de depuración, probablemente notarás que la biblioteca de pruebas de React solo renderiza el árbol de componentes completo para ambos componentes. Esto se debe a que al usuario final no le interesa el árbol de componentes, sino solo el HTML que se muestra.

Entonces, la biblioteca de pruebas de React genera todo el HTML que es importante para el usuario y, por lo tanto, para la prueba.

Todas las pruebas del controlador de devolución de llamada para los componentes Item y SearchForm verifican únicamente si se han llamado las funciones. No se produce ninguna re-renderización de React, ya que todos los componentes se prueban de forma aislada sin gestión de estado, lo cual ocurre únicamente en el componente App. Las pruebas reales con RTL comienzan.

Más arriba en el árbol de componentes, donde se pueden evaluar los cambios de estado y los efectos secundarios. Por lo tanto, a continuación, presentaré las pruebas de integración.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
 - Recapitular todos los [cambios del código fuente](#)² ¹ de esta sección. • Lea más sobre [React Testing Library](#)² ².
 - Lea más sobre [las funciones de búsqueda](#)² ³. • Utilice `toHaveBeenCalledWith` junto a `toHaveBeenCalledTimes` para hacer que sus afirmaciones sean más precisas. A prueba de balas. • Agregue pruebas para sus componentes `List` y `InputWithLabel`.

Pruebas de integración: Componente

La biblioteca de pruebas de React se adhiere a una única filosofía central: en lugar de probar los detalles de implementación de los componentes de React, prueba cómo interactúan los usuarios con la aplicación y si funciona como se espera.

Esto se vuelve especialmente poderoso para las pruebas de integración.

Necesitaremos proporcionar algunos datos antes de probar el componente `App`, ya que solicita datos desde una API remota después de su renderizado inicial. Dado que usamos `axios` para la obtención de datos en el componente `App`, tendremos que simularlo con `Vitest` al principio del archivo de prueba:

```
src/App.test.jsx
```

```
...
```

```
importar axios desde 'axios';
```

```
...
```

```
vi.mock('axios');
```

```
...
```

A continuación, implemente los datos que desea que se devuelvan de la solicitud de API simulada con una promesa de JavaScript y úsela para la simulación de `Axios`. Después, podemos renderizar nuestro componente y asumir que se simulan los datos correctos para nuestra solicitud de API:

² <https://tinyurl.com/2tcf5pve>

¹<https://tinyurl.com/2kmmzrtv>

²<https://bit.ly/30KueQH>

³<https://bit.ly/3jjUw2t>

src/App.test.jsx

...

```
describe('App', () => { it('obtiene  
correctamente los datos', () => { const promise =  
  Promise.resolve({ data: { hits: stories, }, });
```

```
    axios.get.mockImplementationOnce(() => promesa);
```

```
    render(<Aplicación />);
```

```
    pantalla.debug(); }); });
```

Ahora usaremos la función auxiliar `waitFor` de la biblioteca de pruebas de React para esperar hasta que la promesa se resuelva tras el renderizado inicial del componente. Con `async/await`, podemos implementar esto como código síncrono. La función de depuración de RTL es útil porque muestra los elementos del componente App antes y después de la solicitud:

src/App.test.jsx

```
describe('App', () => { it('obtiene  
correctamente los datos', async () => { const promise =  
  Promise.resolve({ data: { hits: stories, }, });
```

```
    axios.get.mockImplementationOnce(() => promesa);
```

```
    render(<Aplicación />);
```

```
    pantalla.debug();
```

```
    esperar waitFor(async () => esperar promesa);
```

```
    pantalla.debug();
```

```
}); });
```

En la salida de depuración, vemos que el indicador de carga se renderiza para la primera función de depuración, pero no para la segunda. Esto se debe a que la obtención de datos y la re-renderización del componente se completan después de resolver la promesa en nuestra prueba con `waitFor`. Afirmemos el indicador de carga en este caso:

```
src/App.test.jsx
```

```
describe('App', () => { it('obtiene
  los datos correctamente', async () => { const promise =
    Promise.resolve({ data: {

      hits: historias, }, });

    axios.get.mockImplementationOnce(() => promesa);

    render(<Aplicación />);

    esperar(pantalla.queryByText(/Cargando/)).toBeInTheDocument();

    esperar(waitFor(async () => esperar promesa);

    esperar(pantalla.queryByText(/Cargando/)).toBeNull(); }); });
```

Dado que estamos probando un elemento devuelto que está ausente, esta vez usamos la función `queryByText` de RTL en lugar de la función `getByText`. Usar `getByText` en este caso generaría un error, ya que no se encuentra el elemento; sin embargo, con `queryByText`, el valor simplemente devuelve nulo.

De nuevo, usamos la expresión regular `/Cargando/` en lugar de la cadena `'Cargando'`. Para usar una cadena, tendríamos que usar explícitamente `'Cargando...'` en lugar de `'Cargando'`. Con una expresión regular, no necesitamos proporcionar la cadena completa, solo necesitamos que coincida con una parte de ella.

A continuación, podemos confirmar si los datos obtenidos se procesan como se espera o no:

src/App.test.jsx

```
describe('App', () => { it('obtiene
  los datos correctamente', async () => { const promise =
    Promise.resolve({ data: {

      hits: historias, }, });

  axios.get.mockImplementationOnce(() => promesa);

  render(<Aplicación />);

  esperar(pantalla.queryByText(/Cargando/)).toBeInTheDocument();

  esperar(waitFor(async () => esperar promesa);

  esperar(pantalla.queryByText(/Cargando/)).toBeNull();

  esperar(pantalla.getByText('Reaccionar')).toBeInTheDocument();
  esperar(pantalla.getByText('Redux')).toBeInTheDocument();
  esperar(pantalla.getAllByText('Descartar').length).toBe(2); }); });
```

El [camino feliz](#)² Ahora se prueba la obtención de datos. De igual forma, podemos probar la ruta incorrecta en caso de una solicitud de API fallida. La promesa debe rechazarse y el error debe detectarse con un bloque try/catch:

src/App.test.jsx

```
describe('App', () => { it('obtuvo
  los datos con éxito', async () => {
    ...
  });

  it('falla al obtener los datos', async () => { const
    promise = Promise.reject();

    axios.get.mockImplementationOnce(() => promesa);

    render(<Aplicación />);
```

² <https://bit.ly/3jiAbuB>

```

    esperar(pantalla.getByText(/Cargando/)).toBeInTheDocument();

    intentar
      { esperar waitFor(async () => esperar promesa); } atrapar
    (error) {
      esperar(pantalla.queryByText(/Cargando/)).toBeNull();
      esperar(pantalla.queryByText(/salió mal/)).toBeInTheDocument();

    }
  });
});

```

Puede haber cierta confusión sobre cuándo usar `getBy` o las variantes de búsqueda `queryBy`. Como regla general, use `getBy` para elementos individuales y `getAllBy` para múltiples elementos. Si busca elementos que no están presentes, use `queryBy` (o `queryAllBy`). En este código, preferí usar `queryBy` por motivos de alineación y legibilidad.

Ahora que sabemos que la obtención inicial de datos funciona para nuestro componente App, podemos pasar a probar las interacciones del usuario. Hasta ahora, solo hemos probado las acciones del usuario en los componentes secundarios, activando eventos sin estado ni efectos secundarios. A continuación, eliminaremos un elemento de la lista después de que los datos se hayan obtenido correctamente. Dado que el elemento con "Jordan Walke" es el primero que se muestra en la lista, se eliminará si hacemos clic en el primer botón "Descartar".

src/App.test.jsx

```

describe('Aplicación', () => {

  ...

  it('elimina una historia', async () => { const
    promise = Promise.resolve({ data: { hits:
      stories, }, });

    axios.get.mockImplementationOnce(() => promesa);

    render(<Aplicación />);

    esperar waitFor(async () => esperar promesa);

    esperar(pantalla.getAllByText('Descartar').length).toBe(2);
    esperar(pantalla.getByText('Jordan Walke')).toBeInTheDocument();
  });
});

```

```
fireEvent.click(pantalla.getAllByText('Descartar')[0]);

esperar(pantalla.getAllByText('Descartar').length).toBe(1);
esperar(pantalla.queryByText('Jordan Walke')).toBeNull(); }); });
```

Para probar la función de búsqueda, configuramos la simulación de manera diferente, porque manejamos la solicitud inicial, más otra solicitud una vez que el usuario busca más historias mediante un término de búsqueda específico:

src/App.test.jsx

```
describe('Aplicación', () => {

  ...

  it('busca historias específicas', async () => { const reactPromise =
    Promise.resolve({ data: { hits: stories, }, });

    const anotherStory = { título:
      'JavaScript', url: 'https://
      en.wikipedia.org/wiki/JavaScript', autor: 'Brendan Eich',
      num_comentarios: 15, puntos:
      10,

      ID de objeto:
      3, };

    constante javascriptPromesa = Promesa.resolve({
      datos:
        { visitas: [otraHistoria], }, });

    axios.get.mockImplementation((url) => { si
      (url.includes('React')) {
        devolver reactPromise;
      }
    })
```



```

    si (url.includes('JavaScript')) { devolver
      javascriptPromesa;
    }

    lanzar Error(); }); }); });

```

En lugar de simular la solicitud una vez con Vitest (`mockImplementationOnce`), ahora simulamos varias solicitudes (`mockImplementation`). Según la URL entrante, la solicitud devuelve la lista inicial (historias relacionadas con "React") o la nueva lista (historias relacionadas con "JavaScript"). Si proporcionamos una URL incorrecta a la solicitud, la prueba genera un error de confirmación. Como antes, rendericemos el componente App:

src/App.test.jsx

```

describe('Aplicación', () => {

  ...

  it('busca historias específicas', async () => { const reactPromise =
    Promise.resolve({ ... });

    const otraHistoria = { ... };

    constante javascriptPromise = Promesa.resolve({ ... });

    axios.get.mockImplementation((url) => {
      ...
    });

    // Render inicial

    render(<Aplicación />);

    // Primera obtención de datos

    esperar esperar(async () => esperar reactPromise);

    esperar(pantalla.queryByDisplayValue('React')).toBeInTheDocument();
    esperar(pantalla.queryByDisplayValue('JavaScript')).toBeNull();

    esperar(pantalla.queryByText('Jordan Walke')).toBeInTheDocument();

```

```

    esperar( pantalla.queryByText('Dan Abramov, Andrew Clark')
    ).toBeInTheDocument();
    expect(screen.queryByText('Brendan Eich')).toBeNull(); }); });

```

Estamos resolviendo la primera promesa para el renderizado inicial. Esperamos que el campo de entrada represente "React" y que los dos elementos de la lista representen a los creadores de React y Redux. También nos aseguramos de que Aún no se han renderizado historias relacionadas con JavaScript. A continuación, cambie el valor del campo de entrada activando un evento y verificando que el nuevo valor se renderice desde el componente App a todos sus componentes secundarios en el campo de entrada.

src/App.test.jsx

```

describe('Aplicación', () => {

  ...

  it('busca historias específicas', async () => {

    ...

    esperar(pantalla.queryByText('Jordan Walke')).toBeInTheDocument();

    esperar(pantalla.queryByText('Dan Abramov, Andrew Clark')
    ).toBeInTheDocument();
    expect(screen.queryByText('Brendan Eich')).toBeNull();

    // Interacción del usuario -> Búsqueda

    fireEvent.change(pantalla.queryByDisplayValue('Reaccionar'), {
      objetivo:
        { valor: 'JavaScript', }, });

    esperar(pantalla.queryByDisplayValue('React')).toBeNull();

    esperar(pantalla.queryByDisplayValue('JavaScript') ).toBeInTheDocument(); }); });

```

Finalmente, podemos enviar esta solicitud de búsqueda activando un evento de envío con el botón. El nuevo término de búsqueda se utiliza desde el estado del componente App, por lo que la nueva URL busca historias relacionadas con JavaScript que ya hemos simulado:

src/App.test.jsx

```
describe('Aplicación', () => {  
  
  ...  
  
  it('busca historias específicas', async () => {  
  
    ...  
  
    esperar(pantalla.queryByDisplayValue('React')).toBeNull();  
  
    esperar(pantalla.queryByDisplayValue('JavaScript') ).toBeInTheDocument();  
  
    fireEvent.click(pantalla.queryByText('Enviar'));  
  
    // Segunda obtención de datos  
  
    esperar esperar(async () => esperar javascriptPromise);  
  
    expect(screen.queryByText('Jordan Walke')).toBeNull();  
  
    esperar( screen.queryByText('Dan Abramov, Andrew  
    Clark') ).toBeNull();  
    expect(screen.queryByText('Brendan Eich')).toBeInTheDocument(); }); });
```

Brendan Eich se presenta como el creador de JavaScript, mientras que los creadores de React y Redux se eliminan.

Esta prueba representa un escenario completo en un solo caso. Podemos completar cada paso (obtención inicial, cambio del valor del campo de entrada, envío del formulario y recuperación de nuevos datos de la API) con las herramientas que hemos utilizado.

La biblioteca de pruebas de React con Vitest es la combinación más popular para las pruebas de React. RTL proporciona herramientas de prueba relevantes, mientras que Vitest ofrece un marco de pruebas general para suites de pruebas, casos de prueba, aserciones y capacidades de simulación. Si necesita una alternativa a RTL, considere probar [Enzyme²](https://www.robinwieruch.de/react-testing-jest-enzyme/) por Airbnb.

² <https://www.robinwieruch.de/react-testing-jest-enzyme/>

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
– Recapitular todos los [cambios del código fuente](#)² de esta sección.
- Lea más sobre [la biblioteca de pruebas de React en React](#)² . • Lea más sobre [las pruebas E2E en React](#)² .
- Mientras continúa con las próximas secciones, mantenga sus pruebas en verde y agregue nuevas pruebas cuando necesario.

Prueba de instantáneas

Las pruebas de instantáneas son una forma más sencilla de probar los componentes de React y su estructura. En esencia, una prueba de instantánea crea una instancia de la salida del componente renderizado como elementos HTML y su estructura. Esta instantánea se compara con la misma instantánea en la siguiente prueba para obtener más información sobre cómo cambió el componente renderizado y mostrar por qué las pruebas fallaron en la diferencia. Puedes aceptar o rechazar cualquier diferencia en tu código fuente hasta que el componente funcione correctamente.

Las pruebas de instantáneas son sencillas y se centran menos en los detalles de implementación del componente. Realicemos una prueba de instantáneas para nuestro componente SearchForm:

src/App.test.jsx

```
describe('Formulario de búsqueda', () => {

  ...

  it('renderiza una instantánea', () => {
    const { contenedor } = render(<SearchForm {...searchFormProps} />);
    esperar(contenedor.firstChild).toMatchSnapshot(); }); });
```

Ejecuta las pruebas con `npm run test` y verás que se ha creado una nueva carpeta `"src/_snapshots_"` en tu carpeta de proyecto. Al igual que con la función de depuración de RTL , hay una instantánea del componente SearchForm renderizado como una estructura de elemento HTML en el archivo. A continuación, accede al archivo `src/App.jsx` y modifica el HTML. Por ejemplo, intenta eliminar el texto en negrita del componente SearchForm:

² <https://tinyurl.com/mr2udtdx>
² <https://tinyurl.com/mt8ym9an>
² <https://www.robinwieruch.de/react-testing-library/>
² <https://www.robinwieruch.de/react-testing-cypress/>

src/App.jsx

```
const SearchForm = ({término de
  búsqueda,
  onSearchInput, acción
  de búsqueda, }) =>
( <acción
  del formulario={acción de búsqueda}>
  <EntradaConEtiqueta
    id="buscar"
    valor={término de búsqueda}
    está enfocado
    onInputChange={onSearchInput}
  >
    Buscar:
  </EntradaConEtiqueta>

  <button type="submit" deshabilitado={!searchTerm}>
    Entregar
  </botón>
</form> );
```

Después de la siguiente prueba, la línea de comando debería verse similar a la siguiente:

Línea de comandos

- Esperado - 3

+ Recibido + 1

```
    <etiqueta
      para="buscar"
    >
-      <fuerte>
-        Buscar:
-      </strong>
+      Buscar:
    </etiqueta>
```

Las instantáneas 1 fallaron

Este es un caso típico de una prueba de instantánea defectuosa. Cuando la estructura HTML de un componente se modifica involuntariamente, la prueba de instantánea nos informa en la línea de comandos. Para solucionarlo, iríamos a

El archivo `src/App.jsx` y edita el componente `SearchForm`. Para realizar cambios intencionales, presiona "u" en la línea de comandos para realizar pruebas interactivas y se creará una nueva instantánea. Pruébalo y observa cómo cambia el archivo de instantáneas en tu carpeta `src/_snapshots_`.

Vitest almacena las instantáneas en una carpeta para validar las diferencias en futuras pruebas. Los usuarios pueden compartir estas instantáneas entre equipos mediante plataformas de control de versiones como Git. Así garantizamos que el DOM se mantenga intacto.

Las pruebas instantáneas son útiles para configurar pruebas rápidamente en React, aunque es mejor evitar usarlas exclusivamente. En su lugar, úsalas para componentes que no se actualizan con frecuencia, son menos complejos y donde es más fácil comparar los resultados de los componentes.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
 - Recapitular todos los [cambios del código fuente](#)² ¹ desde esta sección. •

Agregue una prueba de instantánea para cada uno de los demás componentes y verifique el contenido de su carpeta `src/_snapshots_`.

² <https://tinyurl.com/nhzuv8by>

² ¹<https://tinyurl.com/y89ru5y9>

Estructura del proyecto React

Con varios componentes de React en un solo archivo, quizás te preguntes por qué no los pusimos en archivos diferentes desde el principio. Ya tenemos varios componentes en el archivo `src/App.jsx` que se pueden definir en sus propios archivos/carpetas (a veces también llamados módulos). Para el aprendizaje, es más práctico mantener estos componentes en un solo lugar. A medida que tu aplicación crezca, considera dividirlos en varios archivos/carpetas/módulos para que escale correctamente. Antes de reestructurar nuestro proyecto de React, repasemos [las instrucciones de importación y exportación de JavaScript²](#). porque importar y exportar archivos son dos conceptos fundamentales en JavaScript que debes aprender antes de React.

Es importante tener en cuenta que no existe una forma correcta de estructurar una aplicación React, ya que estas evolucionan naturalmente junto con los requisitos del proyecto. Realizaremos una refactorización simple de la estructura de carpetas y archivos de este proyecto para comprender el proceso. Posteriormente, se incluirá un artículo importante como ejercicio sobre la organización de carpetas y archivos en proyectos React.

En la línea de comando en la carpeta de su proyecto, cree los siguientes archivos para todos nuestros componentes en la carpeta `src/`:

Línea de comandos

```
toque src/list.jsx src/input-with-label.jsx src/search-form.jsx
```

Mueva cada componente del archivo `src/App.jsx` a su propio archivo, excepto el componente `List`, que debe compartir su ubicación con el componente `Item` en el archivo `src/list.jsx`. Luego, en cada archivo, asegúrese de importar React y exportar el componente que se necesita usar. Por ejemplo, en el archivo `src/list.jsx`:

`src/lista.jsx`

```
constante Lista = ({ lista, onRemoveItem }) => (  
  <ul>  
    {lista.map((elemento) => (  
      <Artículo  
        clave={item.objectID}  
        elemento={item}  
        onRemoveItem={onRemoveItem} /> ))}  
    </>  
  <ul>  
  
);  
  
constante Item = ({ item, onRemoveItem }) => (  
  <li>  
    <span>
```

² <https://www.robinwieruch.de/javascript-import-export/>

```

      <a href={item.url}>{item.title}</a>

      <span>{item.author}
      <span>{item.num_comments}
      <span>{item.points}
      <span>
        <button type="button" onClick={() => onRemoveItem(item)}>
          Despedir
        </botón>
      </li>
    );

    exportar { Lista };

```

Dado que solo el componente Lista usa el componente Elemento, podemos mantenerlo en el mismo archivo. Si esto cambia porque el componente Elemento se usa en otro lugar, podemos asignarle su propio archivo. A continuación, el componente InputWithLabel también obtiene su archivo dedicado:

src/entrada-con-etiqueta.jsx

```

importar * como React desde 'react';

constante EntradaConEtiqueta = ({
  id,
  valor,
  tipo = 'texto',
  onChange,
  isFocused,
  hijos, }) =>
{ const
  inputRef = React.useRef();

  React.useEffect(() => { if
    (isFocused && inputRef.current)
    { inputRef.current.focus();
    }
  }, [está enfocado]);

  devolver (
    <>
      <label htmlFor={id}>{hijos}</label> <input

```



```

      ref={inputRef} id={id}
      tipo={tipo}
      valor={valor}

      onChange={onInputChange} />

</>

); };

```

```
exportar { InputWithLabel };

```

El componente SearchForm del archivo src/search-form.jsx debe importar el componente InputWithLabel. Al igual que el componente Item, podríamos haber dejado el componente InputWithLabel junto a SearchForm; sin embargo, nuestro objetivo es que el componente InputWithLabel sea reutilizable con otros componentes:

```
src/formulario-de-búsqueda.jsx
```

```
importar { InputWithLabel } desde './input-with-label';

```

```

const SearchForm = ({término de búsqueda, onSearchInput, acción de búsqueda}) => (
  <form action={AcciónDeBúsqueda}>
    <EntradaConEtiqueta
      id="buscar"
      valor={término de búsqueda}
      está enfocado
      onInputChange={onSearchInput}
    >
      <strong>Buscar:</strong>
    </EntradaConEtiqueta>

    <button type="submit" deshabilitado={!searchTerm}>
      Entregar
    </botón>
  </form>
);

```

```
exportar { Formulario de búsqueda };

```

El componente App debe importar todos los componentes que necesita para renderizar. No necesita importar InputWithLabel, ya que solo se usa para el componente SearchForm.

```
src/App.jsx
```

```
importar * como React desde 'react';
importar axios desde 'axios';

importar { SearchForm } de './search-form'; importar { List }
de './list';

...

constante Aplicación = () => {
  ...
};

exportar aplicación predeterminada;
```

Los componentes que se usan en otros componentes ahora tienen su propio archivo. Si un componente se debe usar como componente reutilizable (p. ej., InputWithLabel), recibe su propio archivo. Solo si un componente (p. ej., Item) está dedicado a otro componente (p. ej., List), lo mantenemos en el mismo archivo. A partir de aquí, existen varias estrategias para estructurar la jerarquía de carpetas/archivos. Una opción es crear una carpeta para cada componente:

Estructura del proyecto

```
- lista/
-- index.jsx -
formulario de búsqueda/
-- index.jsx -
entrada-con-etiqueta/
--index.jsx
```

El archivo index.jsx contiene los detalles de implementación del componente, mientras que otros archivos en la misma carpeta tienen diferentes responsabilidades, como estilo, pruebas y tipos:

Estructura del proyecto

```
- lista/
-- index.jsx --
estilo.css --
prueba.js
-- tipos.js
```

Si se utiliza CSS-in-JS, donde no se necesita ningún archivo CSS, aún se puede tener un archivo style.js separado para todos los componentes con estilo:

Estructura del proyecto

```
- lista/ --  
index.jsx --  
estilo.js --  
prueba.js --  
tipos.js
```

A veces, necesitaremos cambiar de una estructura de carpetas orientada a la tecnología a una estructura de carpetas orientada al dominio, especialmente a medida que el proyecto crece. La carpeta `Universalshared/` se comparte entre componentes específicos del dominio:

Estructura del proyecto

```
- mensajes.jsx -  
usuarios.jsx -  
compartido/  
-- botón.jsx --  
entrada.jsx
```

Si escala esto a la estructura de carpetas de nivel más profundo, cada componente también tendrá su propia carpeta en una estructura de proyecto orientada al dominio:

Estructura del proyecto

```
- mensajes/ --  
index.jsx --  
style.css  
-- test.js --  
types.js -  
usuarios/  
-- index.jsx --  
style.css -- test.js  
-- types.js -  
compartido/  
  
-- botón/  
--- index.jsx ---  
estilo.css ---  
prueba.js ---  
tipos.js -- entrada/  
--- index.jsx  
--- estilo.css
```

```
--- prueba.js  
--- tipos.js
```

Hay muchas maneras de estructurar tu proyecto React, desde proyectos pequeños hasta grandes: desde estructuras de carpetas simples hasta complejas; anidación de carpetas de uno a dos niveles; carpetas dedicadas a estilos, tipos y pruebas, junto con la lógica de implementación. No hay una forma correcta de estructurar carpetas/archivos. Sin embargo, en los ejercicios, encontrarás mi enfoque de 5 pasos para estructurar un proyecto React.

Después de todo, los requisitos de un proyecto evolucionan con el tiempo, al igual que su estructura. Si mantener todos los recursos en un solo archivo parece correcto, no hay ninguna regla que lo prohíba.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² ³ .
 - Recapitular todos los [cambios del código fuente](#)² de esta sección.
- Lea más sobre [las estructuras de carpetas de React](#)² .
- Lea más sobre [las arquitecturas React basadas en características](#)² .
- Mantenga la estructura de carpetas actual si se siente seguro. Las secciones siguientes la omitirán, solo...
utilizando el archivo `src/App.jsx`.

² ³<https://tinyurl.com/3rc7sc86>

² <https://tinyurl.com/mumvenae>

² <https://www.robinwieruch.de/react-folder-structure/>

² <https://www.robinwieruch.de/react-feature-architecture/>

Reacción en el mundo real (avanzado)

Hemos cubierto la mayoría de los fundamentos de React, sus características heredadas y las técnicas para el mantenimiento de aplicaciones. Ahora es el momento de profundizar en el desarrollo de características reales de React. Cada una de las siguientes secciones incluirá una tarea. Intenta abordar estas tareas sin las sugerencias opcionales primero, pero ten en cuenta que serán difíciles en tu primer intento. Si necesitas ayuda, usa las sugerencias opcionales o sigue las instrucciones de la sección.

Clasificación

Tarea: Trabajar con una lista de elementos suele incluir interacciones que facilitan el acceso a los datos a los usuarios. Hasta ahora, cada elemento se listaba con sus propiedades. Para que sea explorable, la lista debe permitir ordenar cada propiedad por título, autor, comentarios y puntos en orden ascendente o descendente. Ordenar solo en una dirección es suficiente, ya que ordenar en la otra dirección formará parte de la siguiente tarea.

Consejos opcionales:

- Introduzca un nuevo estado de ordenación en el componente Aplicación o Lista.
- Para cada propiedad (p. ej. , título, autor, puntos, núm_comentarios), implemente un botón HTML que establezca el estado de ordenación para esta propiedad.
- Use el estado de ordenación para aplicar una función de ordenación adecuada en la lista.
- Use una biblioteca de utilidades como [Lodash²](https://lodash.com) . Se recomienda su función sortBy .

A hand-drawn sketch of a user interface for searching and sorting data. At the top, there is a search bar with the text "Search:" followed by an input field containing "React" and a "Submit" button. Below the search bar, there are three buttons labeled "Topic", "Upvotes", and "Comments", each with a diamond-shaped sort icon to its right. Below these buttons is a large rectangular box representing a list of items, with several horizontal lines indicating the rows of the list.

Bien, ¡comencemos con esta tarea! Trataremos la lista de datos como una tabla. Cada fila representa un elemento de la lista y cada columna representa una propiedad del elemento. La introducción de encabezados debería proporcionar al usuario más información sobre cada columna:

² <https://lodash.com>

src/App.jsx

```

constante Lista = ({ lista, onRemoveItem }) => (
  <ul>
    <li style={{ display: 'flex' }}> <span
      style={{ width: '40%' }}>Título <span style={{ width: '30%' }}
      >Autor <span style={{ width: '10%' }}>Comentarios <span
      style={{ width: '10%' }}>Puntos <span style={{ width: '10%' }}
      >Acciones </li>

    {lista.map((elemento) => (
      <Artículo
        clave={item.objectID}
        elemento={item}
        onRemoveItem={onRemoveItem} /> ))}
    </
  <ul>

);

```

Usamos el estilo en línea para el diseño más básico. Para que el diseño del encabezado coincida con el de las filas, asigne también un diseño a las filas del componente Item:

src/App.jsx

```

constante Item = ({ item, onRemoveItem }) => ( <li
  style={{ display: 'flex' }}> <span style={{ width:
    '40%' }}>
    <a href={item.url}>{item.title}</a> <span

    style={{ width: '30%' }}>{item.author} <span style={{ width: '10%' }}
    >{item.num_comments} <span style={{ width: '10%' }}>{item.points} <span
    style={{ width: '10%' }}> <button type="button" onClick={() =>
    onRemoveItem(item)}>

    Despedir
    </botón>
  </li>

);

```

Durante la implementación, eliminaremos los atributos de estilo, ya que ocupan mucho espacio y saturan la lógica de implementación (por lo que se extraen al CSS correcto). Sin embargo, te recomiendo que los guardes.

El componente Lista gestionará el nuevo estado de ordenación. Esto también se puede hacer en el componente Aplicación, pero al final, solo el componente Lista lo necesita, por lo que podemos transferir el estado directamente a él. El estado de ordenación se inicializa con el estado "NONE", por lo que los elementos de la lista se muestran en el orden en que se obtienen de la API. Además, añadiremos un nuevo controlador para establecer el estado de ordenación con una clave específica:

src/App.jsx

```
constante Lista = ({ lista, onRemoveItem }) => {
  constante [ordenar, establecerOrdenar] = React.useState('NINGUNO');

  constante handleSort = (clave de clasificación) => {
    setSort(clave de clasificación);
  };

  devolver (
    ...

  );
};
```

En el encabezado del componente Lista, los botones pueden ayudarnos a establecer el estado de clasificación para cada columna/propiedad. Se utiliza un controlador en línea para introducir la clave específica de ordenación (sortKey). Al hacer clic en el botón de la columna "Título", "TÍTULO" se convierte en el nuevo estado de ordenación:

src/App.jsx

```
constante Lista = ({ lista, onRemoveItem }) => {
  ...

  devolver (
    <ul>
      <li>
        <span>
          <button type="button" onClick={() => handleSort('TÍTULO')}>
            Título
          </botón>
        <span>

          <button type="button" onClick={() => handleSort('AUTOR')}>
            Autor
          </botón>

        </span>
      </li>
    </ul>
  );
};
```

```

    <span>
      <button type="button" onClick={() => handleSort('COMENTARIO')}>
        Comentarios
      </botón>

    <span>
      <button type="button" onClick={() => handleSort('POINT')}>
        Agujas
      </button>

    <span>Acciones </li>

    {lista.map((elemento) => </ ... )}
  <ul>

); };

```

La gestión de estados para la nueva función está implementada, pero aún no vemos nada al hacer clic en nuestros botones. Esto se debe a que el mecanismo de ordenación no se ha aplicado a la lista real. Ordenar un array con JavaScript no es trivial, ya que cada primitiva de JavaScript (por ejemplo, cadena, booleano, número) presenta casos extremos cuando un array se ordena por sus propiedades. Usaremos una biblioteca llamada [Lodash](#)². Para solucionar esto, se incluyen muchas funciones de utilidad de JavaScript (por ejemplo, `sortBy`). Primero, instálelo mediante la línea de comandos:

Línea de comandos

```
npm instalar lodash
```

En segundo lugar, en la parte superior del archivo, importe la función de utilidad para ordenar:

```

src/App.jsx

importar * como React desde 'react'; importar
axios desde 'axios'; importar { sortBy }
desde 'lodash';

...

```

En tercer lugar, cree un objeto JavaScript (también llamado diccionario en este caso) con todas las asignaciones posibles de `sortKey` y la función `sort`. Cada clave de ordenación específica se asigna a una función que ordena la lista entrante.

Ordenar por 'NINGUNO' devuelve la lista sin ordenar; ordenar por 'PUNTO' devuelve una lista y sus elementos ordenados por la propiedad de puntos, y así sucesivamente:

² <https://lodash.com>

src/App.jsx

```
const ORDENAR =
  { NINGUNO: (lista) => lista,
    TÍTULO: (lista) => sortBy(lista, 'título'), AUTOR: (lista) =>
    sortBy(lista, 'autor'), COMENTARIO: (lista) => sortBy(lista,
    'num_comentarios').reverse(), PUNTO: (lista) => sortBy(lista, 'puntos').reverse(), };
```

```
constante Lista = ({ lista, onRemoveItem }) => {
  ...
};
```

Con el estado de ordenamiento (sortKey) y todas las posibles variaciones de ordenamiento (SORTS) a nuestra disposición, podemos ordenar la lista antes de mapearla:

src/App.jsx

```
constante Lista = ({ lista, onRemoveItem }) => {
  constante [ordenar, establecerOrdenar] = React.useState('NINGUNO');
```

```
  constante handleSort = (clave de clasificación) => {
    setSort(clave de clasificación); };
```

```
  const sortFunction = SORTS[ordenar]; const
  sortedList = sortFunction(lista);
```

```
  devolver (
    <ul>
      ...

      {sortedList.map((elemento) => (
        <Artículo
          clave={item.objectID}
          elemento={item}
          onRemoveItem={onRemoveItem} /> )}}
    </
  <ul>
```

```
); };
```

Tarea terminada y aquí viene el resumen: Primero, extrajimos la función de ordenación del diccionario por su `sortKey` (estado), luego la aplicamos a la lista antes de asignarla para representar cada componente `Item`. Segundo, representamos los botones HTML como columnas de encabezado para ofrecer interacción a nuestros usuarios.

Luego, añadimos detalles de implementación para cada botón modificando el estado de ordenación. Finalmente, usamos el estado de ordenación para ordenar la lista.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor² .
 - Recapitulación de todos los [cambios en el código fuente](#)³ de esta sección.
- Lea más sobre [Lodash](#)³ ¹. ¿Por qué

usamos propiedades numéricas como `puntos` y `num_comments` para una ordenación inversa? Usa tus habilidades de estilo para brindarle al usuario información sobre la ordenación activa. Este mecanismo...

puede ser tan sencillo como darle al botón de clasificación activo un color diferente.

² <https://tinyurl.com/57ybtsmu>

³ <https://tinyurl.com/2tke4skw>

³ ¹<https://lodash.com>

Ordenamiento inverso

Tarea: La función de ordenamiento funciona, pero el ordenamiento solo incluye una dirección. Implemente un ordenamiento inverso al hacer doble clic en un botón de ordenamiento, de modo que se alterne entre ordenamiento normal (ascendente) e inverso (descendente).

Consejos opcionales:

- Tenga en cuenta que la clasificación inversa o normal podría ser simplemente otro estado (por ejemplo, `isReverse`) al lado del clave de ordenación.
- Establezca el nuevo estado en el controlador `handleSort` según la ordenación anterior. • Utilice el nuevo estado `isReverse` para ordenar la lista con la función de ordenación del diccionario con la función `reverse()` aplicada opcionalmente desde matrices de JavaScript.

Empecemos. La dirección de ordenación inicial funciona tanto para cadenas como para ordenaciones numéricas, como la ordenación inversa para números de JavaScript, que los ordena de mayor a menor. Ahora necesitamos otro estado para saber si la ordenación es inversa o normal:

`src/App.jsx`

```
constante Lista = ({ lista, onRemoveItem }) => { constante
  [ordenar, setSort] = React.useState({
    sortKey: 'NINGUNO',
    isReverse: falso, });
  ...
};
```

A continuación, proporcione al controlador de clasificación la lógica para ver si los activadores `sortKey` entrantes son una clasificación normal o inversa. Si la clave de ordenamiento es la misma que la del estado, debería ser un ordenamiento inverso, pero solo si el estado de ordenamiento no estaba ya invertido:

`src/App.jsx`

```
constante Lista = ({ lista, onRemoveItem }) => { constante
  [sort, setSort] = React.useState({ sortKey: 'NONE',
    isReverse: falso, });

  const handleSort = (sortKey) => { const
    isReverse = sort.sortKey === sortKey && !sort.isReverse;
```

```

    setSort({ sortKey: sortKey, isReverse: isReverse }); };

const sortFunction = SORTS[sort.sortKey]; const
sortedList = sortFunction(lista);

devolver (
  ...

); };

```

Por último, dependiendo del nuevo estado `isReverse` , aplique la función de ordenamiento del diccionario con o sin el método inverso de JavaScript integrado para matrices:

`src/App.jsx`

```

constante Lista = ({ lista, onRemoveItem }) => { constante
  [sort, setSort] = React.useState({ sortKey: 'NONE',
    isReverse: false, });

  const handleSort = (sortKey) => { const
    isReverse = sort.sortKey === sortKey && !sort.isReverse;

    setSort({ sortKey, isReverse }); };

  constante sortFunction = SORTS[sort.sortKey];
  constante sortedList = sort.isReverse
    ? sortFunction(lista).reverse() :
    sortFunction(lista);

  devolver (
    ...

  ); };

```

¡La ordenación inversa ya está operativa! ¡Felicitaciones! Ya tienes una lista totalmente ordenable. Por cierto: para el objeto pasado a la función de actualización de estado, usamos una notación abreviada de inicializador de objetos:

Patio de juegos de código

```
constante nombre = 'Robin';
```

```
const usuario =
  { nombre: nombre, };
```

```
console.log(usuario); //
{ nombre: "Robin" }
```

Cuando el nombre de la propiedad en su objeto es el mismo que el nombre de su variable, puede omitir el par clave/valor y solo escribir el nombre:

Patio de juegos de código

```
constante nombre = 'Robin';
```

```
const usuario =
  { nombre, };
```

```
console.log(usuario); //
{ nombre: "Robin" }
```

Si es necesario, lea más sobre [los inicializadores de objetos de JavaScript](#)³ ².

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor³ ³.
 - Recapitulación de todos los [cambios en el código fuente](#)³ De esta sección.
- Considere la desventaja de mantener el estado de ordenación en la Lista en lugar del componente Aplicación. Si no lo sabe, ordene la lista por "Título" y busque otras historias después. ¿Qué cambiaría si el estado de ordenación estuviera en el componente Aplicación?
- Usa tus habilidades de diseño para ofrecer al usuario información sobre la ordenación activa y su estado inverso. Podría ser un SVG con una flecha hacia arriba o hacia abajo junto a cada botón de ordenación activo.

³ ²<https://mzl.la/2XuN651>

³ ³<https://tinyurl.com/y96a9sha>

³ <https://tinyurl.com/529w99nz>

Recordar últimas búsquedas

Tarea: Recordar los últimos cinco términos de búsqueda que llegaron a la API y proporcionar un botón para navegar rápidamente entre búsquedas. Al hacer clic en los botones, se recuperarán las historias del término de búsqueda.

Consejos opcionales:

No use un nuevo estado para esta función. En su lugar, reutilice el estado de la URL y la función de actualización del estado `setUrl` para obtener historias de la API. Adáptelos a múltiples URL como estado y para configurar múltiples URL con `setUrls`. La última URL de `urls` se puede usar para obtener los datos y las últimas cinco URL de `urls` se pueden usar para mostrar los botones.

Empecemos. Primero, refactorizaremos todas las funciones de actualización de estado de URL a URLs y de `setUrl` a `setUrls`. En lugar de inicializar el estado con una URL como cadena, lo convertiremos en un array con la URL inicial como única entrada:

src/App.jsx

```
const Aplicación = () => {
```

```
  ...
```

```
  constante [urls, setUrls] = React.useState([ ` $
    {API_ENDPOINT}${searchTerm}`, ]);
```

```
  ...
```

```
};
```

En segundo lugar, en lugar de usar el estado actual de la URL para obtener datos, se usa la última entrada de la matriz de URL. Si se agrega otra URL a la lista, se usa para obtener datos:

src/App.jsx

```

constante Aplicación = () => {

  ...

  const handleFetchStories = React.useCallback(async () => { dispatchStories({ tipo:
    'STORIES_FETCH_INIT' });

    intenta
    { const lastUrl = urls[urls.length - 1]; const resultado
    = await axios.get(lastUrl);

    dispatchStories({ tipo:
      'STORIES_FETCH_SUCCESS', carga
      útil: result.data.hits, }); } catch

    { dispatchStories({ tipo: 'STORIES_FETCH_FAILURE' });

    } }, [urls]);

  ...
};

```

Y tercero, en lugar de almacenar la cadena de URL como estado con la función de actualización de estado, concatene la nueva URL usando el método concat con las URL anteriores en una matriz para el nuevo estado:

src/App.jsx

```

constante Aplicación = () => {

  ...

  constante searchAction = () => {
    constante url = `${API_ENDPOINT}${searchTerm}`;
    setUrls(urls.concat(url)); };

  ...
};

```

Con cada búsqueda, se almacena otra URL en nuestro estado de URLs. A continuación, se renderiza un botón para cada una de las últimas cinco URLs. Incluiremos un nuevo controlador universal para estos botones, y cada uno pasa una URL específica con un controlador en línea más específico:


```
src/App.jsx
```

```
const getLastSearches = (urls) => urls.slice(-5);
```

```
...
```

```
const App = () => {
```

```
  ...
```

```
  const handleLastSearch = (url) => { // hacer algo };
```

```
  const últimasBúsquedas = obtenerÚltimasBúsquedas(urls);
```

```
  devolver (
```

```
    <div>
```

```
      Mis historias de hackers
```

```
      <Formulario de búsqueda ... />
```

```
      {lastSearches.map((url) => (
```

```
        <botón
```

```
          clave={url}
```

```
          tipo="botón" al
```

```
          hacer clic={() => handleLastSearch(url)}
```

```
        >
```

```
          {url} </
```

```
        botón> ))}
```

```
    ...
```

```
  </div>
```

```
);
```

A continuación, en lugar de mostrar la URL completa de la última búsqueda en el botón como texto del botón, muestre solo el término de búsqueda reemplazando el punto final de la API con una cadena vacía:

src/App.jsx

```
constante extractSearchTerm = (url) => url.replace(API_ENDPOINT, "");

constante obtenerÚltimasBúsquedas = (urls) =>
  urls.slice(-5).map((url) => extractSearchTerm(url));

...

constante Aplicación = () => {
  ...

  const últimasBúsquedas = obtenerÚltimasBúsquedas(urls);

  devolver (
    <div>
      ...

      {lastSearches.map((término de búsqueda) => (
        <botón
          clave={término de
            búsqueda}
          tipo="botón" al hacer clic={() => handleLastSearch(término de búsqueda)}
        >
          {término de
            búsqueda} </
        button> )})
      ...
    </div>

  ); };
```

La función `getLastSearches` ahora devuelve términos de búsqueda en lugar de URL. El término de búsqueda se pasa al controlador en línea en lugar de la URL. Al mapear la lista de URL en `getLastSearches`, podemos extraer el término de búsqueda de cada URL dentro del método `map` del array. Para simplificarlo, también podría verse así:

src/App.jsx

```
constante obtenerÚltimasBúsquedas = (urls) =>
  urls.slice(-5).map(extraerTérminoDeBúsqueda);
```

Ahora proporcionaremos funcionalidad para el nuevo controlador que usa cada botón, ya que al hacer clic en uno de ellos se activará otra búsqueda. Dado que usamos el estado de las URL para obtener datos y sabemos que la última URL siempre se usa para obtenerlos, concatenamos una nueva URL a la lista de URL para activar otra solicitud de búsqueda:

src/App.jsx

```
constante Aplicación = () => {
  ...

  const handleLastSearch = (término de búsqueda) => {
    constante url = `${API_ENDPOINT}${searchTerm}`;
    setUrls(urls.concat(url)); };

  ...
};
```

Si compara la lógica de implementación de este nuevo controlador con la de handleSearchSubmit, podrá observar algunas funciones comunes. Extraiga estas funciones comunes a un nuevo controlador y a una nueva función de utilidad extraída:

src/App.jsx

```
const getUrl = (término de búsqueda) => `${API_ENDPOINT}${término de búsqueda}`;

...

constante Aplicación = () => {
  ...

  const handleSearch = (término de búsqueda) =>
    { const url = getUrl(término de búsqueda);
      setUrls(urls.concat(url)); };

  const searchAction = () =>
    { handleSearch(término de
      búsqueda); };
```

```
const handleLastSearch = (término de búsqueda) =>
  { handleSearch(término de
    búsqueda); };

...

};
```

La nueva función de utilidad se puede usar en otra parte del componente App. Si extrae funcionalidad que puede ser utilizada por dos partes, verifique siempre si puede ser utilizada por un tercero:

src/App.jsx

```
constante Aplicación = () => {
  ...

  // importante: aún envuelve el valor devuelto en [] const [urls, setUrls]
  = React.useState([getUrl(searchTerm)]);

  ...

};
```

La funcionalidad debería funcionar, pero presenta problemas o falla si se usa el mismo término de búsqueda varias veces, ya que searchTerm se usa para cada elemento de botón como atributo clave . Para que la clave sea más específica, concaténala con el índice del array mapeado.

src/App.jsx

```
constante Aplicación = () => {
  ...

  devolver (
    <div>
      ...

      {lastSearches.map((término de búsqueda, índice) => (
        <botón
          clave={término de búsqueda +
            índice} tipo="botón"
          al hacer clic={() => handleLastSearch(término de búsqueda)}
        >
          {término de
            búsqueda} </
            button> ))}
    )}
```

```

    ...
  </div>

); };
```

No es la solución ideal, ya que el índice no es una clave estable (sobre todo al añadir elementos a la lista); sin embargo, no deja de funcionar en este caso. La función ya funciona, pero puedes añadir más mejoras de experiencia de usuario siguiendo las instrucciones a continuación.

Más tareas:

- (1) No mostrar la búsqueda actual como un botón, solo las cinco búsquedas anteriores. Consejo: Adaptar la función `getLastSearches`.
- (2) No mostrar búsquedas duplicadas. Buscar "React" dos veces no debería generar dos búsquedas diferentes. Sugerencia: adapte la función `getLastSearches`.
- (3) Establezca el valor del campo de entrada del componente `SearchForm` con el último término de búsqueda si uno de los botones. Se hace clic en los botones.

La fuente de los cinco botones generados es la función `getLastSearches`. En ella, tomamos el array de URL y devolvemos las últimas cinco entradas. Ahora modificaremos esta función de utilidad para que devuelva las últimas seis entradas en lugar de cinco, eliminando la última, para que la búsqueda actual no se muestre como botón. Posteriormente, solo se mostrarán como botones las cinco búsquedas anteriores:

```
src/App.jsx
```

```

const obtenerÚltimasBúsquedas = (urls) =>
  URL

  .slice(-6) .slice(0,
  -1) .map(extraerTérminoDeBúsqueda);
```

Si se ejecuta la misma búsqueda dos o más veces seguidas, aparecen botones duplicados, lo cual probablemente no sea el comportamiento deseado. Sería aceptable agrupar búsquedas idénticas en un solo botón si fueran consecutivas. También resolveremos este problema en la función de utilidad. Antes de separar la matriz en las cinco búsquedas anteriores, agrupe las búsquedas idénticas:

src/App.jsx

```

constante obtenerÚltimasBúsquedas = (urls) =>
  URL
    .reduce((resultado, url, índice) => { const
      searchTerm = extractSearchTerm(url);

      si (índice === 0)
        { devolver resultado.concat(searchTerm);
        }

      const previousSearchTerm = resultado[resultado.length - 1];

      si (término de búsqueda === término de búsqueda anterior) {
        devolver resultado; }
      else
        { devolver resultado.concat(searchTerm);
        }

    } },

    []).slice(-6).slice(0, -1);

```

La función reduce comienza con un array vacío como resultado. La primera iteración concatena el término de búsqueda extraído de la primera URL con el resultado. Cada término de búsqueda extraído se compara con el anterior. Si el término de búsqueda anterior es diferente del actual, concatena el término de búsqueda con el resultado. Si los términos de búsqueda son idénticos, devuelve el resultado sin añadir nada.

El campo de entrada del componente SearchForm debe configurarse con el nuevo término de búsqueda si se hace clic en uno de los últimos botones de búsqueda. Esto se puede solucionar mediante la función de actualización de estado para el valor específico utilizado en el componente SearchForm.

src/App.jsx

```

constante Aplicación = () => {
  ...

  const handleLastSearch = (término de búsqueda) =>
    { setSearchTerm(término de búsqueda);

    handleSearch(término de búsqueda); };

  ...
};

```

Por último, extraiga el nuevo contenido renderizado de la función de esta sección como un componente independiente, para mantener el componente de la aplicación liviano:

src/App.jsx

```
const Aplicación = () => {
  ...

  const últimasBúsquedas = obtenerÚltimasBúsquedas(urls);

  devolver (
    <div>
      ...

      <Formulario de búsqueda ... />

      <Últimas búsquedas
        últimasBúsquedas={últimasBúsquedas}
        enÚltimaBúsqueda={manejarÚltimaBúsqueda} />

      <hr />

      ...
    </div>

  ); };

const ÚltimasBúsquedas = ({ últimasBúsquedas, enÚltimaBúsqueda }) => (
  <>
    {lastSearches.map((término de búsqueda, índice) => (
      <botón
        clave={término de búsqueda + índice}
        tipo="botón" al
        hacer clic={() => onLastSearch(término de búsqueda)}
      >
        {término de
        búsqueda} </
      button> ))) </>
  );
```

Esta función no fue fácil. Se requirieron muchos conocimientos básicos de React y JavaScript para lograrla. Si no tuviste problemas para implementarla tú mismo o para seguir las instrucciones...

Instrucciones, estás muy bien preparado. Si tuviste algún problema, no te preocupes demasiado. Quizás incluso hayas encontrado otra solución y te haya resultado más sencilla que la que mostré aquí.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor³ .
 - Recapitulación de todos los [cambios en el código fuente](#)³ de esta sección.
- Lea más sobre [la agrupación en JavaScript](#)³ .

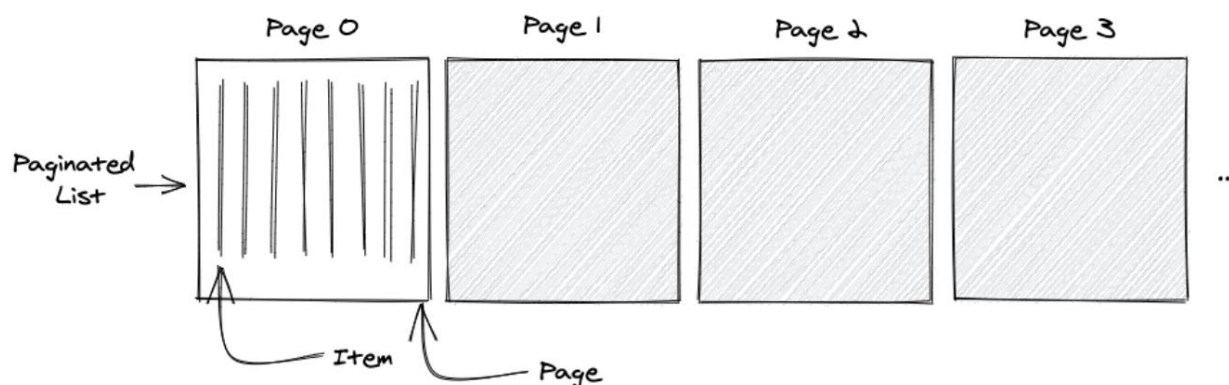
³ <https://tinyurl.com/25u4ns35>

³ <https://tinyurl.com/axtabwcj>

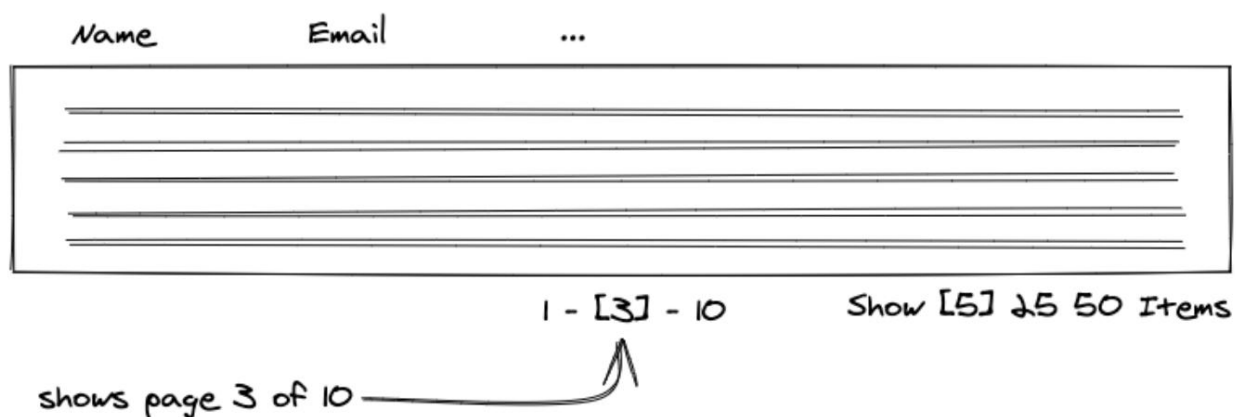
³ <https://www.robinwieruch.de/javascript-groupby/>

Búsqueda paginada

Buscar noticias populares mediante la API de Hacker News es solo un paso hacia un motor de búsqueda completamente funcional, y existen muchas maneras de optimizar la búsqueda. Examine con más detalle la estructura de datos y observe cómo [la API de Hacker News³](https://hn.algolia.com/api) devuelve más que una lista de resultados. En concreto, devuelve una lista paginada. La propiedad `page`, que es 0 en la primera respuesta, puede usarse para obtener más listas paginadas como resultados. Solo necesita pasar la siguiente página con el mismo término de búsqueda a la API.

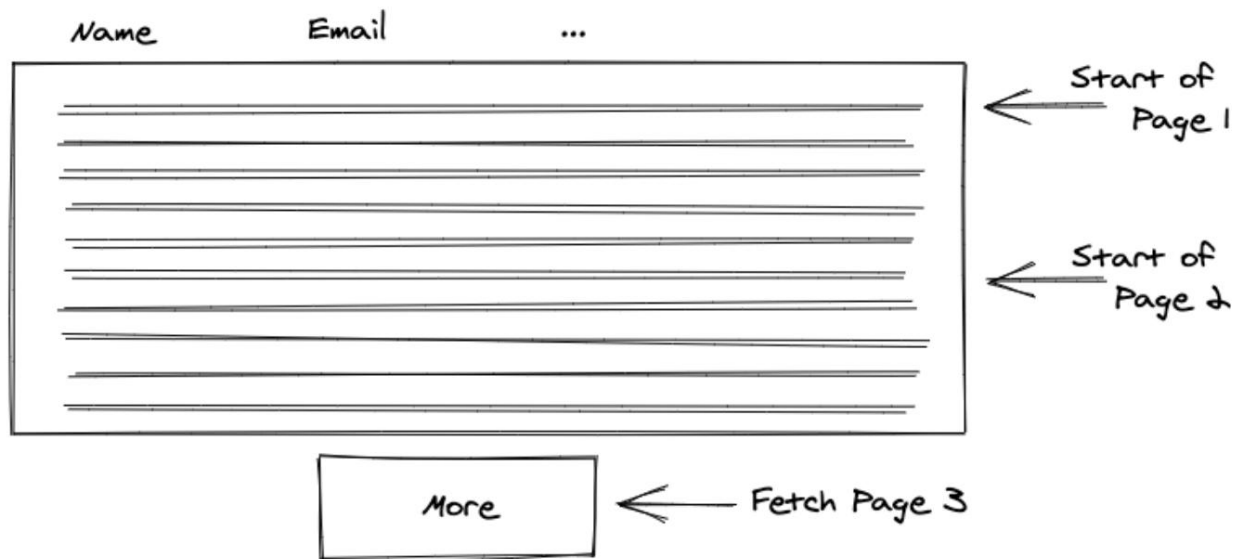


A continuación se muestra cómo implementar una búsqueda paginada con la estructura de datos de Hacker News. Si está acostumbrado a la paginación en otras aplicaciones, es posible que tenga en mente una fila de botones del 1 al 10, donde la página seleccionada se resalta 1-[3]-10 y al hacer clic en uno de los botones se obtiene y muestra este subconjunto de datos.



En cambio, implementaremos la función como paginación infinita. En lugar de mostrar una sola lista paginada al hacer clic en un botón, mostraremos todas las listas paginadas como una sola lista con un botón para acceder a la página siguiente. Cada lista paginada adicional se concatena al final de la lista.

³ <https://hn.algolia.com/api>



Tarea: En lugar de obtener solo la primera página de una lista, amplíe la funcionalidad para obtener las páginas siguientes. Implemente esto como paginación infinita al hacer clic en un botón.

Consejos opcionales:

- Ampliar el API_ENDPOINT con los parámetros necesarios para la búsqueda paginada.
- Almacenar la página del resultado como estado tras la búsqueda de datos.
- Obtener la primera página (0) de datos con cada búsqueda.
- Obtener la página siguiente (página + 1) con cada solicitud adicional generada con un nuevo HTML botón.

¡Hagámoslo! Primero, extendamos la constante de la API para que pueda procesar datos paginados más adelante. Convertiremos esta constante:

src/App.jsx

```
const API_ENDPOINT = 'https://hn.algolia.com/api/v1/search?query=';
```

```
const getUrl = (término de búsqueda) => `${API_ENDPOINT}${término de búsqueda}`;
```

En una constante API componible con sus parámetros:

src/App.jsx

```
constante API_BASE = 'https://hn.algolia.com/api/v1'; constante
API_SEARCH = '/búsqueda'; constante
PARAM_SEARCH = 'consulta=';

// cuidado: note el ? en el medio
const getUrl = (término de búsqueda) => `
  {API_BASE}${BÚSQUEDA_API}?${PARAM_SEARCH}${término de búsqueda}`;
```

Afortunadamente, no necesitamos ajustar los puntos finales de la API en otras partes de la aplicación, ya que extrajimos una función `getUrl` común para ello. Sin embargo, hay un punto donde debemos abordar esta lógica en el futuro:

src/App.jsx

```
constante extractSearchTerm = (url) => url.replace(API_ENDPOINT, "");
```

En los siguientes pasos, no será suficiente reemplazar la base de nuestro punto final de API, que ya no está en nuestro código. Con más parámetros para el punto final de API, la URL se vuelve más compleja. Cambiará de X a Y:

Patio de juegos de código

```
// X
https://hn.algolia.com/api/v1/search?query=react

// Y
https://hn.algolia.com/api/v1/search?query=react&page=0
```

Es mejor extraer el término de búsqueda extrayendo todo lo que está entre `?` y `&`. Tenga en cuenta también que el parámetro de consulta está justo después del `?` y todos los demás parámetros, como la página, lo siguen:

src/App.jsx

```
constante extractSearchTerm = (url) =>
  url.substring(url.lastIndexOf('?') + 1, url.lastIndexOf('&'));
```

También es necesario reemplazar la clave (`consulta=`) , dejando solo el valor (`searchTerm`):

src/App.jsx

```
constante extractSearchTerm = (url) =>
  URL
    .substring(url.lastIndexOf('?') + 1, url.lastIndexOf('&')) .replace(PARAM_SEARCH,
    "");
```

Básicamente, recortaremos la cadena hasta que dejemos solo el término de búsqueda:

Código Playground.jsx

```
// URL
https://hn.algolia.com/api/v1/search?query=react&page=0
```

```
// URL después de la subcadena
query=react
```

```
// URL después de reemplazar
reaccionar
```

A continuación, el resultado devuelto por la API de Hacker News nos entrega los datos de la página :

src/App.jsx

```
constante Aplicación = () => {
  ...

  const handleFetchStories = React.useCallback(async () => { dispatchStories({ tipo:
    'STORIES_FETCH_INIT' });

    intenta
      { const lastUrl = urls[urls.length - 1]; const resultado
        = await axios.get(lastUrl);

        dispatchStories({ tipo:
          'STORIES_FETCH_FAILURE', carga útil:
            { lista:
              result.data.hits, página:
              result.data.page, }, }); } catch

      { dispatchStories({ tipo: 'STORIES_FETCH_FAILURE' });

      }, [urls]);
```

```
...  
};
```

Necesitamos almacenar estos datos para realizar búsquedas paginadas más tarde:

src/App.jsx

```
const storiesReducer = (estado, acción) => {  
  cambiar (acción.tipo) {  
    caso 'STORIES_FETCH_INIT':  
      ...  
    caso 'HISTORIAS_OBTENCIÓN_ÉXITO':  
      devolver {  
        ...estado,  
        isLoading: falso, isError:  
        falso, datos:  
        acción.payload.list, página:  
        acción.payload.page, };  
  
    caso 'FALLO DE BÚSQUEDA DE HISTORIAS':  
      ...  
    caso 'REMOVE_STORY':  
      ...  
    por defecto:  
      lanzar nuevo Error();  
  
  } };
```

```
constante Aplicación = () => {  
  ...  
  
  const [historias, dispatchStories] = React.useReducer( storiesReducer, { datos:  
    [], página: 0, isLoading:  
    false, isError: false } );  
  
  ...  
};
```

Ampliar el punto final de la API con el nuevo parámetro de página . Este cambio se vio respaldado por nuestras optimizaciones prematuras previas, al extraer el término de búsqueda de la URL:

src/App.jsx

```
const API_BASE = 'https://hn.algolia.com/api/v1'; const API_SEARCH =
'/búsqueda'; const PARAM_SEARCH =
'consulta='; const PARAM_PAGE = 'página=';

// cuidado: note el ? y el & en el medio const getUrl =
(searchTerm, page) =>
`${API_BASE}${API_SEARCH}?${PARAM_SEARCH}${término de búsqueda}&${PARAM_PAGE}${página}`;
```

A continuación, debemos ajustar todas las invocaciones de getUrl pasando el argumento de página . Dado que la búsqueda inicial y la última siempre obtienen la primera página (0), pasamos esta página como argumento a la función para recuperar la URL correspondiente:

src/App.jsx

```
const App = () => {
  ...

  const [urls, setUrls] = React.useState([getUrl(searchTerm, 0)]);

  ...

  const handleSearch = (término de búsqueda, página) => { const
    url = getUrl(término de búsqueda, página);
    setUrls(urls.concat(url)); };

  const searchAction = () => {
    handleSearch(término de búsqueda,
    0); };

  const handleLastSearch = (término de búsqueda) =>
    { setSearchTerm(término de búsqueda);

    handleSearch(término de búsqueda,
    0); };

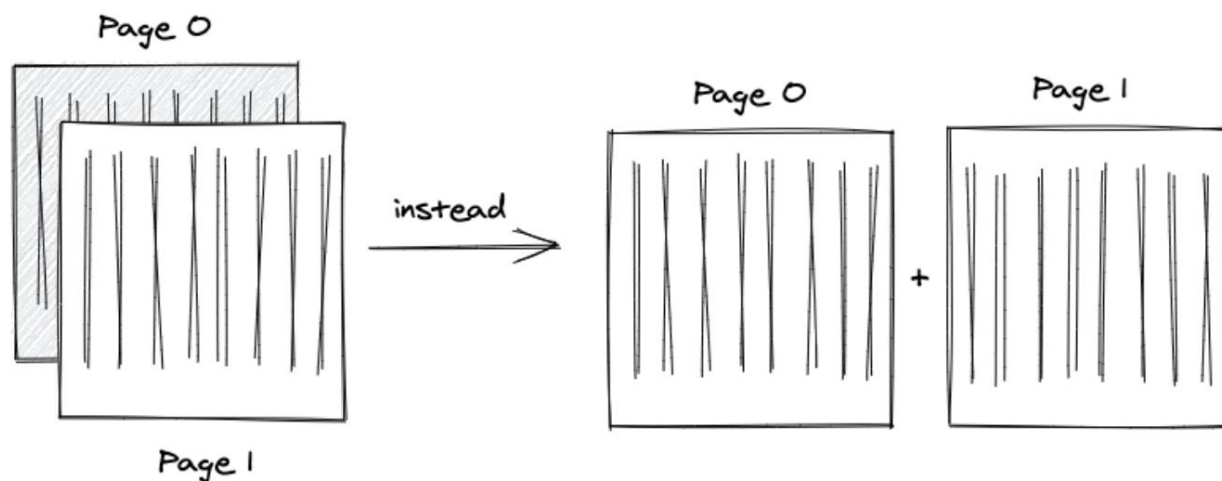
  ...
};
```

Para obtener la página siguiente cuando se hace clic en un botón, necesitaremos incrementar el argumento de la página en este nuevo controlador:

src/App.jsx

```
constante Aplicación = () => {  
  ...  
  
  constante handleMore = () => {  
    const lastUrl = urls[urls.length - 1]; const searchTerm  
    = extractSearchTerm(lastUrl); handleSearch(searchTerm,  
    stories.page + 1); };  
  
  ...  
  
  devolver (  
    <div>  
      ...  
  
      {historias.isLoading ?  
        ( <p>Cargando ...</p> ) :  
        (  
          <List list={historias.data} onRemoveItem={handleRemoveStory} /> )}  
  
      <button type="button" onClick={manejarMás}>  
        Más  
      </botón> </  
    div>  
  
  ); };
```

Hemos implementado la obtención de datos con el argumento de página dinámica . Las búsquedas inicial y final siempre usan la primera página, y cada obtención con el nuevo botón "Más" usa una página incrementada. Sin embargo, hay un error crucial al probar la función: las nuevas búsquedas no amplían la lista anterior, sino que la reemplazan por completo.



Resolvemos esto en el reductor evitando el reemplazo de datos actuales con datos nuevos , concatenando las listas paginadas:

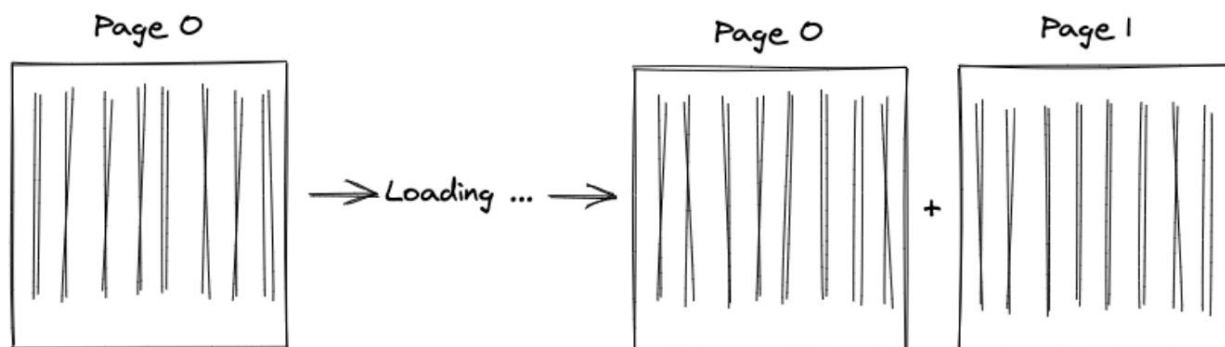
src/App.jsx

```
const storiesReducer = (estado, acción) => {
  cambiar (acción.tipo) {
    caso 'STORIES_FETCH_INIT':
      ...
    caso 'HISTORIAS_OBTENCIÓN_ÉXITO':
      devolver {
        ...estado,
        isLoading: falso, isError:
        falso,
        datos:
          acción.carga.página === 0 ?
            acción.carga.lista :
            estado.datos.concat(acción.carga.lista),
        página: acción.carga.página, };

    caso 'FALLO DE BÚSQUEDA DE HISTORIAS':
      ...
    caso 'REMOVE_STORY':
      ...
    por defecto:
      lanzar nuevo Error();
  };
};
```

La lista mostrada crece tras obtener más datos con el nuevo botón. Sin embargo, todavía hay un

El parpadeo dificulta la experiencia de usuario. Al obtener datos paginados, la lista desaparece un momento porque el indicador de carga aparece y reaparece tras la resolución de la solicitud.



El comportamiento deseado es renderizar la lista (que inicialmente está vacía) y reemplazar el botón "Más" con el indicador de carga solo para las siguientes solicitudes. Esta es una refactorización común de la interfaz de usuario para el renderizado condicional cuando la tarea evoluciona de una lista única a listas paginadas:

src/App.jsx

```
constante Aplicación = () => {
  ...

  devolver (
    <div>
      ...

      <Lista lista={historias.data} onRemoveItem={handleRemoveStory} />

      {historias.isLoading ?
        ( <p>Cargando ...</p> ) :
        (
          <button type="button" onClick={handleMore}>
            Más
          </botón>
        )
      }
    </div>

  );
};
```

Ahora es posible obtener datos actualizados de historias populares. Al trabajar con API de terceros, siempre es recomendable explorar su interfaz. Cada API remota devuelve estructuras de datos diferentes, por lo que sus funciones pueden variar.

Ceremonias:

- Compare su código fuente con el código [fuente](#) del autor³.
 - Recapitulación de todos los [cambios en el código fuente](#)³¹ de esta sección.
- Revise la [documentación de la API de Hacker News](#)³¹¹: ¿Hay alguna forma de obtener más elementos de una lista para una página simplemente añadiendo parámetros adicionales al punto final de la API?
- Consulte el inicio de esta sección, donde se habla de paginación y paginación infinita. ¿Cómo implementarías un componente de paginación normal con botones del 1 al [3] al 10, donde cada botón recupera y muestra solo una página de la lista?
- En lugar de tener un solo botón "Más", ¿cómo se implementaría la paginación infinita con una técnica de desplazamiento infinito? En lugar de hacer clic en un botón para ir a la página siguiente explícitamente, el desplazamiento infinito podría ir a la página siguiente una vez que la ventana gráfica del navegador llegue al final de la lista mostrada.

³ <https://tinyurl.com/yb86rh32>

³¹ <https://tinyurl.com/ypjw5ny4>

³¹¹<https://hn.algolia.com/api>

Implementación de una aplicación React

Ahora es el momento de lanzar tu aplicación React al mundo. Hay muchas maneras de implementar una aplicación React en producción y muchos proveedores que ofrecen este servicio. Lo simplificaremos, reduciéndolo a un solo proveedor, después de lo cual podrás explorar otros proveedores de alojamiento por tu cuenta.

Proceso de construcción

Hasta ahora, todo lo que hemos hecho ha sido la etapa de desarrollo de la aplicación, cuando el servidor de desarrollo se encarga de todo: empaqueta todos los archivos en una sola aplicación y los sirve en el host local de tu máquina local. Como resultado, nuestro código no está disponible para nadie más.

El siguiente paso es llevar la aplicación a la fase de producción alojándola en un servidor remoto (denominado implementación), lo que la hace accesible para los usuarios. Antes de que una aplicación pueda publicarse, debe empaquetarse como una aplicación esencial. Se eliminan el código redundante, el código de prueba y las duplicaciones. También se implementa un proceso llamado minificación que reduce el código. tamaño una vez más.

Afortunadamente, las optimizaciones y el empaquetado, también llamado agrupamiento, vienen con las herramientas de compilación en Vite. Primero, construya su aplicación en la línea de comando:

Línea de comandos

npm ejecutar compilación

Esto crea una nueva carpeta "dist/" en tu proyecto con la aplicación incluida. Podrías usar esta carpeta e implementarla en un proveedor de hosting ahora, pero usaremos un servidor local para simular este proceso antes de empezar. En la línea de comandos, sirve tu aplicación con el servidor HTTP local de Vite:

Línea de comandos

vista previa de ejecución de npm

Se presenta una URL que proporciona acceso a su aplicación optimizada, empaquetada y alojada. Se envía a través de una dirección IP local disponible en su red local, lo que significa que alojamos la aplicación en nuestra máquina local.

Implementar en Firebase

Tras crear una aplicación completa en React, el último paso es la implementación. Es el punto clave para materializar tus ideas, desde aprender a programar hasta producir aplicaciones. Usaremos Firebase Hosting para la implementación.

Firebase funciona con React, así como con la mayoría de las bibliotecas y frameworks como Angular y Vue. Primero, instala la CLI de Firebase globalmente en los módulos de nodo:

Línea de comandos

```
npm install -g herramientas de firebase
```

Usar una instalación global de Firebase CLI nos permite implementar aplicaciones sin preocuparnos por la dependencia del proyecto. Para cualquier paquete de nodo instalado globalmente, recuerde actualizarlo a una versión más reciente con el mismo comando siempre que sea necesario:

Línea de comandos

```
npm install -g herramientas de firebase
```

Si aún no tienes un proyecto de Firebase, regístrate para obtener una [cuenta de Firebase](https://console.firebase.google.com)³¹² Y crea un nuevo proyecto allí. Luego, puedes asociar la CLI de Firebase con la cuenta de Firebase (cuenta de Google):

Línea de comandos

```
inicio de sesión de Firebase
```

Se mostrará una URL en la línea de comandos que puedes abrir en un navegador o mediante la CLI de Firebase. Elige una cuenta de Google para crear un proyecto de Firebase y otorga a Google los permisos necesarios.

Regrese a la línea de comandos para verificar el inicio de sesión. A continuación, vaya a la carpeta del proyecto y ejecute el siguiente comando, que inicializa un proyecto de Firebase para las funciones de alojamiento de Firebase:

Línea de comandos

```
inicio de base de fuego
```

Luego, elige la opción de Hosting. Si te interesa usar otra herramienta además de Firebase Hosting, añade otras opciones:

³¹²<https://console.firebase.google.com>

Línea de comandos

¿Qué funciones de Firebase quieres configurar para este directorio?

Firestore: Configurar reglas de seguridad e indexar archivos para Firestore Functions: Configurar un directorio de Cloud Functions y sus archivos Hosting: Configurar archivos para Firebase Hosting... Alojamiento: Configurar implementaciones de acciones de GitHub Almacenamiento: Configurar un archivo de reglas de seguridad para Cloud Storage

Google detecta todos los proyectos de Firebase asociados a una cuenta tras iniciar sesión. Sin embargo, para este proyecto, comenzaremos con un nuevo proyecto en la plataforma Firebase:

Línea de comandos

Primero, asociemos este directorio de proyecto con un proyecto de Firebase.

Puedes crear varios alias de proyecto ejecutando `firebase use --add`, pero por ahora solo configuraremos un proyecto predeterminado.

? Por favor, seleccione una opción:

Utilice un proyecto existente
 Crear un nuevo proyecto
 Agregar Firebase a un proyecto existente de Google Cloud Platform
 No configure un proyecto predeterminado

Hay algunos pasos de configuración adicionales que definir. En lugar de usar la carpeta `public/` predeterminada, usaremos la carpeta `dist/` de Vite. Como alternativa, si configura la agrupación con una herramienta como Webpack, puede elegir el nombre adecuado para la carpeta de compilación:

Línea de comandos

¿Qué directorio público quieres usar? `dist` ¿Configurar como una aplicación de página única (reescribir todas las URL a `/index.html`)? Sí ¿Configurar compilaciones e implementaciones automáticas con GitHub? No Escribí `dist/index.html`

La aplicación Vite crea una carpeta `dist/` tras ejecutar `npm run build` por primera vez. Esta carpeta contiene todo el contenido fusionado de las carpetas `public/` y `src/`. Al ser una aplicación de una sola página, queremos redirigir al usuario al archivo `index.html` para que el enrutador de React pueda gestionar el enrutamiento del lado del cliente.

La inicialización de Firebase ya está completa. Este paso creó algunos archivos de configuración para Firebase Hosting en la carpeta de tu proyecto. Puedes leer más sobre ellos en [la documentación de Firebase](https://firebase.google.com/docs/hosting)³¹³ Para configurar redirecciones, una página 404 o encabezados. Finalmente, implementa tu aplicación React con Firebase en la línea de comandos:

³¹³<https://bit.ly/3DVgbpG>

Línea de comandos

Implementación de Firebase

Después de una implementación exitosa, debería ver un resultado similar con el identificador de su proyecto:

Línea de comandos

Consola del proyecto: <https://console.firebase.google.com/project/my-react-project-abc123/overview>

URL de alojamiento: <https://my-react-project-abc123.firebaseio.com>

Visita ambas páginas para observar los resultados. El primer enlace te lleva al panel de control de tu proyecto de Firebase, donde verás un nuevo panel para Firebase Hosting. El segundo enlace te lleva a tu aplicación React implementada.

Si ves una página en blanco para tu aplicación React implementada, asegúrate de que el par clave-valor público en `firebase.json` esté configurado como `dist` (o el nombre que hayas elegido para esta carpeta). En segundo lugar, verifica que hayas ejecutado el script de compilación de tu aplicación React con `npm run build`. Por último, consulta el área [oficial de solución de problemas para implementar aplicaciones Vite en Firebase](#)³¹. Pruebe otra implementación con Firebase Deploy.

Ceremonias:

- Lea más sobre [Firebase Hosting](#)³¹.
 - [Conecte su dominio a su aplicación implementada de Firebase](#)³¹.
- Opcional: Si desea tener un servidor en la nube administrado, consulte [DigitalOcean](#)³¹ o [Hetzner](#)³¹. Es más trabajo, pero permite más control. También puedes conectarlo a algo como [Coolify](#)³¹ para obtener una mejor experiencia de usuario.

³¹ <https://bit.ly/3Sp2Xsn>

³¹ <https://bit.ly/3lXypAC>

³¹ <https://bit.ly/3phFxdp>

³¹ <https://m.do.co/c/fb27c90322f3>

³¹ <https://www.hetzner.com/>

³¹ <https://coolify.io/>

Describir

Hemos llegado al final de "El Camino a React", y espero que hayan disfrutado de la lectura y les haya ayudado a familiarizarse con React. Si les gustó el libro, compártanlo con sus amigos interesados en aprender más sobre React. También pueden encontrar una reseña en [Amazon](#)³² o [Goodreads](#)³²¹. Sería muy apreciado.

Desde aquí, te recomiendo ampliar la aplicación para crear tus propios proyectos de React antes de empezar con otro libro, curso o tutorial. Pruébala durante una semana, llévala a producción implementándola y ponte en contacto conmigo o con otros para mostrarla. Siempre me interesa ver lo que han creado mis lectores y aprender cómo puedo ayudarles.

Si buscas extensiones para tu aplicación, te recomiendo varias rutas de aprendizaje una vez que hayas dominado los fundamentos:

- Marco: Explore [Next.js](#)³²² Para crear aplicaciones React más sofisticadas. Incluye componentes y funciones de servidor de React, renderizado del lado del servidor (SSR), enrutamiento basado en archivos y más. Es el siguiente paso natural tras dominar React como biblioteca. Si eliges este camino, consulta mi curso " [El camino hacia el futuro](#)".
- Enrutamiento: implemente el enrutamiento en su aplicación con [React Router](#)³² . Hasta ahora, solo hemos creado una aplicación de una sola página, pero a medida que su proyecto crezca, React Router le ayudará a administrar varias páginas en diferentes URL, todo sin necesidad de solicitudes de servidor adicionales.
- Organización del código: si aún no lo ha hecho, vuelva a consultar el capítulo sobre organización del código y aplique esas [prácticas recomendadas](#)³² . Estructurar adecuadamente sus componentes ayudará con la mantenibilidad, reutilización y escalabilidad a medida que su aplicación crece.
- Herramientas con Webpack y Babel: Usamos Vite para configurar el proyecto de este libro, pero en algún momento querrás comprender las herramientas que lo sustentan para crear proyectos sin depender de Vite. Recomiendo empezar con un [Webpack mínimo](#)³² configuración antes de explorar configuraciones adicionales.
- Conexión a una base de datos y/o autenticación: A medida que tus aplicaciones React crecen, el almacenamiento de datos persistente se vuelve esencial. Una base de datos te permite conservar datos entre sesiones y compartirlos entre usuarios. Firebase es una de las maneras más sencillas de implementar una base de datos sin tener que escribir una aplicación backend. Mi libro, «[El camino a Firebase](#)»³² , Proporciona una guía paso a paso sobre el uso de la autenticación de Firebase y la integración de bases de datos en React.

³² <https://amzn.to/2JHlP42> ³²¹<https://tinyurl.com/4bhcssu7> ³²²<https://nextjs.org/> ³²³<https://www.road-to-next.com/> ³² <https://www.robinwieruch.de/react-router/> ³² <https://www.robinwieruch.de/react-folder-structure/> ³² <https://www.robinwieruch.de/minimal-react-webpack-babel-setup/> ³² <https://www.roadtofirebase.com/>

- Conexión a un backend: Hasta ahora, solo hemos solicitado datos de una API de terceros. Puedes ir más allá creando tu propio backend que se conecte a una base de datos y gestione la autenticación y la autorización. En "[El camino hacia GraphQL](#)"³², Enseño cómo usar GraphQL para la comunicación cliente-servidor, conectar un backend a una base de datos, gestionar sesiones de usuario e interactuar con un backend mediante una API de GraphQL. • Gestión de estado: En este libro, hemos gestionado el estado local dentro de los componentes de React, lo cual es suficiente para muchas aplicaciones. Sin embargo, las aplicaciones más grandes pueden beneficiarse de soluciones externas de gestión de estado. Abordo la más popular en mi libro, "[The Road to Redux](#)"³². Pruebas : En este libro, solo hemos abordado superficialmente las pruebas. Si no está familiarizado con las pruebas de aplicaciones web, [profundice en el tema](#)³³ en pruebas unitarias y de integración, especialmente utilizando React Testing Library para pruebas de componentes y Cypress para pruebas de extremo a extremo.
- Comprobación de tipos: Anteriormente, usamos brevemente TypeScript en React. TypeScript ayuda a prevenir errores y mejora la experiencia del desarrollador. Considere profundizar en él para robustecer sus aplicaciones JavaScript; incluso podría decidir migrar a TypeScript por completo. • Componentes de interfaz de usuario: Muchos principiantes introducen bibliotecas de componentes de interfaz de usuario (como Material UI) demasiado pronto. Es mejor aprender primero a crear elementos esenciales de la interfaz de usuario (menús desplegables, casillas de verificación, diálogos) usando elementos HTML estándar y el estado de React. Una vez que domines estos conceptos básicos, adoptar una biblioteca de componentes de interfaz de usuario será mucho más fácil. • React Native: [React Native](#)³³¹ Te permite crear aplicaciones móviles para iOS y Android con React. Una vez que te familiarices con React, la transición a React Native es relativamente sencilla, ya que comparten conceptos básicos. Las principales diferencias radican en los componentes de diseño móvil, las herramientas de desarrollo y las API específicas de cada plataforma.

Te invito a visitar mi [sitio web](#)³³² para más temas sobre desarrollo web e ingeniería de software.

También puedes [suscribirte a mi Newsletter](#)³³³ Para estar al tanto de nuevos artículos, libros y cursos. Si solo has leído el libro y quieres explorar más, visita el [sitio web oficial del curso](#)³³.

Gracias por leer El camino para reaccionar.

Atentamente, Robin Wieruch

³² <https://www.roadtographql.com/>

³² <https://www.roadtoredux.com/>

³³ <https://www.robinwieruch.de/react-testing-tutorial/>

³³¹<https://reactnative.dev/>

³³²<https://www.robinwieruch.de>

³³³<https://rwieruch.substack.com/>

³³ <https://www.roadtoreact.com/>